

Digital Design SET111

Lecture (01)- Course Introduction

Dr. Ahmed Reda Mohamed¹

¹The MoE Key Laboratory of Artificial Intelligence & Bio-Circuits and Systems Laboratory (**BiCASL**), Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.

Outlines



- ☐ **Course Information**
- ☐ **Course Assessment**
- ☐ **Course Content**
- ☐ **Course Long-Term-Vision**
- ☐ **IC Industry In Egypt**
- ☐ **References**

Outlines



- ☐ **Course Information**
- ☐ **Course Assessment**
- ☐ **Course Content**
- ☐ **Course Long-Term-Vision**
- ☐ **IC Industry In Egypt**
- ☐ **References**

Course Information



☐ Lecture's Time:

☐ 10.30: 12.30@ Tuesday

☐ Lecture's Place:

☐ A308

☐ Lectures Materials, Quiz, Assignment, and Project:

☐ Please, check ECU-LMS

☐ Please, check Microsoft team

☐ Contacts:

☐ Email : A.reda@ecu.edu.eg , Ahmed.reda@sjtu.edu.cn

☐ Whatsapp: 01020202427



Outlines



- ☐ Course Information
- ☒ Course Assessment
- ☐ Course Content
- ☐ Course Long-Term-Vision
- ☐ IC Industry In Egypt
- ☐ References

Course Assessment



☐ Final Exam [40%]

- ☐ You must get > **30%** of the paper grade or you will **fail** in the course regardless of your course work.

☐ Lab, Quiz, Project, and Assignment [30%]

- ☐ You must attend all course sessions otherwise you will **lose** some degrees

☐ Midterm [30%]

- ☐ 1st midterm [20%] → 7th week
- ☐ 2nd midterm [10%] → 12th week

☐ Project [bonus]

- ☐ 3:5 students



Forbidden

Copy or cheat during
Exams and even doing the
Quiz and **Assignment**

Outlines



- ☐ Course Information
- ☐ Course Assessment
- ☒ Course Content
- ☐ Course Long-Term-Vision
- ☐ IC Industry In Egypt
- ☐ References

Course Content (1/2)



☐ Course Objective

- ☐ Overview of important related topics on digital design systems to expand students' horizons about the digital design of integrated circuit or discrete element.
The course consists of the **six** modules.

☐ Module 1: Introductory Concepts.

- ☐ Digital and Analog Quantities, Binary Digits, Logic Levels, and Digital Waveforms, Programmable and Fixed-Function Logic Devices.

☐ Module 2: Number Systems, Operations, and Codes.

- ☐ Decimal, Binary , Octal, Hexadecimal Numbers and their conversion. Complements of Binary Numbers. Logic gates.

☐ Module 3: Boolean Algebra and Logic Simplification.

- ☐ Laws and Rules of Boolean Algebra, DeMorgan's Theorems, Boolean Expressions and Truth Tables, Karnaugh Map minimization.

Course Content (2/2)



□ Course Objective

- Overview of important related topics on digital design systems to expand students' horizons about the digital design of integrated circuit or discrete element.
The course consists of the **six** modules.

□ Module 4: Combinational Logic.

- Implementing Combinational Logic, Universal NAND and NOR Gates, Adder, Comparators, Decoders, Encoders, Code Converters, Multiplexers and Demultiplexers, Parity Generators/Checkers.

□ Module 5: Synchronous Sequential Logic

- Latch, Flip-Flop, Shift Register, Finite State Machine, Counters

□ Module 6: Programmable Logic

- Simple Programmable Logic Devices (SPLDs), Complex Programmable Logic Devices (CPLDs), Field-Programmable Gate Arrays (FPGAs)

Outlines



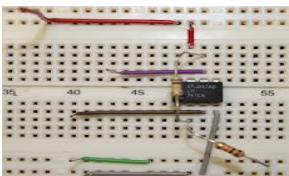
- ☐ Course Information
- ☐ Course Assessment
- ☐ Course Content
- ☐ **Course Long-Term-Vision**
- ☐ IC Industry In Egypt
- ☐ References



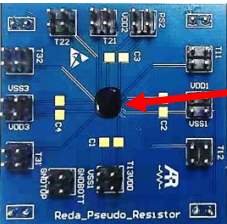
Course Long-Term-Vision (1/3)

Discrete Element

Vey large Scale Integrated Circuit



Bread Board

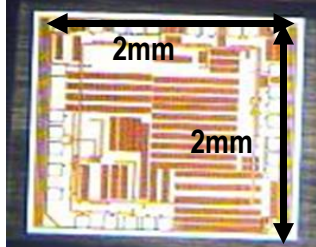


Chip-On-Board (COB)

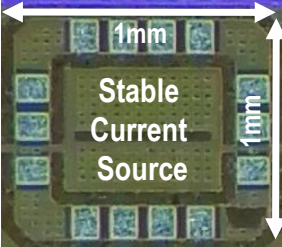


Printed Circuit Board (PCB)

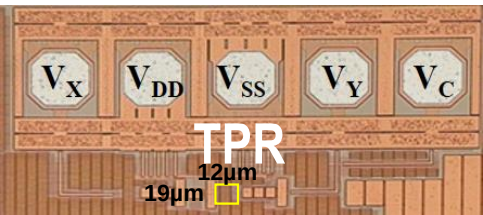
- ☐ Through-hole
- ☐ Surface mount



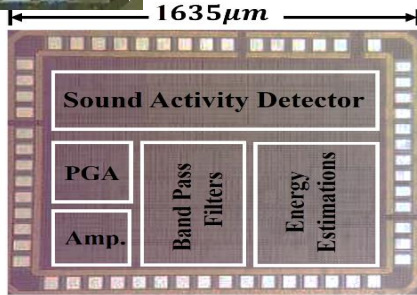
2mm 2mm



1mm 1mm
Stable Current Source

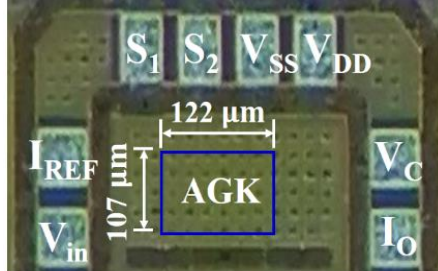


V_X V_{DD} V_{SS} V_Y V_C
TPR
12µm
19µm



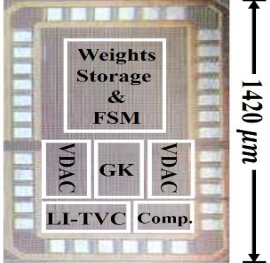
1635µm 800µm

Sound Activity Detector
PGA
Band Pass Filters
Energy Estimations
Amp.



1420µm 107µm

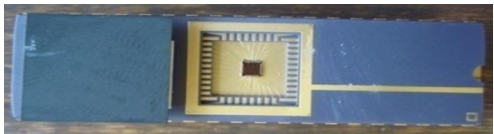
S_1 S_2 V_{SS} V_{DD}
 I_{REF} V_{in} V_C I_O
122µm
AGK



1420µm 800µm

Weights Storage & FSM
VDAC GK VDAC
LI-TVC Comp.

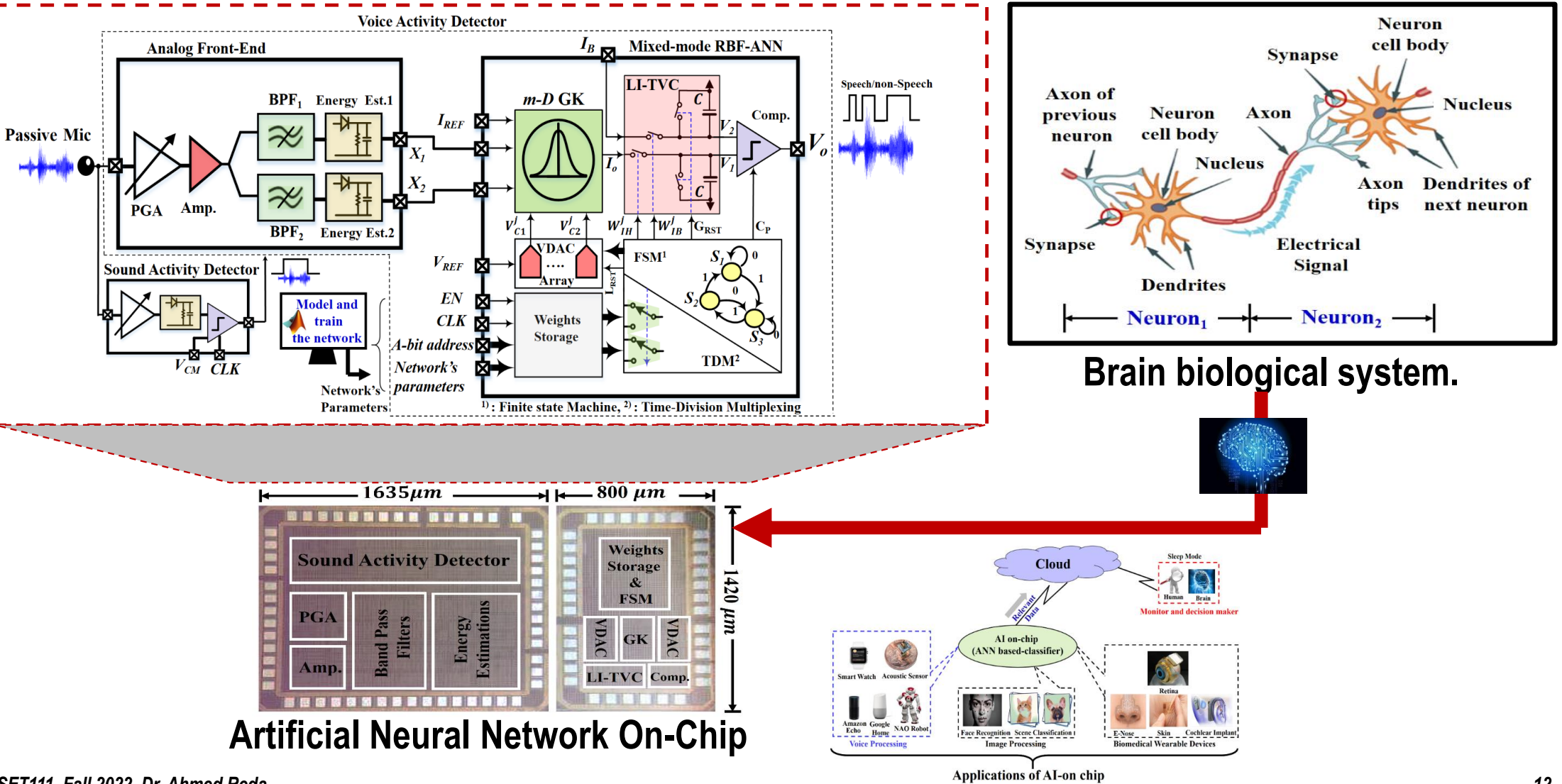
A. Reda



MOSIS (Granted)

Item	Discrete Element	Very large scale Integrated Circuit
Area	Large	Small
Configuration	Easy	Complex
Price	Cheap	Expensive
Applications	Small production	Mass production
Power consumption	High	Low

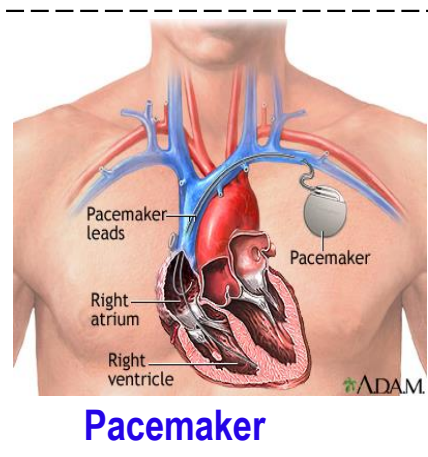
Course Long-Term-Vision (2/3)



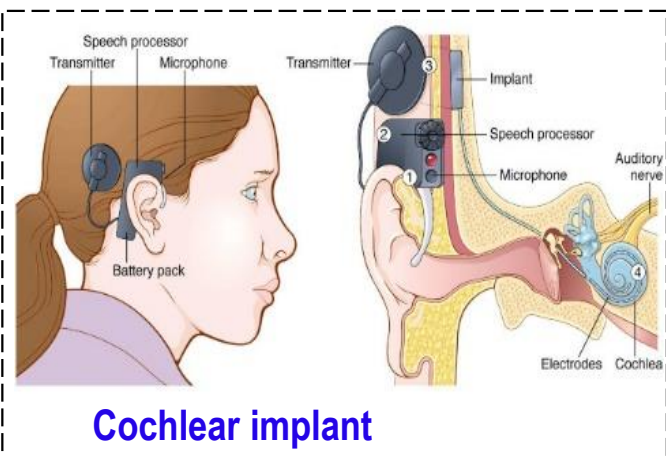


Course Long-Term-Vision (4/4)

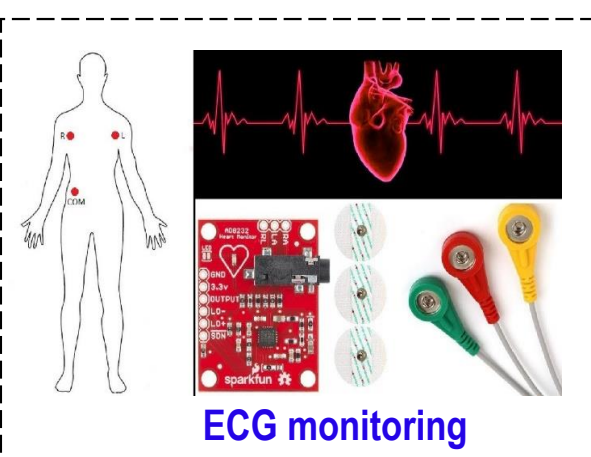
Electronics in Bio-medical Devices



Pacemaker



Cochlear implant

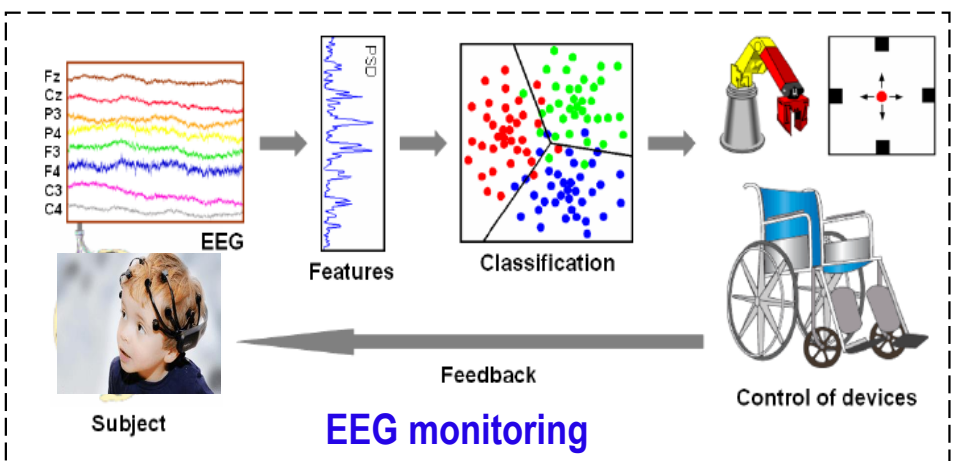


ECG monitoring

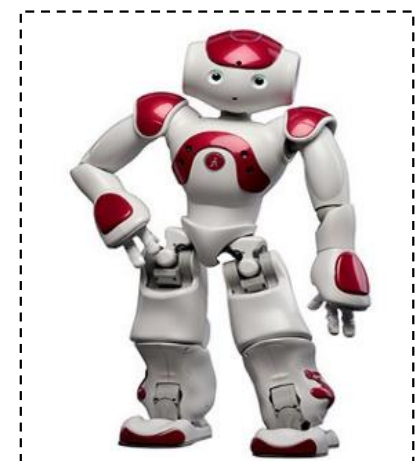
ECG	ElectroCardio-Gram
PPG	Photo-Plethy-smo Graphy
EEG	Electro-encephalo-gram



PPG monitoring



EEG monitoring



Outlines



- ☐ Course Information
- ☐ Course Assessment
- ☐ Course Content
- ☐ Course Long-Term-Vision
- ☒ IC Industry In Egypt
- ☐ References

IC Industry In Egypt



Outlines



- ☐ **Course Information**
- ☐ **Course Assessment**
- ☐ **Course Content**
- ☐ **Course Long-Term-Vision**
- ☐ **IC Industry In Egypt**
- ☐ **References**

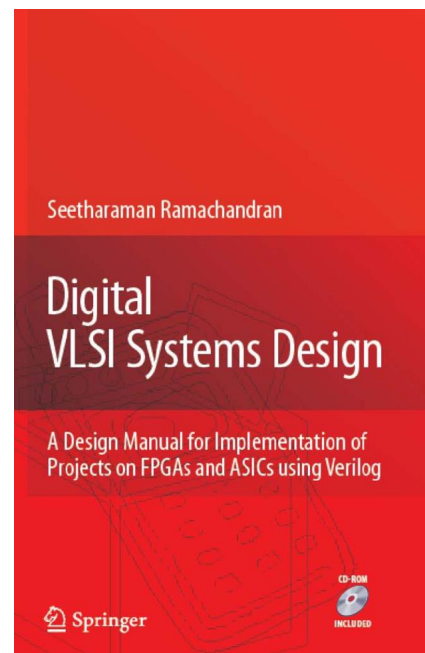
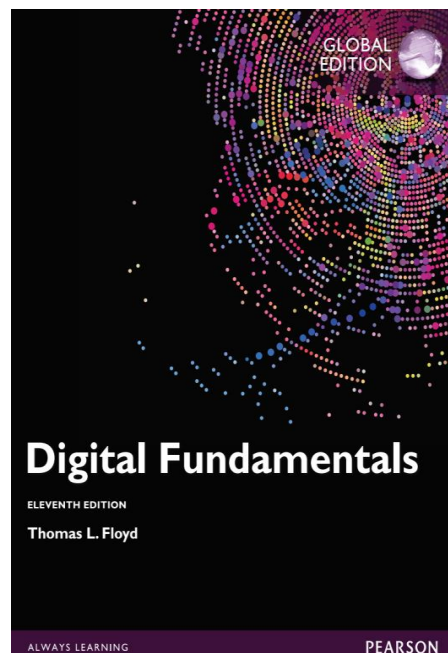
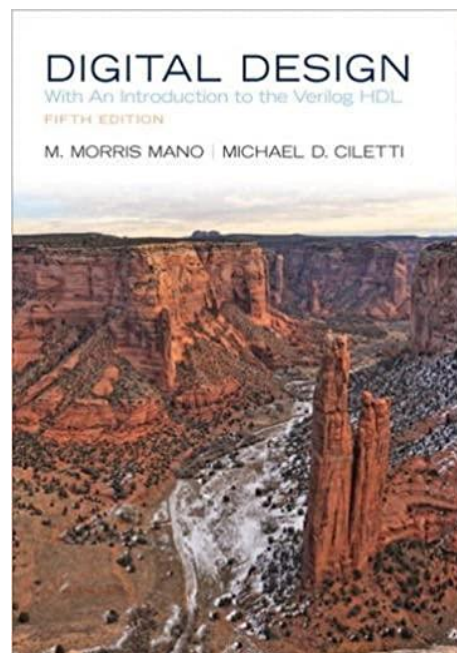
References

- ❑ M. M. Mano and M. D. Ciletti, *Digital design With an Introduction to the Verilog HDL*, 5th Ed., Prentice-Hall, Upper Saddle River, NJ, 2013.
- ❑ T. L. Floyd, *Digital Fundamentals*, 11th Ed., Pearson Education Limited, 2015.
- ❑ S. Ramachandran, *Digital VLSI Systems Design A Design Manual for Implementation of Projects on FPGAs and ASICs Using Verilog*, Springer, 2007.

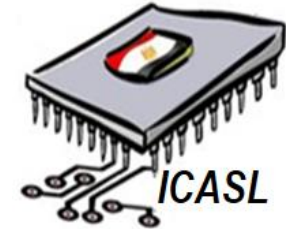
Stay on target
[100%]



- ❑ Lectures
- ❑ Sections
- ❑ Labs
- ❑ Project



Thanks!



Digital Design SET 111

Lecture (02)- Introductory Concepts

Dr. Ahmed Reda Mohamed¹

¹The MoE Key Laboratory of Artificial Intelligence & Bio-Circuits and Systems Laboratory (**BiCASL**), Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.

Outlines

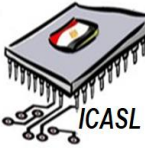


- ☐ **Digital and Analog Quantities**
- ☐ **Binary Digits, Logic Levels, and Digital Waveforms**
- ☐ **Basic Logic Functions**
- ☐ **Programmable Logic**
- ☐ **Fixed-Function Logic Device**

Outlines

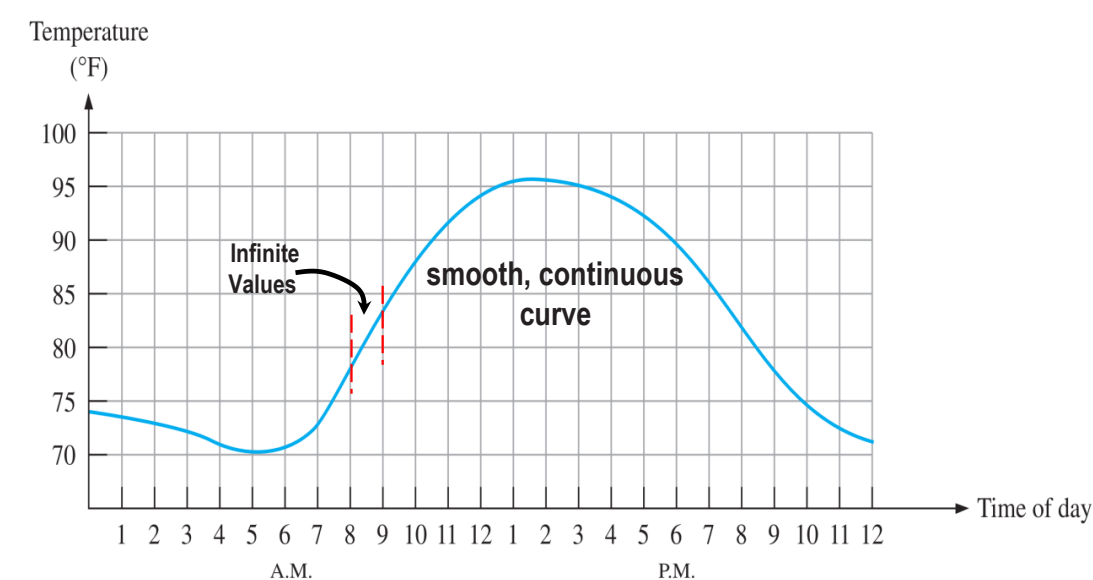


- ☐ **Digital and Analog Quantities**
- ☐ **Binary Digits, Logic Levels, and Digital Waveforms**
- ☐ **Basic Logic Functions**
- ☐ **Programmable Logic**
- ☐ **Fixed-Function Logic Device**

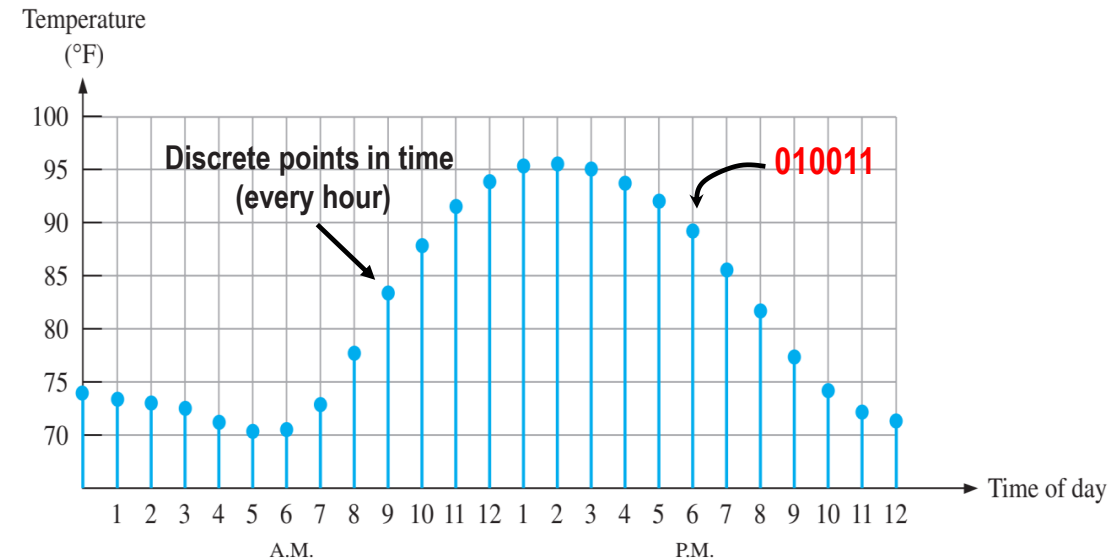


Digital and Analog Quantities (1/3)

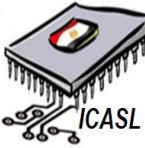
- ❑ Electronic circuits can be divided into **two** broad categories:
 - ❑ **Analog electronics** involves quantities with **continuous** values.
 - ❑ **Digital electronics** involves quantities with **discrete** values.



- ❑ Most things that can be measured quantitatively occur in **nature** in **analog form**



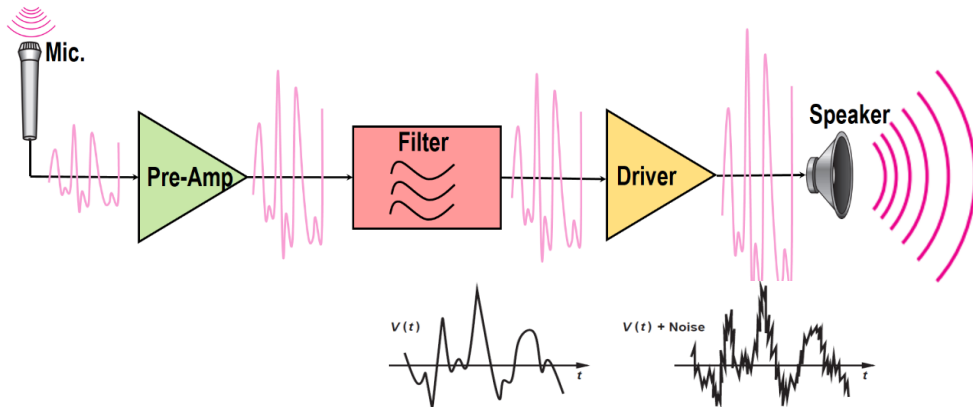
- ❑ Sampled-value representation (**quantization**) of the **analog quantity**
- ❑ Each value represented by a **dot** can be digitized by representing it as a **digital code** (series of 1s and 0s).



Digital and Analog Quantities (2/3)

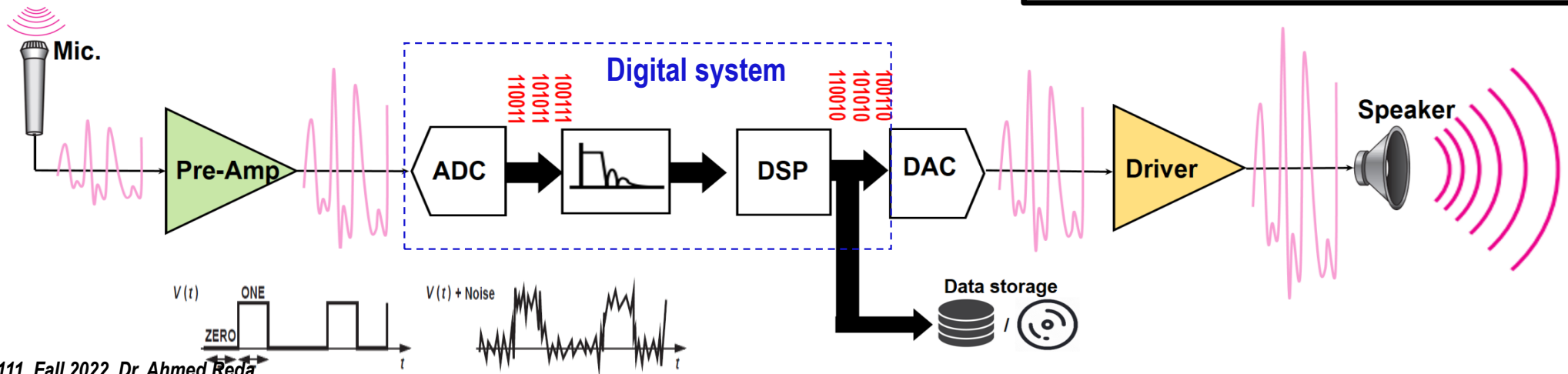
□ Analog system

Sound/Speech



□ Analog and digital system (mixed-mode system)

Sound/Speech



Digital System Advantages

High immunity to noise (Robust).
Stored in a digital memories.
Easily compressed and encrypted.



Digital System Disadvantages

Sampling may cause loss of information.
A/D and D/A consume high power.
Processing is more complex.



Digital and Analog Quantities (3/3)



Mechatronics

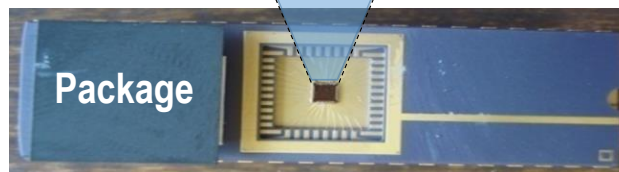
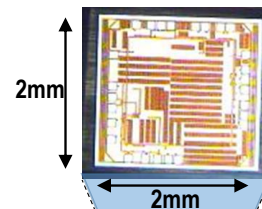


Robotic arm



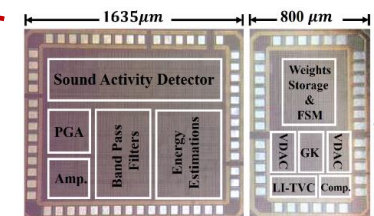
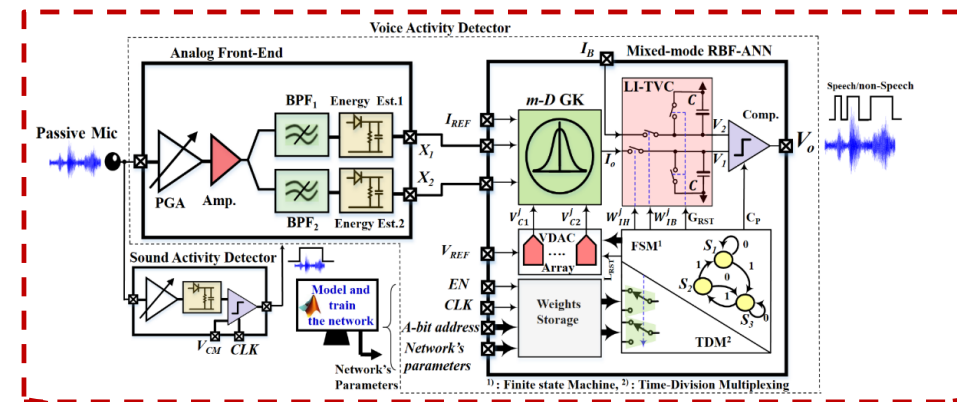
Automotive assembly line

Power Electronics



A. Reda

Biomedical Applications



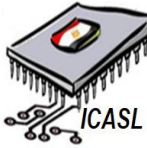
A. Reda

Outlines



- ☐ Digital and Analog Quantities
- ☐ Binary Digits, Logic Levels, and Digital Waveforms
- ☐ Basic Logic Functions
- ☐ Programmable Logic
- ☐ Fixed-Function Logic Devices

Binary Digits, Logic Levels, and Digital waveforms (1/5)



- ❑ Digital electronics involves circuits and systems use only **two** possible states:

HIGH → **1** and **LOW** → **0**

- ❑ The **two-state** number system is called **binary**.
- ❑ Combinations of the **two states** is called **codes** to represent, numbers, symbols, Alphabetic characters, and Instructions .

101101010 Code
↑
Bit

❑ Binary Digits

- ❑ Each of the two digits in the **binary** system, **1** and **0**, is called a **bit**, which is a contraction of the words **binary digit**.
- ❑ In the **positive logic**, **1** is represented by a **HIGH** and a **0** is represented by a **LOW**
- ❑ In the **negative logic**, **1** is represented by a **LOW** and a **0** is represented by a **HIGH**

Binary Digits, Logic Levels, and Digital waveforms (2/5)



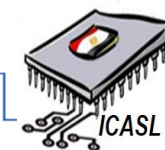
□ Logic Levels

□ The **voltages** used to represent a **High**→1 and a **Low**→0 are called **logic levels**.

□ Ideally, **High** level is represented by a **single voltage value**. Also, **Low** level is represented by another voltage value.



Binary Digits, Logic Levels, and Digital waveforms (2/5)

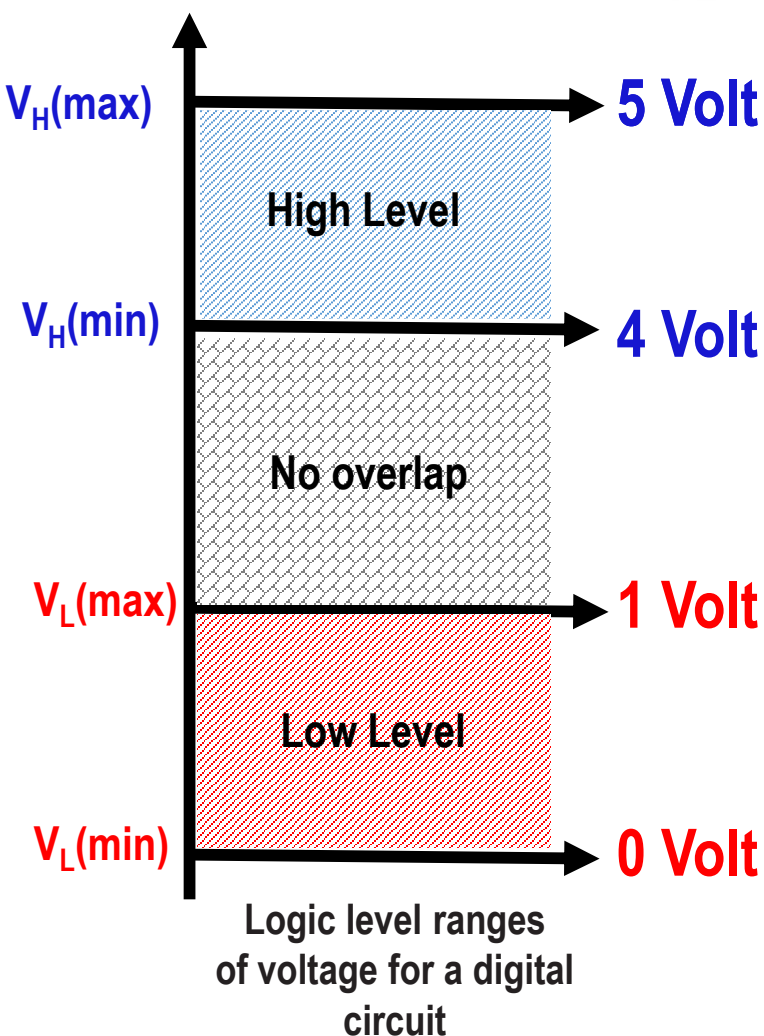


Logic Levels

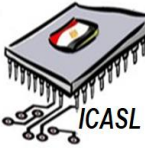
❑ The **voltages** used to represent a **High**→1 and a **Low**→0 are called **logic levels**.

❑ In a **practical** digital circuit, a **HIGH** can be any voltage between a specified minimum value and a specified maximum value. Likewise, a **LOW** can be any voltage between a specified minimum and a specified maximum.

❑ There can be **no overlap** between the accepted range of **HIGH** levels and the accepted range of **LOW** levels.

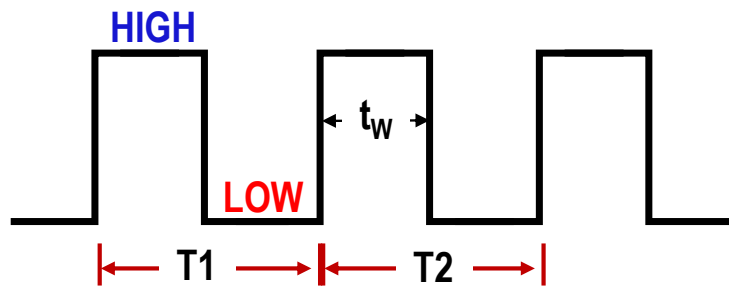


Binary Digits, Logic Levels, and Digital waveforms (3/5)

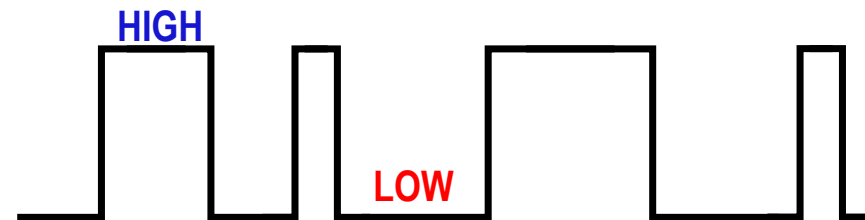


□ Digital Waveforms

- consist of voltage levels that are changing back and forth between the **HIGH** and **LOW** levels.
- are composed of **series of pulses**, sometimes called **pulse trains**



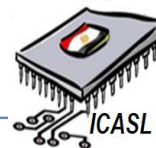
Periodic Waveform



Non-Periodic Waveform

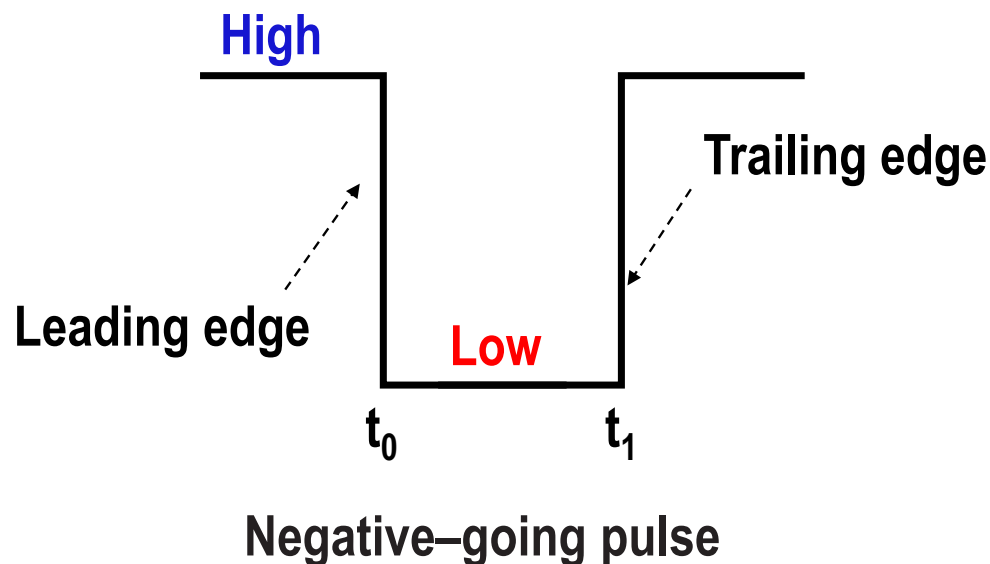
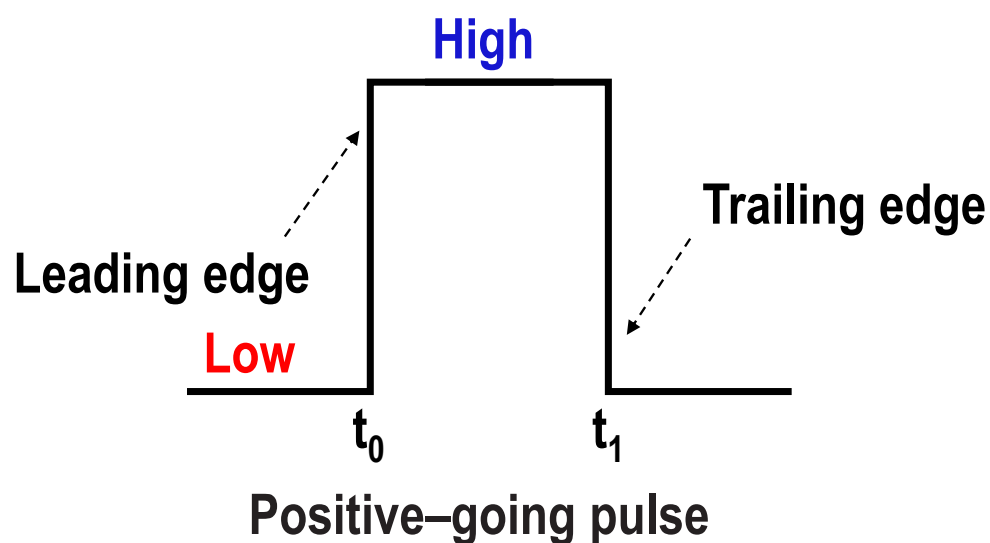
- Period = $T_1 = T_2 = \dots = T$
 - Frequency = $1/T$ (Hz)
 - An important characteristic of a periodic digital waveform is its **duty cycle**, which is the ratio of the **pulse width** (t_W) to the **period** (T)
- Duty Cycle = $\left(\frac{T_W}{T}\right) * 100 \%$

Binary Digits, Logic Levels, and Digital waveforms (4/5)

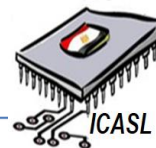


□ Pulse's Characteristics

- A pulse has **two** edges: a **leading edge** that occurs first at time t_0 and a **trailing edge** that occurs last at time t_1

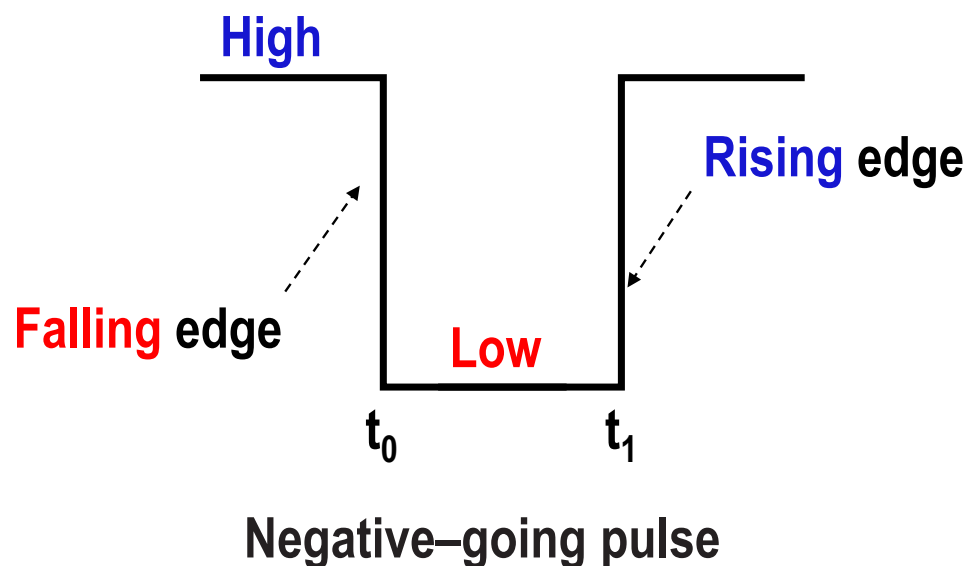
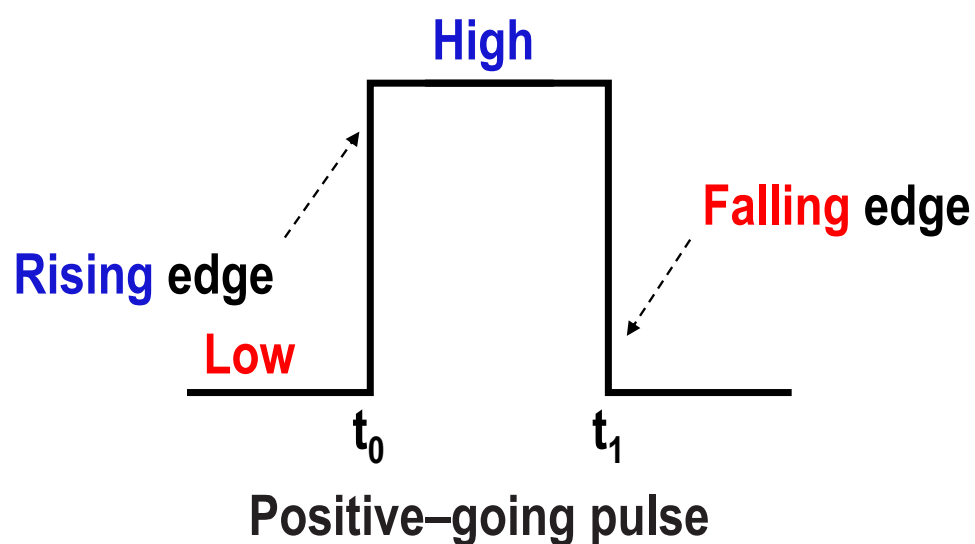


Binary Digits, Logic Levels, and Digital waveforms (4/5)

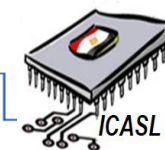


□ Pulse's Characteristics

- A pulse has **two** edges: a **leading edge** that occurs first at time t_0 and a **trailing edge** that occurs last at time t_1
- In ideal pulse, the **rising** and **falling** edges are assumed to change in **zero** time.

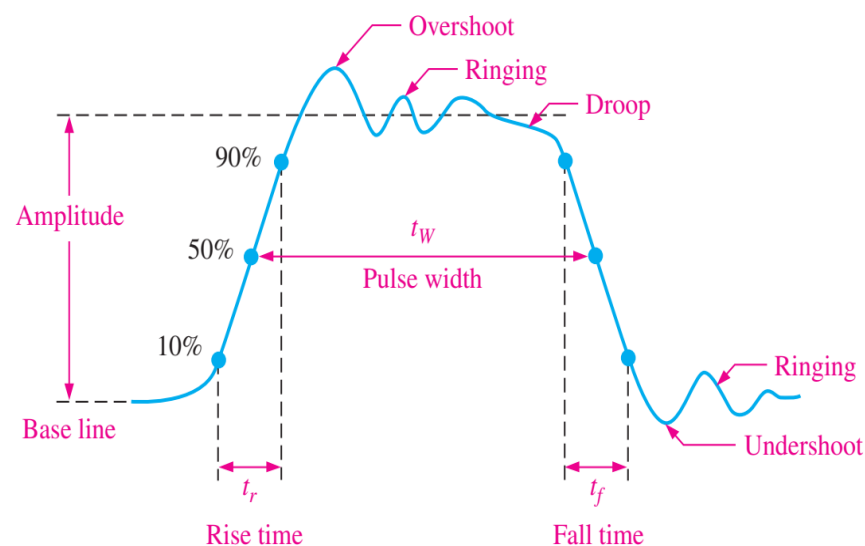
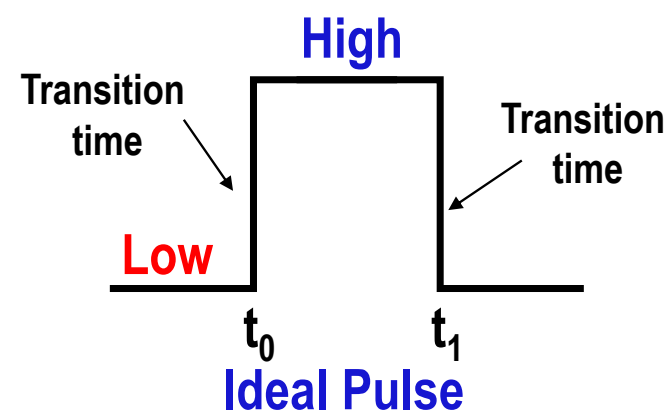


Binary Digits, Logic Levels, and Digital waveforms (5/5)



❑ Pulse's Characteristics

- ❑ In practice, these transitions **never** occur instantaneously.
- ❑ **Rise time** (t_r) from **10%** : **90%** of the pulse amplitude.
- ❑ **Fall time** (t_f) from **90%** : **10%** of the pulse amplitude.
- ❑ **Pulse width** (t_w) from **50%** points on the rising and falling edges.
- ❑ The **overshoot** and **ringing** are produced by stray **inductive** and **capacitance** elements.
- ❑ The droop can be caused by stray circuit **resistance**.



Non-Ideal (Practical) Pulse

Outlines

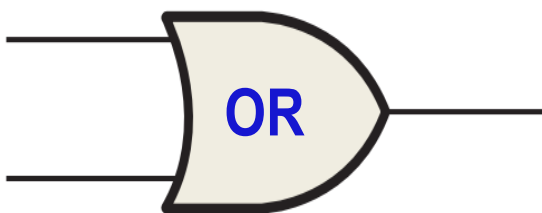
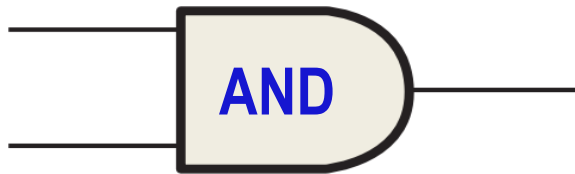
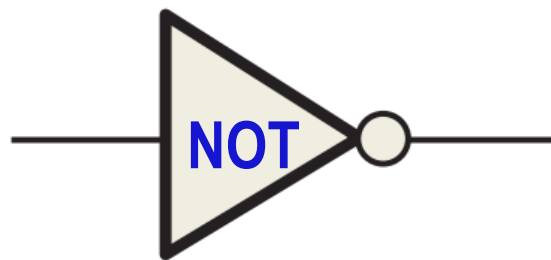


- ☐ Digital and Analog Quantities
- ☐ Binary Digits, Logic Levels, and Digital Waveforms
- ☐ Basic Logic Functions
- ☐ Programmable Logic
- ☐ Fixed-Function Logic Devices

Basic Logic Functions (1/2)



- ❑ The **three** basic logic functions AND, OR, and NOT

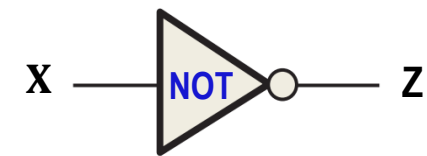


Form various other
types of more complex
logic functions

- ❑ Comparison
- ❑ Arithmetic
- ❑ Code conversion
- ❑ Encoding
- ❑ Decoding
- ❑ Data selection
- ❑ Counting
- ❑ Storage

Basic Logic Functions (2/2)

- ❑ The **NOT** function changes one logic level to the **opposite** logic level
- ❑ The **AND** function produces a **HIGH** output **only** when all the inputs are HIGH
- ❑ The **OR** function produces a **HIGH** output when one **or** more inputs are HIGH



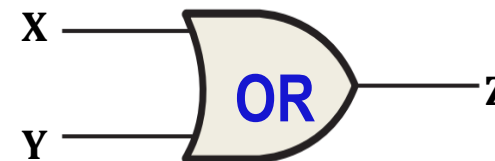
$$Z = \bar{X}$$

X	Z
0	1
1	0



$$Z = X.Y = XY$$

X	Y	Z
0	0	0
0	1	0
1	0	0
1	1	1



$$Z = X + Y$$

X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	1

I/P
O/P

Note

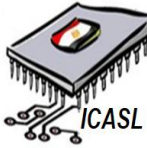
- ❑ Truth table is a combinations of binary variables
- ❑ # of rows = 2^n , where n = # of input variables

Outlines

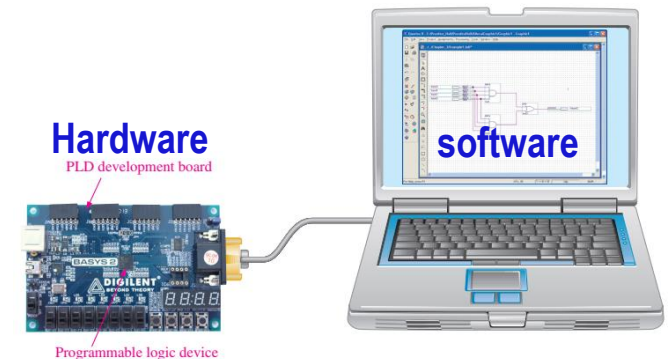


- ☐ **Digital and Analog Quantities**
- ☐ **Binary Digits, Logic Levels, and Digital Waveforms**
- ☐ **Basic Logic Functions**
- ☐ **Programmable Logic**
- ☐ **Fixed-Function Logic Devices**

Programmable Logic (1/3)

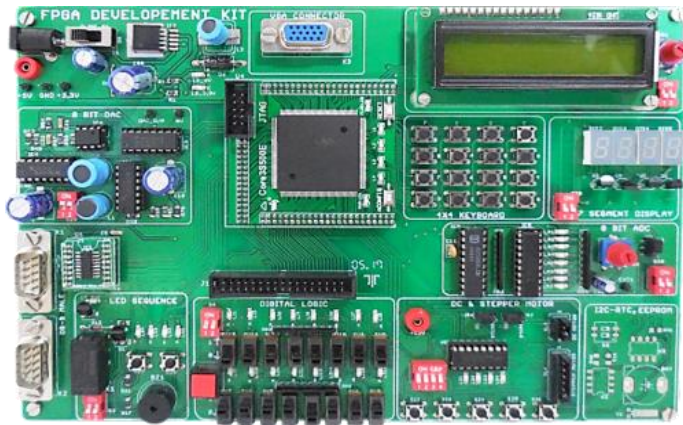


- ❑ Programmable logic requires both **hardware** and **software**.
- ❑ Programmable logic devices can be **programmed** to perform specified logic functions and operations by the **manufacturer** or by the **user**.

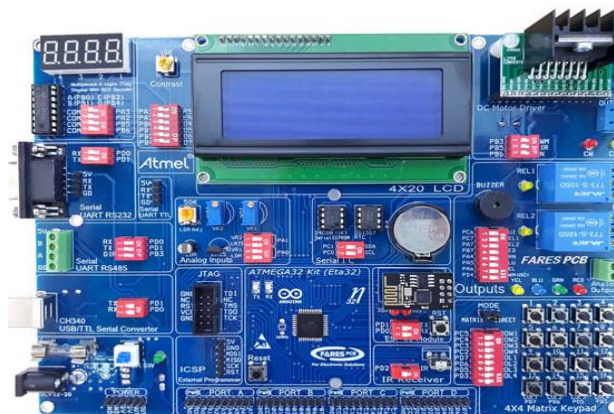


- ❑ **Advantages** of programmable logic.
 - ❑ Designs can be readily changed **without** rewiring or replacing components.
 - ❑ A logic design can generally be implemented **faster** and with **less cost**.
- ❑ **Disadvantage**
 - ❑ It requires **higher cost**.
 - ❑ It is **not efficient** to implement small segments of logic.

Programmable Logic (2/3)



Xilinx Spartan 3E FPGA Development Board



AVR Development Kits

VHDL
Verilog
Assembly
C,C++
Python

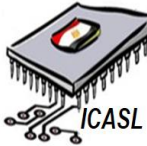


Raspberry Pi RPI4-MODBP-4GB-BULK



Arduino Nano Arduino

Programmable Logic (3/3)



User-programmable logic

PLD

(programmable logic device)

FPGA

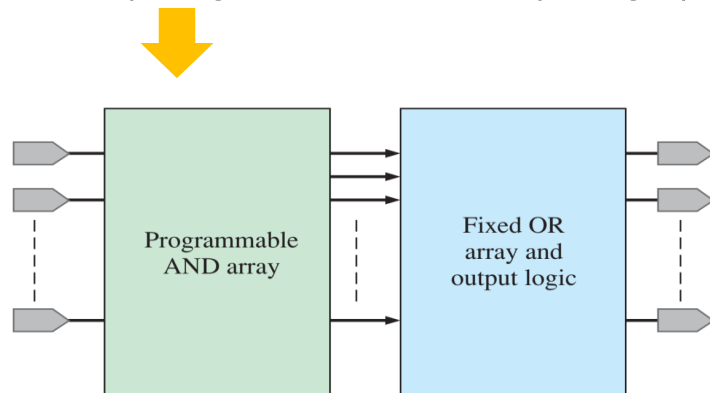
(field-programmable gate array)

Simple PLDs

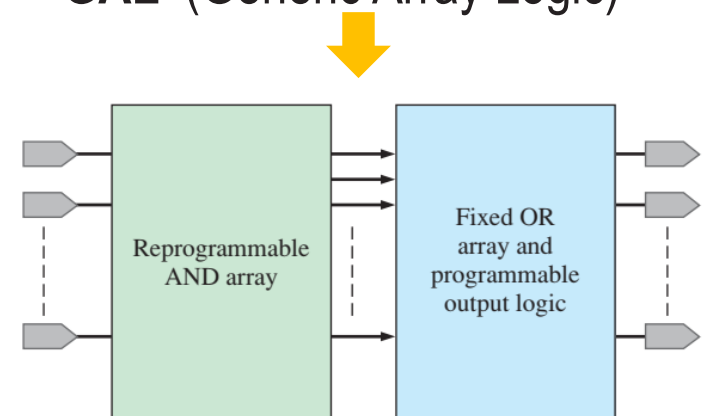
Complex PLDs

PAL (Programmable Array Logic)

GAL (Generic Array Logic)

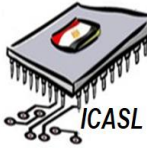


Programmed one time



Reprogrammed many times

Programmable Logic (3/3)



User-programmable logic

PLD

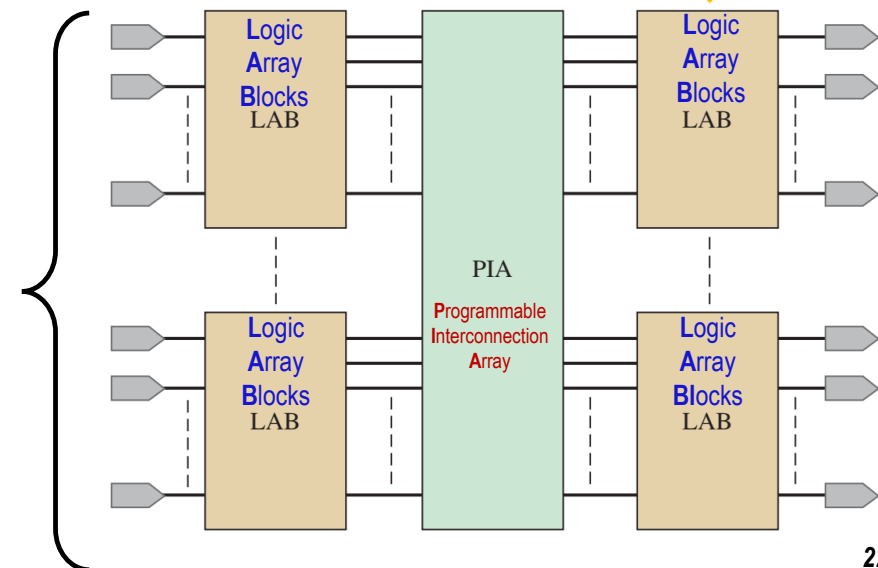
(programmable logic device)

FPGA

(field-programmable gate array)

Simple PLDs

Complex PLDs



- ❑ Put more than one **SPLD** on a **single** chip
- ❑ Containing multiple **SPLDs** and can replace many fixed-function ICs.
- ❑ Implement any of the logic functions (decoders, encoders, multiplexers, demultiplexers, and adders)

Programmable Logic (3/3)



User-programmable logic

PLD

(programmable logic device)

FPGA

(field-programmable gate array)

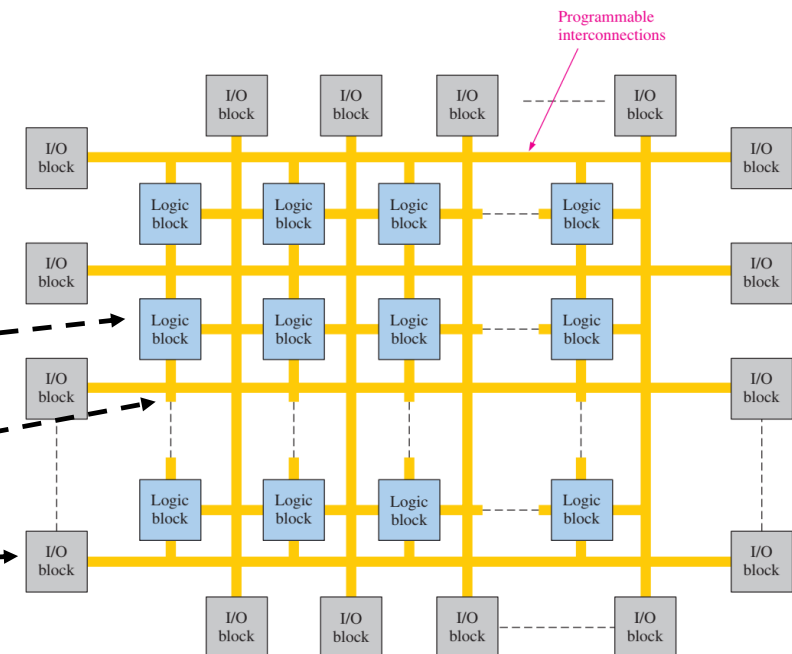
❑ An **FPGA** is generally more complex and has a much higher density than a **CPLD**

❑ **Three** basic elements in an FPGA are

❑ Logic block

❑ Programmable interconnections

❑ Input/output (I/O) blocks



Basic structure of an FPGA

Outlines



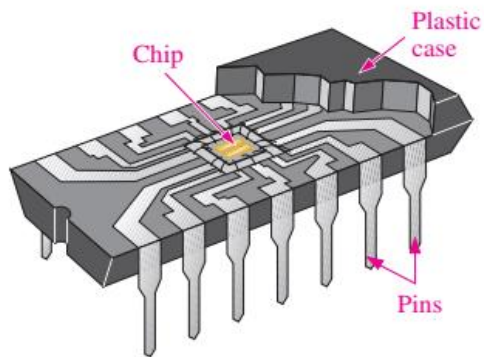
- ☐ Digital and Analog Quantities
- ☐ Binary Digits, Logic Levels, and Digital Waveforms
- ☐ Basic Logic Functions
- ☐ Programmable Logic
- ☐ Fixed-Function Logic Devices

Fixed-Function Logic Devices (1/4)

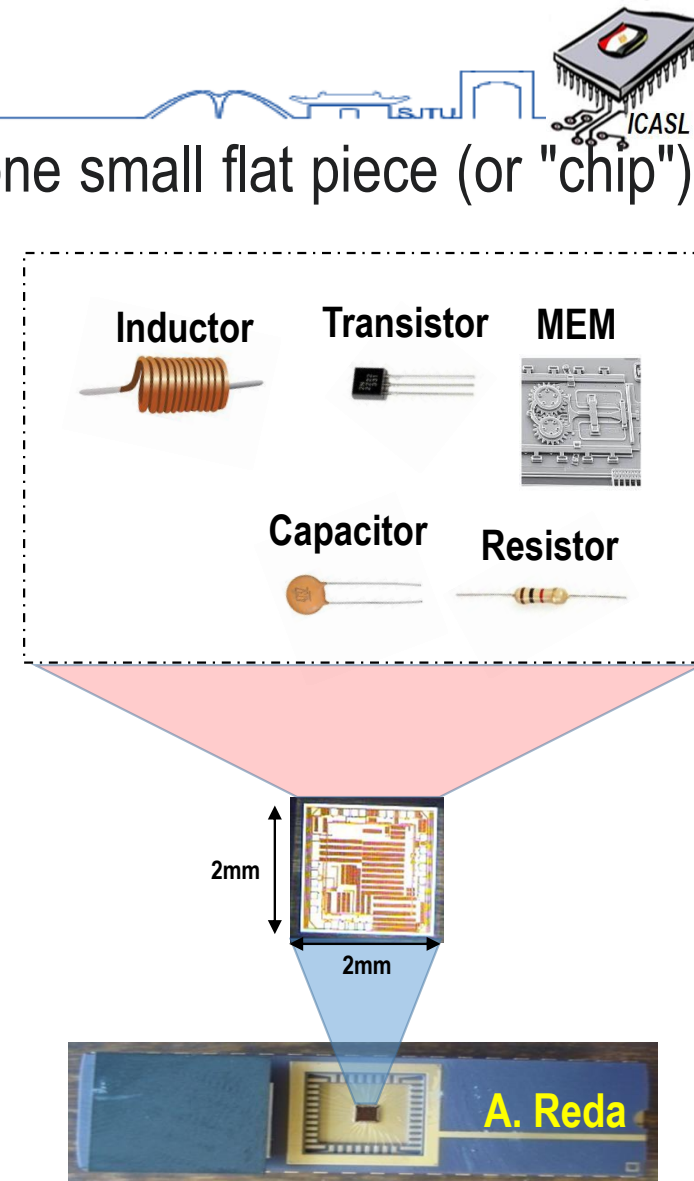
❑ An **integrated circuit** (IC) is a set of electronic circuits on one small flat piece (or "chip") of semiconductor material, usually silicon.

❑ Advantages of IC

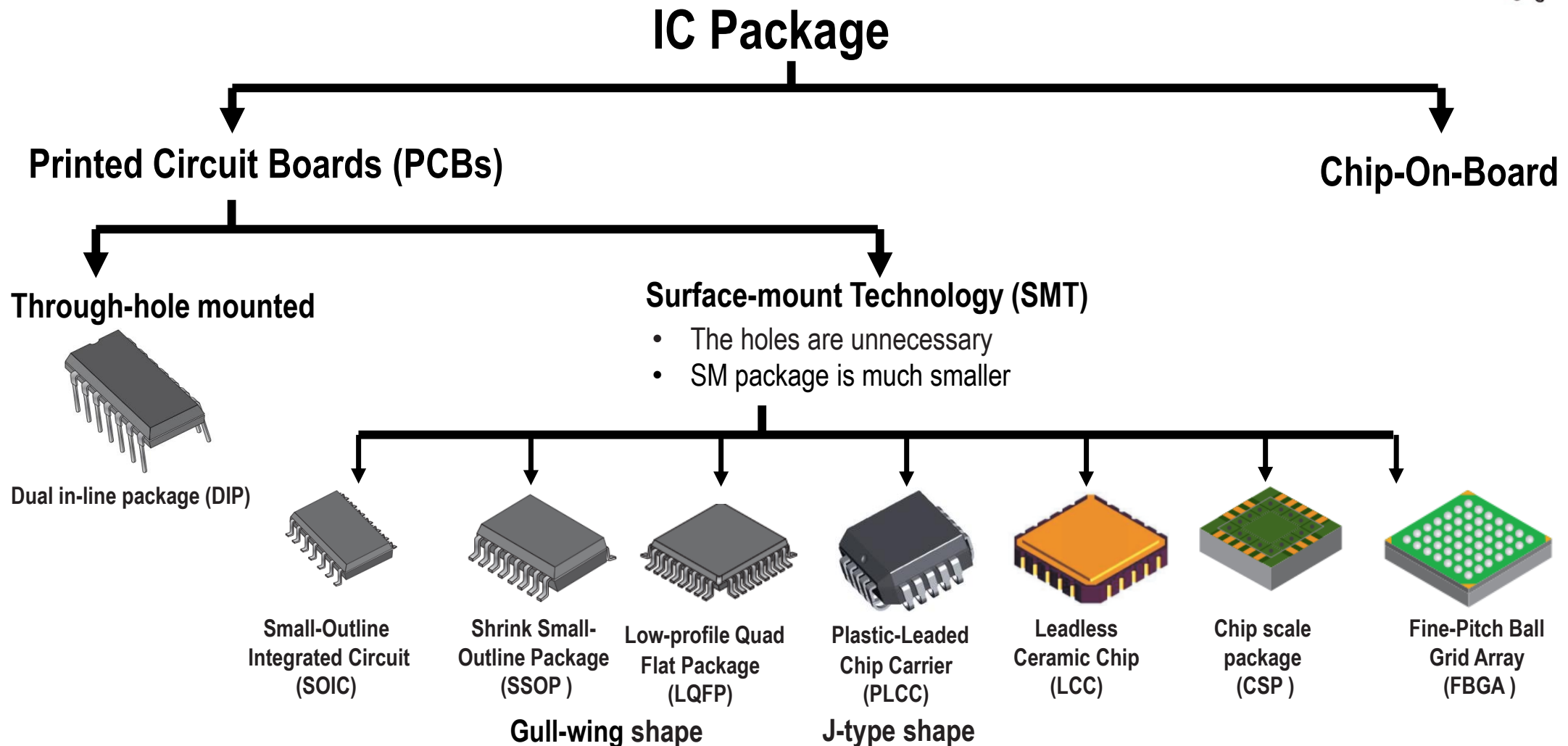
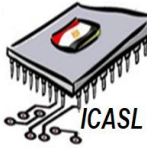
- ❑ Small size
- ❑ High reliability
- ❑ Low cost for mass production
- ❑ Low power consumption



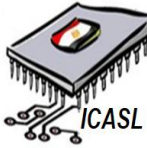
Cutaway view of one type of fixed-function IC package



Fixed-Function Logic Devices (2/4)



Fixed-Function Logic Devices (2/4)



IC Package

Printed Circuit Boards (PCBs)

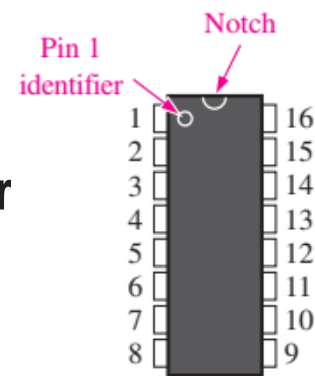
Chip-On-Board

Through-hole mounted

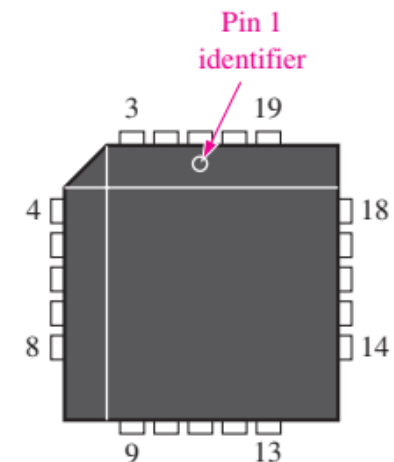
Surface-mount Technology (SMT)

Pin Numbering

- ❑ Looking at the top of the package,
- ❑ Pin 1 is indicated by a small dot, a notch, or beveled edge.
- ❑ The dot is always next to pin 1

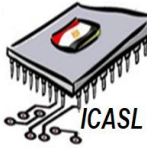


DIP or SSOP



DIP or SSOP

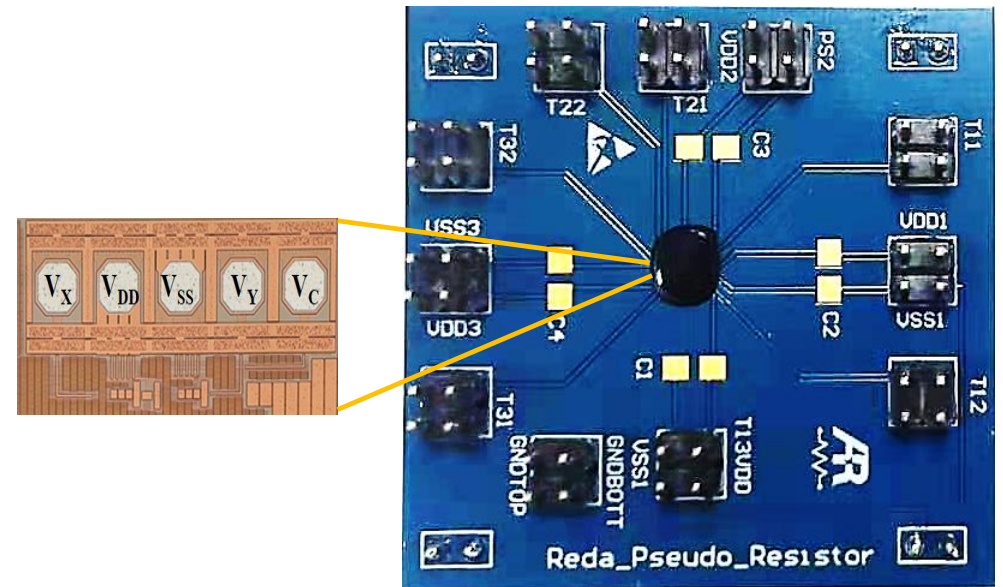
Fixed-Function Logic Devices (2/4)



IC Package

Printed Circuit Boards (PCBs)

Chip-On-Board

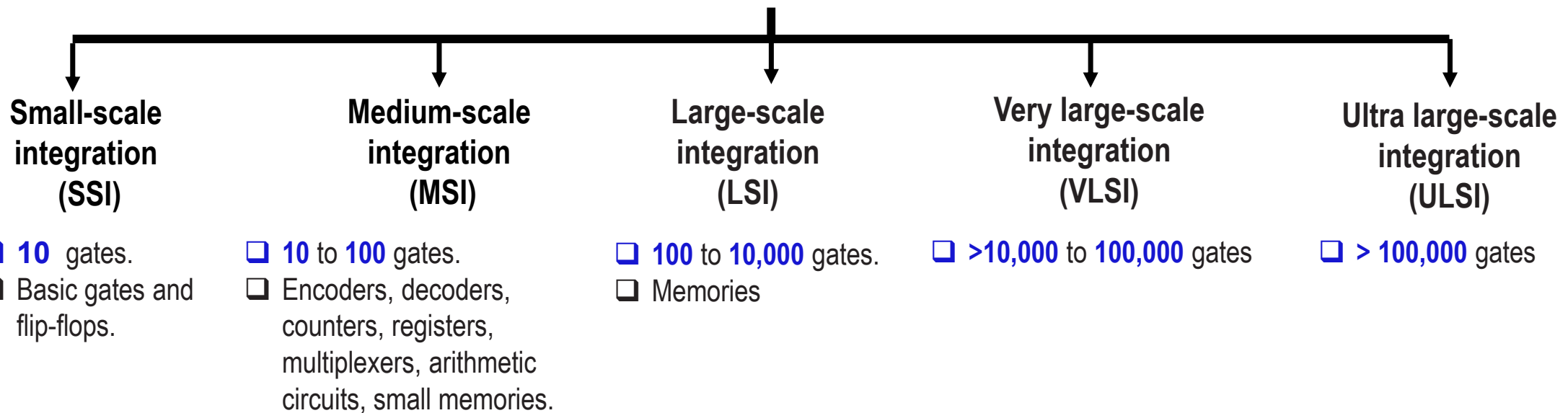


A chip-on-board (COB) is a chip that is mounted directly on a circuit board as opposed to being through-hole or surface-mounted technology.

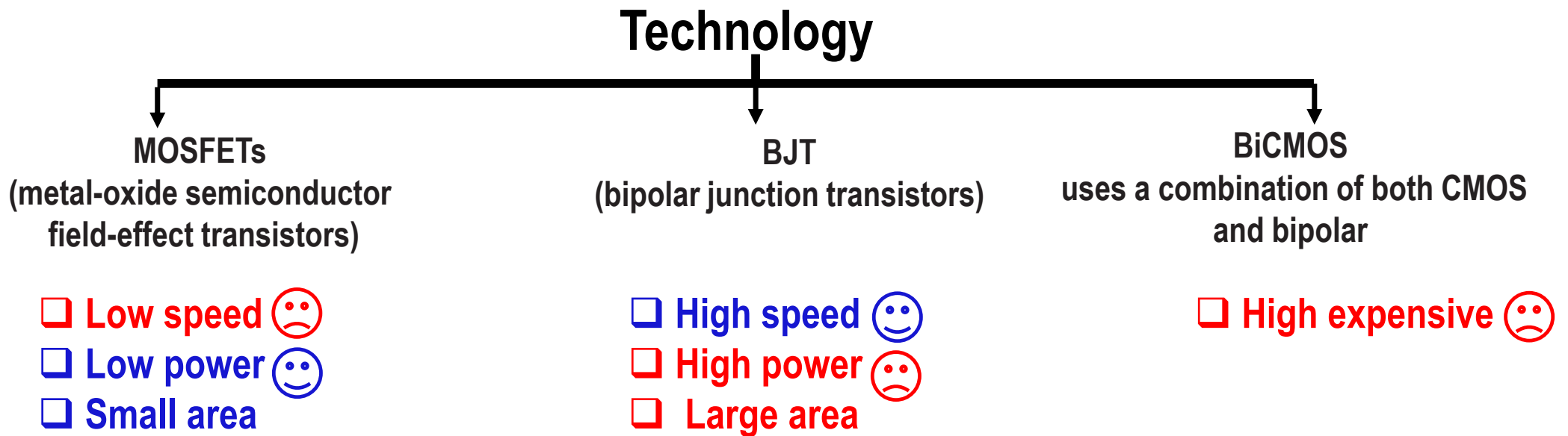
Fixed-Function Logic Devices (3/4)



Complexity



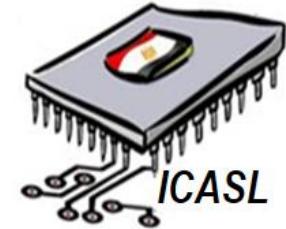
Fixed-Function Logic Devices (4/4)



☐ Advantage of CMOS technology 😊

- ☐ Cheapest technology for high density transistor.
- ☐ Memory can be implemented density.
- ☐ Fully restored level.
- ☐ Very low static power.
- ☐ Symmetrical device (symmetrical rise and fall time)

Thanks!



Digital Design SET 111

Lecture (03)- Number Systems, Operations, and Codes

Dr. Ahmed Reda Mohamed¹

¹The MoE Key Laboratory of Artificial Intelligence & Bio-Circuits and Systems Laboratory (**BiCASL**), Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.

Outlines

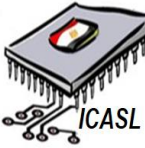


- ☐ **Decimal Number**
- ☐ **Binary Number**
- ☐ **Decimal-to-Binary Conversion**
- ☐ **Binary Arithmetic**
- ☐ **Complements of Binary Numbers**

Outlines



- ☐ **Decimal Number**
- ☐ **Binary Number**
- ☐ **Decimal-to-Binary Conversion**
- ☐ **Binary Arithmetic**
- ☐ **Complements of Binary Numbers**



Decimal Number

- ❑ In the decimal number system, each of the **ten** digits, **0 → 9**, represents a certain quantity.
- ❑ The position of each digit in a **decimal number** has its own represented **weight**.
- ❑ The value of a decimal number is the sum of the digits after each digit has been multiplied by its weight.
- ❑ **Ex:** Express the decimal number **(568.23)₁₀** as a sum of the values of each digit

Weight	10^2	10^1	10^0		10^{-1}	10^{-2}	
	↓	↓	↓		↓	↓	
Decimal number	5	6	8	.	2	3	
The value	$5 \cdot 10^2$	$+$	$6 \cdot 10^1$	$+$	$8 \cdot 10^0$	.	$2 \cdot 10^{-1} + 3 \cdot 10^{-2}$
	500	$+$	60	$+$	8	.	0.2 + 0.03 = (568.23)₁₀
							Decimal number

Outlines



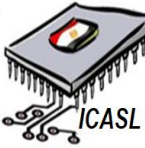
- ☐ Decimal Number
- ☒ Binary Number
- ☐ Decimal-to-Binary Conversion
- ☐ Binary Arithmetic
- ☐ Complements of Binary Numbers

Binary Number (1/3)

- ❑ The **binary** number system is another way to represent quantities
- ❑ The two binary digits (bits) are **1** and **0**.
- ❑ **Ex. Counting in Binary**
 - ❑ **B0** → Least Significant Bit (**LSB**)
 - ❑ **B2** → Most Significant Bit (**MSB**)
 - ❑ Largest decimal number = $2^n - 1$
 - ❑ n : represents the number of bits
 - ❑ Ex, 3 bits ($n = 3$) you can count from 0 to 7

Ex, 2 bits ($n = 2$) you can count from 0 to 3

Decimal Number	Binary Number		
	B2	B1	B0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1



Binary Number (2/3)

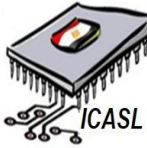
- ❑ In the binary number system, each of the **binary** digits {1,0}, represents a certain quantity.
- ❑ The position of each digit in a **binary number** has its own represented **weight**.
- ❑ The **binary number** can be **converted** into **decimal number** by summing of the digits after each digit has been multiplied by its weight.
- ❑ Ex: Convert the binary whole number $(1101101.1011)_2$ to decimal.

Weight	2^6	2^5	2^4	2^3	2^2	2^1	2^0	.	2^{-1}	2^{-2}	2^{-3}	2^{-4}
Binary number	1	1	0	1	1	0	1	.	1	0	1	1
The value of binary number in decimal	64	+ 32	+ 0	+ 8	+ 4	+ 0	+ 1	+	0.5	+ 0	+ 0.125	+ 0.0625 = $(109.6875)_{10}$

$$(1101101.1011)_2 = (109.6875)_{10}$$

Binary-to-Decimal Conversion

Binary Number (3/3)



Application

- Simple binary counting system.

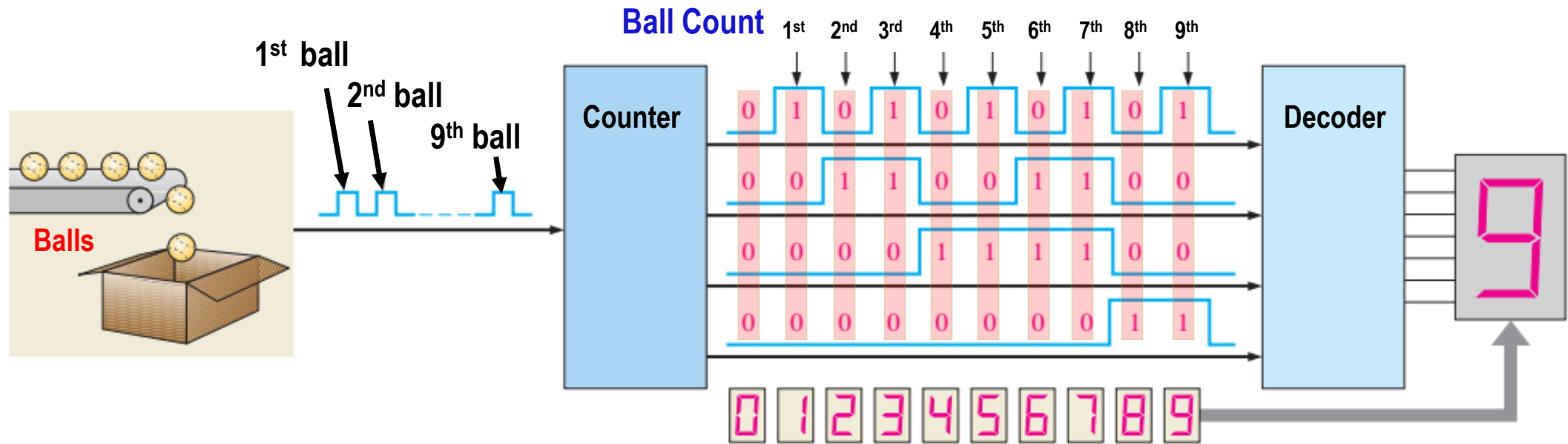
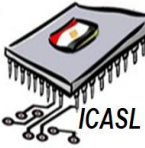


Illustration of a simple binary counting application

Outlines



- ☐ Decimal Number
- ☐ Binary Number
- ☐ **Decimal-to-Binary Conversion**
- ☐ Binary Arithmetic
- ☐ Complements of Binary Numbers



Decimal-to-Binary Conversion (1/2)

- ❑ Decimal number could have **whole** and **fractional** $\rightarrow (45.3125)_{10}$
- ❑ A systematic method of converting **whole** numbers from decimal to binary is the repeated **division-by-2** process until the **quotient** is **0**.
- ❑ The generated **remainders** by each division **form** the **binary** number
 - ❑ Ex.1.- Convert the decimal number $(12)_{10}$ to binary
 - ❑ Ex.2.- Convert the $(45)_{10}$ decimal numbers to binary

Step	quotient	Remainder
12/2	6	0
6/2	3	0
3/2	1	1
1/2	0	1

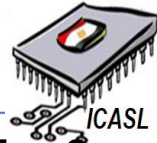
Stop when the Whole-number quotient is 0

1 1 0 0
MSB LSB
 $(12)_{10} = (1100)_2$

Step	quotient	Remainder
45/2	22	1
22/2	11	0
11/2	5	1
5/2	2	1
2/2	1	0
1/2	0	1

Stop when the Whole-number quotient is 0

1 0 1 1 0 1
MSB LSB
 $(45)_{10} = (101101)_2$



Decimal-to-Binary Conversion (2/2)

- ❑ A systematic method of converting **Decimal Fractions** to **Binary** is the **repeated Multiplication by 2** process until the **fractional product is zero** or until the **desired number of decimal places is reached**.
- ❑ The generated carry digits, or **carries**, by the **multiplications** produce the **binary** number
- ❑ EX: convert the decimal number $(0.3125)_{10}$ to binary

Step	Fractional	Carry	MSB			LSB
			0	1	0	1
0.3125×2	0.625	0				
0.625×2	$1.25 \rightarrow 0.25$	1				
0.25×2	0.5	0				
0.5×2	$1.0 \rightarrow 0$	1				

Continue to the desired number of decimal places or stop when the fractional part is all zeros

$$(0.3125)_{10} = (0.0101)_2$$

$$(45.3125)_{10}$$

Step1: Convert the Whole Part
 $(45)_{10} = (101101)_2$

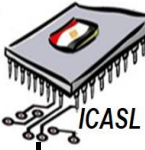
Step:2 Convert the Fraction Part
 $(0.3125)_{10} = (0.0101)_2$

$$(45.3125)_{10} = (101101.0101)_2$$

Outlines



- ☐ Decimal Number
- ☐ Binary Number
- ☐ Decimal-to-Binary Conversion
- ☐ Binary Arithmetic
- ☐ Complements of Binary Numbers



Binary Arithmetic (1/2)

❑ Binary arithmetic is essential in all digital computers and in many other types of digital systems

❑ Binary Addition

❑ The **four** basic rules for adding binary digits (bits)

❑ Ex:- Add the binary numbers $(11)_2$ to $(01)_2$

❑ Ex:- Add the binary numbers $(111)_2$ to $(11)_2$

A	B	Sum	Carry out
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

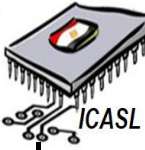
Carry 1 1 1

A	1	1	1
B		1	1
			+
Result	1	0	1 0

Carry 1 1

A	1	1
B	0	1
		+
Result	1	0 0

Carry in	A	B	Sum	Carry out
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Binary Arithmetic (2/2)

❑ Binary arithmetic is essential in all digital computers and in many other types of digital systems

❑ Binary Subtraction

❑ The **four** basic rules for subtracting binary digits (bits)

	A	B	Subtract
	0	0	0
1	0	1	1
	1	0	1
	1	1	0

❑ Special Case:

❑ A **borrow** is required in binary only when you try to subtract a 1 from a 0

❑ Ex:- Subtract $(011)_2$ from $(101)_2$

Right column:

$$1-1=0$$

Middle Column:

Borrow 1 from next column to the left, making a 10 in this column, then $10 - 1 = 1$

Left column:

When a 1 is borrowed a 0 is left, so $0 - 0 = 0$

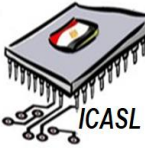
$$\begin{array}{r}
 0 \\
 A 1^1 0 1 \\
 B 0 1 1 - \\
 \hline
 0 1 0
 \end{array}$$

$$\begin{array}{r}
 0 \\
 A 1^1 0 \\
 B 1 - \\
 \hline
 1
 \end{array}$$

Outlines



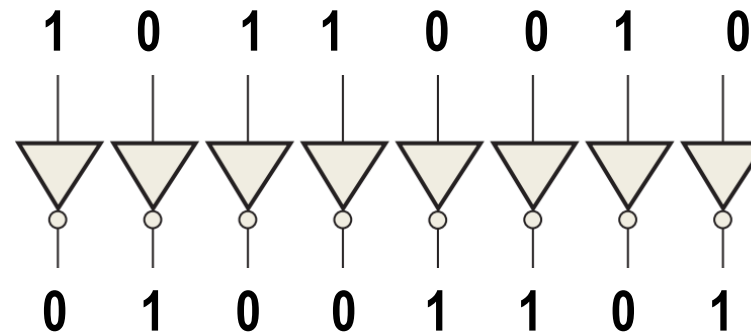
- ☐ Decimal Number
- ☐ Binary Number
- ☐ Decimal-to-Binary Conversion
- ☐ Binary Arithmetic
- ☐ Complements of Binary Numbers



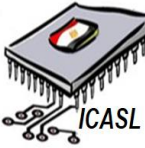
Complements of Binary Numbers (1/3)

- ❑ The **1's complement** and the **2's complement** of a binary number are important because they permit the representation of **negative** numbers.
- ❑ **1's complement**
 - ❑ is obtained by changing each **1** $\rightarrow$ **0** and each **0** to **1**.
 - ❑ Ex:- Find the 1's complement of $(10110010)_2$.
 - ❑ In **hardware**, array of inverters can be used to provide the **1's complement**

Binary Number	1	0	1	1	0	0	1	0
	↓	↓	↓	↓	↓	↓	↓	↓
1's complement	0	1	0	0	1	1	0	1



Hardware Digital circuit for 1's complement



Complements of Binary Numbers (2/3)

□ 2's complement

□ 1st method:

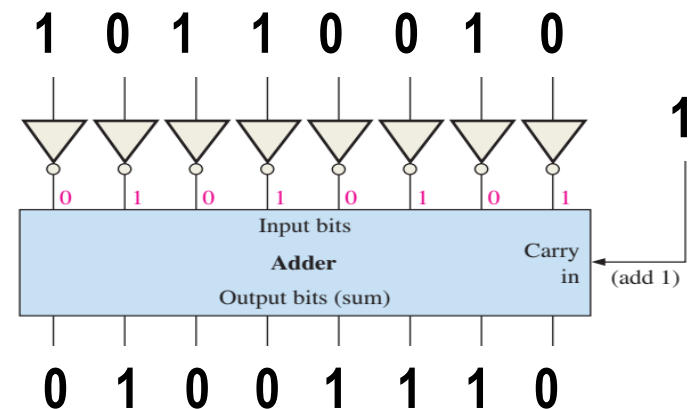
□ 2's complement of a binary number is found by adding 1 to the LSB of the 1's complement

$$\text{2's complement} = (\text{1's complement}) + 1$$

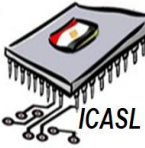
□ Ex.1 . Find the 2's complement of $(10110010)_2$, using the 1st method.

□ In **hardware**, array of inverters can be used with an adder

Binary Number	1	0	1	1	0	0	1	0
	↓	↓	↓	↓	↓	↓	↓	↓
1's complement	0	1	0	0	1	1	0	1
								+
								1
2's complement	0	1	0	0	1	1	1	0



Hardware Digital circuit for 1's complement



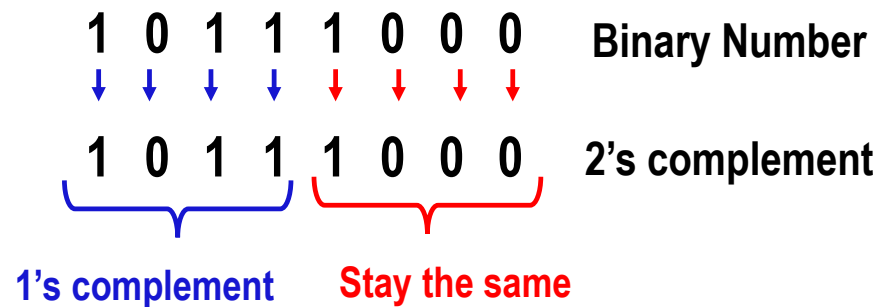
Complements of Binary Numbers (3/3)

□ 2's Complement

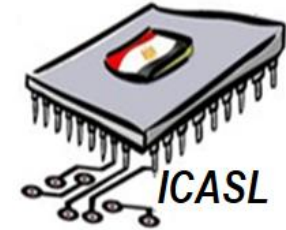
□ 2nd method:-

- Start at the right with the LSB and write the bits as they are up to and including the 1st 1.
- Take the 1's complements of the **remaining bits**.

□ Ex.1 Find the 2's complement of $(10111000)_2$ using the 2nd method



Thanks!



Digital Design SET 111

Lecture (04)-Number Systems, Operations, and Codes

Dr. Ahmed Reda Mohamed¹

¹PhD, the MoE Key Laboratory of Artificial Intelligence & Bio-Circuits and Systems Laboratory (**BiCASL**), Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.

Outlines

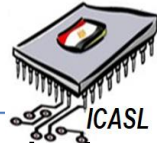


- ☐ **Binary Arithmetic**
- ☐ **Complements of Binary Numbers**
- ☐ **Hexadecimal Numbers**
- ☐ **Octal Numbers**
- ☐ **Conversion Map**

Outlines



- ☐ **Binary Arithmetic**
- ☐ **Complements of Binary Numbers**
- ☐ **Hexadecimal Numbers**
- ☐ **Octal Numbers**
- ☐ **Conversion Map**



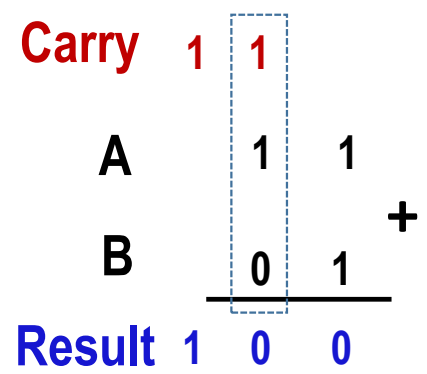
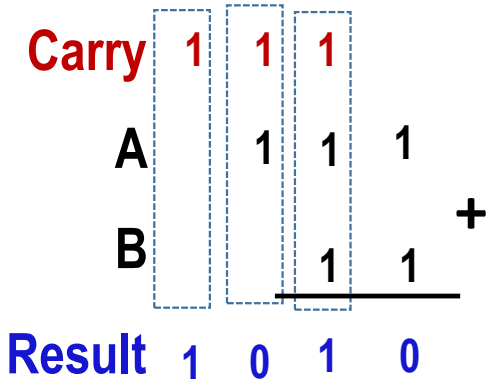
Binary Arithmetic (1/2)

❑ Binary arithmetic is essential in all **digital computers** and in many other types of digital systems

❑ Binary Addition (+)

- ❑ The **four** basic rules for adding binary digits (bits)
- ❑ Ex:- Add the binary numbers $(11)_2$ to $(01)_2$
- ❑ Ex:- Add the binary numbers $(111)_2$ to $(11)_2$

A	+	B	Sum	Carry out
0		0	0	0
0		1	1	0
1		0	1	0
1		1	0	1



Carry in	+	A	+	B	Sum	Carry out
1		0		0	1	0
1		0		1	0	1
1		1		0	0	1
1		1		1	1	1



Binary Arithmetic (2/2)

❑ Binary arithmetic is essential in all digital computers and in many other types of digital systems

❑ Binary Subtraction (-)

❑ The **four** basic rules for subtracting binary digits (bits)

	A	-	B	Subtract
	0		0	0
1	0		1	1
	1		0	1
	1		1	0

❑ Special Case: ?

❑ A **borrow** is required in binary only when you try to subtract a 1 from a 0

❑ Ex:- Subtract $(011)_2$ from $(101)_2$

Right column:

$$1-1=0$$

Middle Column:

Borrow 1 from next column to the left, making a 10 in this column, then $10 - 1 = 1$

Left column:

When a 1 is borrowed a 0 is left, so $0 - 0 = 0$

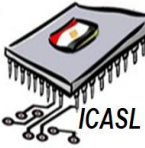
$$\begin{array}{r}
 0 \\
 A 101 \\
 B 011 \\
 \hline
 010
 \end{array}$$

$$\begin{array}{r}
 0 \\
 A 10 \\
 B 1 \\
 \hline
 1
 \end{array}$$

Outlines



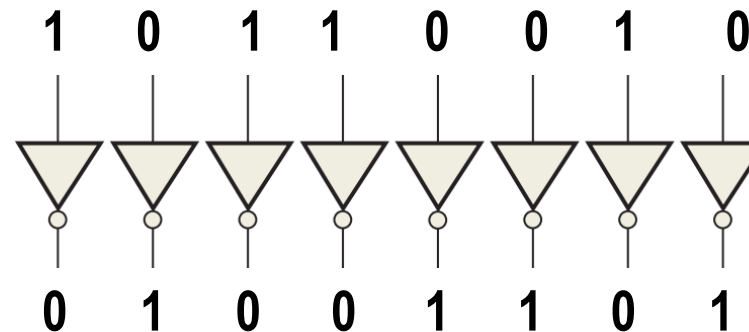
- ☐ Binary Arithmetic
- ☐ Complements of Binary Numbers
- ☐ Hexadecimal Numbers
- ☐ Octal Numbers
- ☐ Conversion Map



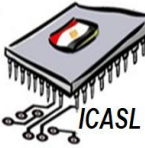
Complements of Binary Numbers (1/5)

- ❑ The **1's complement** and the **2's complement** of a binary number are important because they permit the representation of **negative** numbers.
- ❑ **1's complement**
 - ❑ is obtained by changing each **1** $\rightarrow$ **0** and each **0** to **1**.
 - ❑ Ex:- Find the 1's complement of $(10110010)_2$.
 - ❑ In **hardware**, array of inverters can be used to provide the **1's complement**

Binary Number	1	0	1	1	0	0	1	0
	↓	↓	↓	↓	↓	↓	↓	↓
1's complement	0	1	0	0	1	1	0	1



Hardware Digital circuit for 1's complement



Complements of Binary Numbers (2/5)

□ 2's complement

□ 1st method:

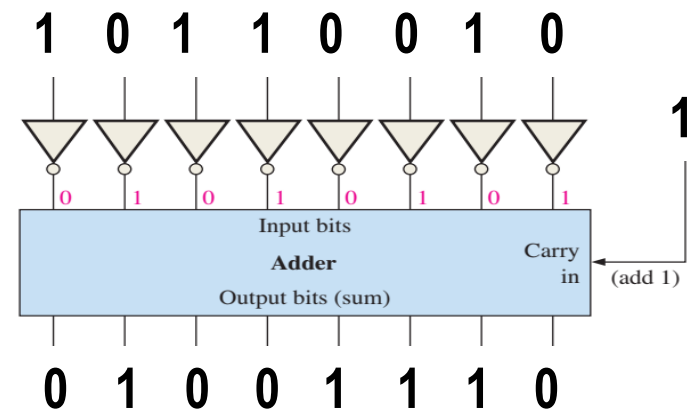
□ 2's complement of a binary number is found by adding 1 to the LSB of the 1's complement

$$\text{2's complement} = (\text{1's complement}) + 1$$

□ Ex.1 . Find the 2's complement of $(10110010)_2$, using the 1st method.

□ In **hardware**, array of inverters can be used with an adder

Binary Number	1	0	1	1	0	0	1	0
	↓	↓	↓	↓	↓	↓	↓	↓
1's complement	0	1	0	0	1	1	0	1
								+
								1
2's complement	0	1	0	0	1	1	1	0



Hardware Digital circuit for 1's complement

Complements of Binary Numbers (3/5)

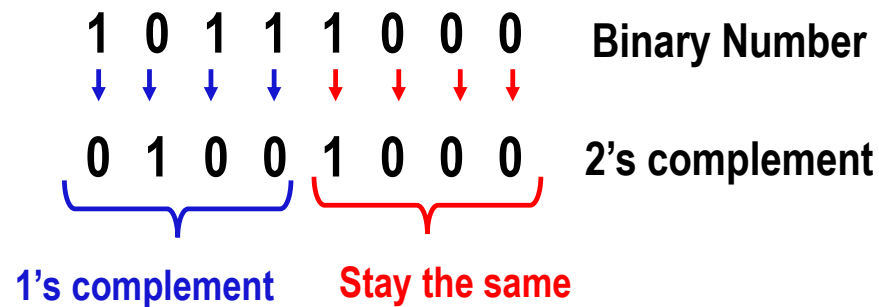


□ 2's Complement

□ 2nd method:-

- Start at the right with the LSB and write the bits as they are up to and including the 1st 1.
- Take the 1's complements of the **remaining bits**.

□ Ex.1 Find the **2's complement** of $(10111000)_2$ using the 2nd method



Complements of Binary Numbers (4/5)



- ❑ The **2's complement** form for representing signed numbers is the most widely used in computers and microprocessor-based systems. **Why?** 
- ❑ **Using 1st complement**, 0 has **two** different representations.

- ❑ $(+0)_{10} = (00000000)_2$

- ❑ $(-0)_{10} = (11111111)_2$

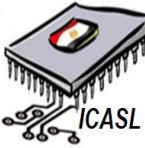
- ❑ **Using 2nd complement**, 0 has only **one** representation.

- ❑ $(+0)_{10} = (00000000)_2$

- ❑ $(-0)_{10} = (00000000)_2$

This is the reason why 2' s
complement is generally used.

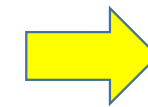




Complements of Binary Numbers (5/5)

❑ **Negative** numbers are the **2's complements** of the corresponding positive numbers.

❑ Ex:- Convert $(-25)_{10}$ into binary using 8-bits.



$(-25)_{10} = (11101111)_2$ 😊

Ans.

Step1: Convert decimal number into binary

Step	quotient	Remainder
25/2	12	1
12/2	6	0
6/2	3	0
3/2	1	1
1/2	0	1

Stop when the Whole-number quotient is 0

1 1 0 0 1

Step2: write the binary number as 8 bits

Step3: find the 1st complements

Step4: find the 2nd complements
= 1st complements+1

	B7	B6	B5	B4	B3	B2	B1	B0
25	0	0	0	1	1	0	0	1
1 st complements	1	1	1	0	0	1	1	0
2 nd complements	1	1	1	0	0	1	1	1

Outlines



- ☐ Binary Arithmetic
- ☐ Complements of Binary Numbers
- ☐ Hexadecimal Numbers
- ☐ Octal Numbers
- ☐ Conversion Map



Hexadecimal Numbers(1/5)

❑ The hexadecimal number system has **16** characters {0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F}

❑ **16** characters = 2^4 ← 4bits

❑ Each **hexadecimal** digit represents a **4-bit** binary number

Decimal Number	Binary Number				Hexadecimal
	B3	B2	B1	B0	
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	2
3	0	0	1	1	3
4	0	1	0	0	4
5	0	1	0	1	5
6	0	1	1	0	6
7	0	1	1	1	7
8	1	0	0	0	8
9	1	0	0	1	9
10	1	0	1	0	A
11	1	0	1	1	B
12	1	1	0	0	C
13	1	1	0	1	D
14	1	1	1	0	E
15	1	1	1	1	F

Hexadecimal Numbers(2/5)

❑ Binary-to-Hexadecimal Conversion

- ❑ Break the binary number into 4-bit groups
- ❑ Starting at the right-most bit and replace each 4-bit group with the equivalent hexadecimal symbol.

❑ Ex.1. Convert the following binary number $(1100101001010111)_2$ to hexadecimal

Ans.

$(1100101001010111)_2 \rightarrow (CA57)_{16}$

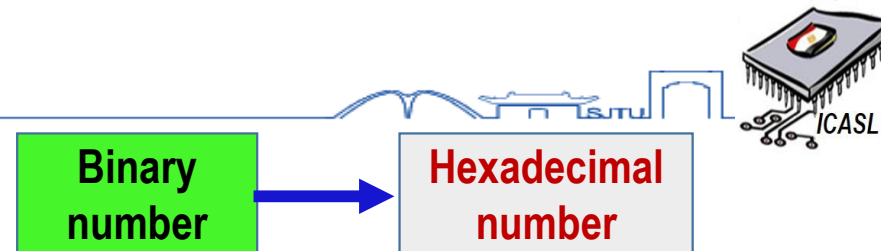
C A 5 7

❑ Ex.2. Convert the following binary number $(111111000101101001)_2$ to hexadecimal

Ans.

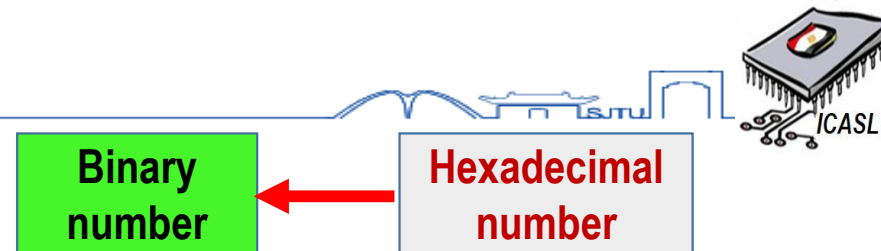
$(00111111000101101001)_2 \rightarrow (3F169)_{16}$

3 F 1 6 9



Hexadecimal Numbers(3/5)

□ Hexadecimal-to-Binary Conversion



- Reverse the process and replace each hexadecimal symbol with the appropriate 4 bits

- Ex.1. Convert the following hexadecimal number $(10A4)_{16}$ to binary number

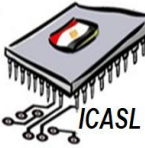
Ans.

$$\begin{array}{cccc} 1 & 0 & A & 4 \\ \hline 0001 & 0000 & 1010 & 0100 \end{array} \quad \longrightarrow \quad (0001000010100100)_2$$

- Ex.2. Convert the following hexadecimal number $(CF8E)_{16}$ to binary number

Ans.

$$\begin{array}{cccc} C & F & 8 & E \\ \hline 1100 & 1111 & 1000 & 1110 \end{array} \quad \longrightarrow \quad (1100111110001110)_2$$



Hexadecimal Numbers(4/5)

❑ Hexadecimal-to-Decimal Conversion

- ❑ **Step1:** convert the hexadecimal number to binary
- ❑ **Step2:** convert the binary to decimal

❑ **Ex.1.** Convert the following hexadecimal number $(1C)_{16}$ to decimal number

Ans.

Step1: $\begin{matrix} 1 & C \\ \hline 0001 & 1100 \end{matrix} \rightarrow (00011100)_2$

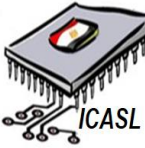
Step2:

Weight	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Binary Number	0	0	0	1	1	1	0	0

$$\text{Value in decimal} = 0 + 0 + 0 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 + 0 = (28)_{10}$$

$$(1C)_{16} = (00011100)_2 = (28)_{10}$$





Hexadecimal Numbers(6/5)

□ Hexadecimal-to-Decimal Conversion

- **Step1:** convert the hexadecimal number to binary
- **Step2:** convert the binary to decimal

□ **Ex.2.** Convert the following hexadecimal number $(A85)_{16}$ to decimal number

Ans.

Step1: $\begin{array}{ccc} A & 8 & 5 \\ \hline 1010 & 1000 & 0101 \end{array} \rightarrow (101010000101)_2$

Step2:

Weight	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Binary Number	1	0	1	0	1	0	0	0	0	1	0	1
Value in decimal	$= 1 \cdot 2^{11} + 0 + 1 \cdot 2^9 + 0 + 1 \cdot 2^7 + 1 \cdot 2^2 + 1 \cdot 2^0 = (2693)_{10}$											

$$(A85)_{16} = (101010000101)_2 = (2693)_{10}$$



Outlines



- ☐ Binary Arithmetic
- ☐ Complements of Binary Numbers
- ☐ Hexadecimal Numbers
- ☐ Octal Numbers
- ☐ Conversion Map

Octal Numbers(1/5)

- ❑ The **Octal** number system has **8** digits {0,1,2,3,4,5,6,7}

❑ $8 = 2^3$ ← 3 bits

- ❑ Each **Octal** digit represents a **3-bit** binary number.

- ❑ It is used **less** frequently than **hexadecimal** 😊
in conjunction with computers and microprocessors

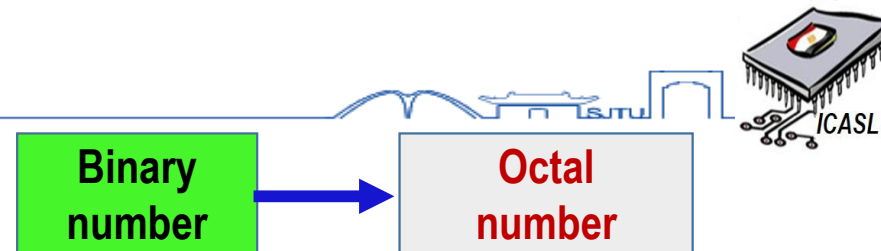


Decimal Number	Binary Number			Octal
	B2	B1	B0	
0	0	0	0	0
1	0	0	1	1
2	0	1	0	2
3	0	1	1	3
4	1	0	0	4
5	1	0	1	5
6	1	1	0	6
7	1	1	1	7

Octal Numbers(2/5)

□ Binary-to-Octal Conversion

- Break the binary number into 3-bit groups
- Starting at the right-most bit and replace each 3-bit group with the equivalent octal number



□ Ex.1. Convert the following binary number $(110101)_2$ to octal

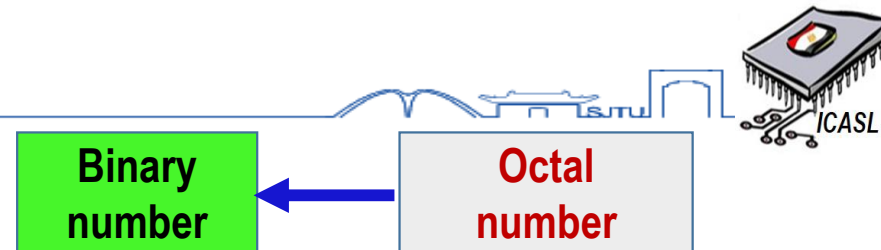
$$\begin{array}{c} (110101)_2 \\ \underbrace{\quad\quad}_6 \underbrace{\quad}_5 \\ \text{Ans.} \quad \longrightarrow \quad (65)_8 \end{array}$$

□ Ex.2. Convert the following binary number $(11010000100)_2$ to octal

$$\begin{array}{c} (011010000100)_2 \\ \underbrace{\quad}_3 \underbrace{\quad}_2 \underbrace{\quad}_0 \underbrace{\quad}_4 \\ \text{Ans.} \quad \longrightarrow \quad (3204)_8 \end{array}$$

Octal Numbers(3/5)

□ Octal-to-Binary Conversion



□ Reverse the process and replace each octal with the appropriate 3 bits

□ Ex.1. Convert the following octal number $(13)_8$ to binary number

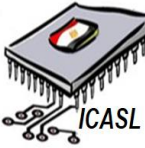
Ans.

$$\begin{array}{cc} 1 & 3 \\ \underbrace{} & \underbrace{} \\ 001 & 011 \end{array} \quad \longrightarrow \quad (001011)_2$$

□ Ex.2. Convert the following octal number $(140)_8$ to binary number

Ans.

$$\begin{array}{ccc} 1 & 4 & 0 \\ \underbrace{} & \underbrace{} & \underbrace{} \\ 001 & 100 & 000 \end{array} \quad \longrightarrow \quad (001100000)_2$$



Octal Numbers(4/5)

□ Octal-to-Decimal Conversion

- **Step1:** convert the Octal number to binary
- **Step2:** convert the binary to decimal

□ **Ex.1.** Convert the following Octal number $(13)_8$ to decimal number

Ans.

Step1:

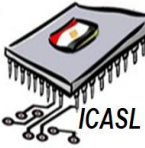
$$\begin{array}{cc} 1 & 3 \\ \underbrace{} & \underbrace{} \\ 001 & 011 \end{array} \longrightarrow = (001011)_2$$

Step2:

Weight	2^5	2^4	2^3	2^2	2^1	2^0
Binary Number	0	0	1	0	1	1
Value in decimal	$= 0 + 0 + 1 \cdot 2^3 + 0 + 1 \cdot 2^1 + 1 \cdot 2^0 = (11)_{10}$					

$$(13)_8 = (001011)_2 = (11)_{10}$$





Octal Numbers(5/5)

□ Octal-to-Decimal Conversion

- **Step1:** convert the Octal number to binary
- **Step2:** convert the binary to decimal

□ **Ex.2.** Convert the following Octal number $(140)_8$ to decimal number

Ans.

Step1: $\begin{matrix} 1 & 4 & 0 \\ \underbrace{} & \underbrace{} & \underbrace{} \\ 001 & 100 & 000 \end{matrix} \longrightarrow = (001100000)_2$

Step2:

Weight	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	↓	↓	↓	↓	↓	↓	↓	↓	↓
Binary Number	0	0	1	1	0	0	0	0	0

Value in decimal = $0 + 0 + 1 \cdot 2^6 + 1 \cdot 2^5 + 0 + 0 + 0 + 0 + 0 + 0 = (96)_{10}$

$$(140)_8 = (001100000)_2 = (96)_{10}$$

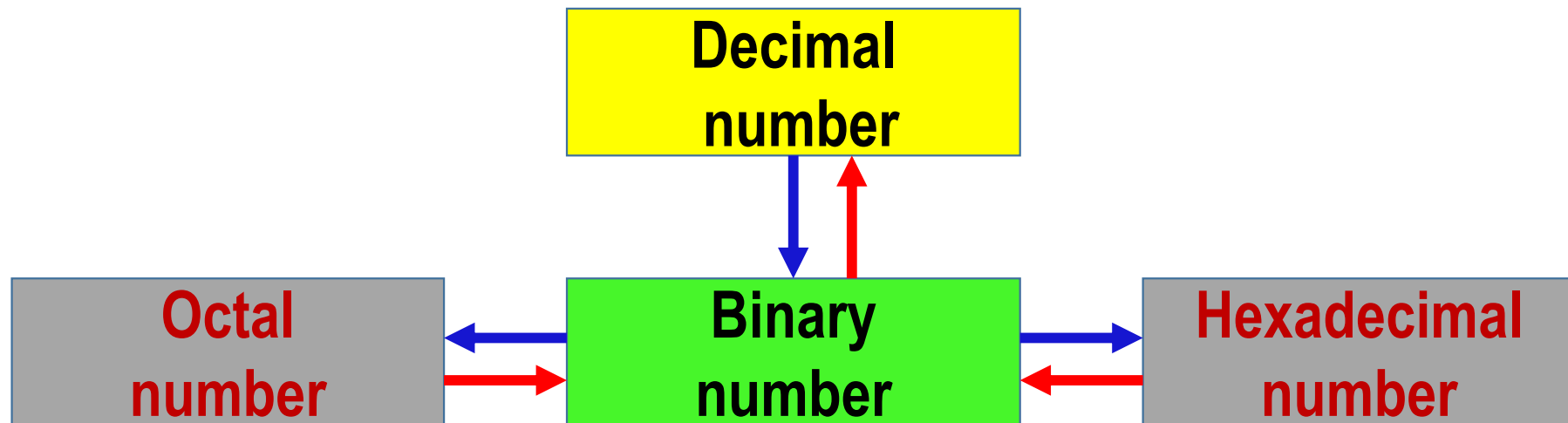


Outlines

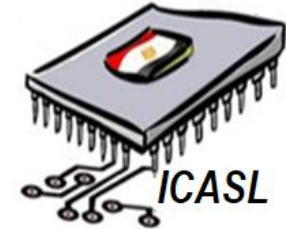


- ☐ Binary Arithmetic
- ☐ Complements of Binary Numbers
- ☐ Hexadecimal Numbers
- ☐ Octal Numbers
- ☐ Conversion Map

Conversion Map



Thanks!



Digital Design (SET 111)

L05

Logic Gates

Dr. Ahmed Reda Mohamed¹

¹PhD, the MoE Key Laboratory of Artificial Intelligence & Bio-Circuits and Systems Laboratory (**BiCASL**), Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.

Outlines



- ☐ The NOT Gate
- ☐ The AND Gate
- ☐ The OR Gate
- ☐ The NAND Gate
- ☐ The NOR Gate
- ☐ The XOR Gate
- ☐ The XNOR Gate

Outlines



- ☐ **The NOT Gate**
- ☐ **The AND Gate**
- ☐ **The OR Gate**
- ☐ **The NAND Gate**
- ☐ **The NOR Gate**
- ☐ **The XOR Gate**
- ☐ **The XNOR Gate**

The NOT Gate(1/2)

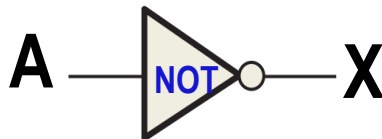


□ Definition

- The NOT Gate performs the operation called **inversion** or **complementation**. $1 \longrightarrow 0$
- The NOT Gate changes **one logic** level to the **opposite** level ,In terms of bits. $0 \longrightarrow 1$

□ Symbol

- **Standard** logic symbols for the inverter (ANSI/IEEE Std. 91-1984/ Std. 91a-1991)



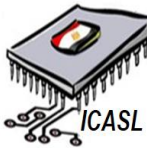
□ NOT Gate Truth Table

- Truth table is a combinations of binary **input** and **output** variables
- # of rows of truth table $= 2^n$, where **n**=# of input variables
- Ex. **n=1** $\rightarrow$ # of rows $= 2^1 = 2$

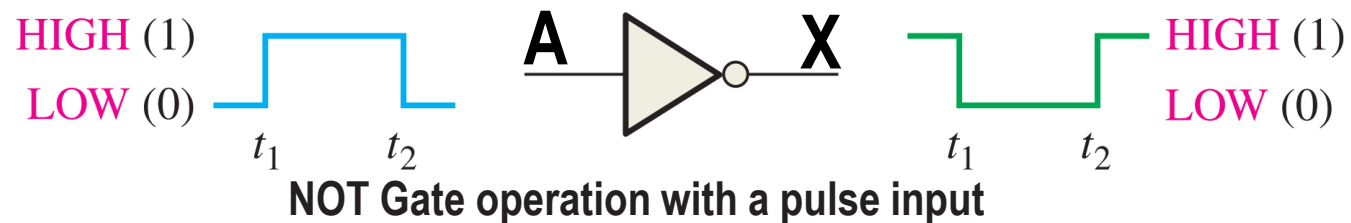
Truth Table

Input	Output
A	X
0	1
1	0

The NOT Gate (2/2)



NOT Gate Operation

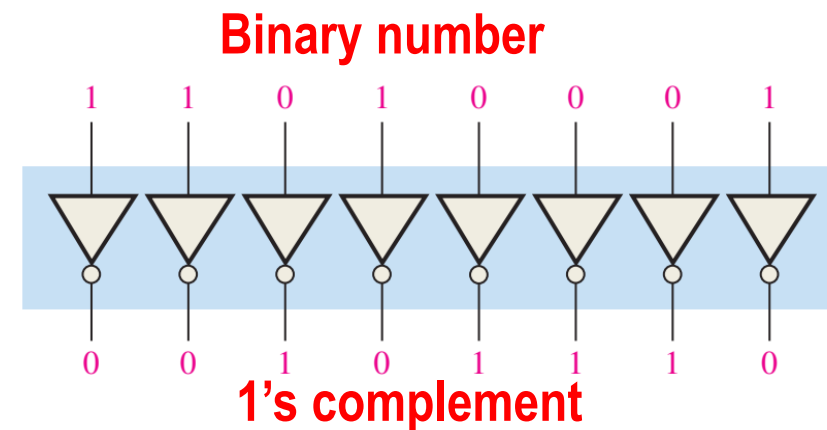


NOT Gate Logic Expression

- The complement of a variable is designated by a **bar** over the letter
- If the input variable is called **A** and the output variable is called **X** → **$X = \bar{A}$**

Application

- 1's complement circuit using inverters.



Outlines



- ☐ The NOT Gate
- ☒ The AND Gate
- ☐ The OR Gate
- ☐ The NAND Gate
- ☐ The NOR Gate
- ☐ The XOR Gate
- ☐ The XNOR Gate

The AND Gate (1/3)

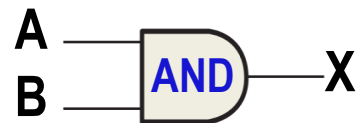


□ Definition

- The **AND** function produces a **HIGH** output only when **all** the inputs are **HIGH**
- Basic **AND** Gate is composed of **two** or more inputs and a single output.

□ Symbol

- Standard logic symbols for the AND



□ AND Gate Truth Table

- Ex. $n=2 \rightarrow \# \text{ of rows } = 2^2 = 4$

Truth Table

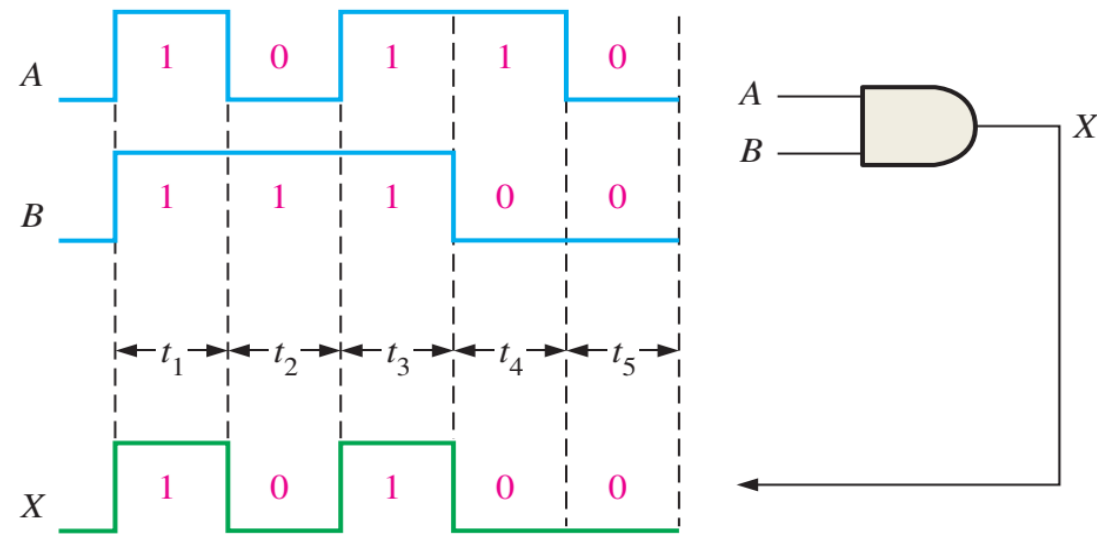
Input		Output
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

The AND Gate (2/3)



□ AND Gate Operation

- Output will be a **HIGH** only when all the inputs are **HIGH**



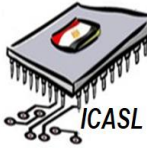
Example of AND gate operation

□ AND Gate Logic Expression

- The Boolean expression for the output of a 2-input AND gate

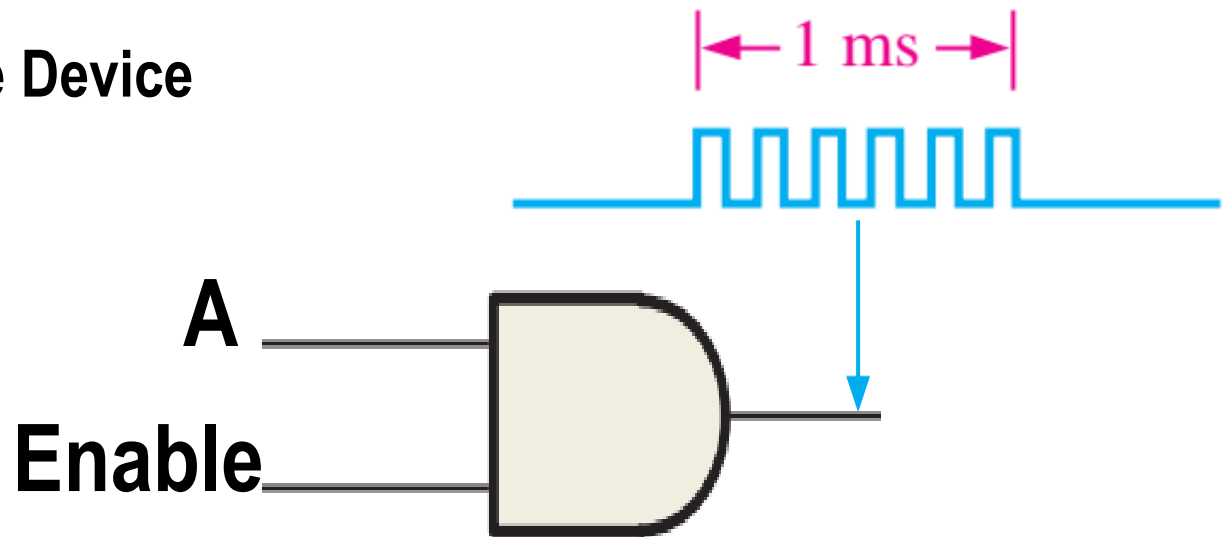
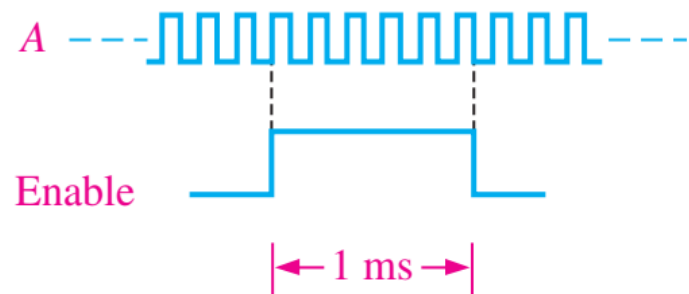
$$X = A \cdot B$$

The AND Gate (3/3)



Application

The AND Gate as an Enable Device



Q1: Build the truth table for a 3-input AND Gate, Determine the total number of possible input combinations, and Draw the symbol.



Outlines



- ☐ The NOT Gate
- ☐ The AND Gate
- ☒ The OR Gate
- ☐ The NAND Gate
- ☐ The NOR Gate
- ☐ The XOR Gate
- ☐ The XNOR Gate

The OR Gate (1/3)

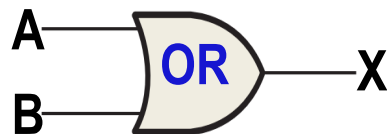


□ Definition

- The **OR** function produces a **HIGH** output when **one or more** inputs are **HIGH**
- Basic **OR** gate is composed of two or more inputs and a single output.

□ Symbol

- Standard logic symbols for the OR



□ OR Gate Truth Table

- Ex. $n=2 \rightarrow \# \text{ of rows } = 2^2 = 4$

Truth Table

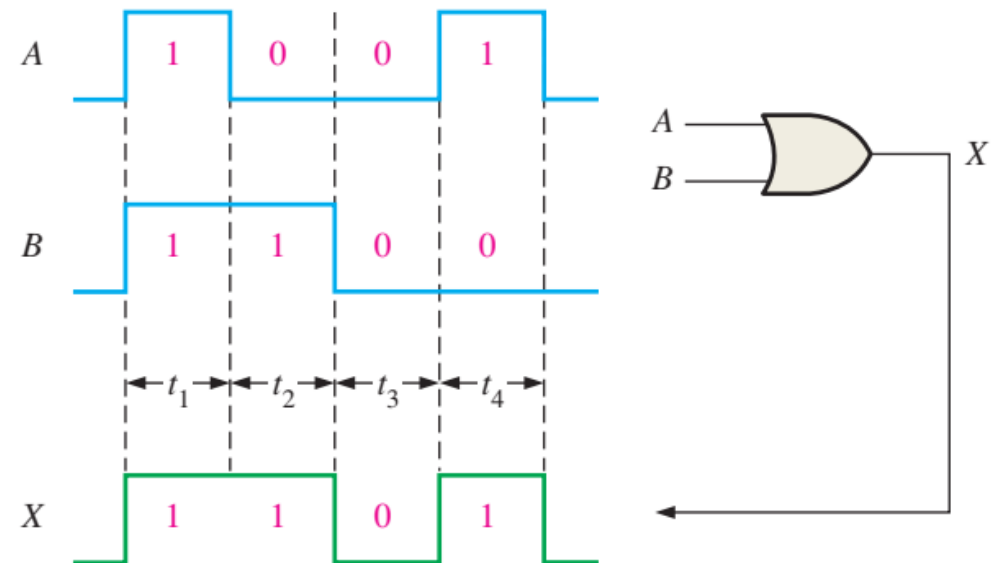
Input		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

The OR Gate (2/3)



OR Gate Operation

- Output will be a **HIGH** when one **or** more inputs are **HIGH**



Example of OR gate operation

OR Gate Logic Expression

- The Boolean expression for the output of a 2-input OR Gate

$$X = A + B$$

The OR Gate (3/3)

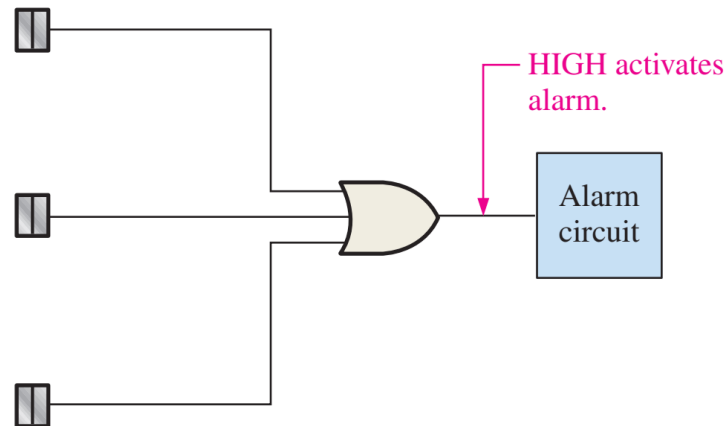


□ Application

- A simplified intrusion detection system

Open door/window
sensors

HIGH = Open
LOW = Closed



Q1: Build the truth table for a 3-input OR Gate, Determine the total number of possible input combinations, and Draw the symbol.

Outlines



- ☐ The NOT Gate
- ☐ The AND Gate
- ☐ The OR Gate
- ☒ The NAND Gate
- ☐ The NOR Gate
- ☐ The XOR Gate
- ☐ The XNOR Gate

The NAND Gate (1/2)

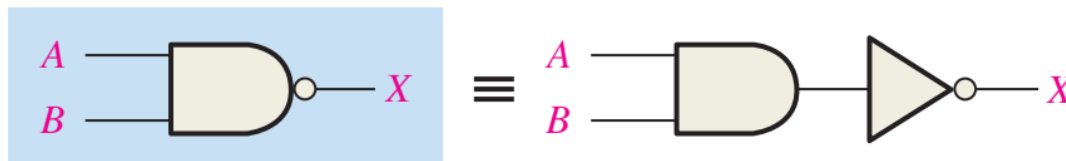


□ Definition

- The term **NAND** is a contraction of NOT-AND → **Universal Gate**
- The **NAND** function produces a **LOW** output only when all the inputs are **HIGH**

□ Symbol

- Standard logic symbols for the NAND



Truth Table

Input		Output
A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

□ NAND Gate Truth Table

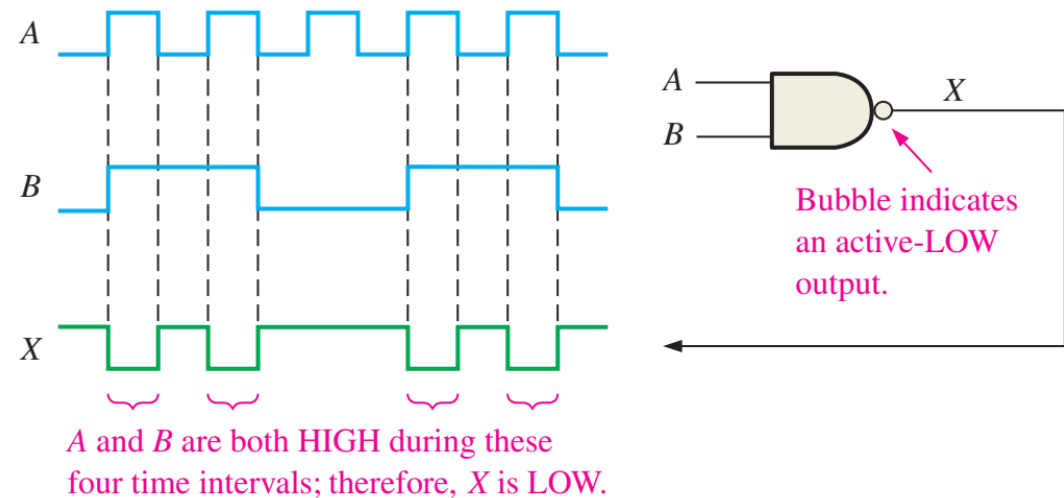
- Ex. $n=2 \rightarrow \# \text{ of rows } = 2^2 = 4$

The NAND Gate (2/2)



□ NAND Gate Operation

- Output will be a **LOW** only when all the inputs are **HIGH**



Example of NAND Gate operation

□ NAND Gate Logic Expression

- The Boolean expression for the output of a 2-input NAND Gate

$$X = \overline{A \cdot B}$$

Outlines



- ☐ The NOT Gate
- ☐ The AND Gate
- ☐ The OR Gate
- ☐ The NAND Gate
- ☒ The NOR Gate
- ☐ The XOR Gate
- ☐ The XNOR Gate

The NOR Gate (1/2)

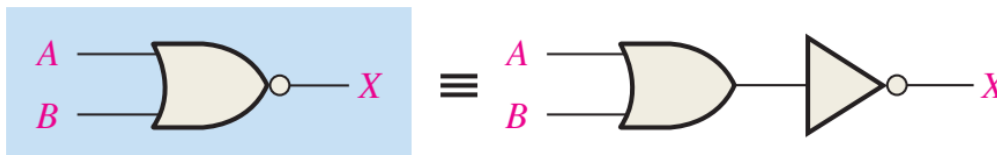


□ Definition

- The term **NOR** is a contraction of NOT-OR → **Universal Gate**
- The **NOR** function produces a **LOW** output when any of its input is **high**

□ Symbol

- Standard logic symbols for the NOR Gate



□ NOR Gate Truth Table

- Ex. $n=2 \rightarrow \# \text{ of rows } = 2^2 = 4$

Truth Table

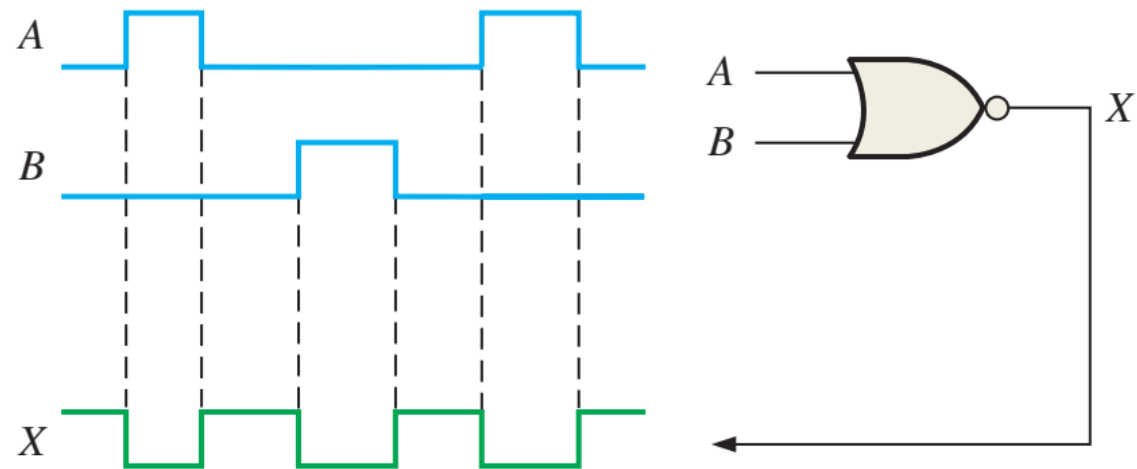
Input		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

The NOR Gate (2/2)



□ NOR Gate Operation

- Output will be a **LOW** when any of its input is **High**



Example of NOR gate operation

□ NOR Gate Logic Expression

- The Boolean expression for the output of a 2-input NOR Gate

$$X = \overline{A + B}$$

Outlines



- ☐ The NOT Gate
- ☐ The AND Gate
- ☐ The OR Gate
- ☐ The NAND Gate
- ☐ The NOR Gate
- ☐ The XOR Gate
- ☐ The XNOR Gate

The XOR Gate (1/2)



Definition

- ❑ The term **XOR** is a contraction of Exclusive-OR
- ❑ The **XOR** function produces a **High** output when both inputs are different

Symbol

- ❑ Standard logic symbols for the XOR



XOR Gate Truth Table

- ❑ Ex. $n=2 \rightarrow \# \text{ of rows } = 2^2 = 4$

Truth Table

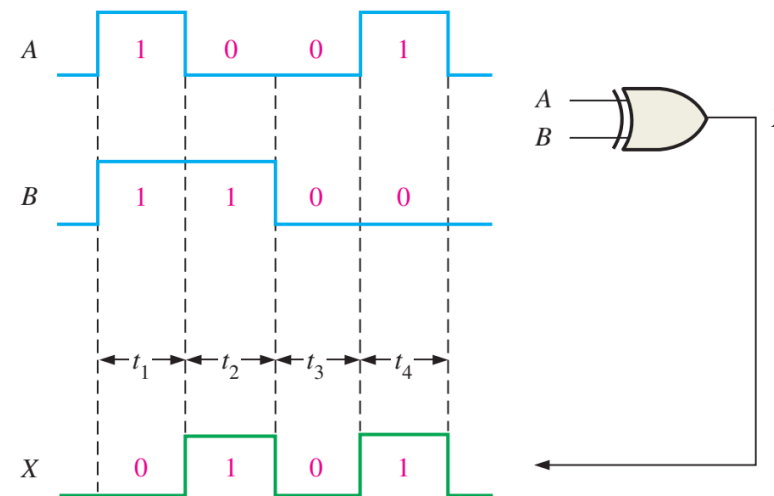
Input		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

The XOR Gate (2/2)



XOR Gate Operation

- Output will be a **High** when both inputs are different



Example of XOR gate operation

XOR Gate Logic Expression

- The Boolean expression for the output of a 2-input XOR Gate

$$X = A \oplus B$$

Outlines



- ☐ The NOT Gate
- ☐ The AND Gate
- ☐ The OR Gate
- ☐ The NAND Gate
- ☐ The NOR Gate
- ☐ The XOR Gate
- ☐ The XNOR Gate

The XNOR Gate (1/2)

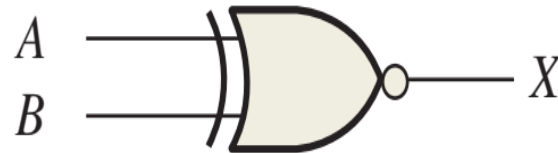


□ Definition

- The term **XNOR** is a contraction of Exclusive-NOR
- The **XNOR** function produces a **High** output when both inputs are the same

□ Symbol

- Standard logic symbols for the XNOR



Truth Table

Input		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

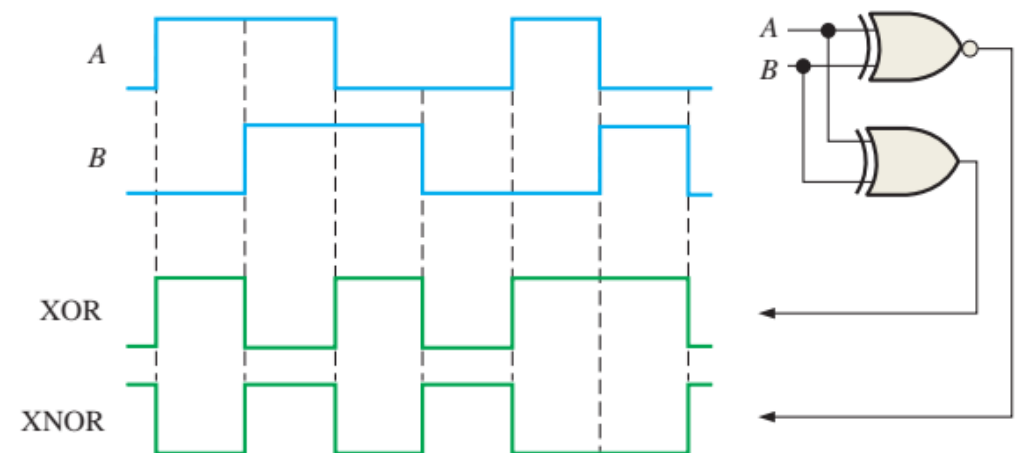
□ XNOR Gate Truth Table

- Ex. $n=2 \rightarrow \# \text{ of rows } = 2^2 = 4$

The XNOR Gate (2/2)

❑ XNOR Gate Operation

- ❑ Output will be a **High** when both inputs are the same



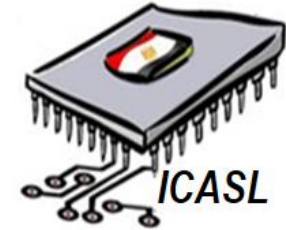
Example of XOR and XNOR gate operation

❑ XNOR Gate Logic Expression

- ❑ The Boolean expression for the output of a 2-input XNOR gate

$$X = A \odot B$$

Thanks!



Digital Design (SET 111)

L06

Boolean Algebra and Logic Simplification

Dr. Ahmed Reda Mohamed¹

¹PhD, the MoE Key Laboratory of Artificial Intelligence & Bio-Circuits and Systems Laboratory (**BiCASL**), Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.

Outlines



- ☐ **Boolean Operations and Expressions**
- ☐ **Laws of Boolean Algebra**
- ☐ **Rules of Boolean Algebra**
- ☐ **DeMorgan's Theorems**
- ☐ **Boolean Expression for a Logic Circuit**

Outlines



- ☐ **Boolean Operations and Expressions**
- ☐ **Laws of Boolean Algebra**
- ☐ **Rules of Boolean Algebra**
- ☐ **DeMorgan's Theorems**
- ☐ **Boolean Expression for a Logic Circuit**

Boolean Operations and Expressions



□ Definition

□ Boolean algebra is the **mathematics** of digital logic

□ Variable and complement are terms widely used in Boolean algebra

□ A **variable** is a symbol used to represent an action, a condition, or data {0 ,1 }

□ The complement is the inverse of a variable and is indicated by a **bar** over the variable. The complement of the variable A is $\overline{A}$

□ Main Boolean Operation

□ Boolean **Addition** equivalent to the OR operation ($F=A+B$)

□ Boolean **Multiplication** is equivalent to the AND operation ($F=A.B$)

Outlines



- ☐ Boolean Operations and Expressions
- ☐ Laws of Boolean Algebra
- ☐ Rules of Boolean Algebra
- ☐ DeMorgan's Theorems
- ☐ Boolean Expression for a Logic Circuit

Laws of Boolean Algebra (1/2)



☐ A) Commutative Law

- ☐ Addition for two variables $\rightarrow A+B=B+A$
- ☐ Multiplication for two variables $\rightarrow A.B=B.A$

☐ B) Associative Law

- ☐ Addition for multiple variables $\rightarrow A+(B+C)=(A+B)+C$
- ☐ Multiplication for multiple variables $\rightarrow A.(B.C)=(A.B).C$

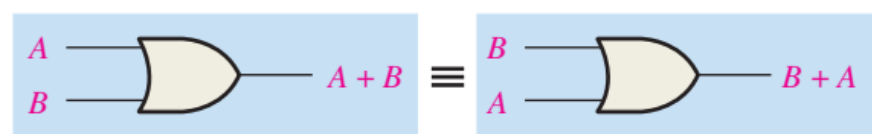
☐ C) Distributive Law

- ☐ The distributive law is written for three variables $\rightarrow A.(B+C)=A.B+A.C$

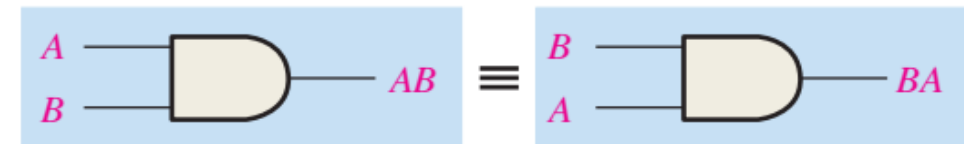
Laws of Boolean Algebra (2/2)



□ A) Commutative Law

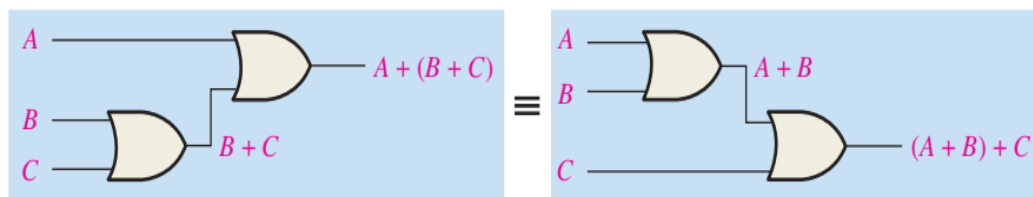


Application of commutative law of addition

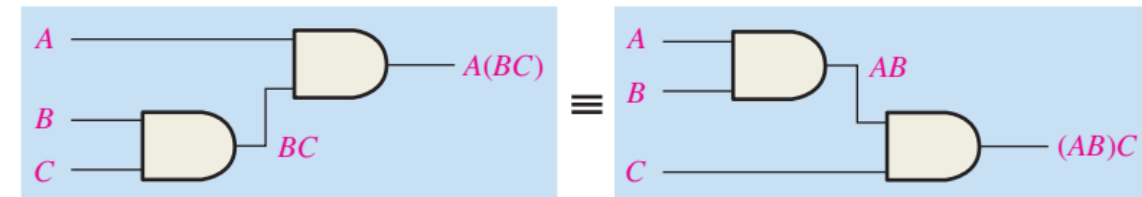


Application of commutative law of multiplication

□ B) Associative Law

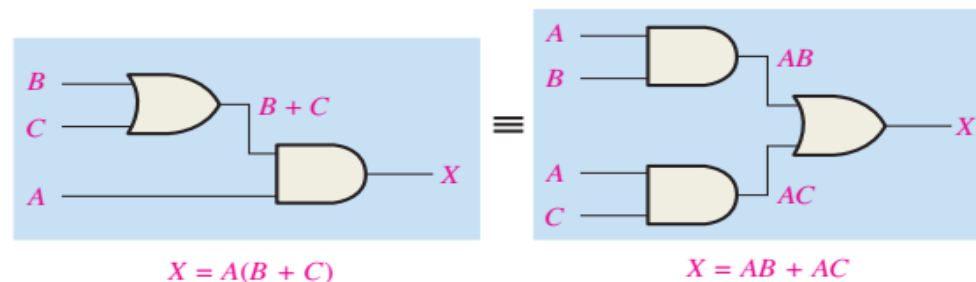


Application of associative law of addition



Application of associative law of multiplication

□ C) Distributive Law



$$X = A(B + C)$$

$$X = AB + AC$$

Application of distributive law

Outlines



- ☐ Boolean Operations and Expressions
- ☐ Laws of Boolean Algebra
- ☐ Rules of Boolean Algebra
- ☐ DeMorgan's Theorems
- ☐ Boolean Expression for a Logic Circuit

Rules of Boolean Algebra



Rule 1: $A+0=A$

Rule 2: $A+1=1$

Rule 3: $A+A=A$

Rule 4: $A+\bar{A}=1$

Rule 5: $A.0=0$

Rule 6: $A.1=A$

Rule 7: $A.A=A$

Rule 8: $A.\bar{A}=0$

Rule 9: $\overline{\bar{A}}=A$



Rule 10: $A+A.B=A$

Rule 11: $(A+B).(A+C)=A+BC$

Rule 12: $(A+\bar{A}B)=A+B$

Outlines



- ☐ Boolean Operations and Expressions
- ☐ Laws of Boolean Algebra
- ☐ Rules of Boolean Algebra
- ☐ DeMorgan's Theorems
- ☐ Boolean Expression for a Logic Circuit

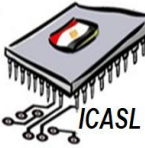
DeMorgan's Theorems (1/2)



- ❑ **DeMorgan**, a mathematician who knew Boole proposed two theorems that are an important part of Boolean algebra
- ❑ **DeMorgan's** theorems provide mathematical verification of the equivalency of the NAND and NOR gates.
- ❑ **DeMorgan's first theorem**
 - ❑ The complement of a **product** of variables is equal to the sum of the complements of the variables.
- ❑ **DeMorgan's second theorem**
 - ❑ The complement of a **sum** of variables is equal to the product of the complements of the variables.

$$\overline{XY} = \overline{X} + \overline{Y}$$

$$\overline{X + Y} = \overline{X} \cdot \overline{Y}$$



DeMorgan's Theorems (2/2)

□ Ex.1. Apply DeMorgan's theorems to the expressions $\overline{X.Y.Z}$

Ans

$$\overline{X.Y.Z} = \overline{X} + \overline{Y} + \overline{Z}$$

□ Ex.2. Apply DeMorgan's theorems to the expressions $\overline{X + Y + Z}$

Ans

$$\overline{X + Y + Z} = \overline{X} . \overline{Y} . \overline{Z}$$

□ Ex.3. Apply DeMorgan's theorems to the expressions $\overline{A . \overline{B} + \overline{C} . D + E . F}$

Ans

$$\overline{A . \overline{B} + \overline{C} . D + E . F} = (\overline{A} + B) . (C + \overline{D}) . (\overline{E} + \overline{F})$$

□ Ex.4. Apply DeMorgan's theorems to the expressions $\overline{(A + B + C) . D}$

Ans

$$\overline{(A + B + C) . D} = \overline{A} . \overline{B} . \overline{C} + \overline{D}$$

Outlines



- ☐ Boolean Operations and Expressions
- ☐ Laws of Boolean Algebra
- ☐ Rules of Boolean Algebra
- ☐ DeMorgan's Theorems
- ☐ Boolean Expression for a Logic Circuit

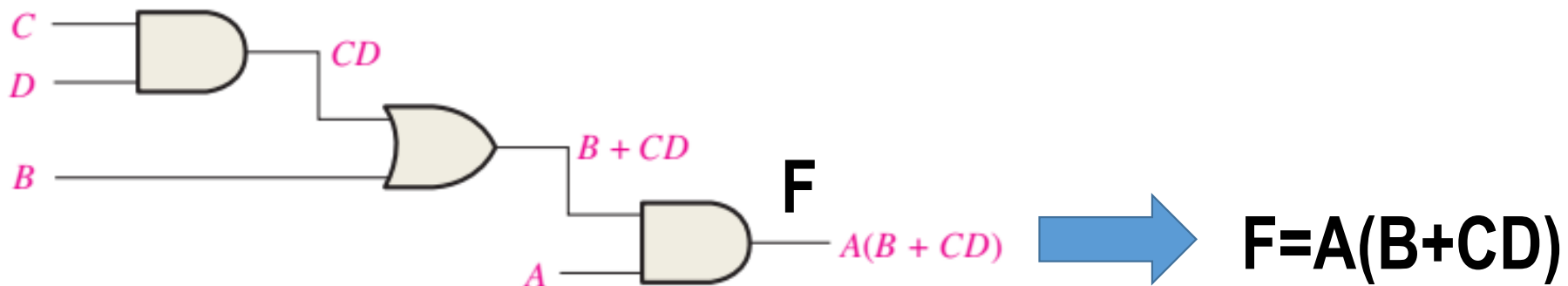
Boolean Expression for a Logic Circuit (1/4)



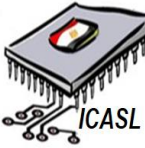
□ Main Steps

- **Begin** at the left-most inputs
- **Move** on toward the final output meanwhile writing the expression for each gate.

□ **EX.1.** Determine the Boolean expression for the following logic circuit.



Boolean Expression for a Logic Circuit (2/4)



□ Ex.1 Using Boolean algebra techniques, simplify this expression

$$AB + A(B + C) + B(B + C)$$

Ans

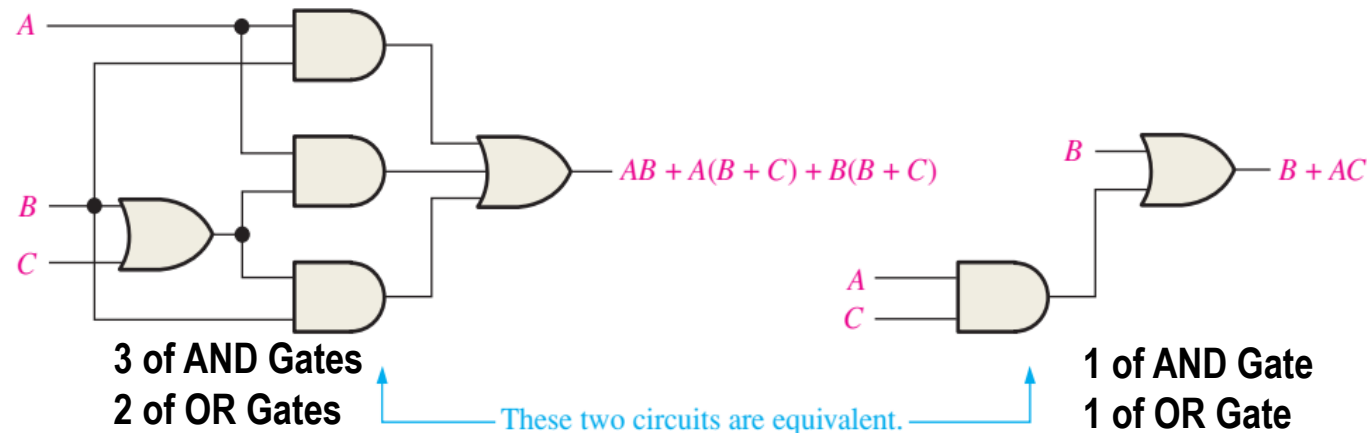
$$AB + A(B + C) + B(B + C) = AB + AB + AC + BB + BC \quad \longrightarrow \quad \text{Where } AB + AB = AB$$

$$= AB + AC + BB + BC \quad \longrightarrow \quad \text{Where } BB = B$$

$$= AB + AC + B + BC \quad \longrightarrow \quad \text{Where } B + BC = B(1 + C) = B$$

$$= AB + AC + B \quad \longrightarrow \quad \text{Where } AB + B = B(1 + A) = B$$

$$= AC + B$$



Boolean Expression for a Logic Circuit (3/4)



□ Ex.2 Using Boolean algebra techniques, simplify this expression

$$[A \overline{B} (C + BD) + \overline{A} \overline{B}]C$$

Ans

$$[A \overline{B} (C + BD) + \overline{A} \overline{B}]C = [A \overline{B} C + A \overline{B} BD + \overline{A} \overline{B}]C$$

$$= A \overline{B} CC + A \overline{B} BDC + \overline{A} \overline{B} C \quad \longrightarrow \quad \text{Where } C.C=C, \overline{B} B=0$$

$$= A \overline{B} C + 0 + \overline{A} \overline{B} C \quad \longrightarrow \quad \text{Where } C.C=C, \overline{B} B=0$$

$$= \overline{B} C (A + \overline{A}) \quad \longrightarrow \quad \text{Where } (A + \overline{A})=1$$

$$= \overline{B} C$$

Boolean Expression for a Logic Circuit (4/4)



□ Ex.3 Using Boolean algebra techniques, simplify this expression

$$\overline{AB + AC} + \overline{A} \overline{B} C$$

Ans

$$\overline{AB + AC} + \overline{A} \overline{B} C = (\overline{A} + \overline{B}).(\overline{A} + \overline{C}) + \overline{A} \overline{B} C$$

$$= (\overline{A} \overline{A} + \overline{A} \overline{C} + \overline{A} \overline{B} + \overline{B} \overline{C}) + \overline{A} \overline{B} C \rightarrow \text{Where } \overline{A} \overline{A} = \overline{A}$$

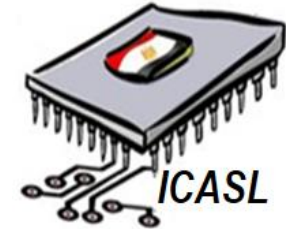
$$= \overline{A} + \overline{A} \overline{C} + \overline{A} \overline{B} + \overline{B} \overline{C} + \overline{A} \overline{B} C \rightarrow \text{Where } \overline{A} (1 + \overline{C}) = \overline{A}$$

$$= \overline{A} + \overline{A} \overline{B} + \overline{B} \overline{C} + \overline{A} \overline{B} C \rightarrow \text{Where } \overline{A} \overline{B} (1 + C) = \overline{A} \overline{B}$$

$$= \overline{A} + \overline{B} \overline{C} + \overline{A} \overline{B} \rightarrow \text{Where } \overline{A} (1 + \overline{B}) = \overline{A}$$

$$= \overline{A} + \overline{B} \overline{C}$$

Thanks!



Digital Design (SET 111)

L07

Boolean Algebra and Logic Simplification

Dr. Ahmed Reda Mohamed¹

¹PhD, the MoE Key Laboratory of Artificial Intelligence & Bio-Circuits and Systems Laboratory (**BiCASL**), Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.

Outlines

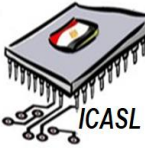


- ☐ **Standard Forms of Boolean Expressions**
- ☐ **Boolean Expressions and Truth Tables**
- ☐ **The Karnaugh Map (K-map)**
- ☐ **Map the standard SOP expression in the K-Map**
- ☐ **K-Map Simplification of SOP Expressions**
- ☐ **K-Map Simplification of Truth Table**
- ☐ **K-Map Simplification of Logic Circuit**

Outlines



- ☐ **Standard Forms of Boolean Expressions**
- ☐ **Boolean Expressions and Truth Tables**
- ☐ **The Karnaugh Map (K-map)**
- ☐ **Map the standard SOP expression in the K-Map**
- ☐ **K-Map Simplification of SOP Expressions**
- ☐ **K-Map Simplification of Truth Table**
- ☐ **K-Map Simplification of Logic Circuit**



Standard Forms of Boolean Expressions

□ All Boolean expressions can be converted into either of **two standard** forms.

□ The sum-of-products (**SOP**) form.

Ex.1. $X = \underbrace{AB}_{G1} + \underbrace{BCD}_{G2} + \underbrace{AC}_{G3}$

G4

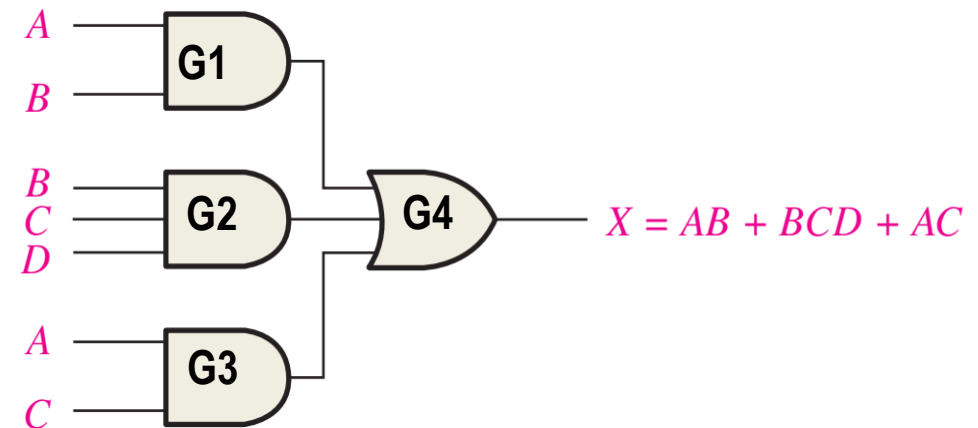
A **SOP** expression is equal to 1.

□ The product-of-sums (**POS**) form.

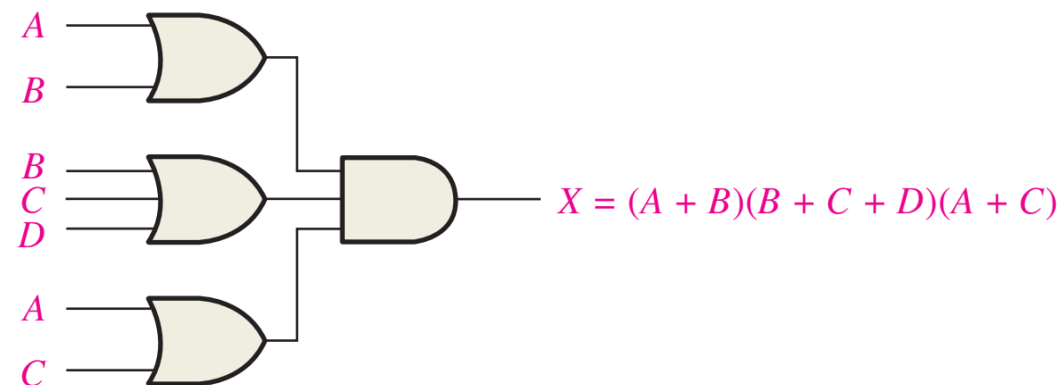
Ex.2. $X = \underbrace{(A + B)}_{G1} \cdot \underbrace{(B + C + D)}_{G2} \cdot \underbrace{(A + C)}_{G3}$

G4

A **POS** expression is equal to 0



Implementation of the **SOP** expression



Implementation of the **POS** expression

Outlines



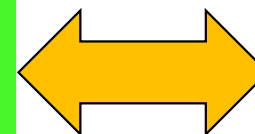
- ☐ Standard Forms of Boolean Expressions
- ☐ **Boolean Expressions and Truth Tables**
- ☐ The Karnaugh Map (K-map)
- ☐ Map the standard SOP expression in the K-Map
- ☐ K-Map Simplification of SOP Expressions
- ☐ K-Map Simplification of Truth Table
- ☐ K-Map Simplification of Logic Circuit

Boolean Expressions and Truth Tables (1/7)



- ❑ The **truth table** is a common way of presenting the logical operation of a circuit.
- ❑ Standard **SOP** or **POS** expressions can be determined from a truth table and vice versa

$$\begin{aligned} X &= AB + BCD + AC && \text{SOP} \\ X &= (A + B). (B + C + D). (A + C) && \text{POS} \end{aligned}$$

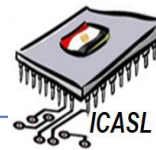


Input			Output
A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

❑ Remember

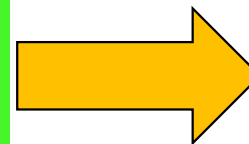
- ❑ A truth table is simply a list of the possible combinations of **input** variable values and the corresponding **output** values (**1 or 0**).
- ❑ For an expression with a domain of **two** variables, there are **four** different combinations of those variables ($2^2 = 4$).
- ❑ For an expression with a domain of **three** variables, there are **eight** different combinations of those variables ($2^3 = 8$).

Boolean Expressions and Truth Tables (2/7)

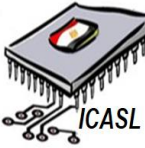


- ❑ The **truth table** is a common way of presenting the logical operation of a circuit.
- ❑ Standard **SOP** or **POS** expressions can be determined from a truth table and vice versa

$$X = AB + BCD + AC \quad \text{SOP}$$
$$X = (A + B). (B + C + D). (A + C) \quad \text{POS}$$



Input			Output
A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



Boolean Expressions and Truth Tables (3/7)

□ Converting SOP Expressions into Truth Table Format

Ex.1 Develop a truth table for the standard SOP expression

$$X = \overline{A} \overline{B} C + A \overline{B} \overline{C} + A B C$$

Ans.

□ 1st Step: Build the truth table

of variables= 3

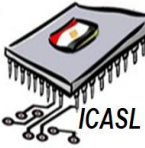
of possible combinations= $2^3=8$

□ 2nd Step: Fill in the table according to SOP

A SOP expression is equal to 1.

$\overline{A}$	$\overline{B}$	C	A	$\overline{B}$	$\overline{C}$	A	B	C
↓	↓	↓	↓	↓	↓	↓	↓	↓
0	0	1	1	0	0	1	1	1

Input			Output	Product Term
A	B	C	X	
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		



Boolean Expressions and Truth Tables (4/7)

□ Converting POS Expressions into Truth Table Format

Ex.1 Develop a truth table for the standard POS expression

$$X = (A + B + C)(A + \overline{B} + C)(A + \overline{B} + \overline{C})(\overline{A} + B + \overline{C})(\overline{A} + \overline{B} + C)$$

Ans.

□ 1st Step: Build the truth table

of variables= 3

of possible combinations= $2^3=8$

□ 2nd Step: Fill in the table according to POS

A POS expression is equal to 0

$$(A + B + C) (A + \overline{B} + C) (A + \overline{B} + \overline{C}) (\overline{A} + B + \overline{C}) (\overline{A} + \overline{B} + C)$$

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
0 0 0 0 1 0 0 1 1 1 0 1 1 1 0

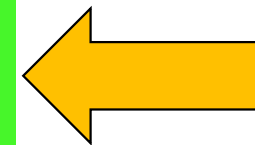
Input			Output	Sum Term
A	B	C	X	
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

Boolean Expressions and Truth Tables (5/7)

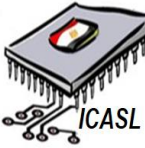


- ❑ The **truth table** is a common way of presenting the logical operation of a circuit.
- ❑ Standard **SOP** or **POS** expressions can be determined from a truth table and vice versa

$$X = AB + BCD + AC \quad \text{SOP}$$
$$X = (A + B). (B + C + D). (A + C) \quad \text{POS}$$



Input			Output
A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



Boolean Expressions and Truth Tables (6/7)

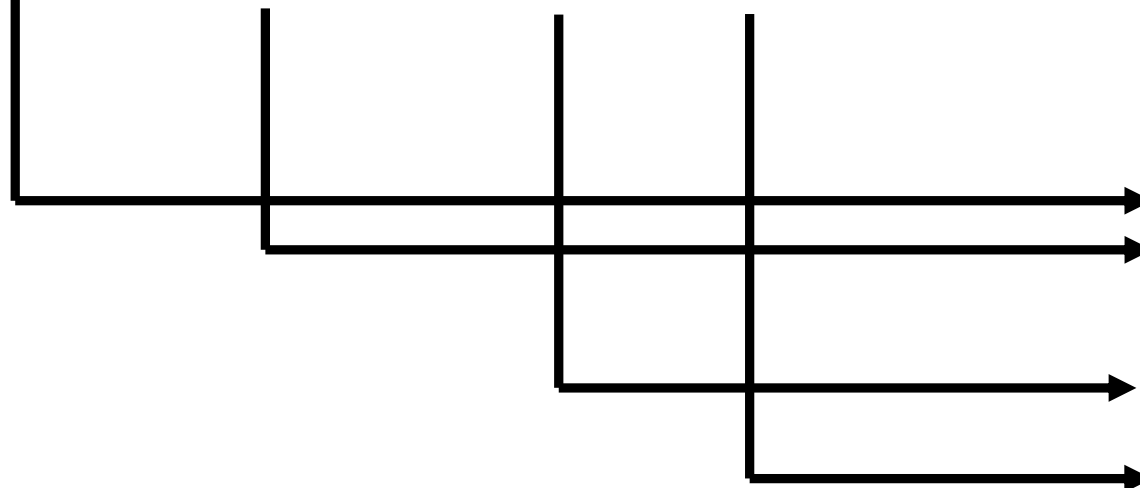
□ Converting Truth Table Format into Standard Expressions

Ex.1 Determine the standard **SOP** expression and the equivalent standard **POS** expression.

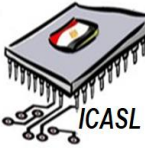
Ans.

□ To get SOP → 1

$$X = \overline{A} B C + A \overline{B} \overline{C} + A B \overline{C} + A B C$$



Input			Output
A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1



Boolean Expressions and Truth Tables (7/7)

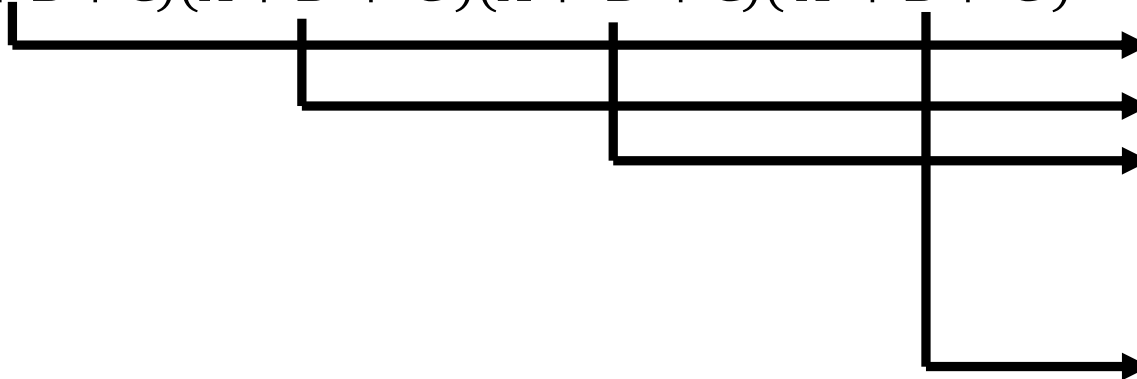
□ Converting Truth Table Format into Standard Expressions

Ex.1 Determine the standard **SOP** expression and the equivalent standard **POS** expression.

Ans.

□ To get POS → 0

$$X = (A + B + C)(A + B + \overline{C})(A + \overline{B} + C)(\overline{A} + B + \overline{C})$$



Input			Output
A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1



SOP and the POS expressions derived in this example are equivalent.

Outlines



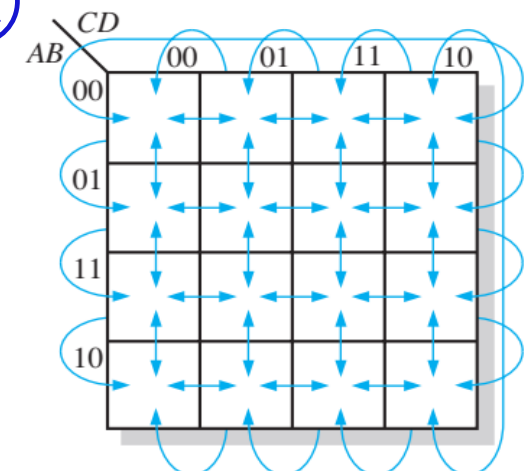
- ☐ Standard Forms of Boolean Expressions
- ☐ Boolean Expressions and Truth Tables
- ☒ **The Karnaugh Map (K-map)**
- ☐ Map the standard SOP expression in the K-Map
- ☐ K-Map Simplification of SOP Expressions
- ☐ K-Map Simplification of Truth Table
- ☐ K-Map Simplification of Logic Circuit

The Karnaugh Map



❑ Definition

- ❑ A **Karnaugh** map provides a systematic method for simplifying Boolean expressions. It provides a “**cookbook**” method for simplification. 😊
- ❑ It is an array of cells in which each cell represents a binary value of the input variables.
- ❑ **Notes:** there is only a single-variable change between adjacent cells. 😊
- ❑ The number of cells in a Karnaugh map is **equal** to the total number of possible input variable combinations= the number of rows in a truth table. 😊
- ❑ Ex.1 2-Variable needs $2^2 = 4$ cells.
- ❑ Ex.1 3-variables needs $2^3 = 8$ cells.
- ❑ Ex. 2 Four variables needs $2^4 = 16$ cells.



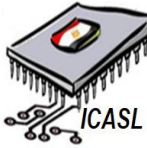
Adjacent cells on a K-Map

Outlines



- ☐ Standard Forms of Boolean Expressions
- ☐ Boolean Expressions and Truth Tables
- ☐ The Karnaugh Map (K-map)
- ☐ **Map the standard SOP expression in the K-Map**
- ☐ K-Map Simplification of SOP Expressions
- ☐ K-Map Simplification of Truth Table
- ☐ K-Map Simplification of Logic Circuit

Map the standard SOP expression in the K-Map (1/5)



- Map the standard SOP expression in the K-Map
 - The 2-variables Karnaugh Map

Ex.1

$$F = A \cdot \overline{B} + \overline{A} \cdot B$$

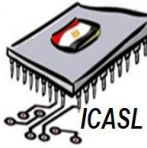
		B	
		0	1
A	0	0	1
	1	1	0

Ex.2

$$F = \overline{A} \cdot \overline{B} + A \cdot B$$

		B	
		0	1
A	0	1	0
	1	0	1

Map the standard SOP expression in the K-Map (2/5)



- Map the standard SOP expression in the K-Map
 - The 3-variables Karnaugh Map

Ex.1

$$F = A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot \overline{B} \cdot \overline{C} + A \cdot \overline{B} \cdot C + \overline{A} \cdot B + B \cdot \overline{C}$$

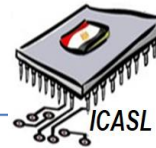
		B			
		00	01	11	10
A	0	1	0	1	1
	1	1	1	0	1

Ex.2

$$F = \overline{A} \cdot \overline{B} \cdot C + \overline{A} \cdot B \cdot \overline{C} + A \cdot B \cdot \overline{C} + ABC$$

		B			
		00	01	11	10
A	0	0	1	0	1
	1	0	0	1	1

Map the standard SOP expression in the K-Map (3/5)



- Map the standard SOP expression in the K-Map
 - The 3-variables Karnaugh Map

Ex.3

$$F = \overline{A} . B . C + A . \overline{B} . C + A . \overline{B} . \overline{C}$$

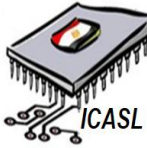
		B			
		00	01	11	10
A	0	0	0	1	0
	1	1	1	0	0

Ex.4

$$F = \overline{A} + A . \overline{B} + A . B . \overline{C}$$

		B			
		00	01	11	10
A	0	1	1	1	1
	1	1	1	0	1

Map the standard SOP expression in the K-Map (4/5)



- Map the standard SOP expression in the K-Map
 - The 4-variables Karnaugh Map

Ex.1

$$F = \overline{A} \cdot \overline{C} \cdot \overline{D} + A \cdot \overline{C} \cdot \overline{D} + \overline{A} \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot C \cdot \overline{D} + A \cdot C \cdot \overline{D}$$

		CD			
		00	01	11	10
A	00	1	1	0	1
	01	1	0	0	1
	11	1	0	0	1
	10	1	0	0	1

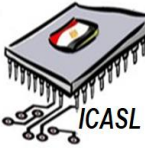
Groupings: {00, 01} (D), {11, 10} (C), {00, 11} (A), {01, 10} (B)

Ex.2

$$F = \overline{B} \cdot \overline{C} + A \cdot \overline{B} + A \cdot B \cdot \overline{C} + A \cdot \overline{B} \cdot C \cdot \overline{D} + \overline{A} \cdot \overline{B} \cdot \overline{C} \cdot D + A \cdot \overline{B} \cdot C \cdot D$$

		CD			
		00	01	11	10
A	00	1	1	0	0
	01	0	0	0	0
	11	1	1	0	0
	10	1	1	1	1

Groupings: {00, 01} (D), {11, 10} (C), {00, 11} (A), {01, 10} (B)



Map the standard SOP expression in the K-Map (5/5)

- Map the standard SOP expression in the K-Map
 - The 4-variables Karnaugh Map

Ex.3

$$F = A + \overline{C} \cdot D + A \cdot C \cdot \overline{D} + \overline{A} \cdot B \cdot C \cdot \overline{D}$$

		CD			
		00	01	11	10
A	00	0	1	0	0
	01	0	1	0	1
	11	1	1	1	1
	10	1	1	1	1

Groupings: {C, D}, {B}, {A}

Ex.4

$$F = \overline{B} \cdot \overline{C} \cdot \overline{D} + \overline{A} \cdot B \cdot \overline{C} \cdot \overline{D} + A \cdot B \cdot \overline{C} \cdot \overline{D} + \overline{A} \cdot \overline{B} \cdot C \cdot D + A \cdot \overline{B} \cdot C \cdot D + \overline{A} \cdot \overline{B} \cdot C \cdot \overline{D} + \overline{A} \cdot B \cdot C \cdot \overline{D} + A \cdot B \cdot C \cdot \overline{D} + A \cdot \overline{B} \cdot C \cdot \overline{D}$$

		CD			
		00	01	11	10
A	00	1	0	1	1
	01	1	0	0	1
	11	1	0	0	1
	10	1	0	1	1

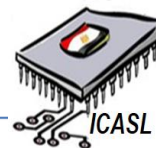
Groupings: {C, D}, {B}, {A}

Outlines



- ☐ Standard Forms of Boolean Expressions
- ☐ Boolean Expressions and Truth Tables
- ☐ The Karnaugh Map (K-map)
- ☐ Map the standard SOP expression in the K-Map
- ☐ **K-Map Simplification of SOP Expressions**
- ☐ K-Map Simplification of Truth Table
- ☐ K-Map Simplification of Logic Circuit

K-Map Simplification of SOP Expressions (1/6)



- ❑ **Fewest possible terms with the Fewest possible variables.** 😊
- ❑ A minimum **SOP** expression is obtained by grouping the **1s**
 - ❑ A group must contain either 1, 2, 4, 8, or 16 cells, which are all powers of **two**. In the case of a 3-variable map, $2^3 = 8$ cells is the maximum group.
 - ❑ **Always** include the **largest** possible number of 1s in a group .
 - ❑ Each 1 on the map must be included in at **least** one group. The 1s already in a group can be **included** in another group as long as the overlapping groups include non-common 1s
 - ❑ Each group of cells containing 1s creates one **product term**.
 - ❑ When all the minimum product terms are derived from the **Karnaugh** map, they are summed to form the minimum **SOP** expression.

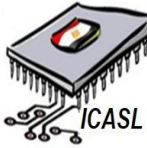
For a 3-variable map

- (1) A 1-cell group yields a 3-variable product term
- (2) A 2-cell group yields a 2-variable product term
- (3) A 4-cell group yields a 1-variable term
- (4) An 8-cell group yields a value of 1 for the expression.

For a 4-variable map

- (1) A 1-cell group yields a 4-variable product term
- (2) A 2-cell group yields a 3-variable product term
- (3) A 4-cell group yields a 2-variable product term
- (4) An 8-cell group yields a 1-variable term
- (5) A 16-cell group yields a value of 1 for the expression

K-Map Simplification of SOP Expressions (2/6)



□ The 2-variables Karnaugh Map

Ex.1
 $F = A \cdot \overline{B} + \overline{A} \cdot B + A \cdot B$



Ex.1
 $F = A + B$

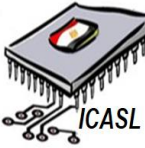
		B	
A	$\overline{A}$	0	1
	1	0	1
A	0	1	1
	1	1	1

Ex.2
 $F = A \cdot \overline{B} + \overline{A} \cdot B + \overline{A} \cdot \overline{B}$



Ex.2
 $F = \overline{A} + \overline{B}$

		B	
A	$\overline{A}$	0	1
	1	1	1
A	0	1	0
	1	1	0



K-Map Simplification of SOP Expressions (3/6)

□ The 3-variables Karnaugh Map

Ex.1

$$F = \overline{A} \cdot \overline{B} \cdot C + \overline{A} \cdot B \cdot \overline{C} + A \cdot B \cdot \overline{C} + ABC$$



Ex.1

$$F = A \cdot B + B \cdot \overline{C} + \overline{A} \cdot \overline{B} \cdot C$$

		B			
		00	01	11	10
A	0	0	1	0	1
	1	0	0	1	1

Groupings: A horizontal group of 1s in the first row (01, 10) and a vertical group of 1s in the second column (00, 10). A horizontal group of 1s in the second row (01, 11) and a vertical group of 1s in the third column (01, 11). A horizontal group of 1s in the third row (01, 11) and a vertical group of 1s in the fourth column (11, 10).

Ex.1

$$F = A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot \overline{B} \cdot \overline{C} + A \cdot \overline{B} \cdot C + \overline{A} \cdot B + B \cdot \overline{C}$$

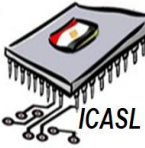


Ex.1

$$F = \overline{C} + \overline{A} \cdot B + A \cdot \overline{B}$$

		B			
		00	01	11	10
A	0	1	0	1	1
	1	1	1	0	1

Groupings: A horizontal group of 1s in the first row (00, 11, 10) and a vertical group of 1s in the second column (00, 10). A horizontal group of 1s in the second row (00, 01, 11) and a vertical group of 1s in the third column (01, 11). A horizontal group of 1s in the third row (00, 01, 11) and a vertical group of 1s in the fourth column (11, 10).



K-Map Simplification of SOP Expressions (4/6)

□ The 3-variables Karnaugh Map

Ex.3

$$F = \overline{A}.B.C + A.\overline{B}.C + A.\overline{B}.\overline{C}$$



Ex.3

$$F = A.\overline{B} + \overline{A}.B.C$$

		B			
		00	01	11	10
A	0	0	0	1	0
	1	1	1	0	0

Groupings: A horizontal group of 1s in the first row (A=0) and a vertical group of 1s in the first column (A=1). A horizontal group of 1s in the second row (A=1) is also shown.

Ex.4

$$F = \overline{A} + A.\overline{B} + A.B.\overline{C}$$



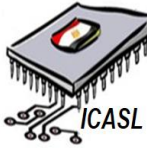
Ex.4

$$F = \overline{A} + \overline{B} + \overline{C}$$

		B			
		00	01	11	10
A	0	1	1	1	1
	1	1	1	0	1

Groupings: A horizontal group of 1s in the first row (A=0), a horizontal group of 1s in the second row (A=1), and a vertical group of 1s in the first column (A=1). A horizontal group of 1s in the first row (A=0) is also shown.

K-Map Simplification of SOP Expressions (5/6)



□ The 4-variables Karnaugh Map

Ex.1

$$F = \overline{A} \cdot \overline{C} \cdot \overline{D} + A \cdot \overline{C} \cdot \overline{D} + \overline{A} \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot C \cdot \overline{D} + A \cdot C \cdot \overline{D}$$



Ex.1

$$F = \overline{D} + \overline{A} \cdot \overline{B} \cdot \overline{C}$$

CD \ AB	00	01	11	10
00	1	1	0	1
01	1	0	0	1
11	1	0	0	1
10	1	0	0	1

Ex.2

$$F = \overline{B} \cdot \overline{C} + A \cdot \overline{B} + A \cdot B \cdot \overline{C} + A \cdot \overline{B} \cdot C \cdot \overline{D} + \overline{A} \cdot \overline{B} \cdot \overline{C} \cdot D + A \cdot \overline{B} \cdot C \cdot D$$

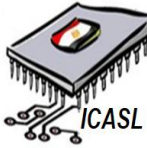


Ex.2

$$F = A \cdot \overline{B} + A \cdot \overline{C} + \overline{B} \cdot \overline{C}$$

CD \ AB	00	01	11	10
00	1	1	0	0
01	0	0	0	0
11	1	1	0	0
10	1	1	1	1

K-Map Simplification of SOP Expressions (6/6)



□ The 4-variables Karnaugh Map

Ex.3

$$F = \overline{B} \cdot \overline{C} \cdot \overline{D} + \overline{A} \cdot B \cdot \overline{C} \cdot \overline{D} + A \cdot B \cdot \overline{C} \cdot \overline{D} + \overline{A} \cdot \overline{B} \cdot C \cdot D + A \cdot \overline{B} \cdot C \cdot D + \overline{A} \cdot \overline{B} \cdot C \cdot \overline{D} + \overline{A} \cdot B \cdot C \cdot \overline{D} + A \cdot B \cdot C \cdot \overline{D} + A \cdot \overline{B} \cdot C \cdot \overline{D}$$



Ex.3

$$F = \overline{D} + \overline{B} \cdot C$$

		C			
		00	01	11	10
A	00	1	0	1	1
	01	1	0	0	1
	11	1	0	0	1
	10	1	0	1	1

Groupings:
 - Group D: { (A=0, B=0, C=0, D=1), (A=0, B=0, C=1, D=1), (A=0, B=1, C=0, D=1), (A=0, B=1, C=1, D=1) }
 - Group B: { (A=0, B=0, C=0, D=1), (A=0, B=0, C=1, D=1), (A=1, B=0, C=0, D=1), (A=1, B=0, C=1, D=1) }
 - Group C: { (A=0, B=0, C=1, D=1), (A=0, B=1, C=1, D=1), (A=1, B=0, C=1, D=1), (A=1, B=1, C=1, D=1) }
 - Group A: { (A=0, B=0, C=0, D=1), (A=1, B=0, C=0, D=1), (A=0, B=1, C=0, D=1), (A=1, B=1, C=0, D=1) }

Outlines



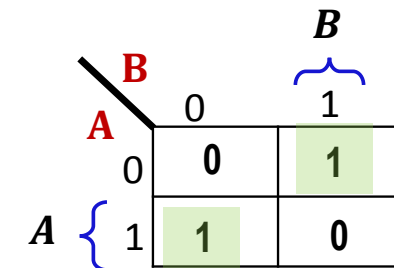
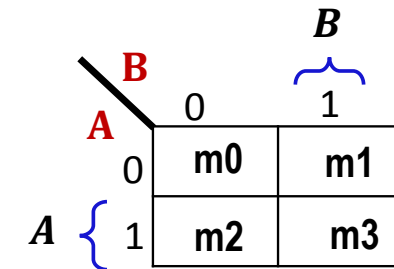
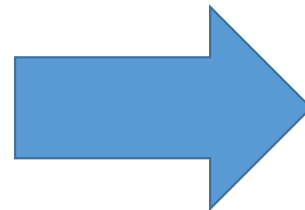
- ☐ Standard Forms of Boolean Expressions
- ☐ Boolean Expressions and Truth Tables
- ☐ The Karnaugh Map (K-map)
- ☐ Map the standard SOP expression in the K-Map
- ☐ K-Map Simplification of SOP Expressions
- ☐ **K-Map Simplification of Truth Table**
- ☐ K-Map Simplification of Logic Circuit

K-Map Simplification of Truth Table (1/3)

□ The 2-variables Karnaugh Map

Truth Table

Input		Output
A	B	F
0	0	0
0	1	1
1	0	1
1	1	0



$$F = A \cdot \overline{B} + \overline{A} \cdot B$$

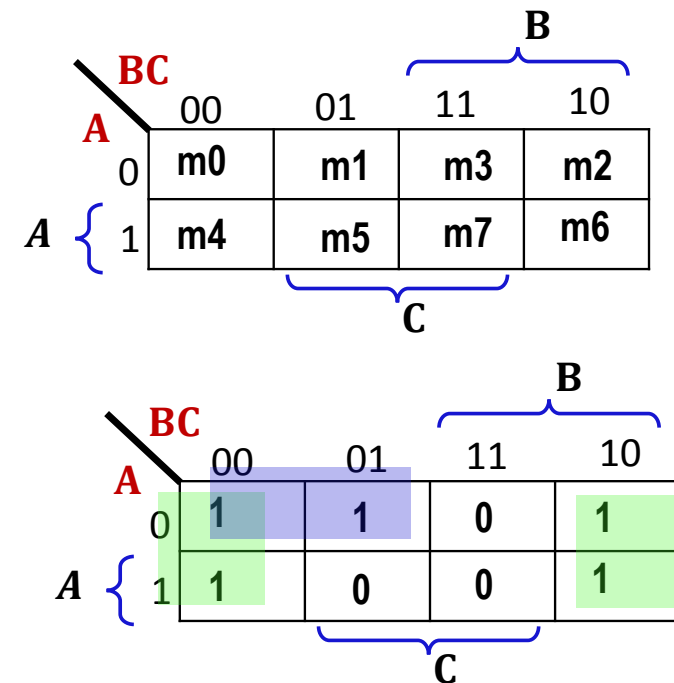
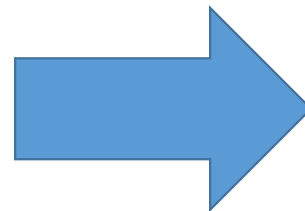
K-Map Simplification of Truth Table (2/3)



□ The 3-variables Karnaugh Map

Truth Table

Input			Output
A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

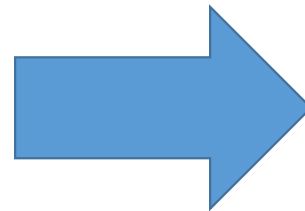


$$F = \overline{C} + \overline{A} \cdot \overline{B}$$

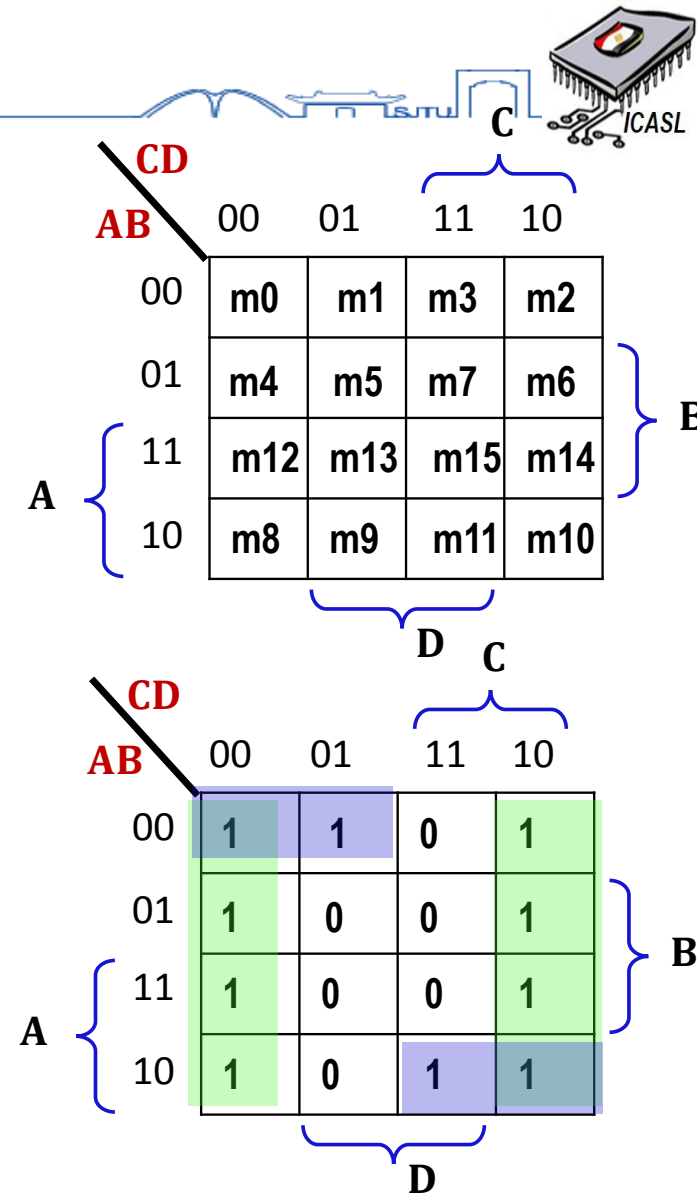
K-Map Simplification of Truth Table (3/3)

- ❑ The 4-variables Karnaugh Map
- Truth Table**

Input				Output
A	B	C	D	F
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0



$$F = \overline{D} + A \cdot \overline{B} \cdot C + \overline{A} \cdot \overline{B} \cdot \overline{C}$$

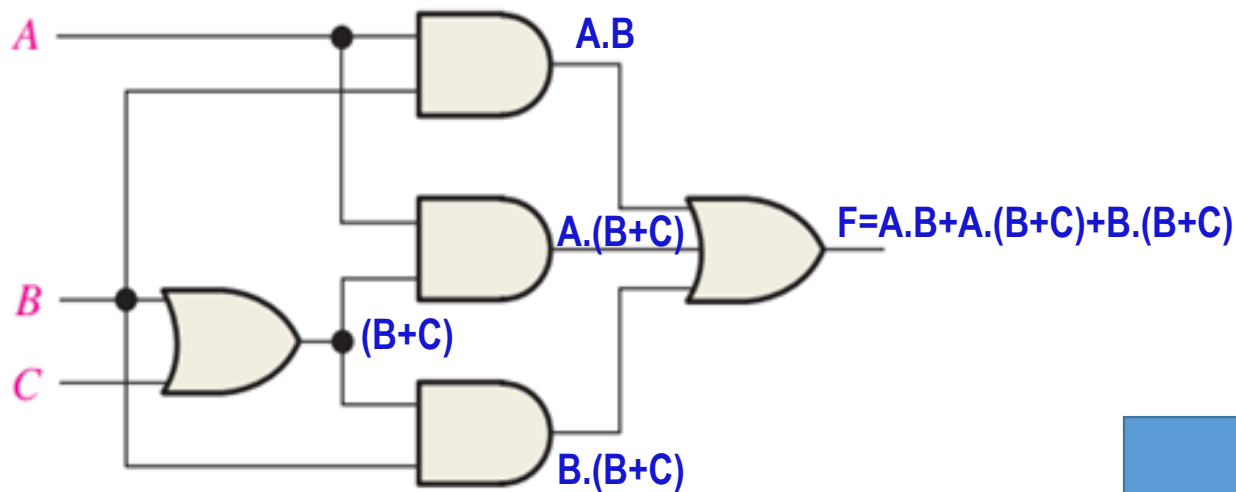


Outlines



- ☐ Standard Forms of Boolean Expressions
- ☐ Boolean Expressions and Truth Tables
- ☐ The Karnaugh Map (K-map)
- ☐ Map the standard SOP expression in the K-Map
- ☐ K-Map Simplification of SOP Expressions
- ☐ K-Map Simplification of Truth Table
- ☐ **K-Map Simplification of Logic Circuit**

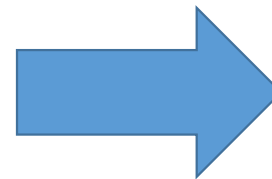
K-Map Simplification of Logic Circuit (4/4)



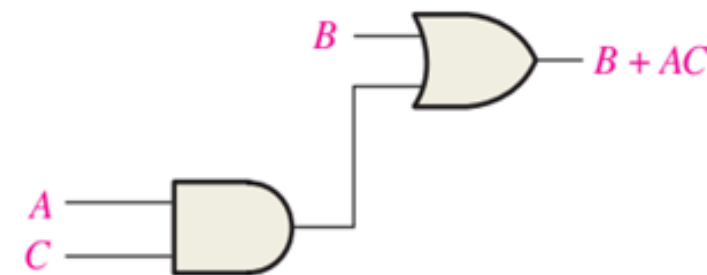
$$F = A.B + A.B + A.C + B.B + B.C$$

$$F = A.B + A.C + B + B.C$$

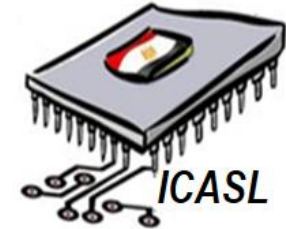
$$F = B + AC$$



		B			
		BC			
A	0	0	0	1	1
	1	0	1	1	1



Thanks!



Digital Design (SET 111)

L08

Combinational Logic Analysis

Dr. Ahmed Reda Mohamed¹

¹PhD, the MoE Key Laboratory of Artificial Intelligence & Bio-Circuits and Systems Laboratory (**BiCASL**), Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.

Outlines



- ☐ Implementation using Logic Gates
- ☐ Implementation using Universal Logic Gates

Outlines

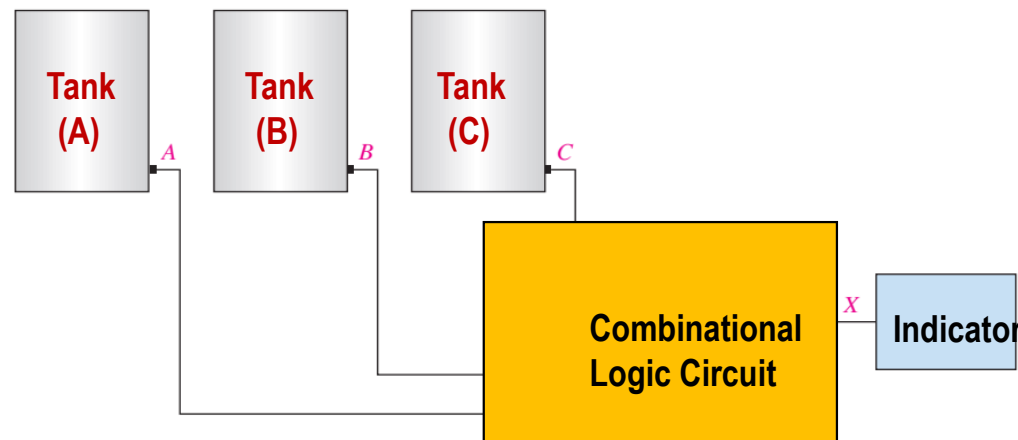


- ☐ Implementation using Logic Gates
- ☐ Implementation using Universal Logic Gates

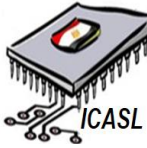
Implementation using Logic Gates



- Ex.1 In a certain chemical-processing plant, a liquid chemical is used in a manufacturing process. The chemical is stored in **three** different tanks. A level sensor in each tank produces a **HIGH** voltage when the level of chemical in the tank drops below a specified point. Design a circuit that monitors the chemical level in each tank and indicates when the level in **any two** of the tanks drops below the specified point.

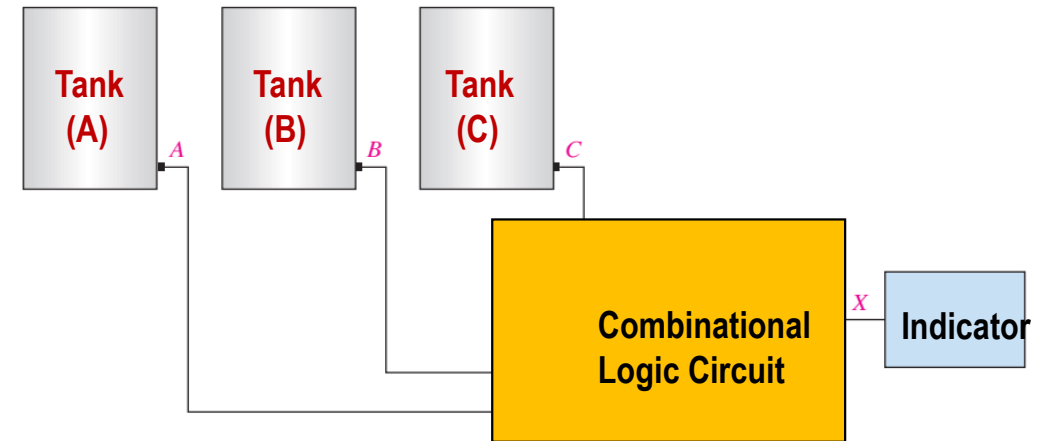


Implementation using Logic Gates



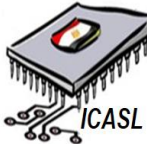
Ans.

Input			Output
A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



		B			
		00	01	11	10
A	0	0	0	1	0
	1	1	1	1	1
		C			

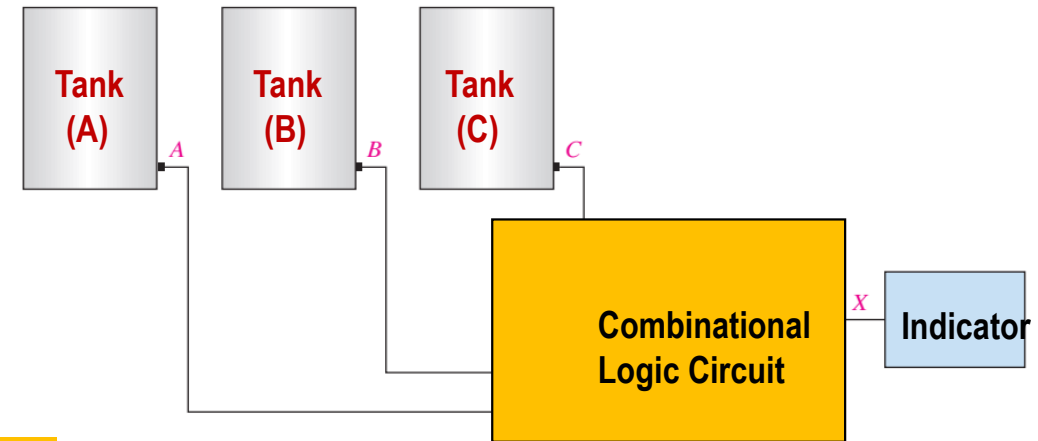
Implementation using Logic Gates



Input			Output
A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

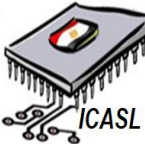
Ans.

$$X = AB + AC + BC$$



		B			
		00	01	11	10
A	0	0	0	1	0
	1	1	1	1	1
		C			

Karnaugh map showing the output X for inputs A, B, and C. The map is a 2x4 grid. The columns are labeled with BC (00, 01, 11, 10) and the rows with A (0, 1). The output values are: (0,00)=0, (0,01)=0, (0,11)=1, (0,10)=0, (1,00)=1, (1,01)=1, (1,11)=1, (1,10)=1. The cells with 1 are highlighted in different colors: (0,11) is blue, (1,00) is green, (1,01) is purple, (1,11) is red, and (1,10) is pink. Brackets indicate groupings: a blue bracket under the 1s in the BC=11 column, a green bracket under the 1s in the A=1 row, and a red bracket under the 1s in the BC=01 and BC=11 columns.



Implementation using Logic Gates

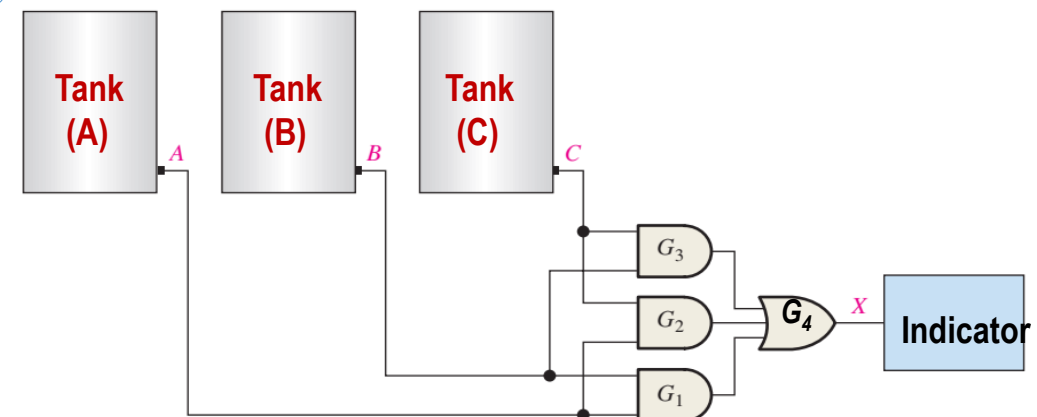
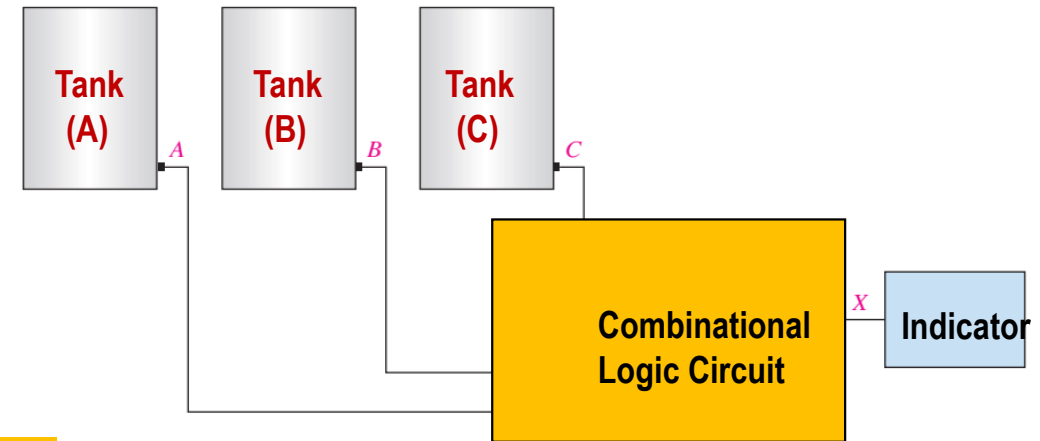
Input			Output
A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

		B			
		00	01	11	10
A	0	0	0	1	0
	1	0	1	1	1

Ans.

$$X = AB + AC + BC$$

G_1 G_2 G_3
 G_4

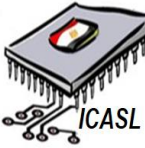


Implementation using Logic Gates

□ Ex.2 Design an XOR gate of two inputs using AND, OR, and NOT gates.

Ans.





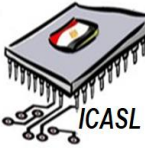
Implementation using Logic Gates

□ Ex.2 Design an XOR gate of two inputs using AND, OR, and NOT gates.

Ans.

Input		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

		<i>B</i>	
		0	1
<i>A</i>	0	0	1
	1	1	0



Implementation using Logic Gates

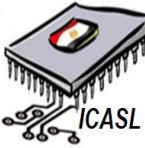
□ Ex.2 Design an XOR gate of two inputs using AND, OR, and NOT gates.

Ans.

Input		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

		B	
		0	1
A	0	0	1
	1	1	0

$$X = A \cdot \overline{B} + \overline{A} \cdot B$$



Implementation using Logic Gates

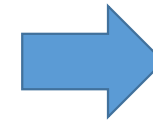
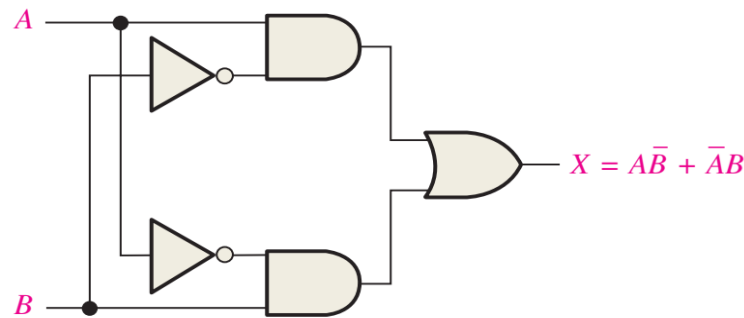
□ Ex.2 Design an XOR gate of two inputs using AND, OR, and NOT gates.

Ans.

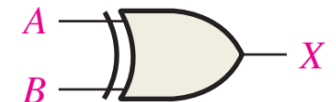
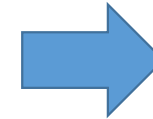
Input		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

A	B	
	0	1
0	0	1
1	1	0

$$X = A \cdot \overline{B} + \overline{A} \cdot B$$



$$X = A \oplus B$$

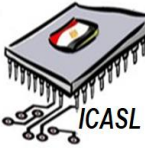


Implementation using Logic Gates

□ Ex.3 Design an XNOR gate of two inputs using AND, OR, and NOT gates.

Ans.





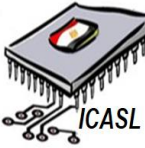
Implementation using Logic Gates

□ Ex.3 Design an XNOR gate of two inputs using AND, OR, and NOT gates.

Ans.

Input		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

		<i>B</i>	
		0	1
<i>A</i>	0	1	0
	1	0	1



Implementation using Logic Gates

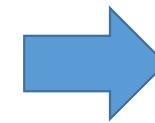
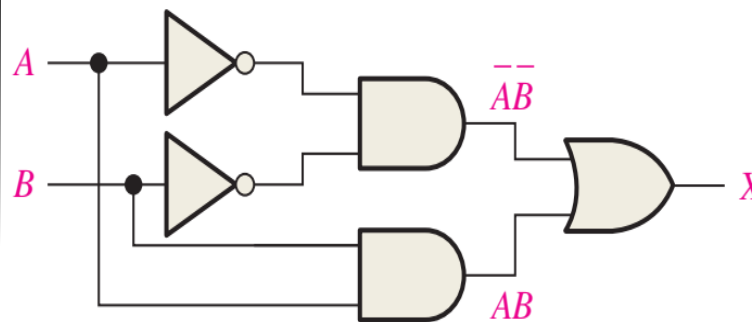
□ Ex.3 Design an XNOR gate of two inputs using AND, OR, and NOT gates.

Ans.

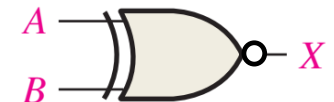
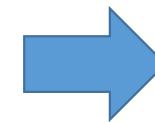
Input		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

A	B	
	0	1
0	1	0
1	0	1

$$X = \overline{A} \cdot \overline{B} + A \cdot B$$



$$X = A \odot B$$



Implementation using Logic Gates



- Ex. 4 Design a logic circuit with four input variables that will only produce a 1 output when **exactly three** input variables are 1s.

Ans.

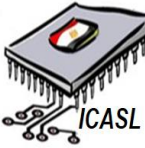
Implementation using Logic Gates



- Ex. 4 Design a logic circuit with four input variables that will only produce a 1 output when **exactly three** input variables are 1s.

Ans.

Input				Output
A	B	C	D	x
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0



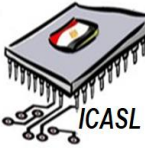
Implementation using Logic Gates

- Ex. 4 Design a logic circuit with four input variables that will only produce a 1 output when **exactly three** input variables are 1s.

Input				Output
A	B	C	D	x
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Ans.

		C			
		00	01	11	10
A	CD	00	01	11	10
	AB	0	0	0	0
	01	0	0	1	0
	11	0	1	0	1
	10	0	0	1	0



Implementation using Logic Gates

- Ex. 4 Design a logic circuit with four input variables that will only produce a 1 output when **exactly three** input variables are 1s.

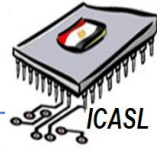
Input				Output
A	B	C	D	x
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Ans.

		C			
		00	01	11	10
A	CD \ AB	00	01	11	10
	00	0	0	0	0
	01	0	0	1	0
	11	0	1	0	1
	10	0	0	1	0

Blue shaded cells (1s) are at (AB, CD) = (01, 11), (11, 01), (11, 10), and (10, 11). Brackets indicate groupings for variables B, C, and D.

$$X = \overline{A} . B . C . D + A . \overline{B} . C . D + A . B . \overline{C} . D + A . B . C . \overline{D}$$



Implementation using Logic Gates

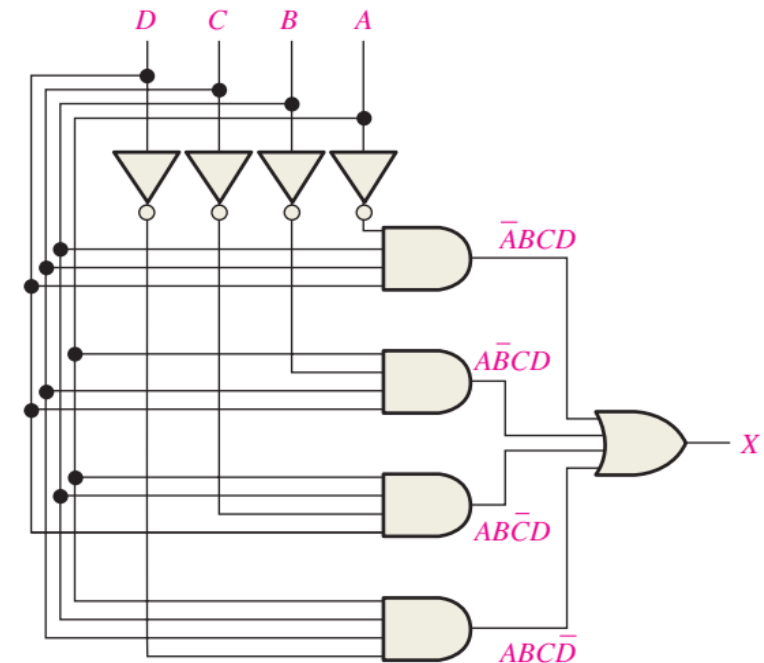
- Ex. 4 Design a logic circuit with four input variables that will only produce a 1 output when exactly three input variables are 1s.

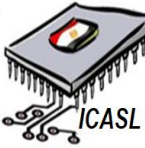
Input				Output
A	B	C	D	x
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Ans.

		C			
		00	01	11	10
A	00	0	0	0	0
	01	0	0	1	0
	11	0	1	0	1
	10	0	0	1	0

$$X = \bar{A} \cdot B \cdot C \cdot D + A \cdot \bar{B} \cdot C \cdot D + A \cdot B \cdot \bar{C} \cdot D + A \cdot B \cdot C \cdot \bar{D}$$





Implementation using Logic Gates

- ❑ Ex. 4 Design a logic circuit with four input variables that will only produce a 1 output when exactly three input variables are 1s.

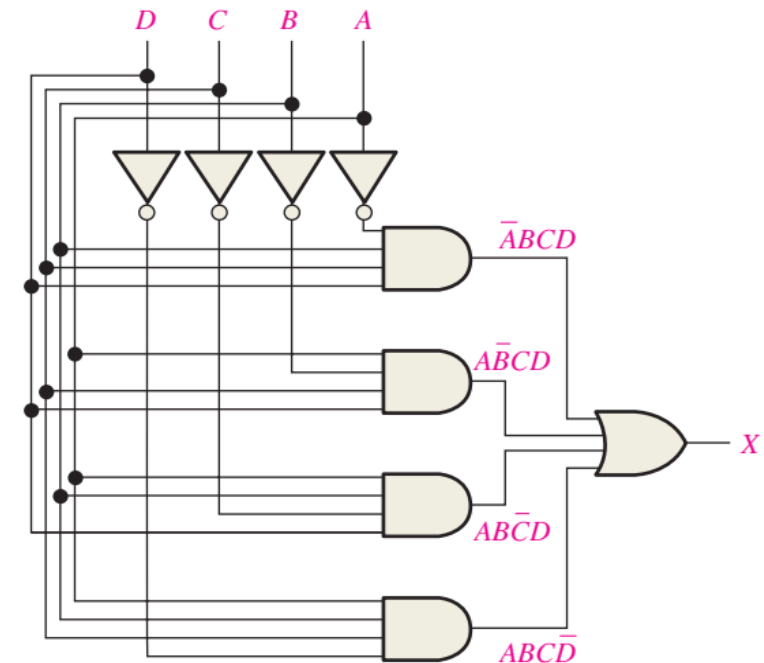
Input				Output
A	B	C	D	x
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Ans.

		C			
		00	01	11	10
A	CD				
	AB	00	01	11	10
	00	0	0	0	0
	01	0	0	1	0
	11	0	1	0	1
	10	0	0	1	0

Brackets in the original image group the rows by A (00, 01, 11, 10) and columns by C (00, 01, 11, 10). The cells with value 1 are at (A=01, C=11), (A=11, C=01), (A=11, C=10), and (A=10, C=11).

$$X = \bar{A} \cdot B \cdot C \cdot D + A \cdot \bar{B} \cdot C \cdot D + A \cdot B \cdot \bar{C} \cdot D + A \cdot B \cdot C \cdot \bar{D}$$



Implementation using Logic Gates



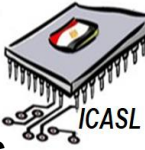
- Ex. 5 Design a logic circuit with four input variables will produce a 1 on its output only when all three inputs are 1s or when all three inputs are 0s

Ans.

Outlines



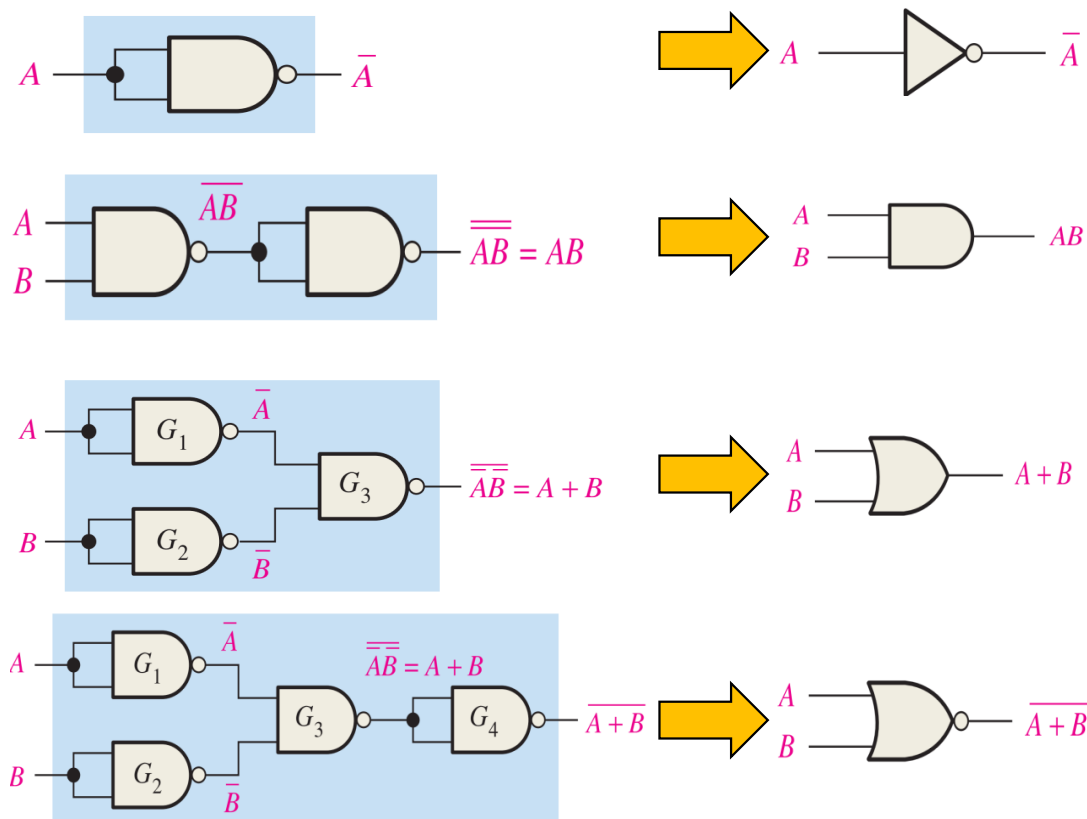
- ❑ Implementation using Logic Gates
- ❑ Implementation using Universal Logic Gates



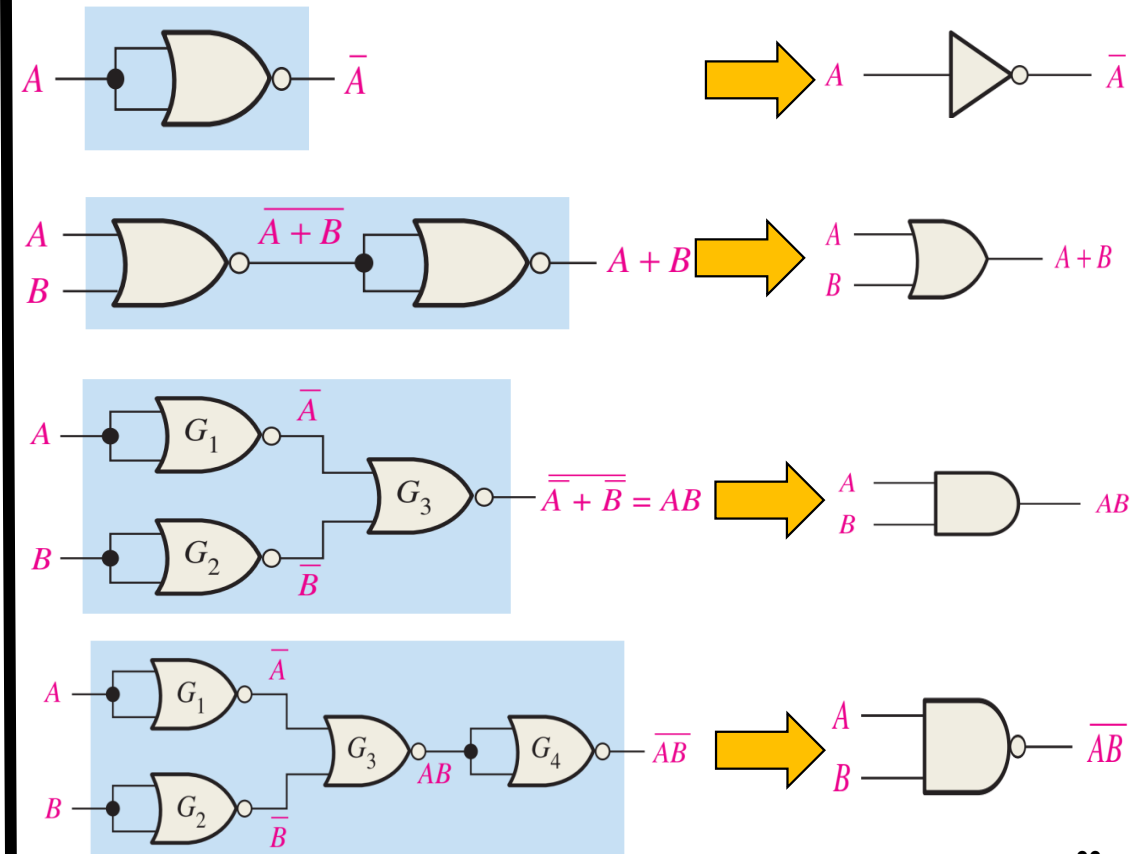
Implementation using Universal Logic Gates

- ❑ The **NAND** gate is a universal gate because it can implement the NOT, the AND, the OR, and the NOR functions.
- ❑ The **NOR** gate is a universal gate because it can implement the NOT, the AND, the OR, and the NOR functions.

NAND Gate

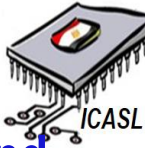


NOR Gate



Implementation using Universal Logic Gates

In order to implement a digital circuit for logic functions using **Universal gates (NAND, NOR and NOT)**.



NAND Gate

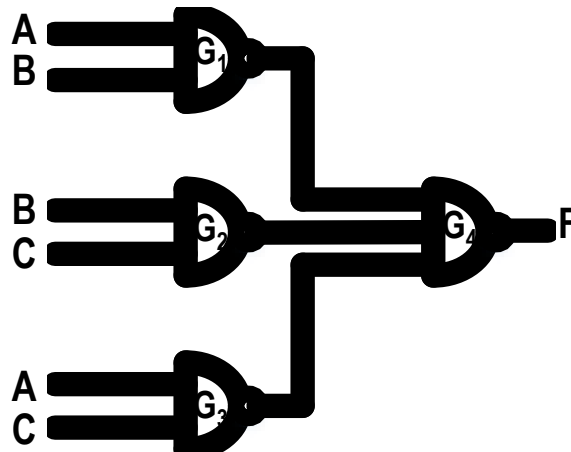
❑ The function must be in form of Sum Of Product (SOP)

Ex: $F = AB + BC + AC$

$$\overline{\overline{F}} = \overline{\overline{AB + BC + AC}}$$

$$\overline{\overline{F}} = \overline{\overline{AB} \cdot \overline{BC} \cdot \overline{AC}}$$

$\underbrace{\overline{AB}}_{G_1} \quad \underbrace{\overline{BC}}_{G_2} \quad \underbrace{\overline{AC}}_{G_3}$
 $\underbrace{\hspace{10em}}_{G_4}$



NOR Gate

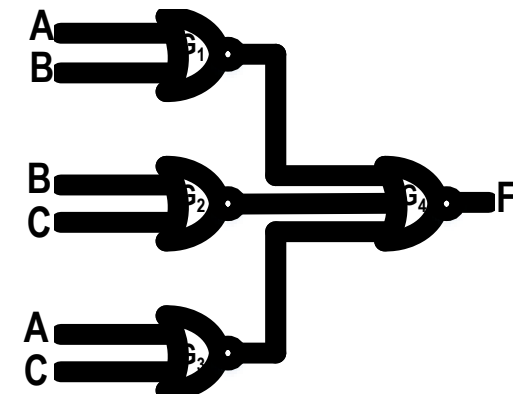
❑ The function must be in form of Product of Sum (POS)

Ex: $F = (A+B) \cdot (B+C) \cdot (A+C)$

$$\overline{\overline{F}} = \overline{\overline{(A+B) \cdot (B+C) \cdot (A+C)}}$$

$$\overline{\overline{F}} = \overline{\overline{(A+B)} + \overline{\overline{(B+C)}} + \overline{\overline{(A+C)}}}$$

$\underbrace{\overline{\overline{(A+B)}}}_{G_1} \quad \underbrace{\overline{\overline{(B+C)}}}_{G_2} \quad \underbrace{\overline{\overline{(A+C)}}}_{G_3}$
 $\underbrace{\hspace{10em}}_{G_4}$



Implementation using Universal Logic Gates



- Ex. 1 Using universal logic gates, implement a three input odd-parity checker using minimum number of transistor .Noted that the output is high when an odd number of 1's.

Ans.

Implementation using Universal Logic Gates



- Ex. 1 Using universal logic gates, implement a three input odd-parity checker using minimum number of transistor .Noted that the output is high when an odd number of 1's.

Ans.





Implementation using Universal Logic Gates

- Ex. 1 Using universal logic gates, implement a three input odd-parity checker using minimum number of transistor .Noted that the output is high when an odd number of 1's.

Ans.

Input			Output
C	B	A	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1





Implementation using Universal Logic Gates

- Ex. 1 Using universal logic gates, implement a three input odd-parity checker using minimum number of transistor .Noted that the output is high when an odd number of 1's.

Ans.

Input			Output
C	B	A	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1





Implementation using Universal Logic Gates

- Ex. 1 Using universal logic gates, implement a three input odd-parity checker using minimum number of transistor .Noted that the output is high when an odd number of 1's.

Ans.

Input			Output
C	B	A	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



		B			
		00	01	11	10
A	0	0	1	0	1
	1	1	0	1	0

Diagram illustrating the truth table for the three-input odd-parity checker. The inputs are A, B, and C. The output is F. The truth table shows that the output is 1 for odd numbers of 1's (001, 010, 100, 111) and 0 for even numbers of 1's (000, 011, 101, 110).



Implementation using Universal Logic Gates

- Ex. 1 Using universal logic gates, implement a three input odd-parity checker using minimum number of transistor .Noted that the output is high when an odd number of 1's.

Ans.

Input			Output
C	B	A	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



A \ BC	B			
	00	01	11	10
0	0	1	0	1
1	1	0	1	0

The output function is obtained by using Sum Of Product (SOP)

$$F = A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot B \cdot \overline{C} + \overline{A} \cdot \overline{B} \cdot C + A \cdot B \cdot C$$

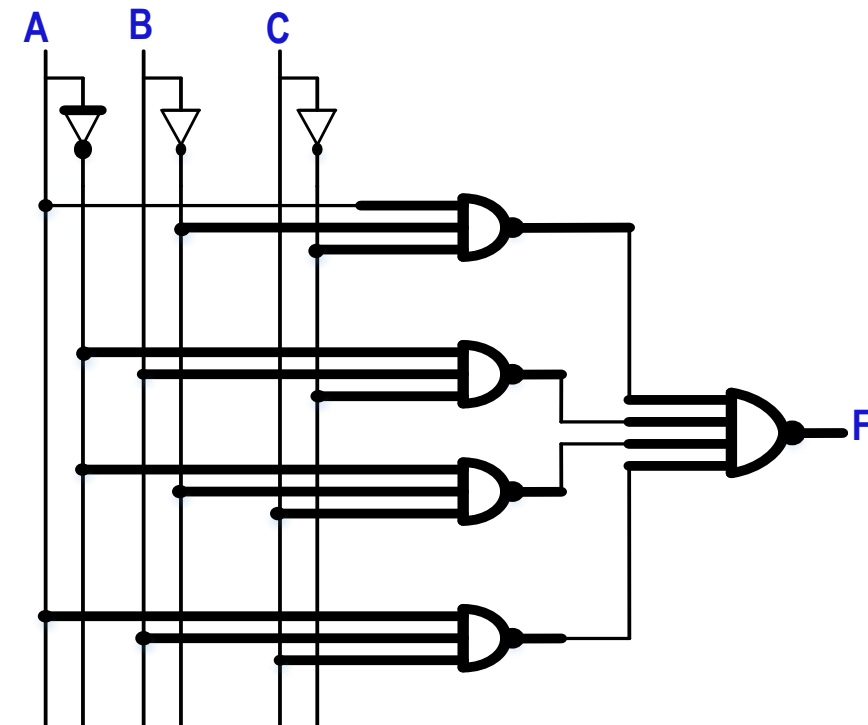
Implementation using Universal Logic Gates



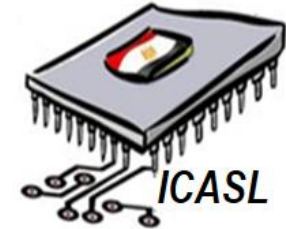
□ Ex. 1 Using universal logic gates, implement a three input odd-parity checker using minimum number of transistor .Noted that the output is high when an odd number of 1's.

Ans.

$$F = A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot B \cdot \overline{C} + \overline{A} \cdot \overline{B} \cdot C + A \cdot B \cdot C$$
$$F = A \cdot \overline{B} \cdot \overline{C} \cdot \overline{A} \cdot B \cdot \overline{C} \cdot \overline{A} \cdot \overline{B} \cdot C \cdot A \cdot B \cdot C$$



Thanks!



Digital Design (SET 111)

L09

Boolean Algebra and Logic Simplification

Dr. Ahmed Reda Mohamed¹

¹PhD, the MoE Key Laboratory of Artificial Intelligence & Bio-Circuits and Systems Laboratory (**BiCASL**), Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.

Outlines

- ☐ Exercise
- ☐ “Don’t Care” Conditions



Outlines

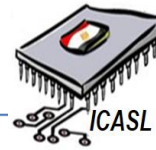
☐ Exercise

☐ “Don’t Care” Conditions



Exercise

□ Ex.1 Use a Karnaugh map to minimize the following standard SOP expression:



$$F = A \cdot \overline{B} \cdot C + \overline{A} \cdot B \cdot C + \overline{A} \cdot \overline{B} \cdot C + \overline{A} \cdot \overline{B} \cdot \overline{C} + A \cdot \overline{B} \cdot \overline{C}$$

Ans

$$F = \overline{B} + \overline{A} \cdot C$$

		B			
		00	01	11	10
A	0	1	1	1	0
	1	1	1	0	0

“Don’t Care” Conditions (1/2)



- ❑ Sometimes a situation arises in which some **input** variable combinations are **not allowed**.
- ❑ That is, for these “**don’t care**” terms either a **1** or a **0** may be assigned to the output; it really does **not matter** since they will never occur.
- ❑ The “don’t care” terms can be used to advantage on the **Karnaugh** map.
- ❑ For each “don’t care” term, an **X** is placed in the cell. When grouping the 1s, the Xs can be treated as 1s to make a larger grouping or as 0s if they cannot be used to advantage.

“Don’t Care” Conditions (2/2)



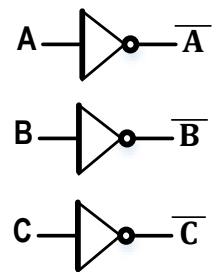
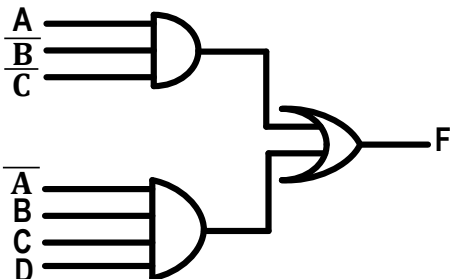
❑ Example.

Truth Table

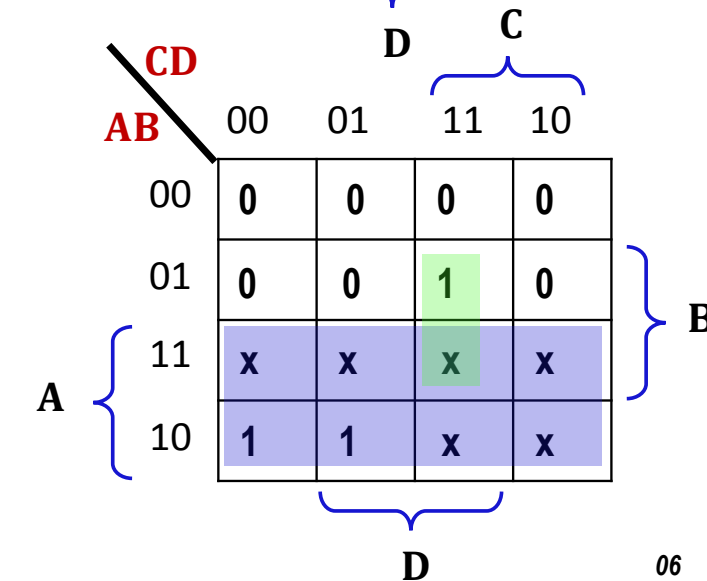
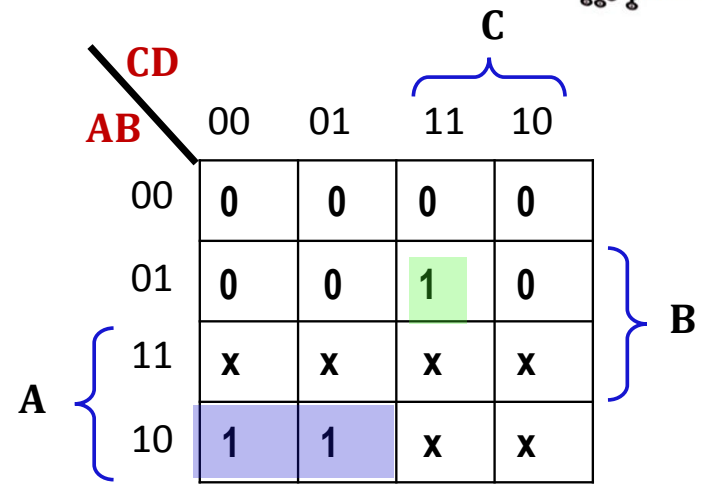
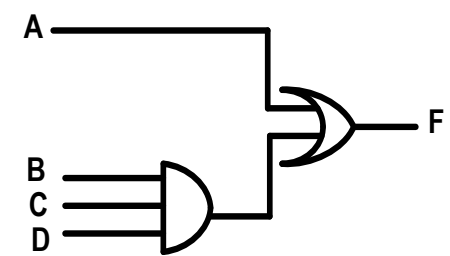
Input				Output
A	B	C	D	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

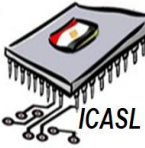
Don't Care

❑ Without don't care
 $F = A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot B \cdot C \cdot D$



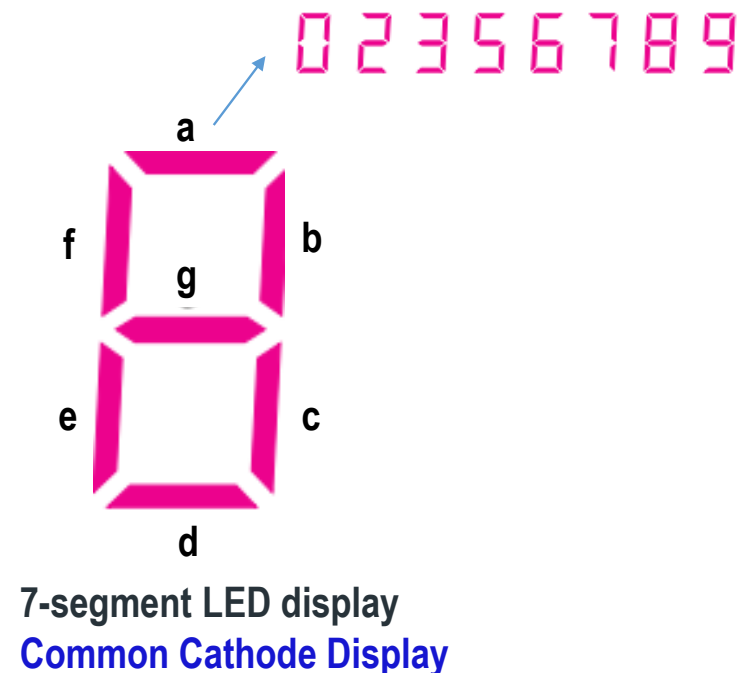
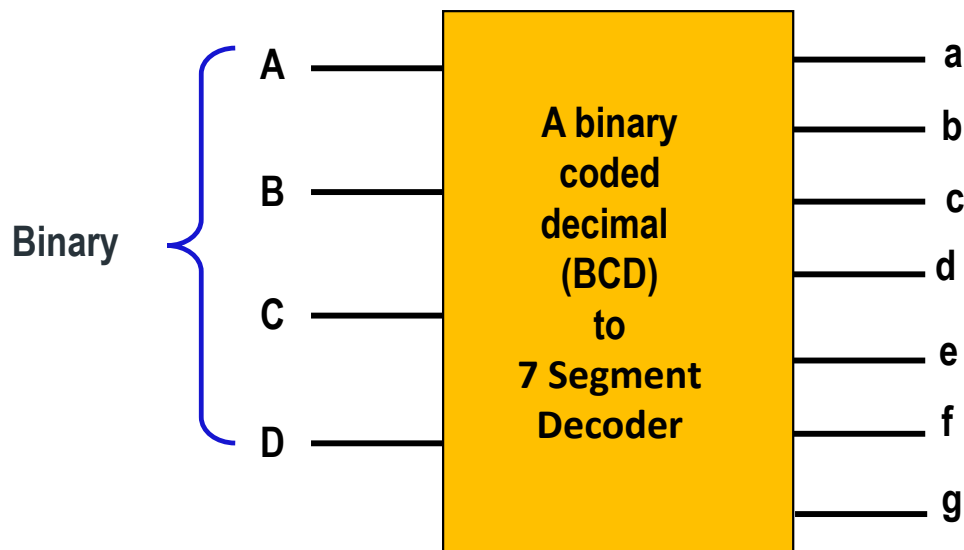
❑ With don't care
 $F = A + B \cdot C \cdot D$





Exercise (1/15)

- In a 7-segment display, each of the seven segments is activated for various digits. For example, segment **a** is activated for the digits 0, 2, 3, 5, 6, 7, 8, and 9, as shown. Since each digit can be represented by a BCD code, **derive** an SOP expression for segment **a** using the variables ABCD and then minimize the expression using K-map.

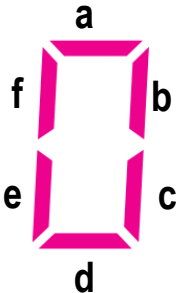




Exercise (2/15)

Truth Table

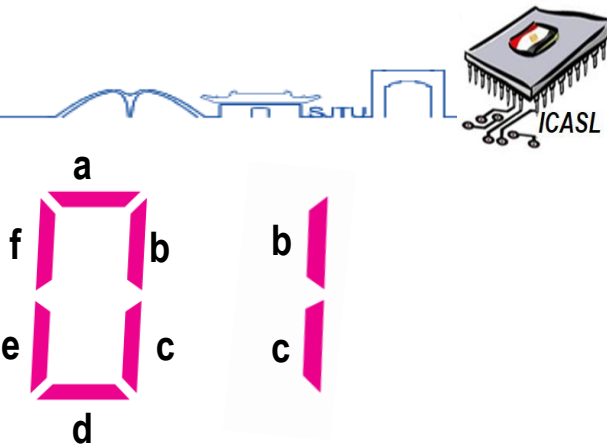
Input				Output						
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1							
0	0	1	0							
0	0	1	1							
0	1	0	0							
0	1	0	1							
0	1	1	0							
0	1	1	1							
1	0	0	0							
1	0	0	1							
1	0	1	0							
1	0	1	1							
1	1	0	0							
1	1	0	1							
1	1	1	0							
1	1	1	1							



Exercise (3/15)

Truth Table

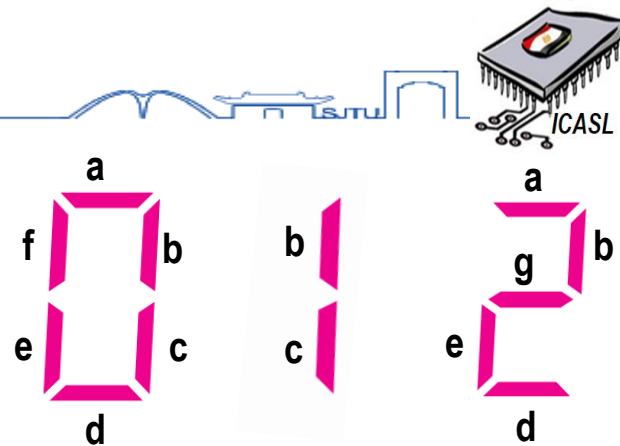
Input				Output						
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0							
0	0	1	1							
0	1	0	0							
0	1	0	1							
0	1	1	0							
0	1	1	1							
1	0	0	0							
1	0	0	1							
1	0	1	0							
1	0	1	1							
1	1	0	0							
1	1	0	1							
1	1	1	0							
1	1	1	1							



Exercise (4/15)

Truth Table

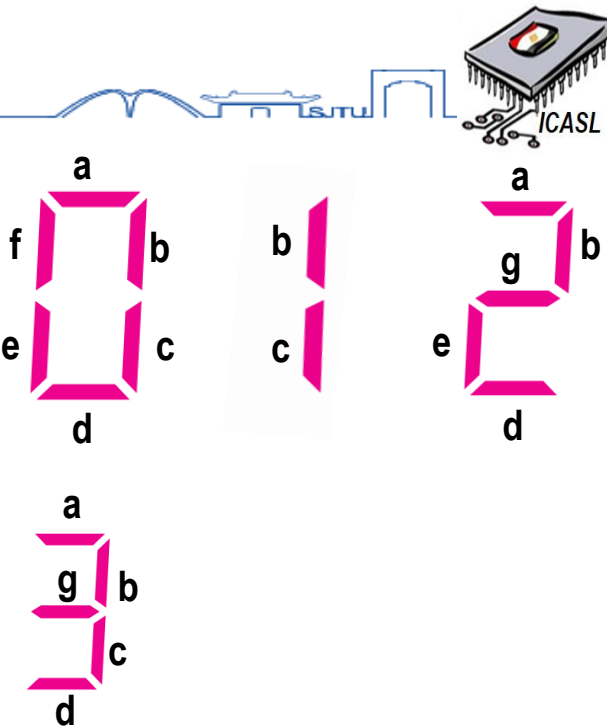
Input				Output						
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1							
0	1	0	0							
0	1	0	1							
0	1	1	0							
0	1	1	1							
1	0	0	0							
1	0	0	1							
1	0	1	0							
1	0	1	1							
1	1	0	0							
1	1	0	1							
1	1	1	0							
1	1	1	1							



Exercise (5/15)

Truth Table

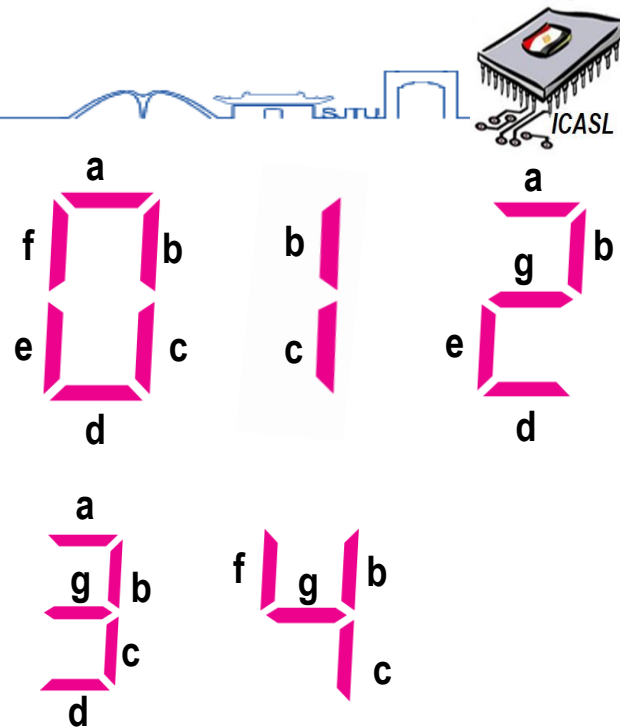
Input				Output						
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0							
0	1	0	1							
0	1	1	0							
0	1	1	1							
1	0	0	0							
1	0	0	1							
1	0	1	0							
1	0	1	1							
1	1	0	0							
1	1	0	1							
1	1	1	0							
1	1	1	1							



Exercise (6/15)

Truth Table

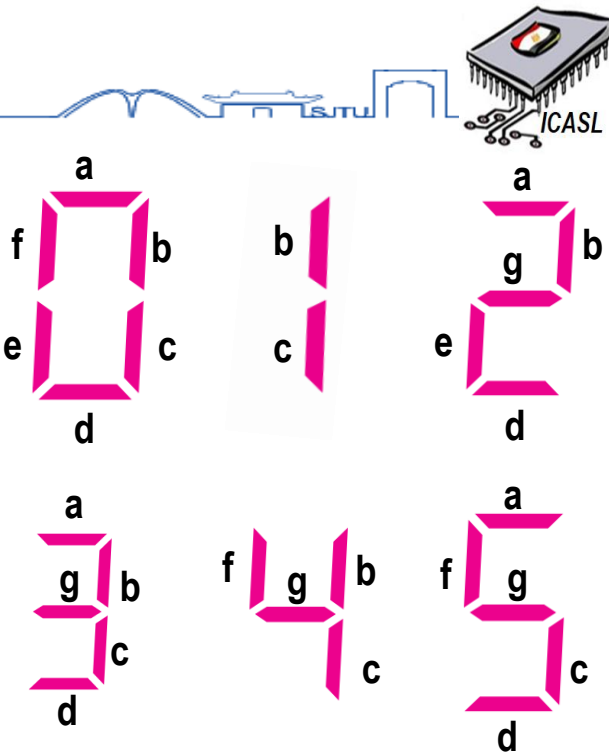
Input				Output						
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1							
0	1	1	0							
0	1	1	1							
1	0	0	0							
1	0	0	1							
1	0	1	0							
1	0	1	1							
1	1	0	0							
1	1	0	1							
1	1	1	0							
1	1	1	1							



Exercise (7/15)

Truth Table

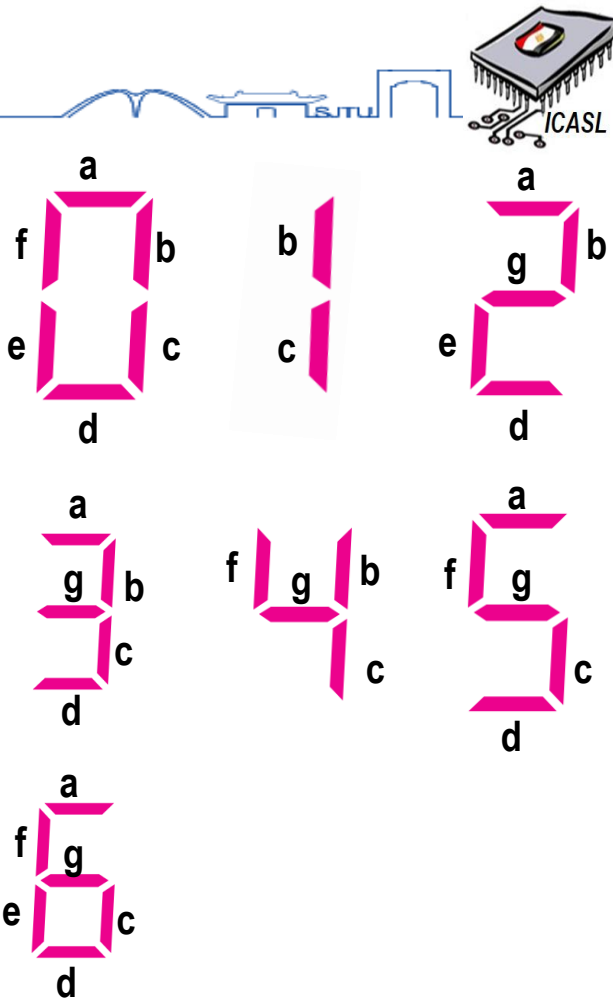
Input				Output						
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0							
0	1	1	1							
1	0	0	0							
1	0	0	1							
1	0	1	0							
1	0	1	1							
1	1	0	0							
1	1	0	1							
1	1	1	0							
1	1	1	1							



Exercise (8/15)

Truth Table

Input				Output						
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1							
1	0	0	0							
1	0	0	1							
1	0	1	0							
1	0	1	1							
1	1	0	0							
1	1	0	1							
1	1	1	0							
1	1	1	1							

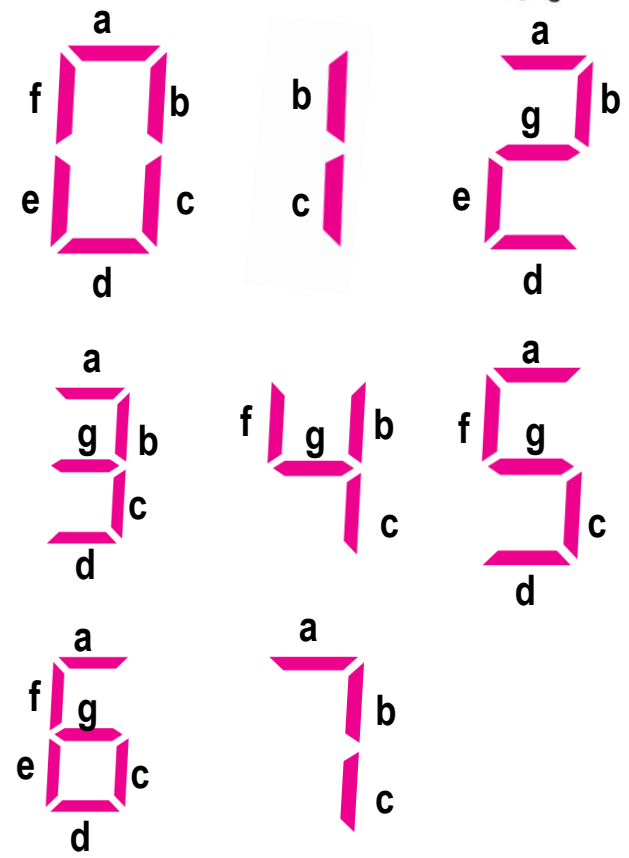




Exercise (9/15)

Truth Table

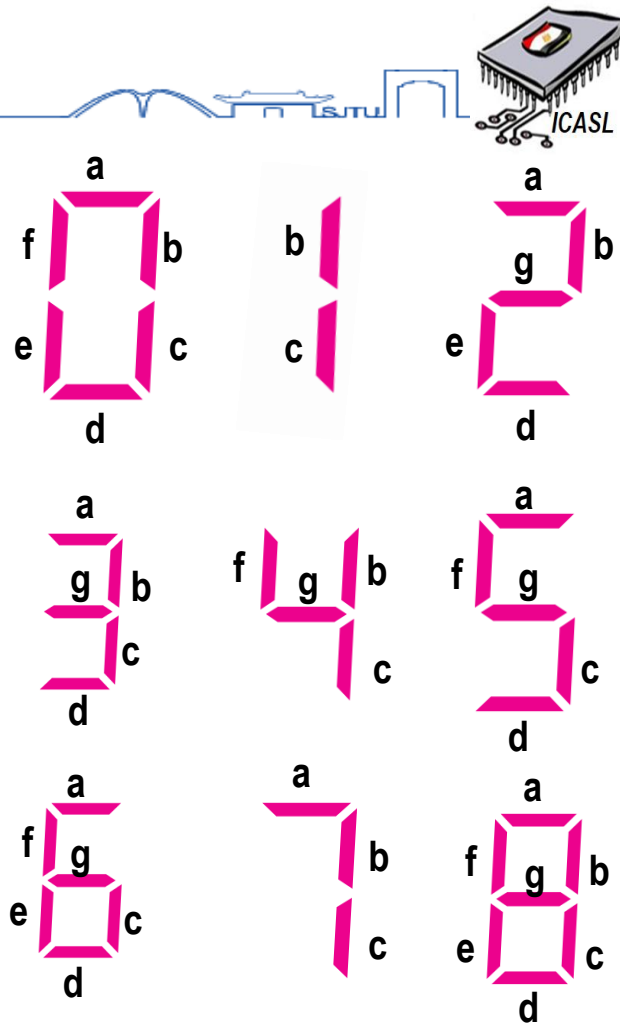
Input				Output						
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0							
1	0	0	1							
1	0	1	0							
1	0	1	1							
1	1	0	0							
1	1	0	1							
1	1	1	0							
1	1	1	1							



Exercise (10/15)

Truth Table

Input				Output						
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1							
1	0	1	0							
1	0	1	1							
1	1	0	0							
1	1	0	1							
1	1	1	0							
1	1	1	1							

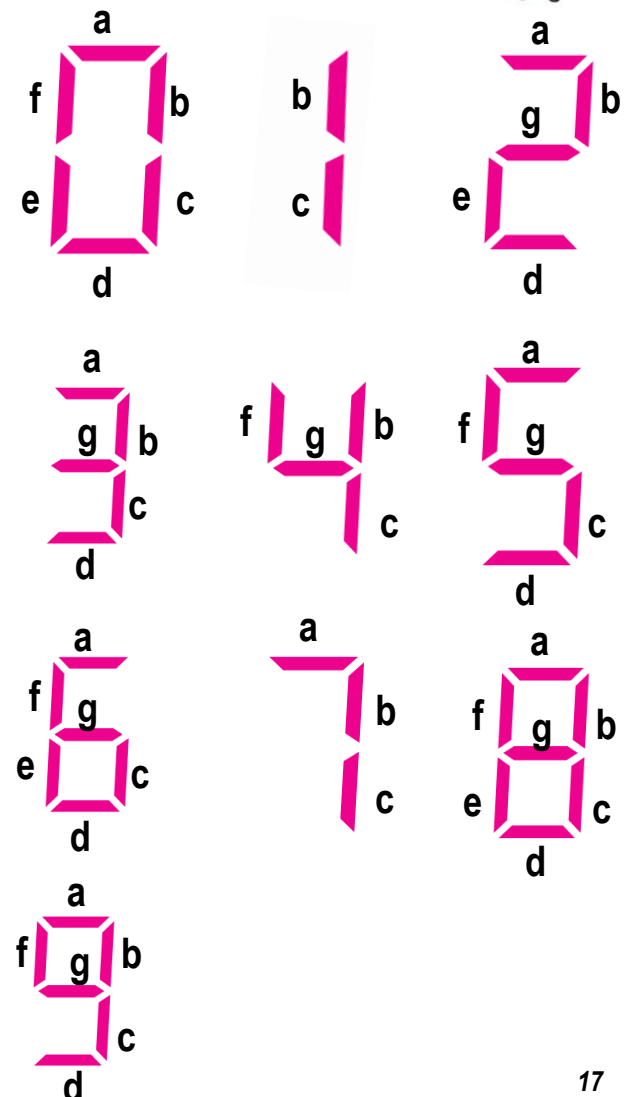




Exercise (11/15)

Truth Table

Input				Output						
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0							
1	0	1	1							
1	1	0	0							
1	1	0	1							
1	1	1	0							
1	1	1	1							

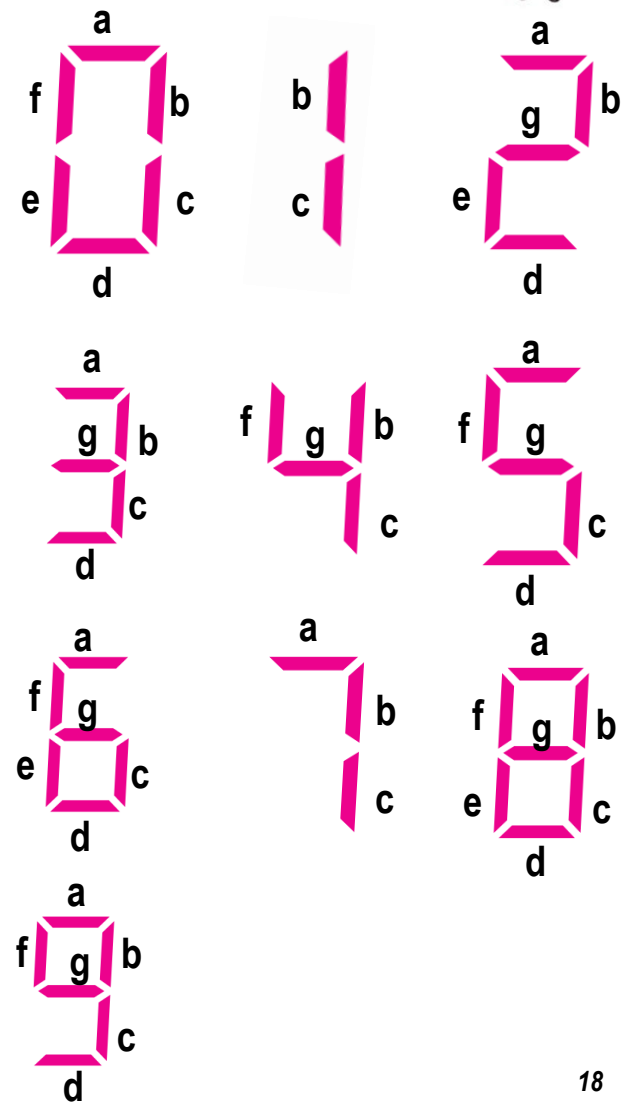


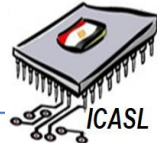


Exercise (12/15)

Truth Table

Input				Output						
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	x	x	x	x	x	x	x
1	0	1	1	x	x	x	x	x	x	x
1	1	0	0	x	x	x	x	x	x	x
1	1	0	1	x	x	x	x	x	x	x
1	1	1	0	x	x	x	x	x	x	x
1	1	1	1	x	x	x	x	x	x	x





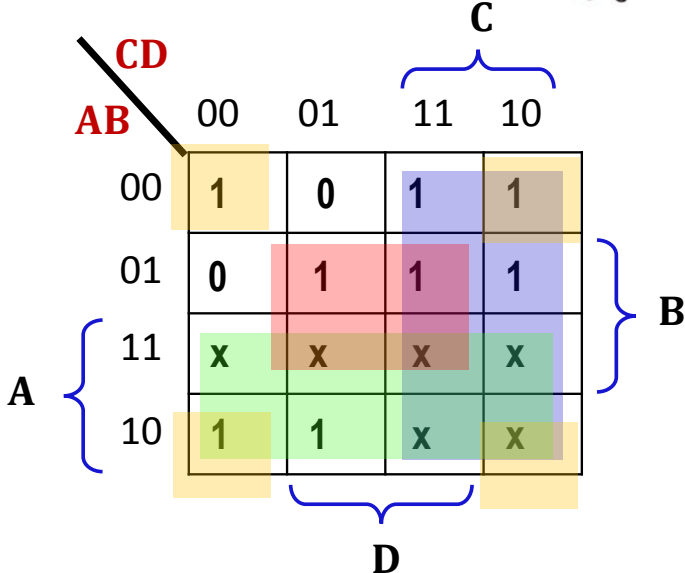
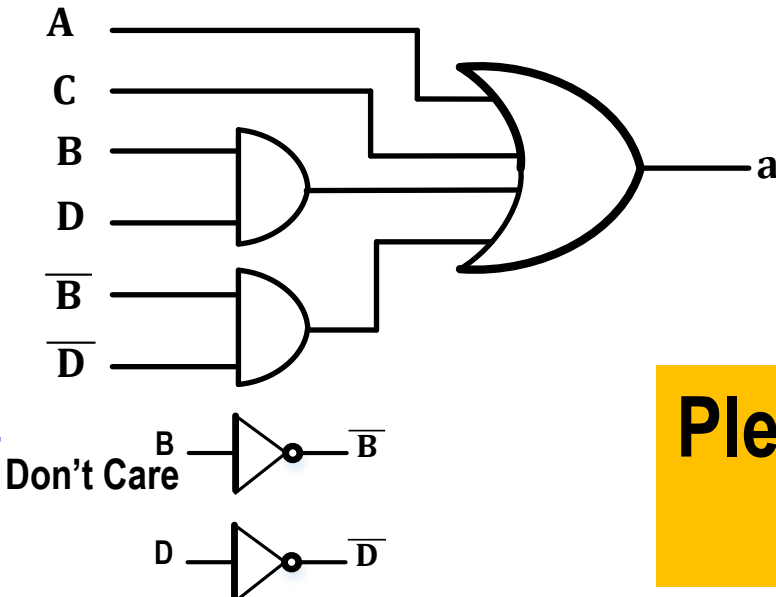
Exercise (13/15)

Truth Table

Input				Output
A	B	C	D	a
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	x
1	0	1	1	x
1	1	0	0	x
1	1	0	1	x
1	1	1	0	x
1	1	1	1	x

□ With don't care

$$a = A + C + B.D + \overline{B}. \overline{D}$$

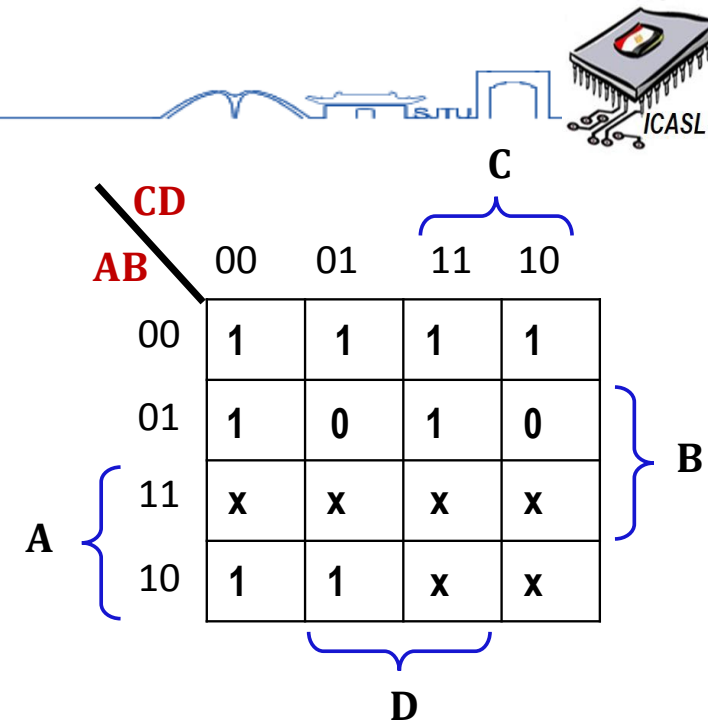


Please, complete for
b, c, d, e, f, g

Exercise (14/15)

Truth Table

Input				Output	
A	B	C	D	$\checkmark$ a	b
0	0	0	0	1	1
0	0	0	1	0	1
0	0	1	0	1	1
0	0	1	1	1	1
0	1	0	0	0	1
0	1	0	1	1	0
0	1	1	0	1	0
0	1	1	1	1	1
1	0	0	0	1	1
1	0	0	1	1	1
1	0	1	0	x	x
1	0	1	1	x	x
1	1	0	0	x	x
1	1	0	1	x	x
1	1	1	0	x	x
1	1	1	1	x	x



□ With don't care

$$b = \overline{B} + \overline{C} \cdot \overline{D} + C \cdot D$$

Please, complete for
b, c, d, e, f, g

Exercise (15/15)

Truth Table

□ With don't care

$$a = A + C + B.D + \overline{B}. \overline{D}$$

$$b = \overline{B} + \overline{C}. \overline{D} + C.D$$

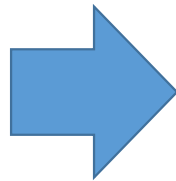
$$c = B + \overline{C} + D$$

$$d = \overline{B}. \overline{D} + C. \overline{D} + B. \overline{C}. D + A$$

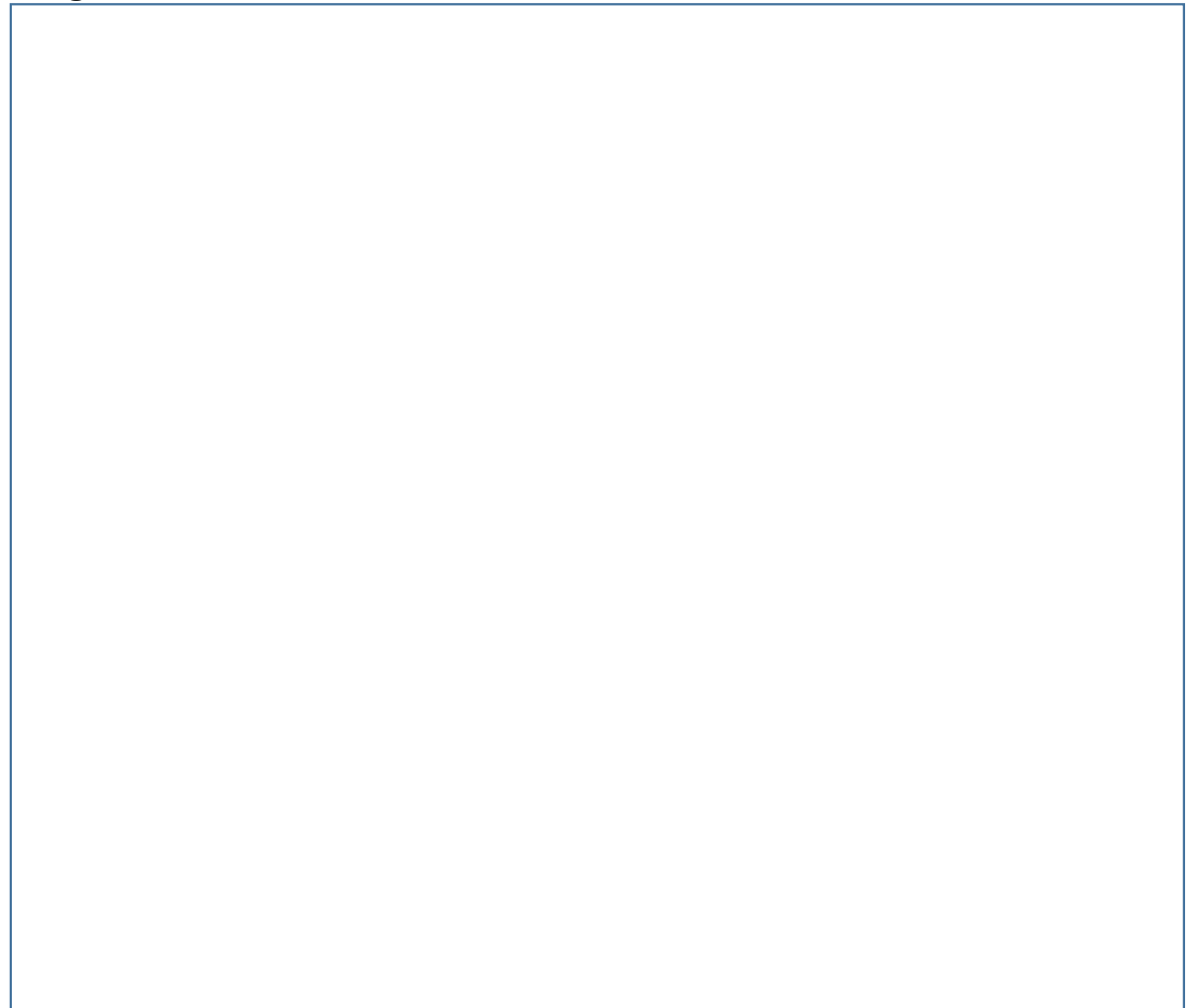
$$e = \overline{B}. \overline{D} + C. \overline{D}$$

$$f = A + \overline{C}. \overline{D} + B. \overline{C} + B. \overline{D}$$

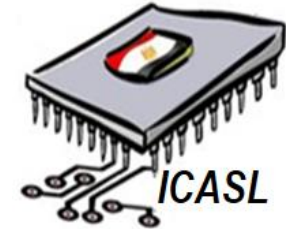
$$g = A + B. \overline{C} + \overline{B}. C + C. \overline{D}$$



Logic Circuit



Thanks!



Digital Design (SET 111)

L10

Functions of Combinational Logic

Dr. Ahmed Reda Mohamed¹

¹PhD, the MoE Key Laboratory of Artificial Intelligence & Bio-Circuits and Systems Laboratory (**BiCASL**), Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.

Outlines



- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☐ Binary Subtractor
- ☐ Parallel Binary Adder/Subtractor
- ☐ Application
- ☐ Decoder
- ☐ Encoder
- ☐ Application
- ☐ Multiplexer
- ☐ De-multiplexer

Outlines

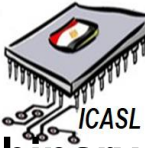


- ☐ **Half and Full Adders**
- ☐ **Parallel Binary Adders**
- ☐ **Binary Subtractor**
- ☐ **Parallel Binary Adder/Subtractor**
- ☐ **Application**
- ☐ **Decoder**
- ☐ **Encoder**
- ☐ **Application**
- ☐ **Multiplexer**
- ☐ **De-multiplexer**

Half and Full Adders

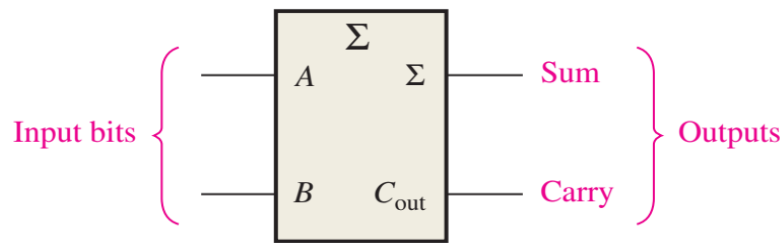


- ☐ **Adders** are important in computers and also in other types of **digital** systems.
- ☐ An **understanding** of the basic adder operation is fundamental to the study of digital systems.
- ☐ After completing this section, you should be able to
 - ☐ **Describe** the function of a **half-adder**
 - ☐ **Draw** a **half-adder** logic diagram
 - ☐ Describe the function of **the full-adder**
 - ☐ Draw a **full-adder** logic diagram using **half-adders**
 - ☐ Implement a **full-adder** using AND-OR logic



Half Adders

- The **half-adder** accepts **two** binary digits on its inputs and produces **two** binary digits on its outputs → a sum bit and a carry bit.



Logic symbol

Truth Table

A	B	C_{out}	Σ
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Σ = sum

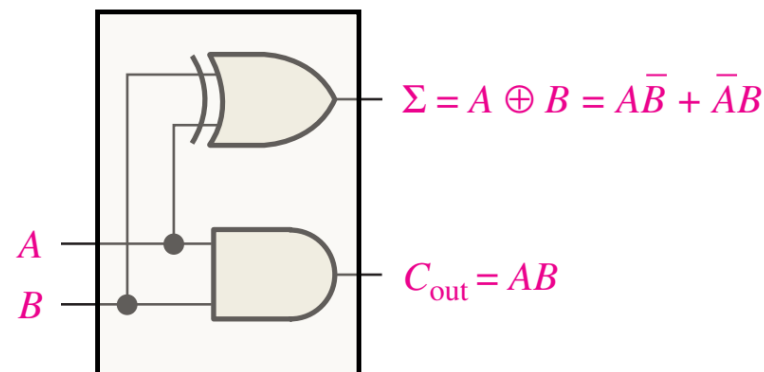
C_{out} = output carry

		B	
		0	1
A	0	0	1
	1	1	0

$$\text{Sum} = \overline{A} \cdot B + A \cdot \overline{B} = A \oplus B$$

		B	
		0	1
A	0	0	0
	1	0	1

$$C_{out} = A \cdot B$$

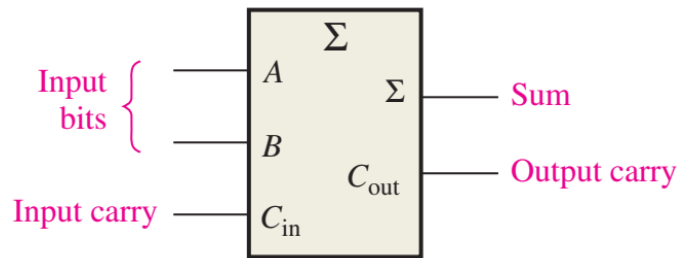


Half-adder logic diagram



Full Adders

- ❑ The **full-adder** accepts three bits (**two** input bits and an **input carry**) and generates a **sum** output and an output carry. The basic difference between a **full-adder** and a **half-adder** is that the full-adder accepts an input carry.



Logic symbol

Truth Table

A	B	C _{in}	C _{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$\begin{aligned}
 \text{Sum} &= A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot \overline{B} \cdot C + A \cdot B \cdot C + \overline{A} \cdot B \cdot \overline{C} \\
 &= A \cdot (\overline{B} \cdot \overline{C} + B \cdot C) + \overline{A} \cdot (\overline{B} \cdot C + B \cdot \overline{C}) \\
 &= A \cdot (\overline{B} \oplus \overline{C}) + \overline{A} \cdot (B \oplus C) \\
 &= A \oplus B \oplus C
 \end{aligned}$$

$$C_{out} = A \cdot C + A \cdot B + B \cdot C$$

$$C_{out} = A \cdot \overline{B} \cdot C + A \cdot B \cdot C + \overline{A} \cdot B \cdot C + A \cdot B \cdot \overline{C} = A \cdot B + (A \oplus B) \cdot C_{in}$$

Note:

XOR Gate

$$A \oplus B = \overline{A} \cdot B + A \cdot \overline{B}$$

XNOR Gate

$$A \odot B = \overline{A} \cdot \overline{B} + A \cdot B$$

$$A \odot B = \overline{A \oplus B}$$

$$\text{Sum} = A \oplus B \oplus C_{in}$$

$$C_{out} = A \cdot B + A \cdot C_{in} + B \cdot C_{in}$$

$$= A \cdot B + (A \oplus B) \cdot C_{in}$$

Full Adders

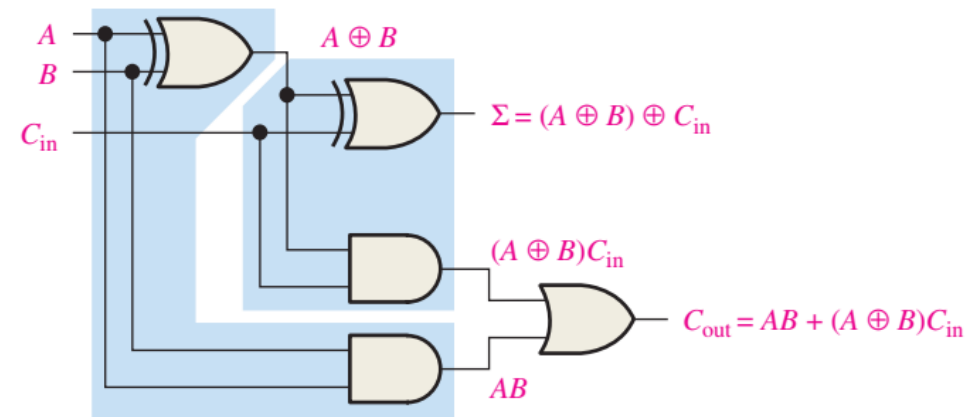
- Full adder implementation using (a) logic circuit and (b) two- half adders

Full adder

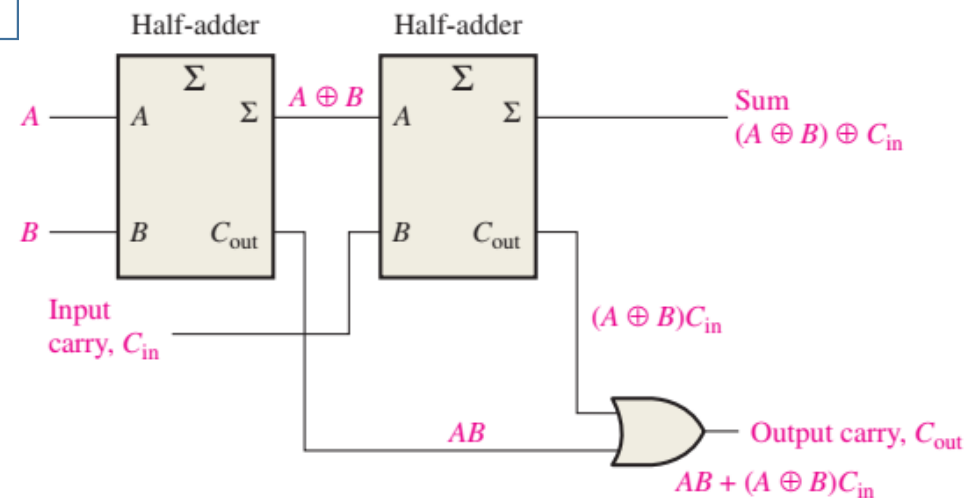
$$\text{Sum} = A \oplus B \oplus C_{in}$$

$$C_{out} = A.B + A.C_{in} + B.C_{in}$$

$$= A.B + (A \oplus B).C_{in}$$



(a) Complete logic circuit for a full-addder



(b) Two half-adders to form a full-addder

Outlines

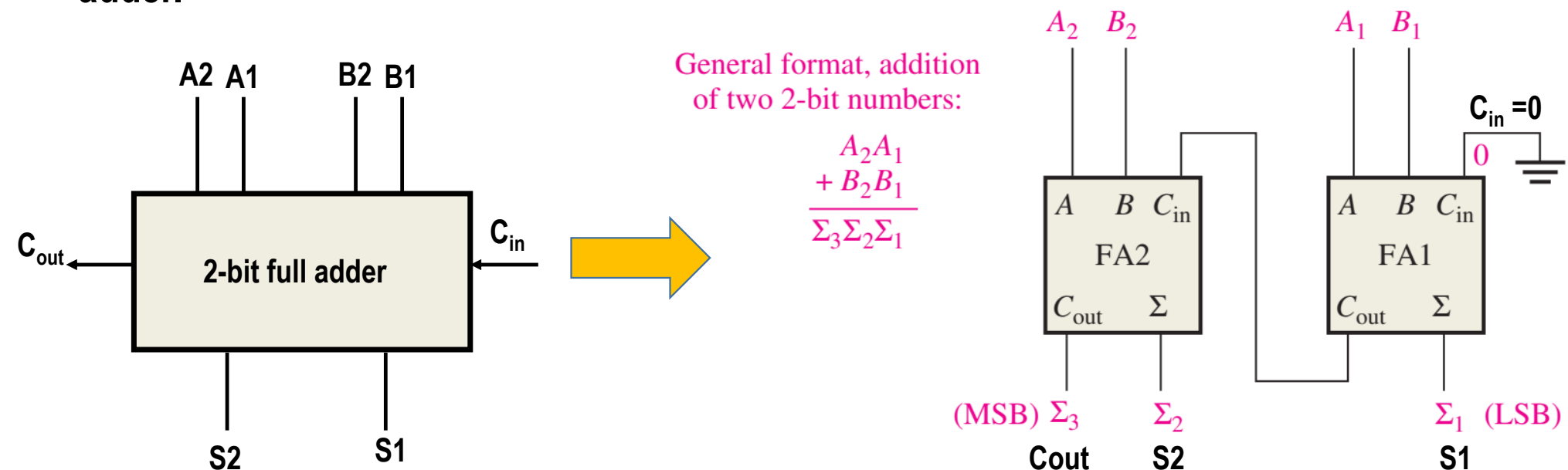


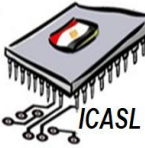
- ☐ Half and Full Adders
- ☒ **Parallel Binary Adders**
- ☐ Binary Subtractor
- ☐ Parallel Binary Adder/Subtractor
- ☐ Comparator
- ☐ Decoder
- ☐ Encoder
- ☐ Multiplexer
- ☐ De-multiplexer



Parallel Binary Adders (1/3)

- ❑ A single **full-adder** is capable of adding **two** 1-bit numbers and an input carry. To add binary numbers with more than one bit, you must use **additional** full-adders.
- ❑ To add two binary numbers, a full-adder (FA) is required for **each bit** in the numbers. So for 2-bit numbers, two adders are needed.
- ❑ The carry output of each adder is connected to the carry input of the **next** higher-order adder.

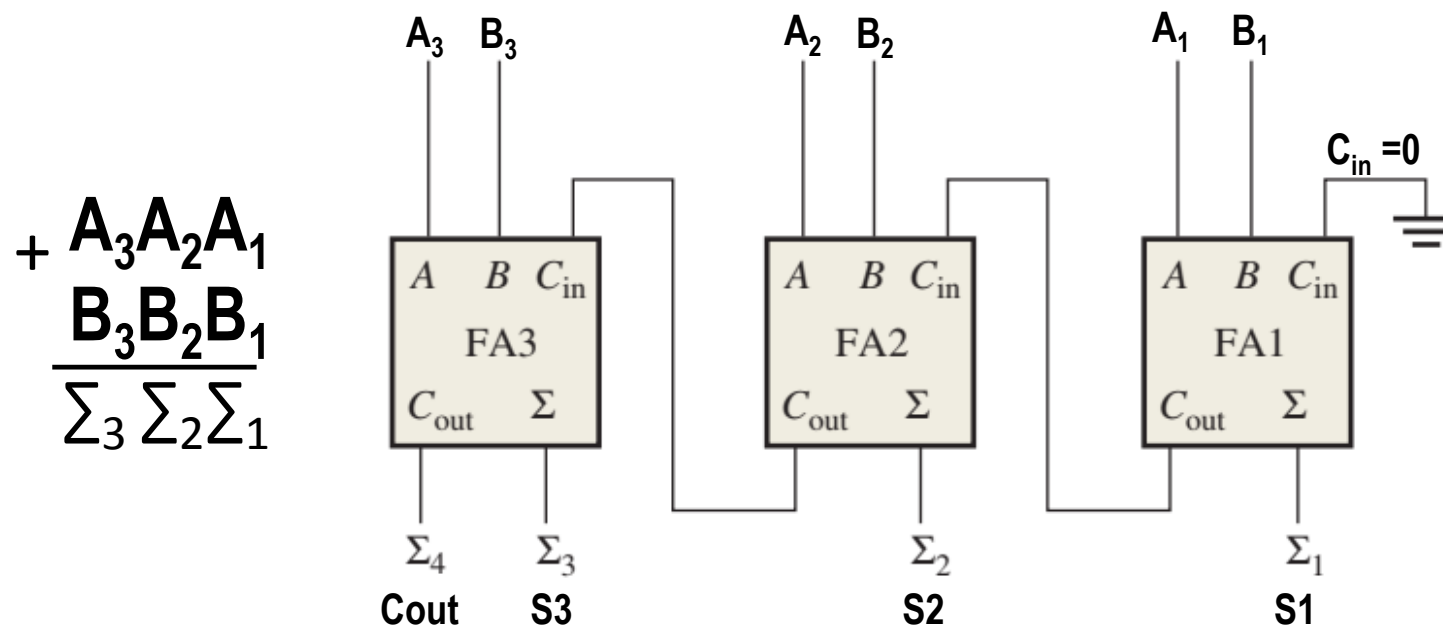




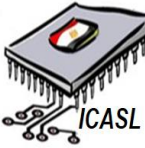
Parallel Binary Adders (1/3)

- ☐ A) Design a 3-bit parallel adder
- ☐ B) show the intermediate carries when the binary numbers 101 and 011 are being added

Ans.



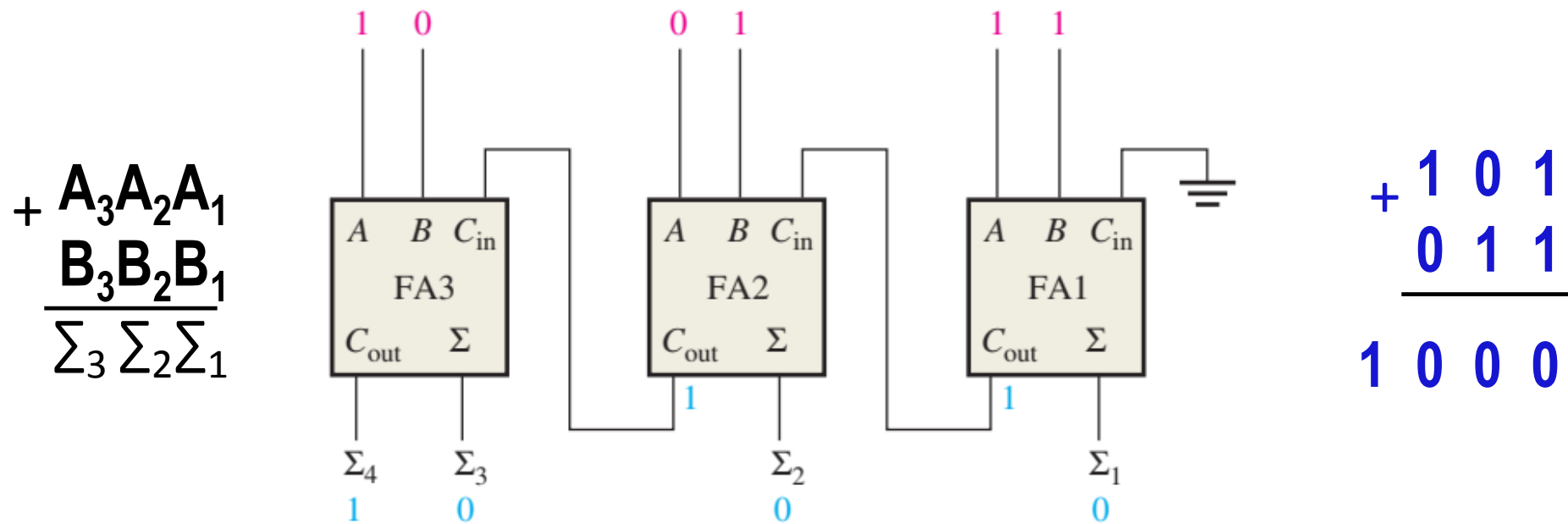
Block diagram of a basic 3-bit parallel adder using two full-adders



Parallel Binary Adders (3/3)

- ☐ A) Design a 3-bit parallel adder
- ☐ B) show the intermediate carries when the binary numbers 101 and 011 are being added

Ans.

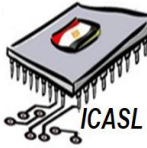


Block diagram of a basic 3-bit parallel adder using two full-adders

Outlines



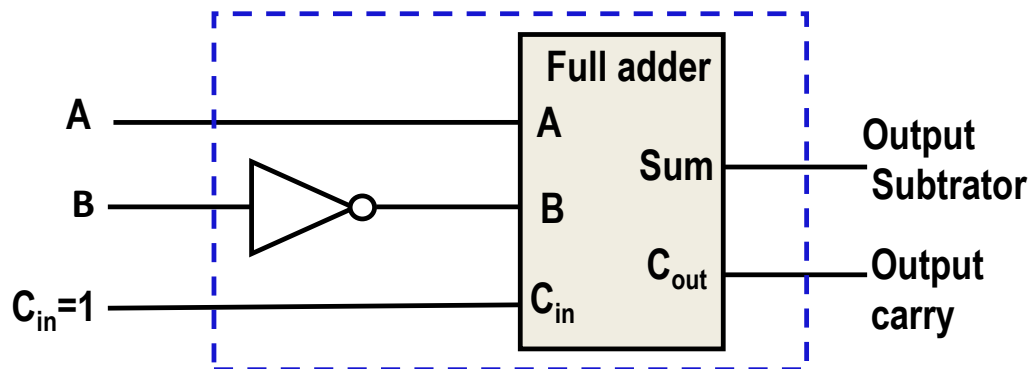
- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☒ **Binary Subtractor**
- ☐ Parallel Binary Adder/Subtractor
- ☐ Comparator
- ☐ Decoder
- ☐ Encoder
- ☐ Multiplexer
- ☐ De-multiplexer



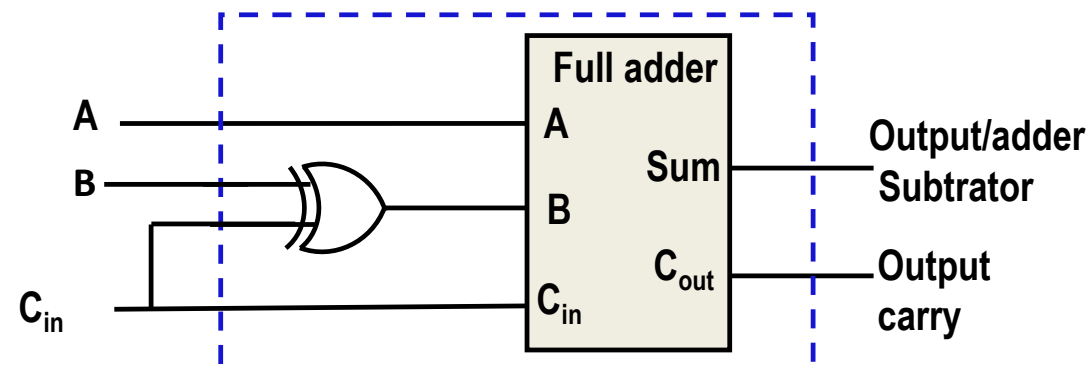
Binary Subtractor

- ❑ The subtraction of binary numbers can be easily done using 2' complements.
- ❑ The subtraction $A - B$ is done by taking the **2's complement of B** and **adding** it to A .
- ❑ The 2's complement can be obtained by taking the 1's complement and adding 1 to the least significant bit (LSB).

Subtrator



Adder/Subtrator



If $C_{in} = 0$, the circuit acts as an Adder

If $C_{in} = 1$, the circuit acts as a Subtractor

Remember that $B \oplus 0 = B$
and $B \oplus 1 = B'$

Outlines

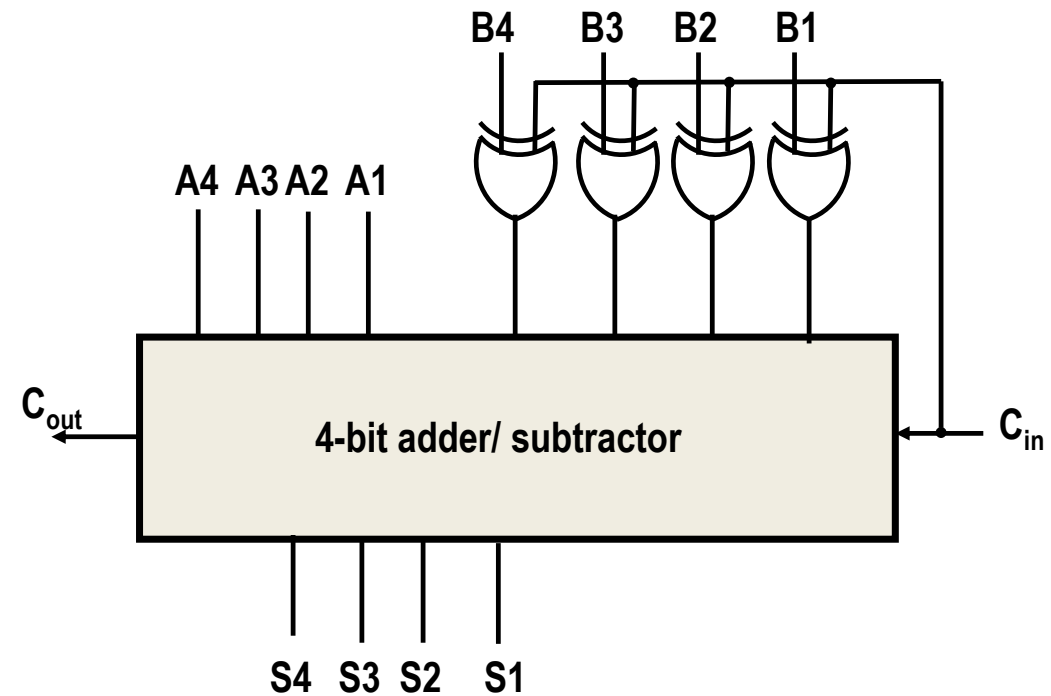
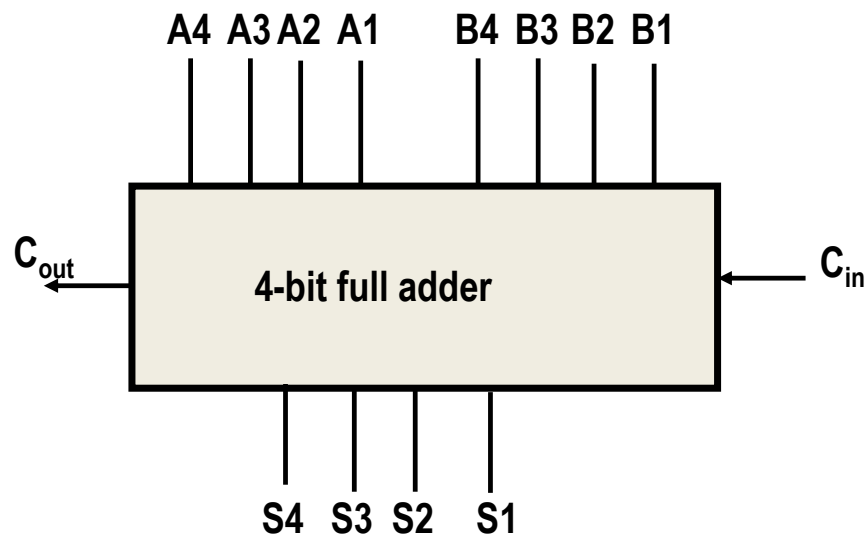


- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☐ Binary Subtractor
- ☒ **Parallel Binary Adder/Subtractor**
- ☐ Comparator
- ☐ Decoder
- ☐ Encoder
- ☐ Multiplexer
- ☐ De-multiplexer

Parallel Binary Adder/Subtractor



- Design a 4-bit parallel adder/Subtractor



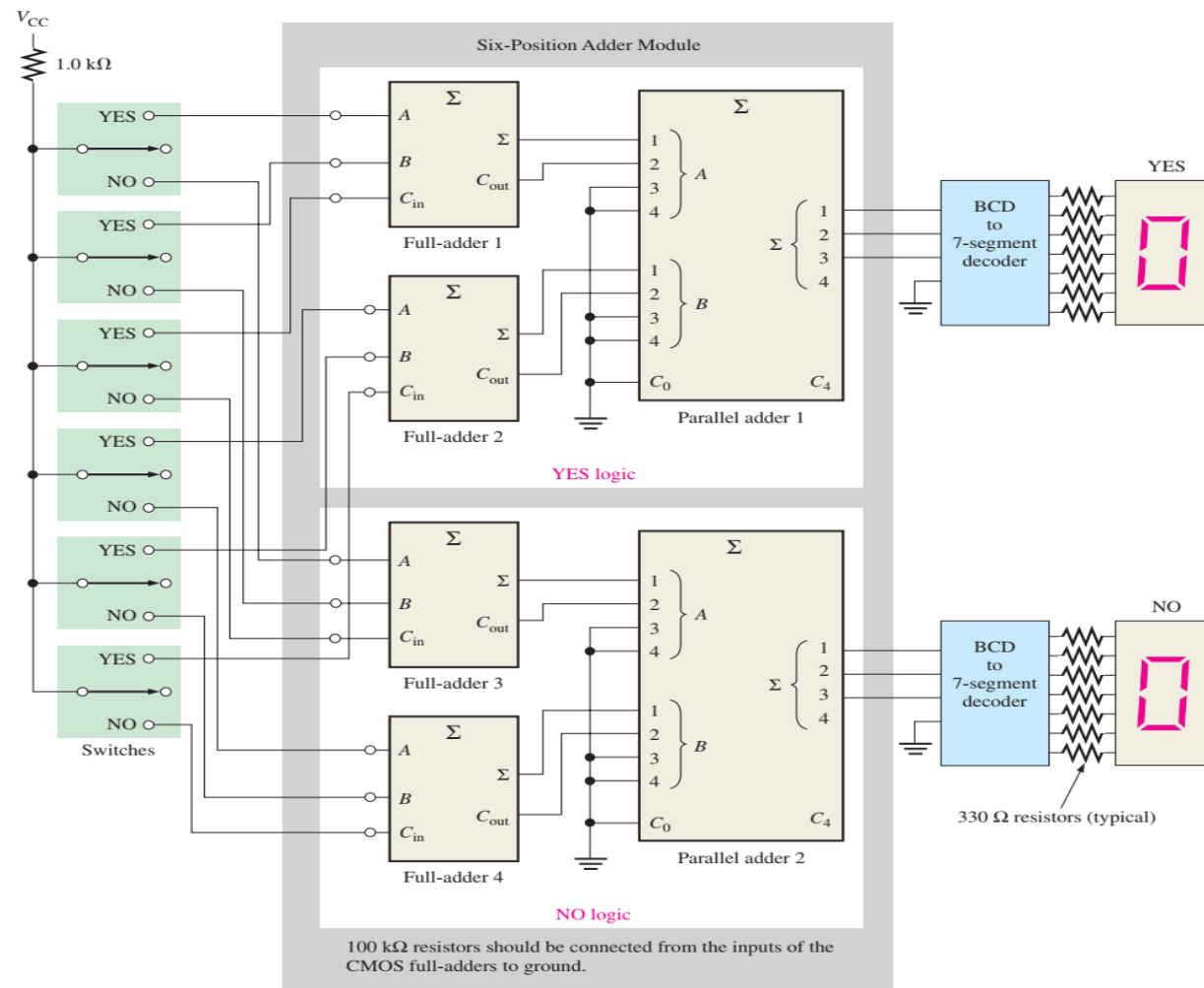
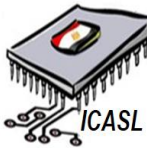
If $C_{in} = 0$, the circuit acts as an Adder $\rightarrow S = A + B$
If $C_{in} = 1$, the circuit acts as a Subtractor $\rightarrow S = A + (-B)$

Application (1/2)



- ❑ An example of **full-adder and parallel adder** application is a simple **voting** system that can be used to simultaneously provide the number of “**yes**” votes and the number of “**no**” votes.
- ❑ This type of system can be used where a group of people are **assembled** and there is a need for immediately determining opinions (for or against), making decisions, or voting on certain issues or other matters.
- ❑ In its **simplest form**, the system includes a switch for “**yes**” or “**no**” selection at each position in the assembly and a digital display for the number of yes votes and one for the number of no votes .
- ❑ The basic system is for a **6-position setup**, but it can be expanded to any number of positions with additional 6-position modules and additional parallel adder and display circuits.

Application (2/2)



A voting system using full-adders and parallel binary adders

Outlines

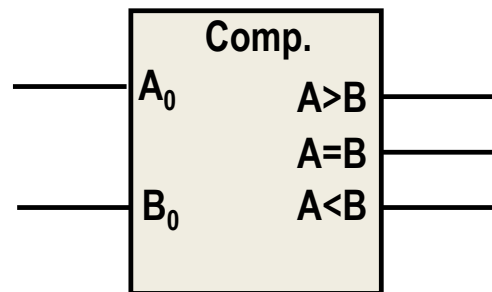


- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☐ Binary Subtractor
- ☐ Parallel Binary Adder/Subtractor
- ☐ **Comparator**
- ☐ Decoder
- ☐ Encoder
- ☐ Application
- ☐ Multiplexer
- ☐ De-multiplexer



Comparators (1/3)

- ❑ The basic function of a comparator is to compare the magnitudes of **two** binary quantities to determine the relationship of those quantities.
- ❑ Ex.1.Design a comparator circuit to compare **two** numbers {A,B} where each number consists of 1 bit.



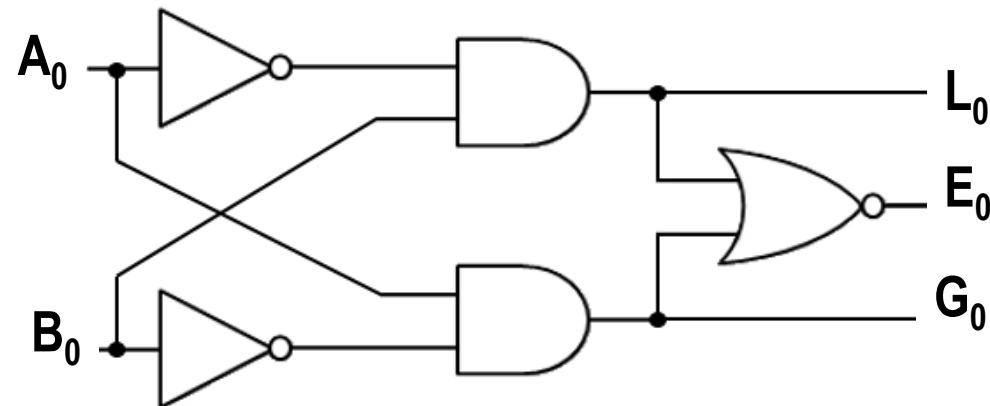
Logic symbol

Ans.

$$G_0 = A_0 \cdot \overline{B_0}$$

$$L_0 = \overline{A_0} \cdot B_0$$

$$E_0 = \overline{A_0} \cdot \overline{B_0} + A_0 \cdot B_0 = \overline{G_0 + L_0}$$



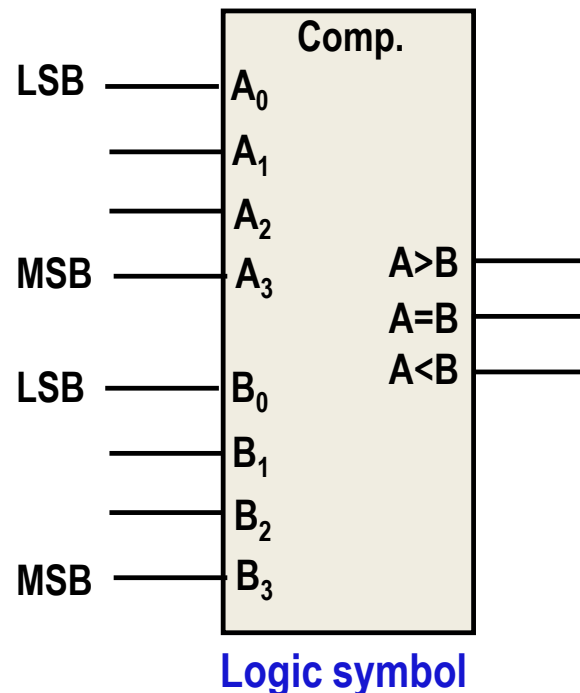
A ₀	B ₀	G ₀	L ₀	E ₀
		A ₀ > B ₀	A ₀ < B ₀	A ₀ = B ₀
0	0	0	0	1
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1



Comparators (2/3)

- Ex2. Design a comparator circuit to compare two numbers {A,B} where each number consists of 4 bit.

Ans.



Note

$$\begin{array}{l} A = 1011 \\ B = 0111 \end{array}$$



$A > B$



$$G = G_3 + E_3G_2 + E_3E_2G_1 + E_3E_2E_1G_0$$

$$\begin{array}{l} A = 1001 \\ B = 1010 \end{array}$$



$A < B$



$$L = L_3 + E_3L_2 + E_3E_2L_1 + E_3E_2E_1L_0$$

$$\begin{array}{l} A = 0101 \\ B = 0101 \end{array}$$



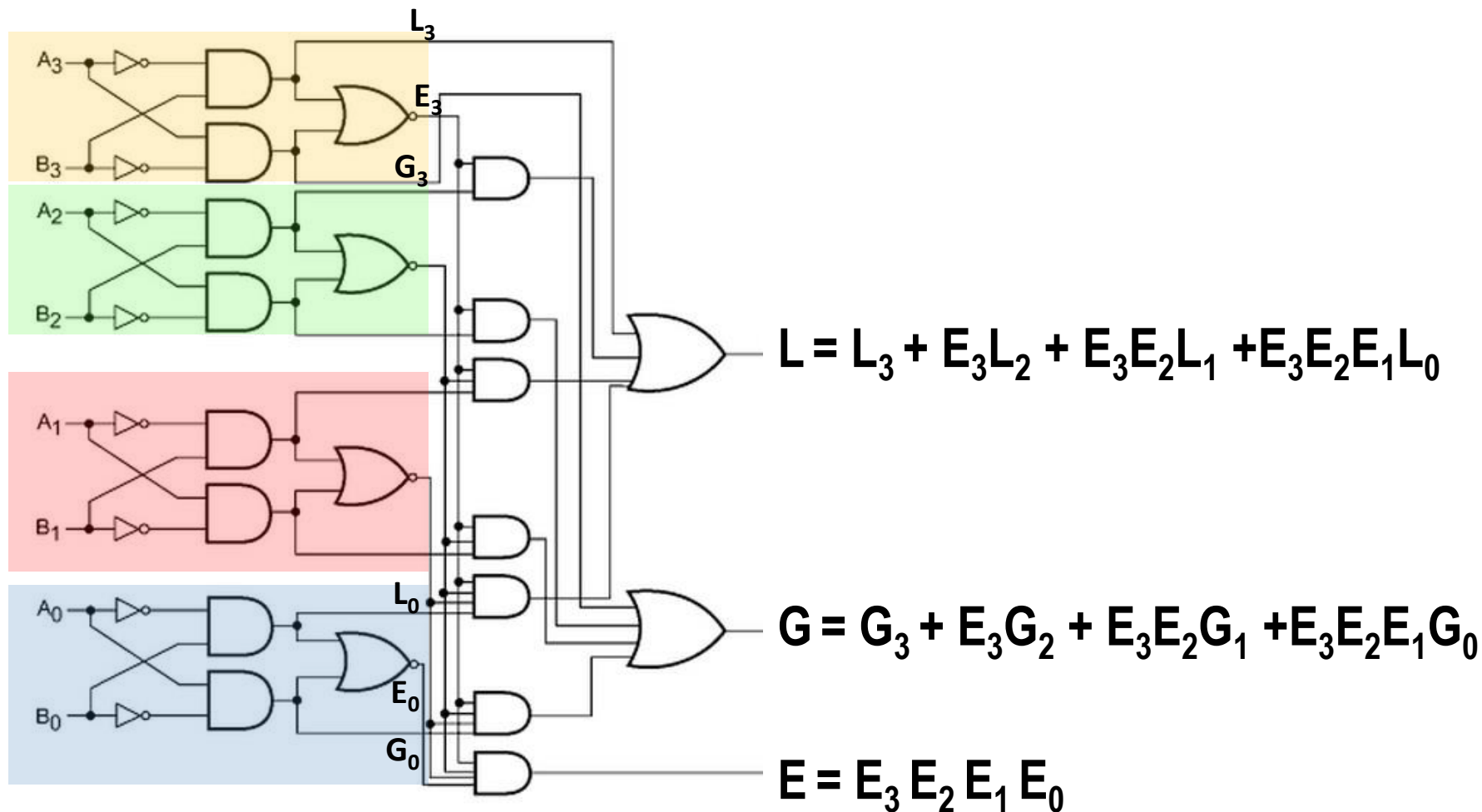
$A = B$



$$E = E_3E_2E_1E_0$$

Those two numbers would be equal if all pairs of their significant digits are equal: $A_3=B_3, A_2=B_2, A_1=B_1, A_0=B_0$

Comparators (3/3)



Outlines

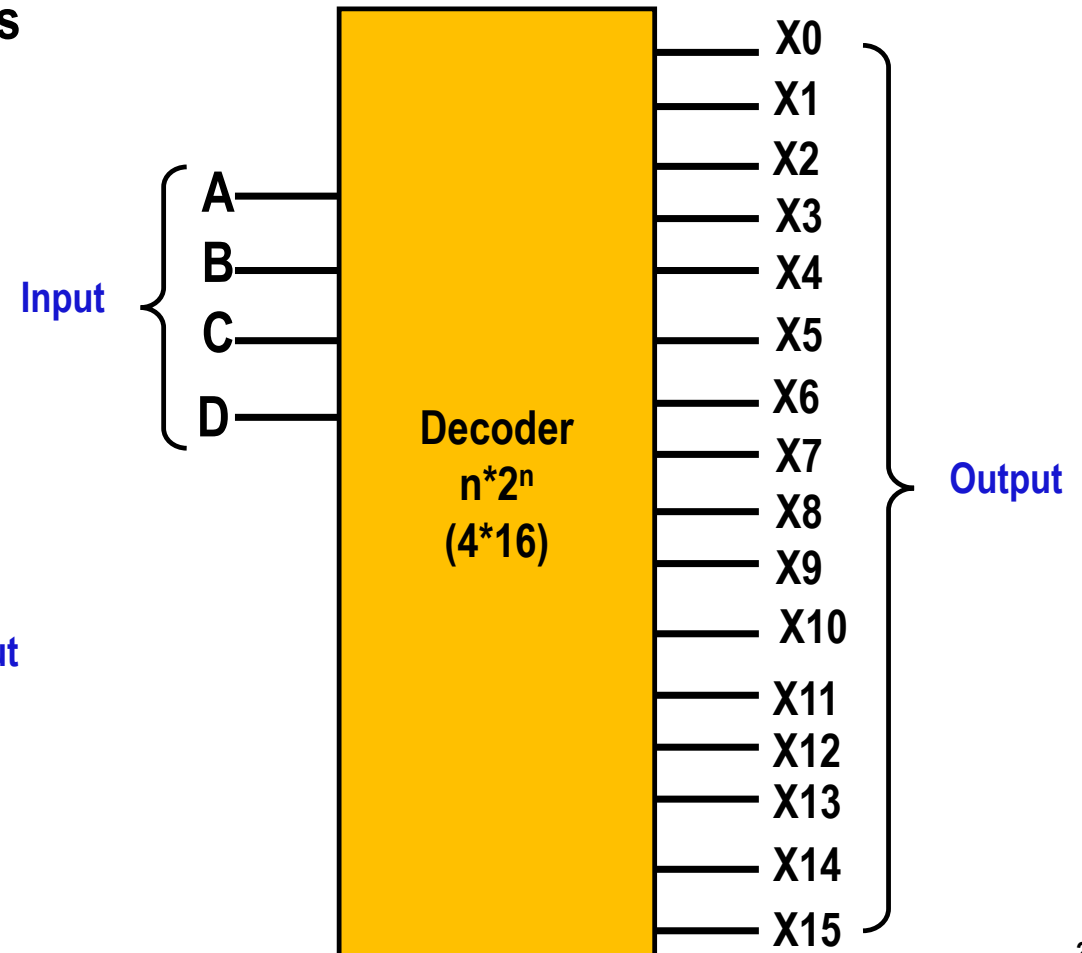
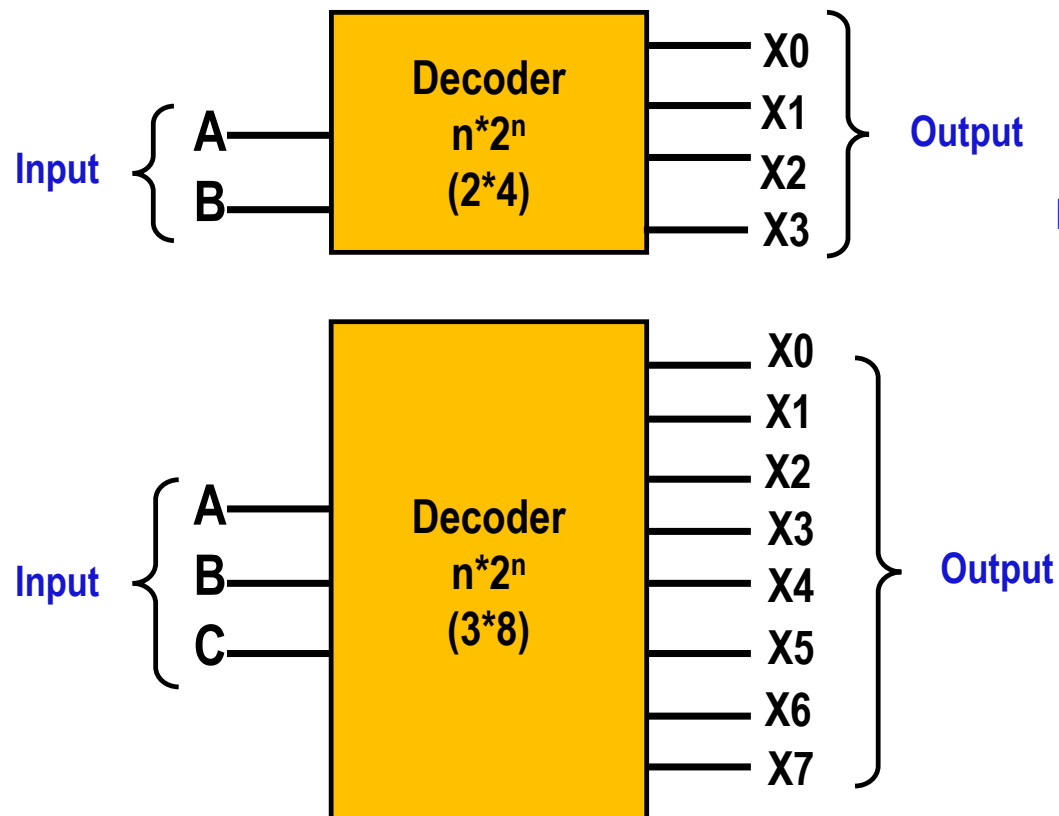


- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☐ Binary Subtractor
- ☐ Parallel Binary Adder/Subtractor
- ☐ Comparator
- ☐ **Decoder**
- ☐ Encoder
- ☐ Multiplexer
- ☐ De-multiplexer



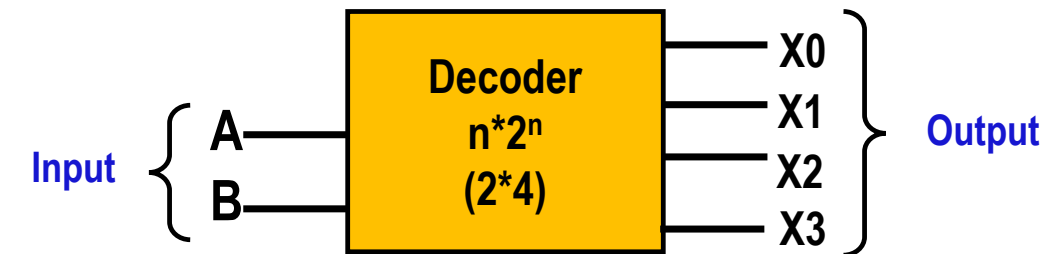
Decoder (1/5)

- ❑ A decoder is a digital circuit that **detects** the presence of a specified combination of bits (code) on its inputs and **indicates the presence of that code** by a specified output level.
- ❑ A decoder has **n** input lines form **2^n** output lines



Decoder (2/5)

□ Ex.1 Design a 2-bit decoder with active-high outputs



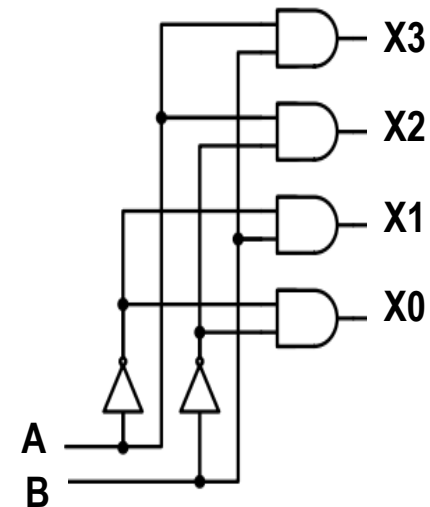
Input		Output			
A	B	X0	X1	X2	X3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

A	B		
	0	1	
0	1	0	X0 = $\overline{A} \cdot \overline{B}$
1	0	0	

A	B		
	0	1	
0	0	1	X1 = $\overline{A} \cdot B$
1	0	0	

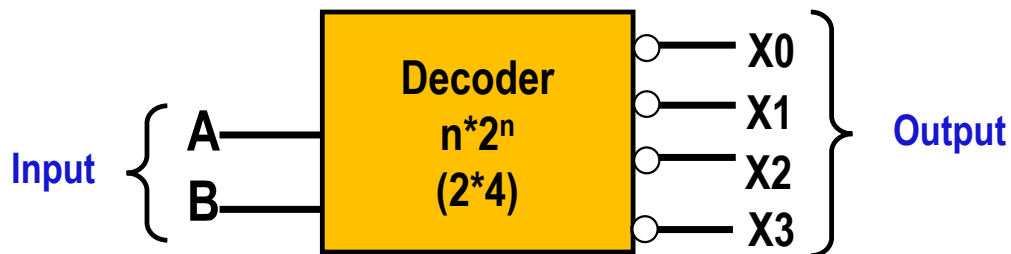
A	B		
	0	1	
0	0	0	
1	1	0	X2 = $A \cdot \overline{B}$

A	B		
	0	1	
0	0	0	
1	0	1	X3 = $A \cdot B$

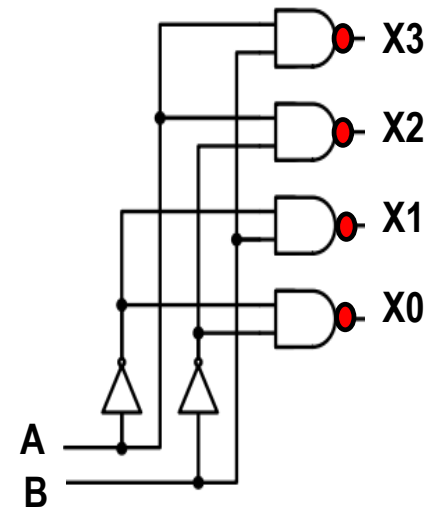


Decoder (3/5)

□ Ex.2 Design a 2-bit decoder with active-Low outputs



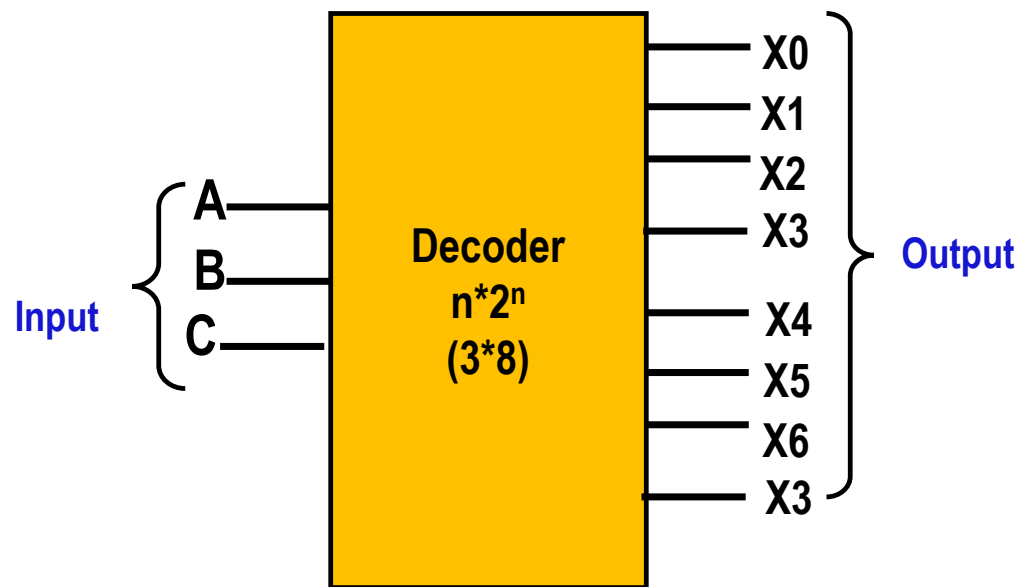
Input		Output			
A	B	X0	X1	X2	X3
0	0	0	1	1	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0



Decoder (4/5)



□ Ex.3 Design a 3-bit decoder with active-high outputs



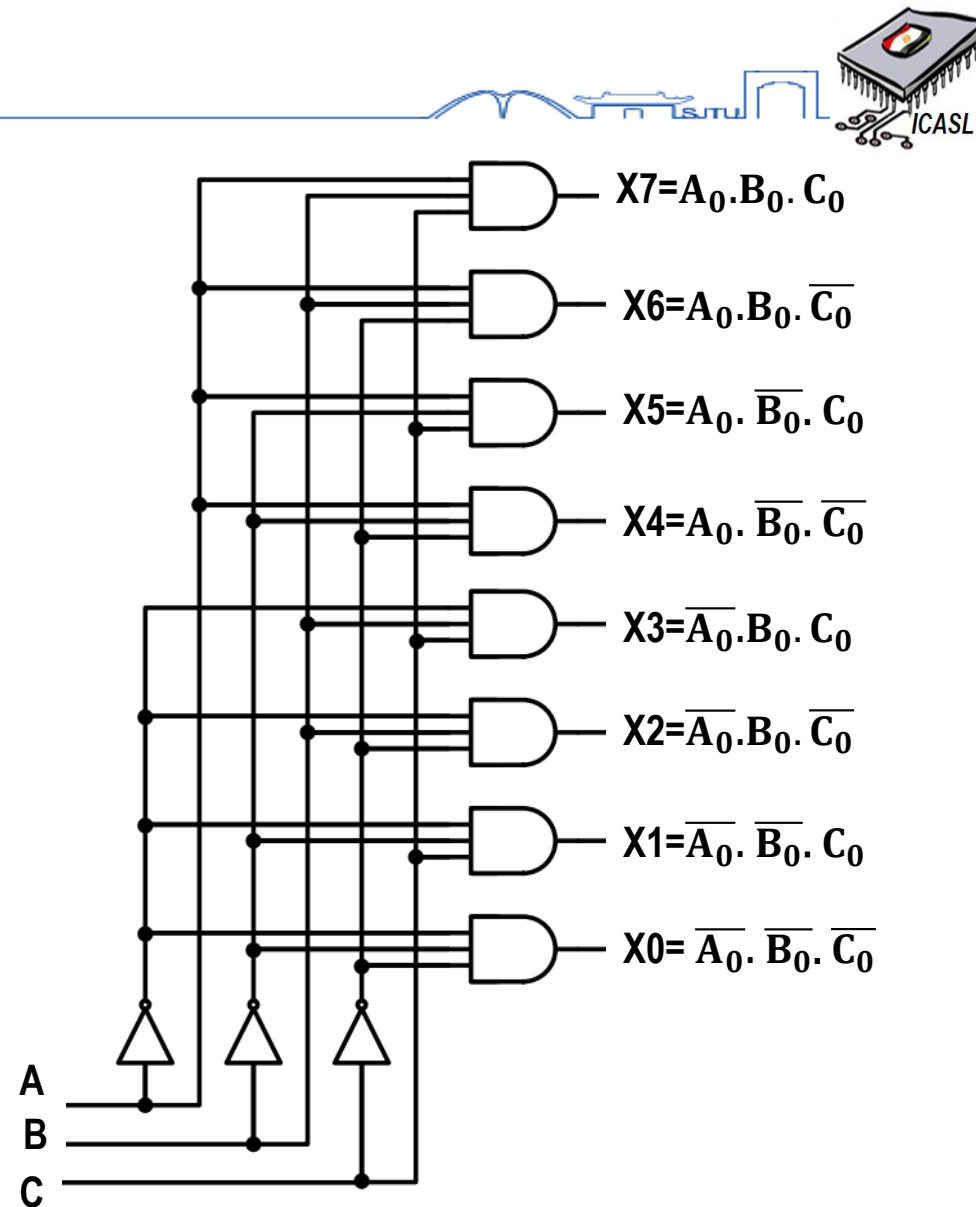
Input			Output							
A	B	C	X0	X1	X2	X3	X4	X5	X6	X7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

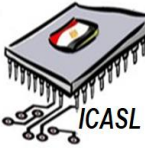
Decoder (5/5)

□ Ex.3 Design a 3-bit decoder with active-high outputs

Input			Output							
A	B	C	X0	X1	X2	X3	X4	X5	X6	X7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

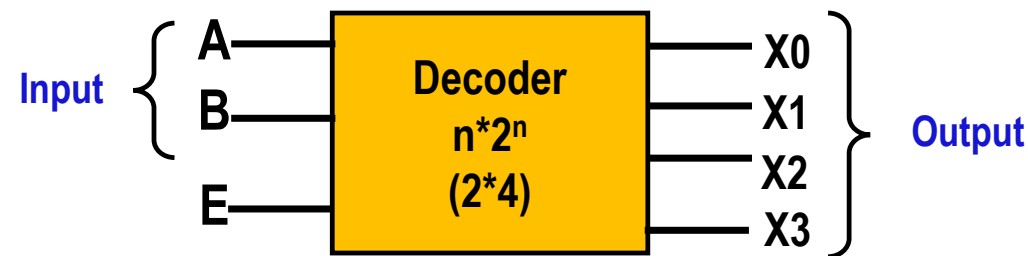
$$\begin{array}{lll}
 X0 = \overline{A_0} \cdot \overline{B_0} \cdot \overline{C_0} & X2 = \overline{A_0} \cdot B_0 \cdot \overline{C_0} & X4 = A_0 \cdot \overline{B_0} \cdot \overline{C_0} & X6 = A_0 \cdot B_0 \cdot \overline{C_0} \\
 X1 = \overline{A_0} \cdot \overline{B_0} \cdot C_0 & X3 = \overline{A_0} \cdot B_0 \cdot C_0 & X5 = A_0 \cdot \overline{B_0} \cdot C_0 & X7 = A_0 \cdot B_0 \cdot C_0
 \end{array}$$





Decoder Expander (1/2)

□ Ex.1 Design a 2-bit decoder with enable line and with active-high outputs



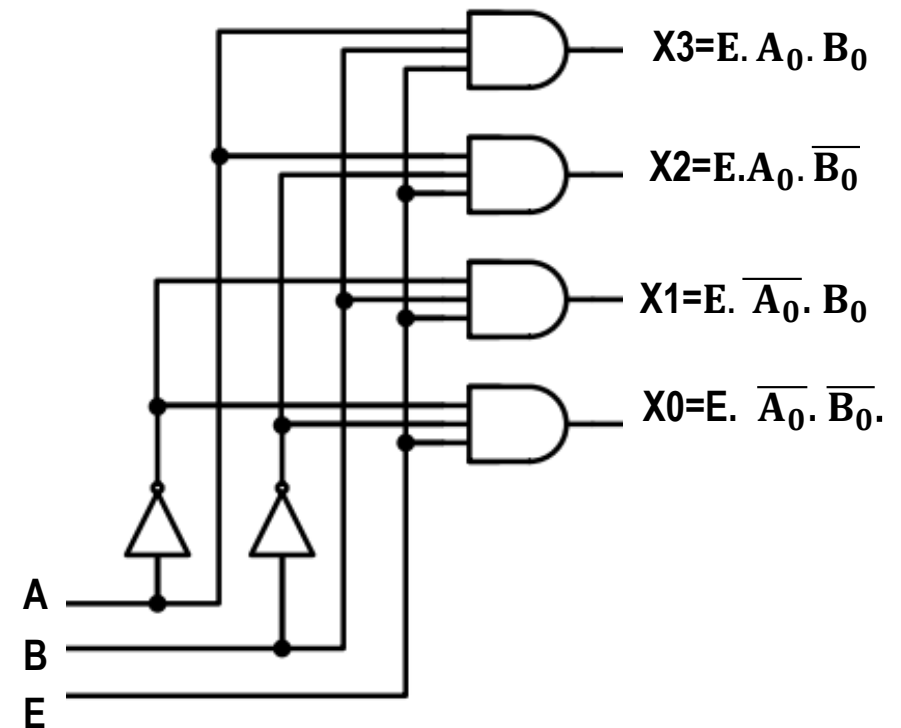
Input			Output			
E	A	B	X0	X1	X2	X3
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	X	X	0	0	0	0

$$X0 = E \cdot \overline{A_0} \cdot \overline{B_0}$$

$$X1 = E \cdot \overline{A_0} \cdot B_0$$

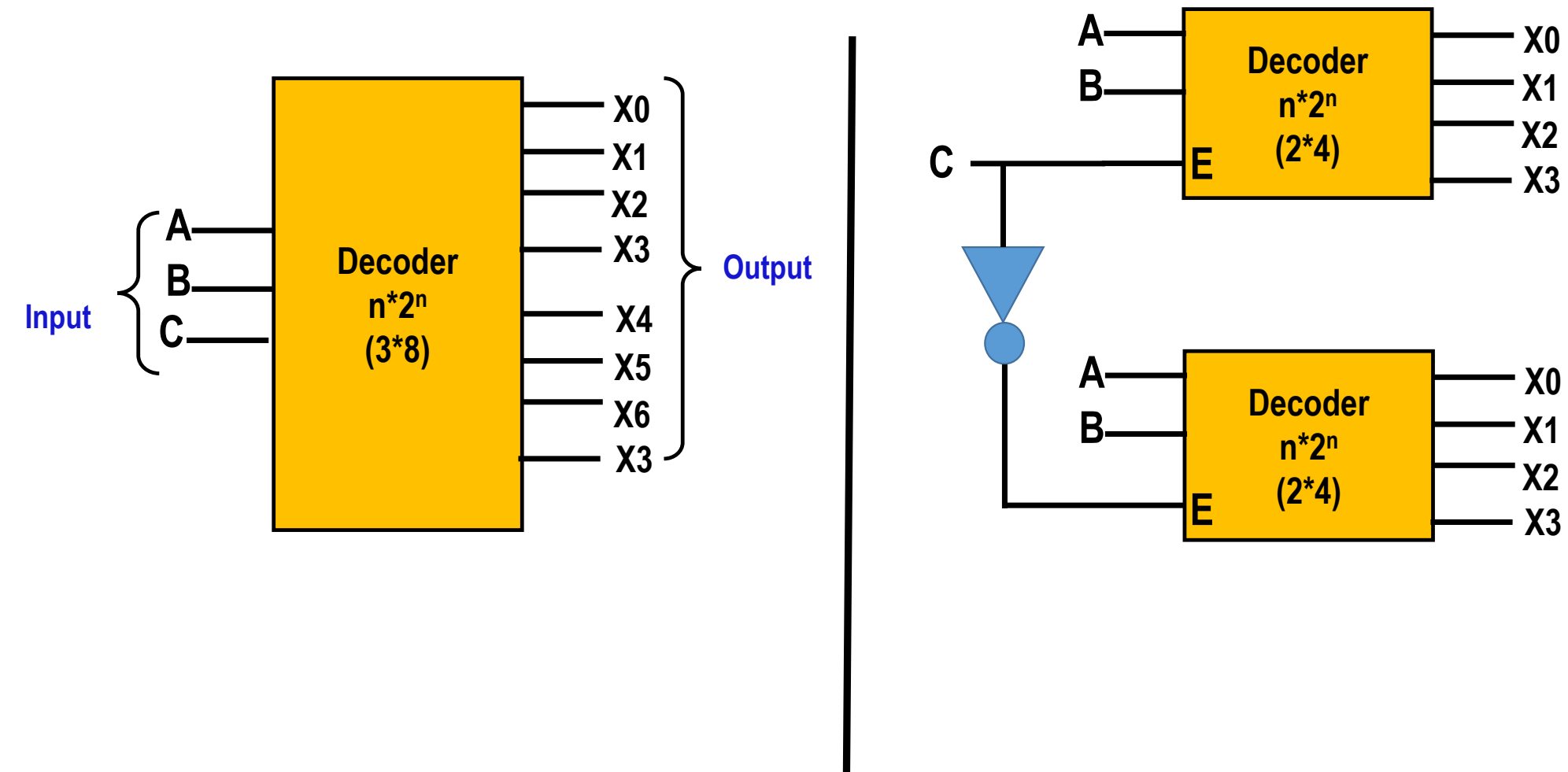
$$X2 = E \cdot A_0 \cdot \overline{B_0}$$

$$X3 = E \cdot A_0 \cdot B_0$$



Decoder Expander (2/2)

- Design a 3-bit decoder with 2-bit decoders whose an enable line.



Outlines



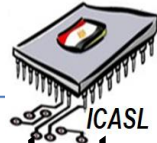
- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☐ Binary Subtractor
- ☐ Parallel Binary Adder/Subtractor
- ☐ Comparator
- ☐ Decoder
- ☒ Encoder
- ☐ Comparator
- ☐ Multiplexer
- ☐ De-multiplexer

Encoder (1/2)



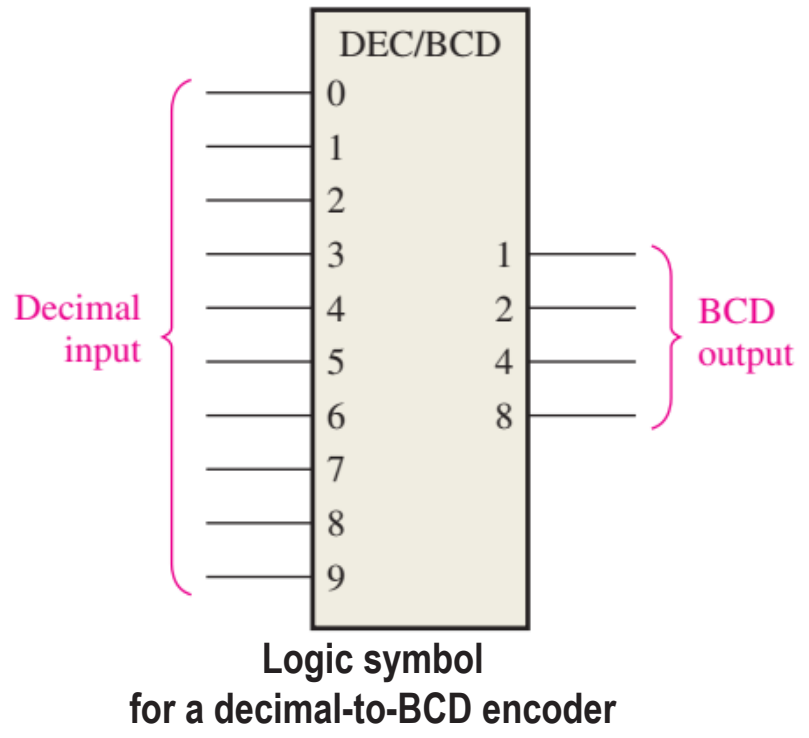
- ☐ An encoder is a combinational logic circuit that essentially performs a “reverse” decoder function.
- ☐ An encoder accepts an active level on **one** of its inputs representing a digit, such as a **decimal** and converts it to a coded output, such as **BCD or binary**.
- ☐ Encoders can also be devised to encode various symbols and alphabetic characters.
- ☐ The process of converting from **familiar** symbols or numbers to a coded format is called *encoding*

☐ Ex. The Decimal-to-BCD Encoder

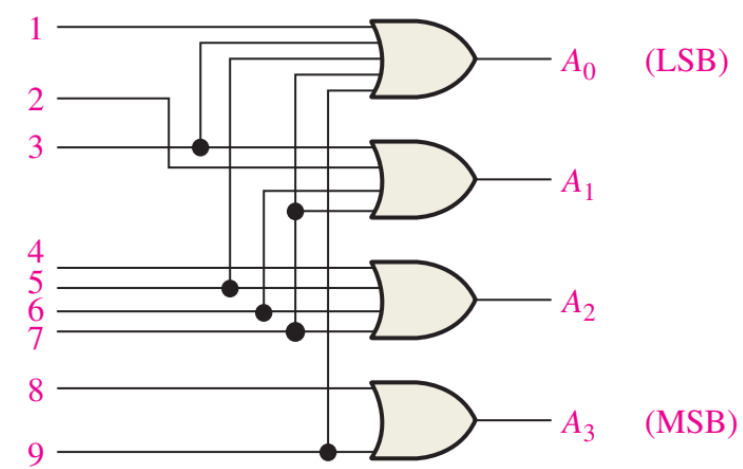


Encoder (2/2)

- ❑ This type of encoder has **ten** inputs—one for each decimal digit—and **four** outputs corresponding to the BCD code



Decimal Digit	BCD Code			
	A ₃	A ₂	A ₁	A ₀
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1



Basic logic diagram of a decimal-to-BCD encoder

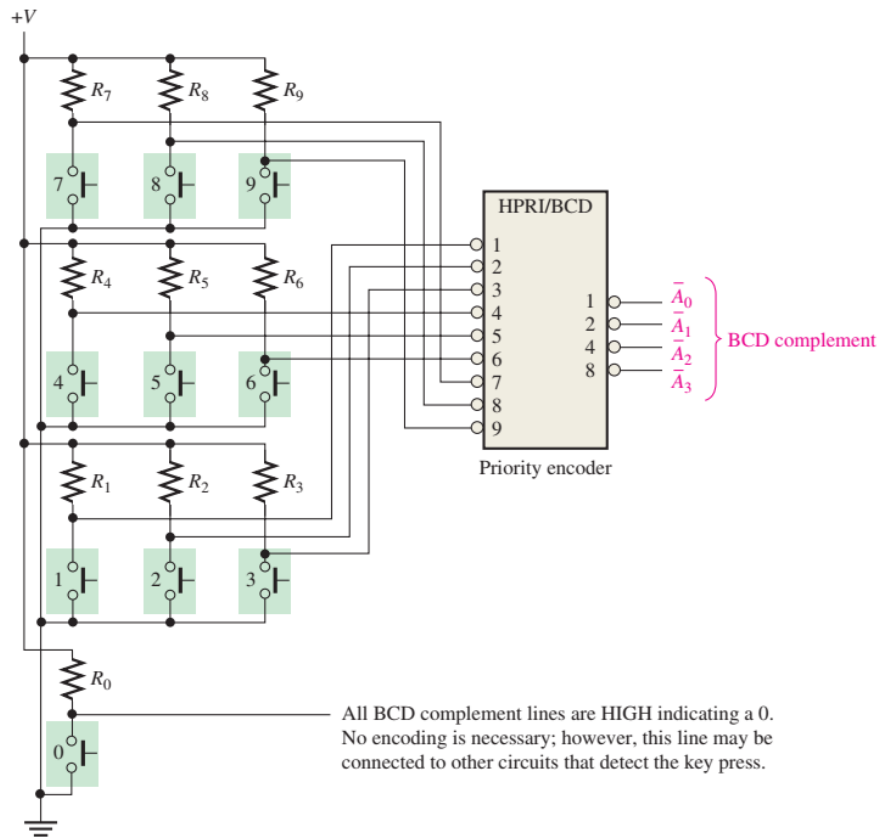
Application



❑ A 7-Segment Display Multiplexer

- ❑ The **ten** decimal digits on a numeric **keypad** must be encoded for processing by the logic circuitry.
- ❑ In this example, when one of the keys is **pressed**, the decimal digit is encoded to the corresponding BCD code.
- ❑ A simple **keyboard** encoder arrangement using a priority encoder can be designed .
- ❑ The keys are represented by **ten** push-button switches, each with a pull-up resistor to +5V.
- ❑ The pull-up resistor ensures that the line is **HIGH** when a key is not depressed.
- ❑ When a key is depressed, the line is connected to ground, and a **LOW** is applied to the corresponding encoder input.
- ❑ The zero key is not connected because the BCD output represents zero when none of the other keys is depressed.

Application



Outlines

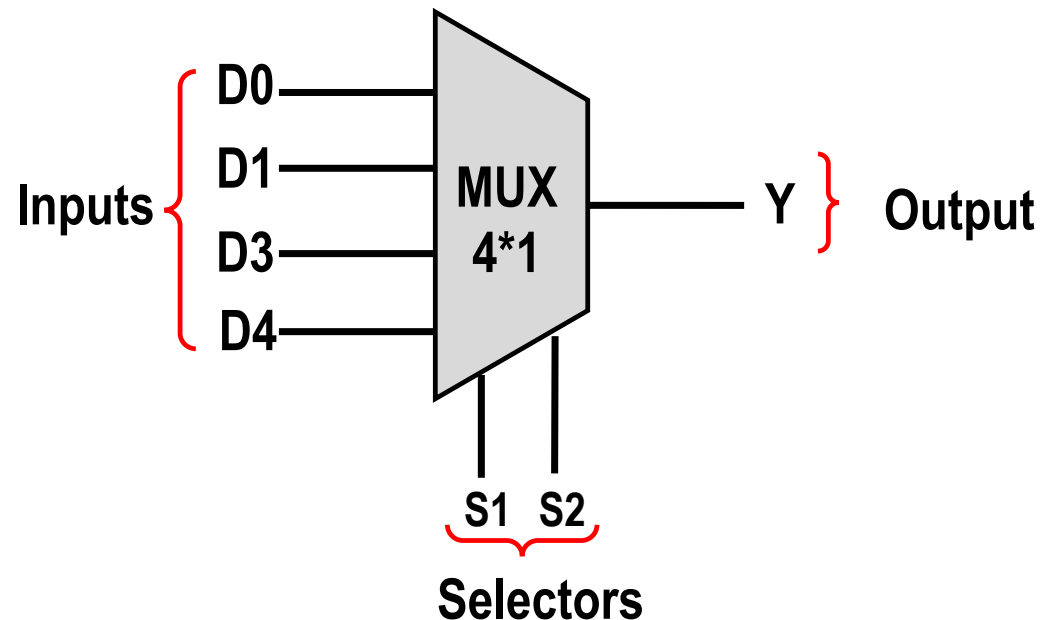
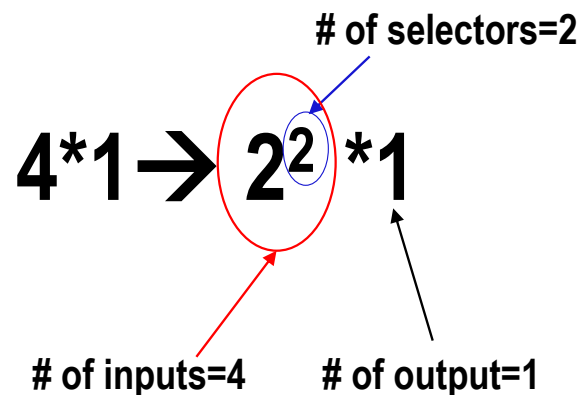


- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☐ Binary Subtractor
- ☐ Parallel Binary Adder/Subtractor
- ☐ Comparator
- ☐ Decoder
- ☐ Encoder
- ☐ Multiplexer
- ☐ De-multiplexer



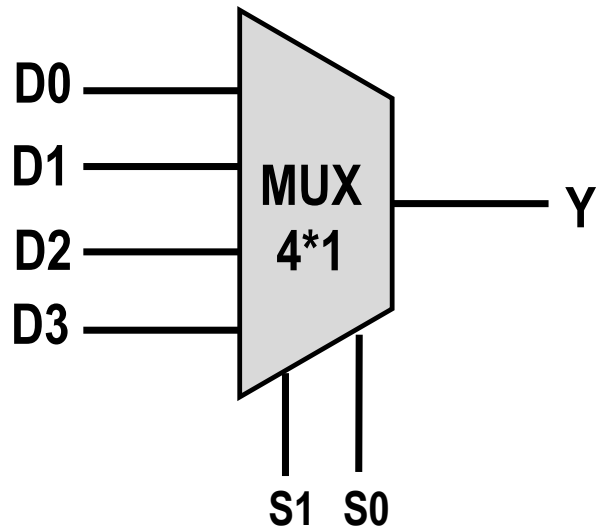
Multiplexers (1/2)

- ❑ A multiplexer (MUX) is a device that allows digital information from several sources to be routed onto a single line for transmission over that line to a common destination.
- ❑ The basic multiplexer has several data-input lines and a single output line.
- ❑ It also has data-select inputs, which permit digital data on any one of the inputs to be switched to the output line.
- ❑ Multiplexers are also known as data selectors.



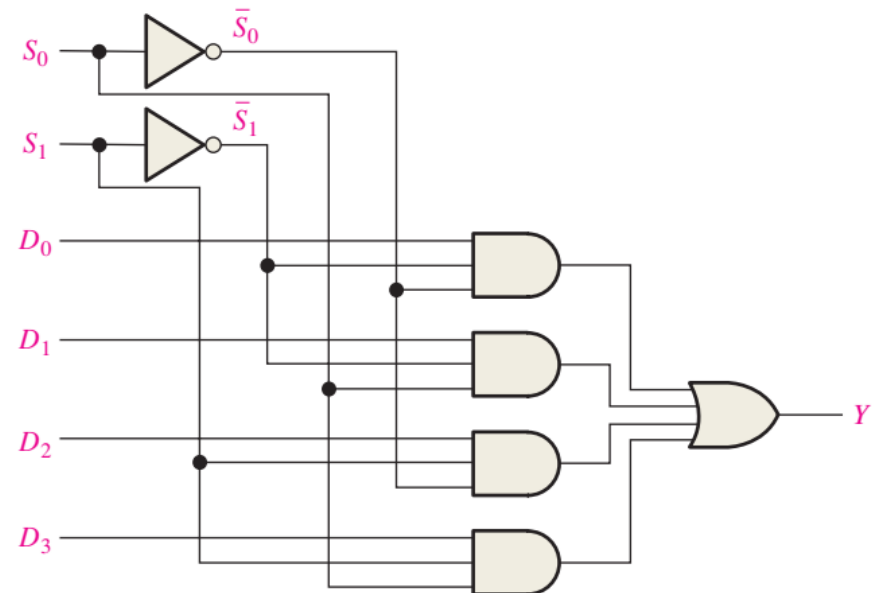
Multiplexers (2/2)

□ Ex1. Design a 4*1 multiplexer using logic gates



Data-Select Inputs		Input Selected
S_1	S_0	
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

$$Y = D_0\bar{S}_1\bar{S}_0 + D_1\bar{S}_1S_0 + D_2S_1\bar{S}_0 + D_3S_1S_0$$

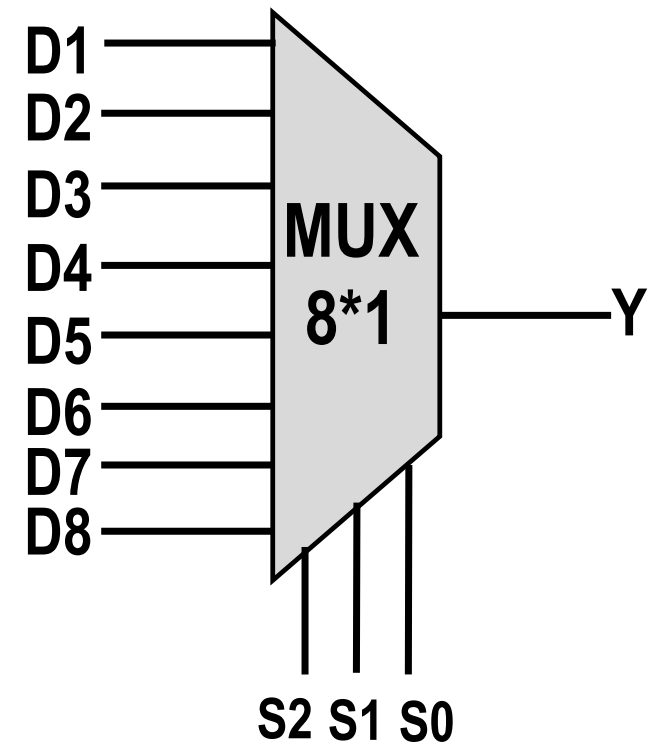
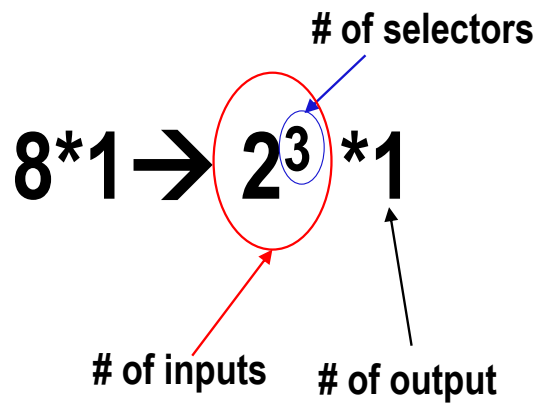


Logic diagram for a 4*1-input multiplexer

Multiplexers (1/2)



□ Ex1. Design a 8*1 multiplexer using logic gates



Multiplexers (1/2)

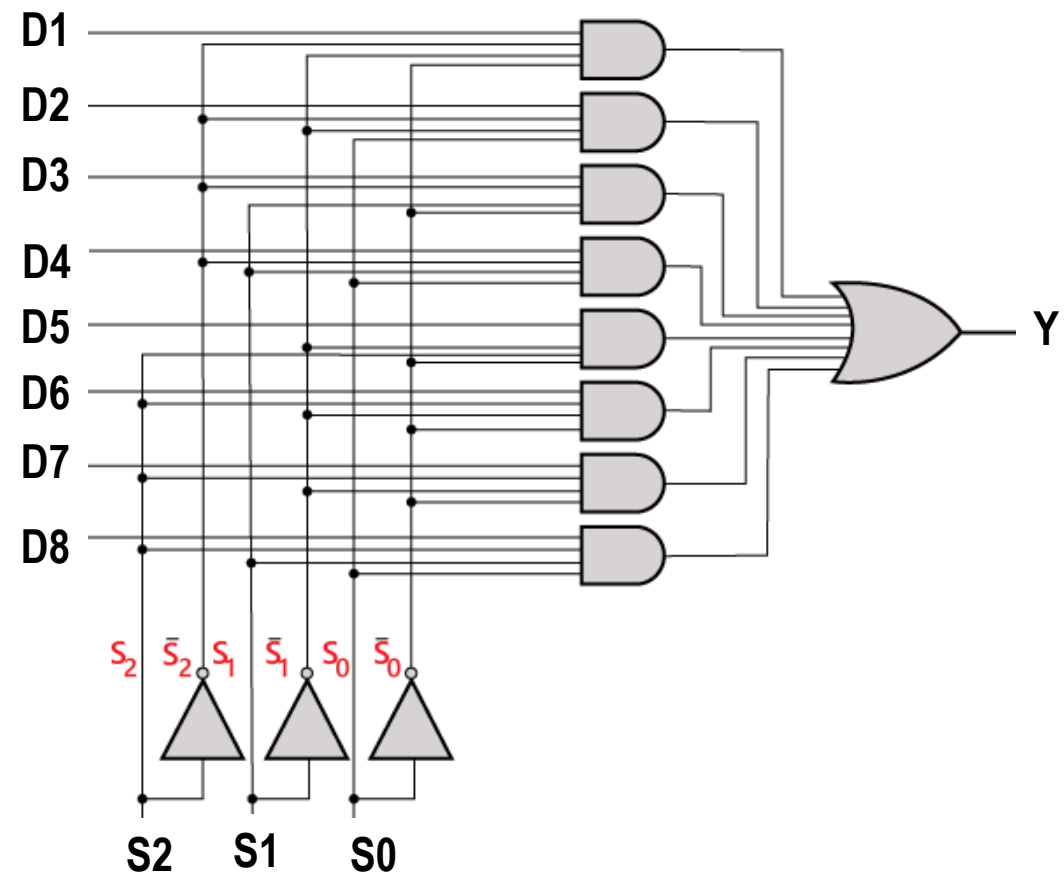


□ Ex1. Design a 8*1 multiplexer using logic gates

Truth table

S2	S1	S0	F
0	0	0	D1
0	0	1	D2
0	1	0	D3
0	1	1	D4
1	0	0	D5
1	0	1	D6
1	1	0	D7
1	1	1	D8

$$Y = D_1 \cdot \bar{S}_2 \cdot \bar{S}_1 \cdot \bar{S}_0 + D_2 \cdot \bar{S}_2 \cdot \bar{S}_1 \cdot S_0 + D_3 \cdot \bar{S}_2 \cdot S_1 \cdot \bar{S}_0 + D_4 \cdot \bar{S}_2 \cdot S_1 \cdot S_0 + D_5 \cdot S_2 \cdot \bar{S}_1 \cdot \bar{S}_0 + D_6 \cdot S_2 \cdot \bar{S}_1 \cdot S_0 + D_7 \cdot S_2 \cdot S_1 \cdot \bar{S}_0 + D_8 \cdot S_2 \cdot S_1 \cdot S_0$$



Logic diagram for a 8*1-input multiplexer

Multiplexers

- Design a 3*1 multiplexer using logic gates



Assignment

Outlines

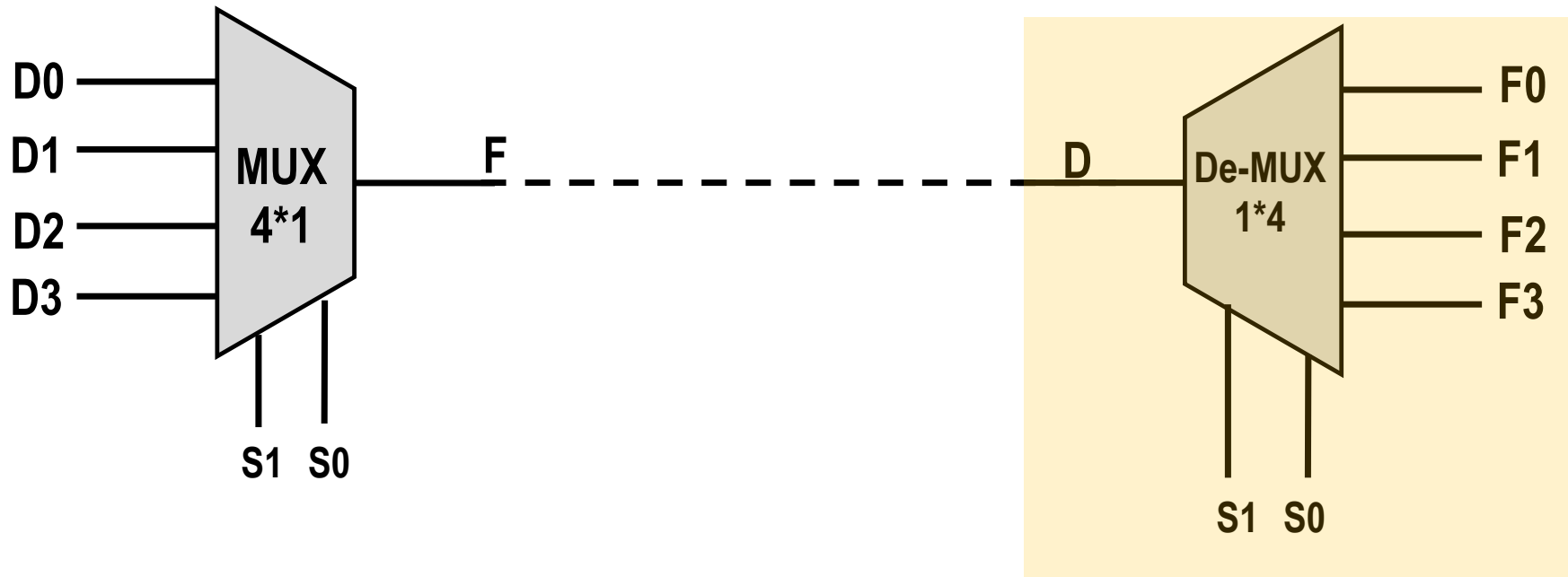


- ☐ Half and Full Adders
- ☐ Parallel Binary Adders
- ☐ Binary Subtractor
- ☐ Parallel Binary Adder/Subtractor
- ☐ Comparator
- ☐ Decoder
- ☐ Encoder
- ☐ Multiplexer
- ☐ De-multiplexer

Demultiplex (1/2)

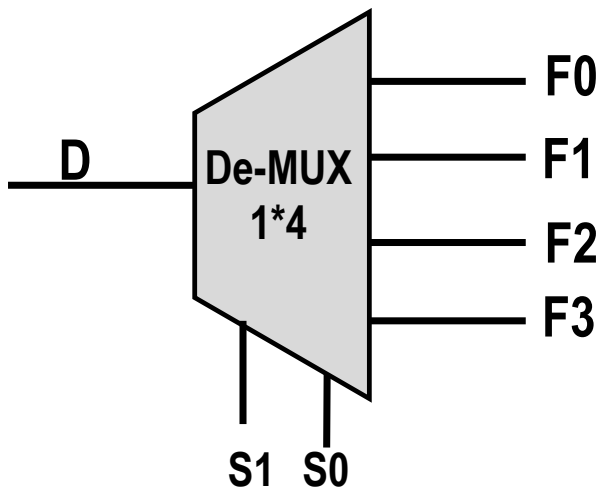


- ❑ A demultiplexer (**DEMUX**) basically reverses the multiplexing function.
- ❑ It takes digital information from one line and distributes it to a given number of output lines.
- ❑ For this reason, the **demultiplexer** is also known as a data distributor.



Demultiplex (2/2)

□ Ex1. Design a 1*4 multiplexer using logic gates



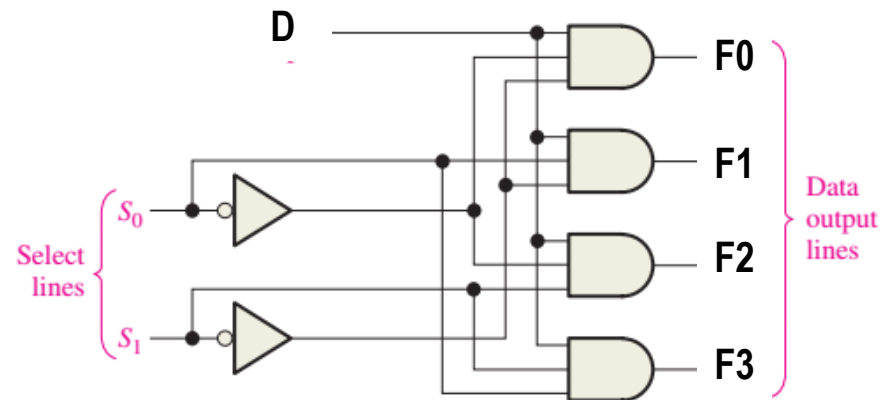
Selectors		Output			
S1	S0	F0	F1	F2	F3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

$$F0 = D \cdot \overline{S_1} \cdot \overline{S_0}$$

$$F1 = D \cdot \overline{S_1} \cdot S_0$$

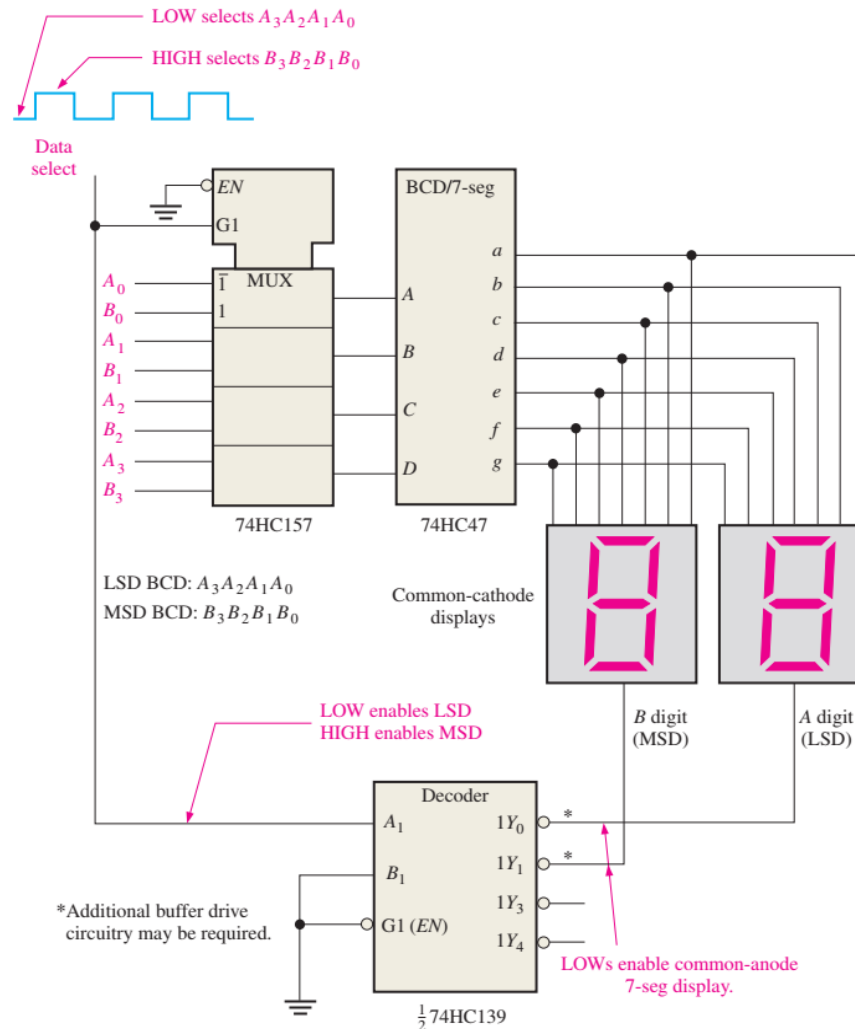
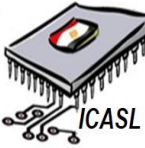
$$F2 = D \cdot S_1 \cdot \overline{S_0}$$

$$F3 = D \cdot S_1 \cdot S_0$$



A 1-line-to-4-line demultiplexer

Application



Simplified 7-segment display multiplexing logic

Parity Generators/Checkers (1/2)



- ❑ Errors can occur as digital codes are being transferred from one point to another within a digital system.
- ❑ The errors take the form of undesired changes in the bits that make up the coded information; that is, a 1 can change to a 0, or a 0 to a 1, **because of component malfunctions or electrical noise.**
- ❑ In most digital systems, the **probability** that even a single bit error will occur is **very small**, and the likelihood that more than one will occur is even smaller.
- ❑ Nevertheless, when an error occurs undetected, it can cause serious problems in a digital system.

A parity bit indicates if the number of 1s in a code is even or odd for the purpose of error detection.

The sum of an even number of 1s is always 0, and the sum of an odd number of 1s is always 1.

Parity Generators/Checkers



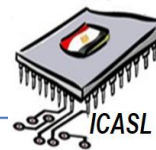
□ Design three input odd-parity checker. The output is **high** when an odd number of 1's.]

(a) Using universal logic gates

(b) Using Logic gate

Ans.

Parity Generators/Checkers



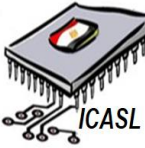
□ Design three input odd-parity checker. The output is **high** when an odd number of 1's.]

(a) Using universal logic gates

(b) Using Logic gate

Ans.





Implementation using Universal Logic Gates

□ Design three input odd-parity checker. The output is **high** when an odd number of 1's.]

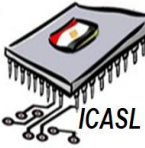
(a) Using universal logic gates

(b) Using Logic gates

Ans.

Input			Output
C	B	A	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1





Implementation using Universal Logic Gates

□ Design three input odd-parity checker. The output is high when an odd number of 1's.]

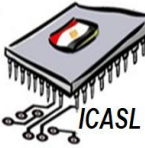
(a) Using universal logic gates

(b) Using Logic gate

Ans.

Input			Output
C	B	A	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1





Implementation using Universal Logic Gates

□ Design three input odd-parity checker. The output is **high** when an odd number of 1's.]

(a) Using universal logic gates

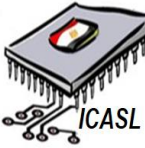
(b) Using Logic gate

Ans.

Input			Output
C	B	A	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



		B			
		00	01	11	10
A	0	0	1	0	1
	1	1	0	1	0



Implementation using Universal Logic Gates

□ Design three input odd-parity checker. The output is **high** when an odd number of 1's.]

(a) Using universal logic gates

(b) Using Logic gate

Input			Output
C	B	A	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Ans.

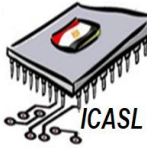


		B			
		00	01	11	10
A	BC	0	1	0	1
	1	1	0	1	0

The output function is obtained by using Sum Of Product (**SOP**)

$$F = A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot B \cdot \overline{C} + \overline{A} \cdot \overline{B} \cdot C + A \cdot B \cdot C$$

Implementation using Universal Logic Gates



□ Design three input odd-parity checker. The output is **high** when an odd number of 1's.]

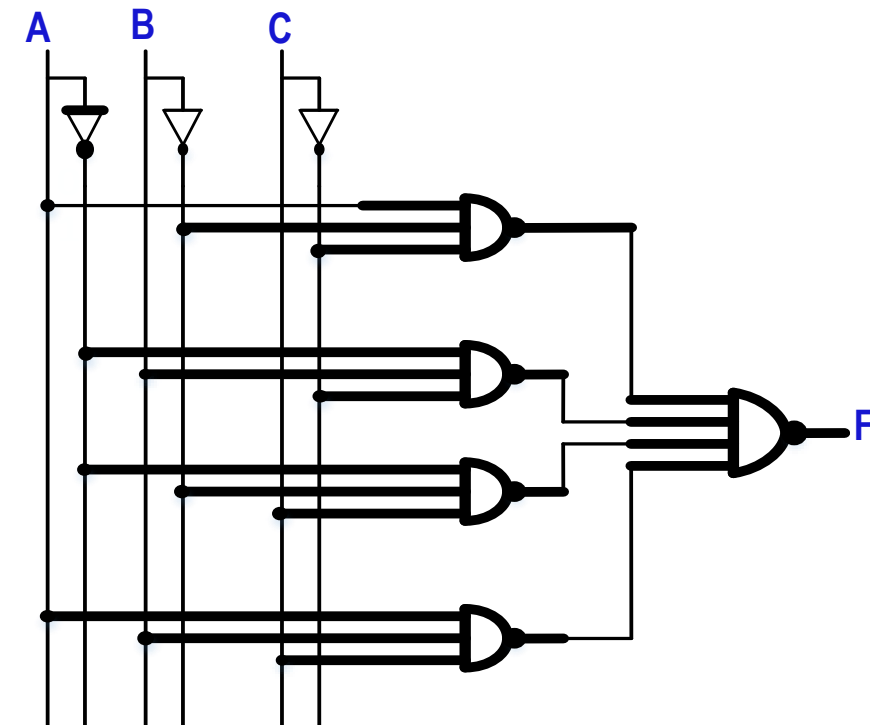
(a) Using universal logic gates

(b) Using Logic gate

Ans.

$$F = A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot B \cdot \overline{C} + \overline{A} \cdot \overline{B} \cdot C + A \cdot B \cdot C$$

$$F = A \cdot \overline{B} \cdot \overline{C} \cdot \overline{A} \cdot B \cdot \overline{C} \cdot \overline{A} \cdot \overline{B} \cdot C \cdot A \cdot B \cdot C$$



Implementation using Universal Logic Gates



□ Design three input odd-parity checker. The output is **high** when an odd number of 1's.]

(a) Using universal logic gates

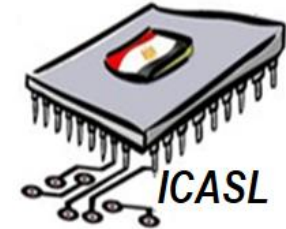
(b) Using Logic gate

Ans.

$$F = A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot B \cdot \overline{C} + \overline{A} \cdot \overline{B} \cdot C + A \cdot B \cdot C$$

Draw the logic gates ?

Thanks!



Digital Design (SET 111)

L11

Latches, Flip-Flops, and Timers

Dr. Ahmed Reda Mohamed¹

¹PhD, the MoE Key Laboratory of Artificial Intelligence & Bio-Circuits and Systems Laboratory (**BiCASL**), Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.

Outlines



- ☐ Types of digital logic circuits
- ☐ Types of Storage Elements
- ☐ SR-Latch
- ☐ Gated SR-Latch
- ☐ D-Latch
- ☐ D-Flip-Flop
- ☐ JK-Flip-Flop
- ☐ Flip-Flop Applications
- ☐ 555 Timer

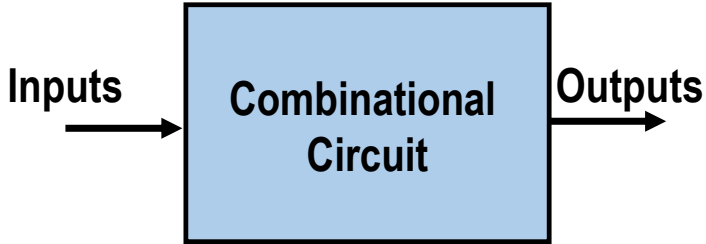
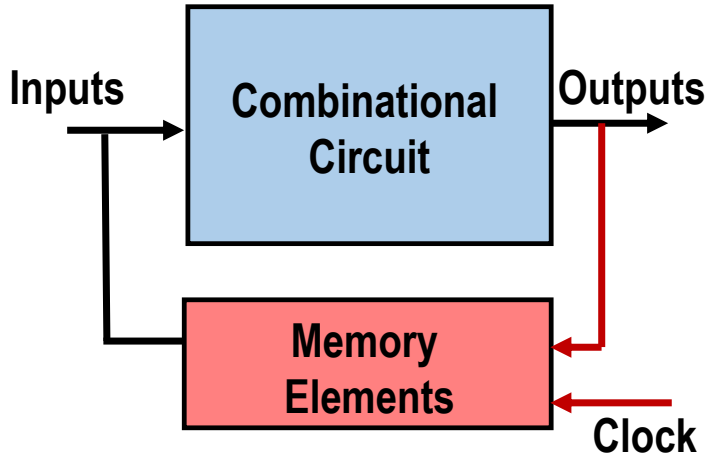
Outlines



- ☐ **Types of digital logic circuits**
- ☐ **Types of Storage Elements**
- ☐ **SR-Latch**
- ☐ **Gated SR-Latch**
- ☐ **D-Latch**
- ☐ **D-Flip-Flop**
- ☐ **JK-Flip-Flop**
- ☐ **Flip-Flop Applications**
- ☐ **555 Timer**

Types of Digital Logic Circuits



Combinational Circuit	Sequential Circuit
 Output depends on the present state of input only. Does not contain memory elements. Does not use the Feedback path Does not require a clock signal. Ex. Adder, Subtractor, MUX, Decoder, Encoder..etc	 Output depends on the present inputs and the past history of inputs. Contains memory elements. Uses Feedback path. Most sequential circuits use a clock signal. Ex. Flip Flops, Registers, Counters, etc

Outlines

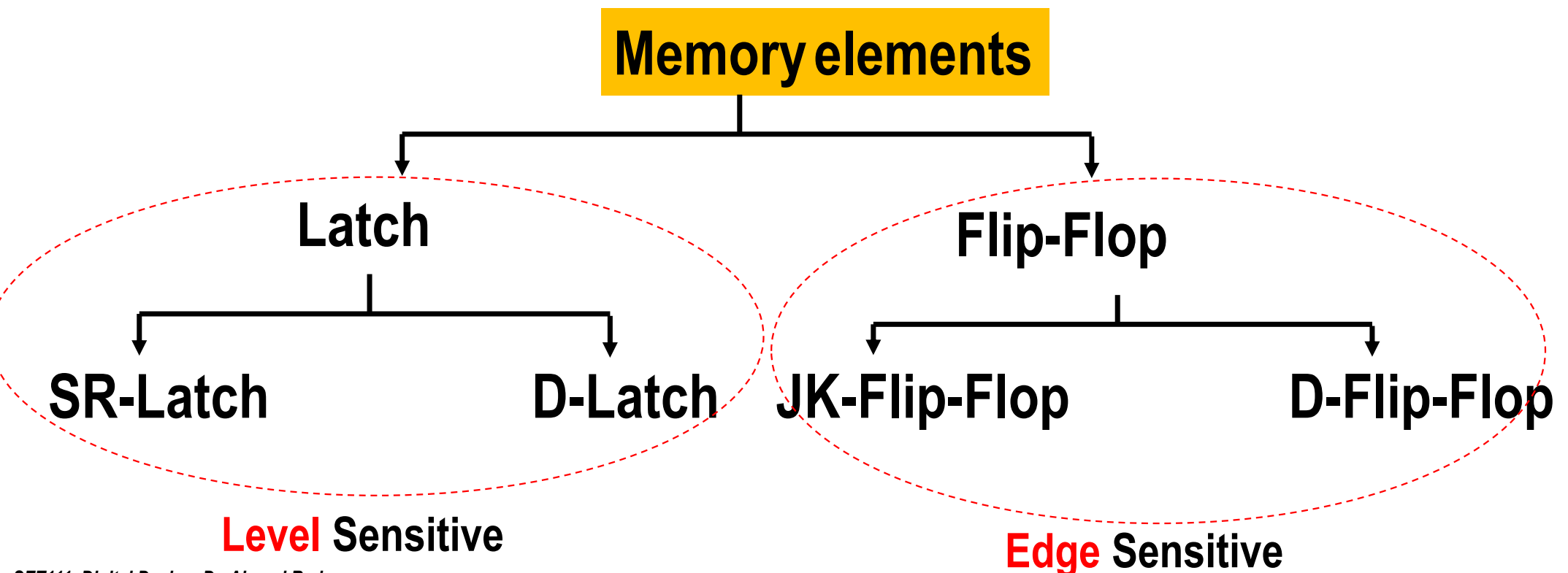


- ☐ Types of digital logic circuits
- ☒ **Types of Storage Elements**
- ☐ SR-Latch
- ☐ Gated SR-Latch
- ☐ D-Latch
- ☐ D-Flip-Flop
- ☐ JK-Flip-Flop
- ☐ Flip-Flop Applications
- ☐ 555 Timer

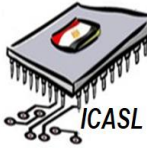
Types of Storage Elements (1/3)



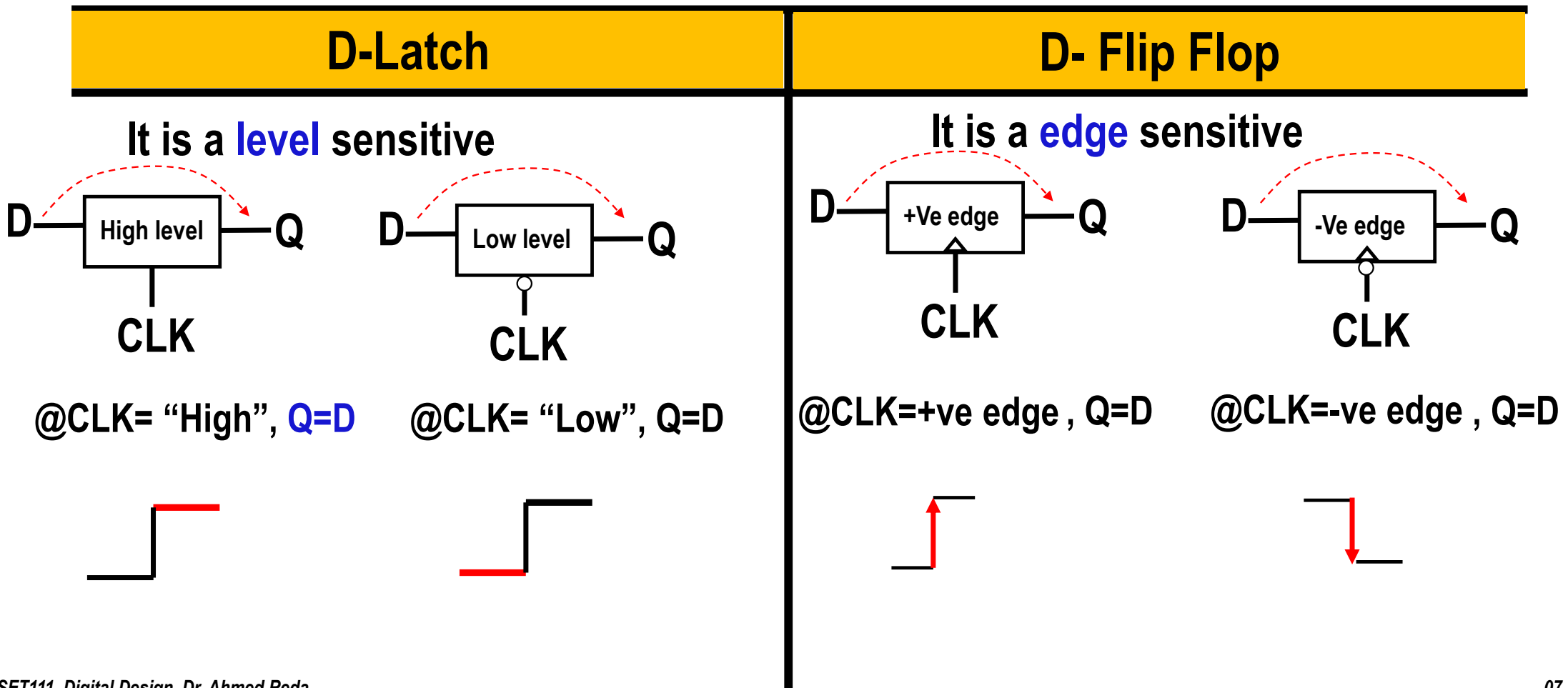
□ **Storage/Memory** elements are needed in most digital logic circuits to hold (**remember**) logic values.



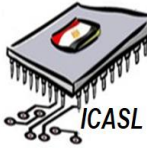
Types of Storage Elements (2/3)



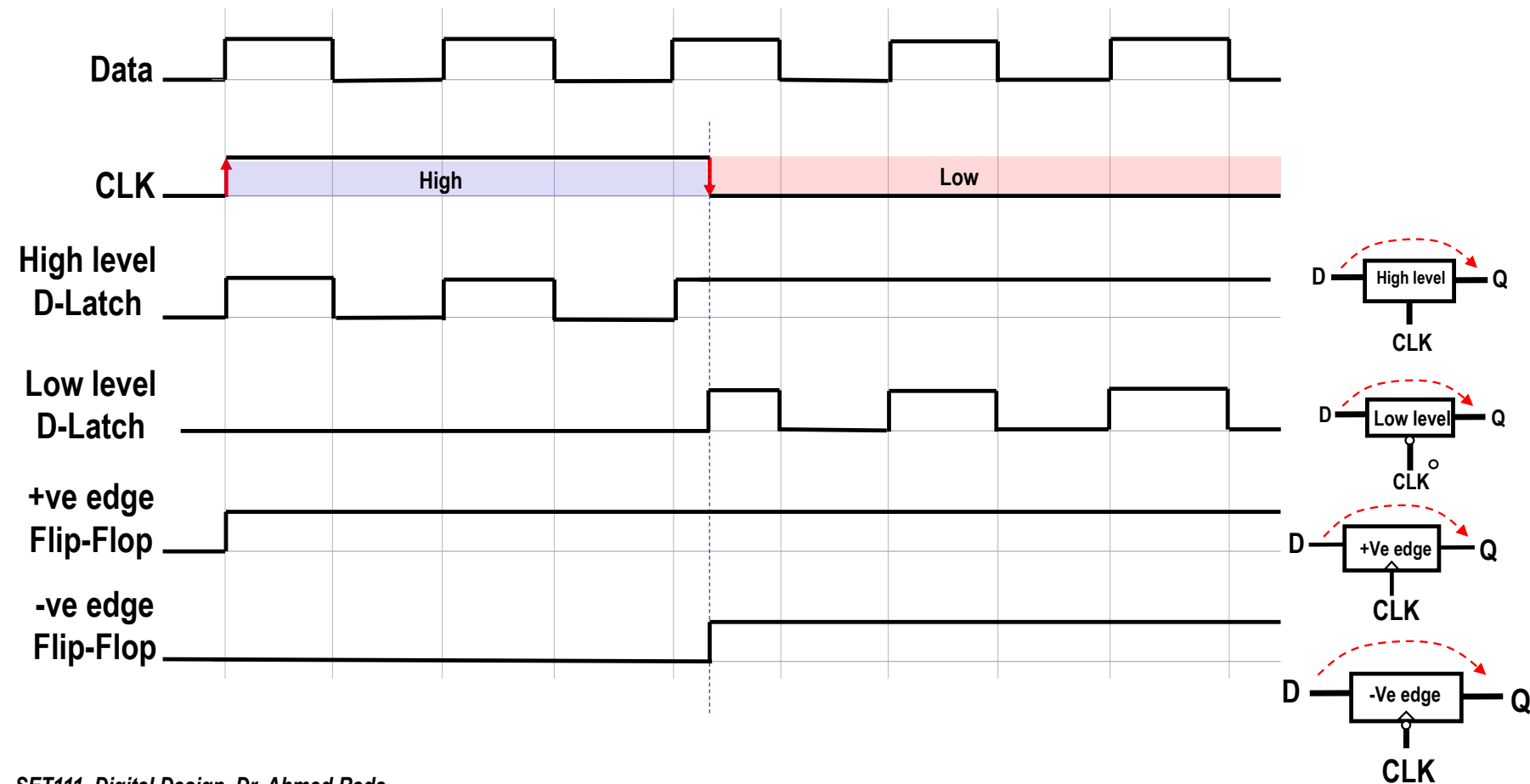
- ❑ To understand the difference between the latch and flip-flop.



Types of Storage Elements (3/3)



□ Timing diagram for D-Latch and D-Flip-flop



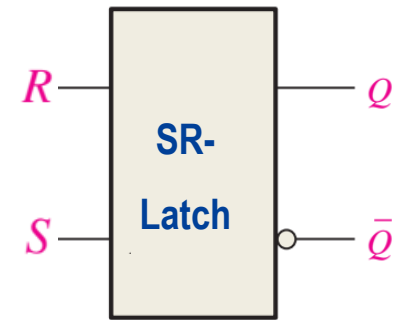
Outlines



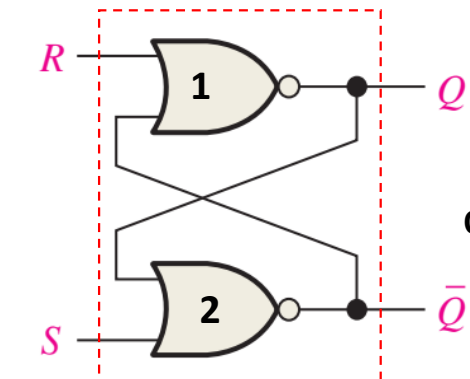
- ☐ Types of digital logic circuits
- ☐ Types of Storage Elements
- ☒ **SR-Latch**
- ☐ Gated SR-Latch
- ☐ D-Latch
- ☐ D-Flip-Flop
- ☐ JK-Flip-Flop
- ☐ Flip-Flop Applications
- ☐ 555 Timer

SR-Latch (1/11)

□ Active-HIGH input S-R latch



Block Diagram



Two cross-coupled NOR gates

Internal Circuit

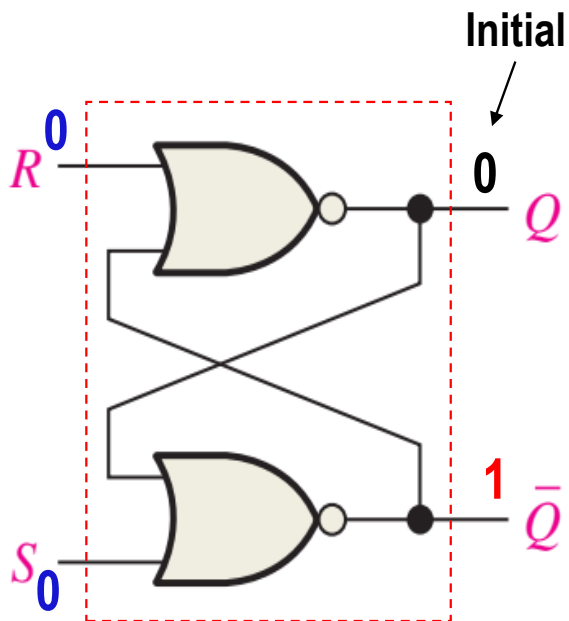
Truth Table

Inputs		Output		Comments
S	R	Q	$\bar{Q}$	
0	0	Q_0	$\bar{Q}_0$	No change, Latch remains in present state
0	1	0	1	RESET
1	0	1	0	SET
1	1	0	0	Invalid Condition

Practically, setting both inputs to 1 is **forbidden!**

SR-Latch (2/11)

❑ Operation of Active-HIGH input S-R latch



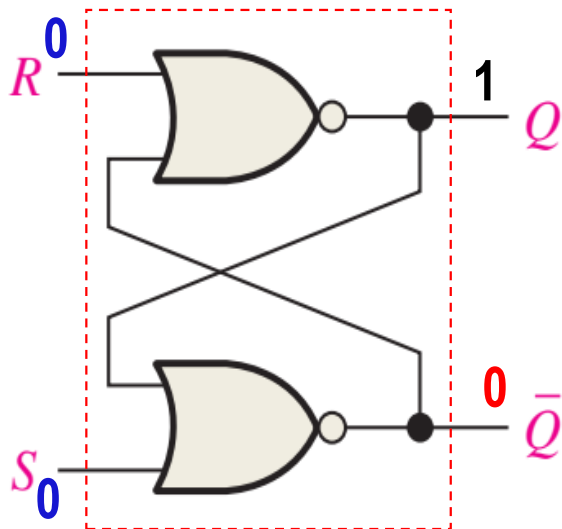
Truth Table

Inputs		Initial Output	Output		Comments
S	R	Q_0	Q	$\overline{Q}$	
0	0	0	0	1	
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

SR-Latch (3/11)



❑ Operation of Active-HIGH input S-R latch



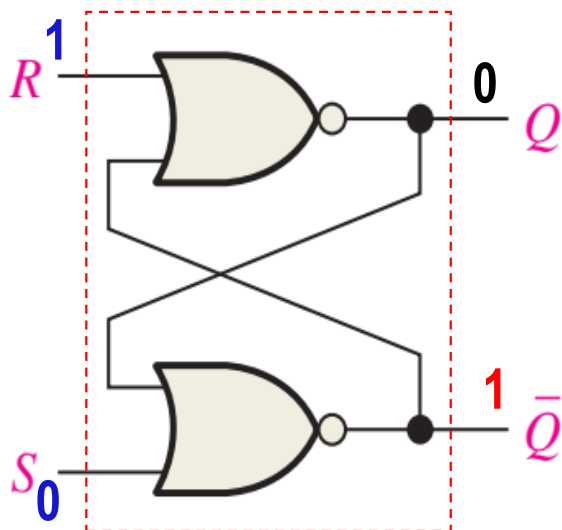
Truth Table

Inputs		Initial Output	Output		Comments
S	R	Q_0	Q	$\overline{Q}$	
0	0	0	0	1	No change, Latch remains in present state
0	0	1	1	0	
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

SR-Latch (4/11)



❑ Operation of Active-HIGH input S-R latch

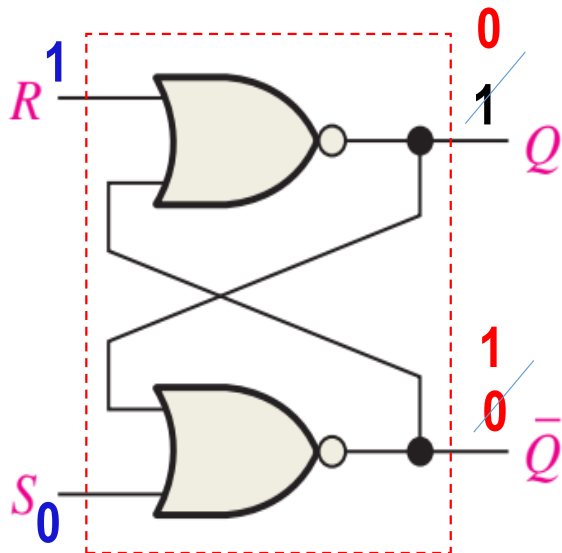


Truth Table

Inputs		Initial Output	Output		Comments
S	R	Q_0	Q	$\overline{Q}$	
0	0	0	0	1	No change, Latch remains in present state
0	0	1	1	0	
0	1	0	0	1	
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

SR-Latch (5/11)

□ Operation of Active-HIGH input S-R latch

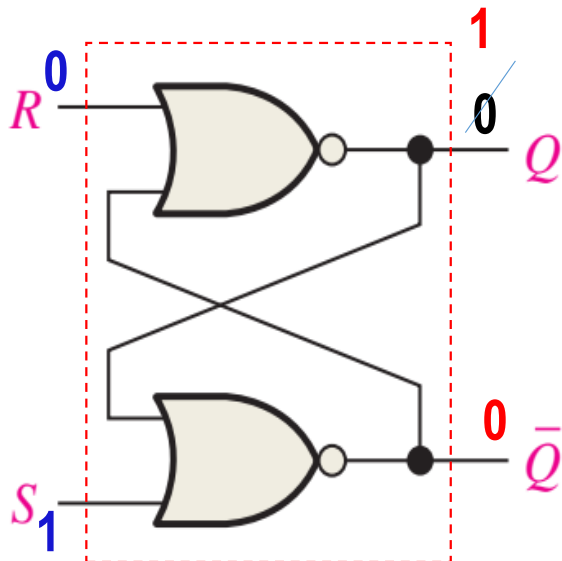


Truth Table

Inputs		Initial Output	Output		Comments
S	R	Q_0	Q	$\overline{Q}$	
0	0	0	0	1	No change, Latch remains in present state
0	0	1	1	0	
0	1	0	0	1	RESET
0	1	1	0	1	
1	0	0			
1	0	1			
1	1	0			
1	1	1			

SR-Latch (6/11)

□ Operation of Active-HIGH input S-R latch

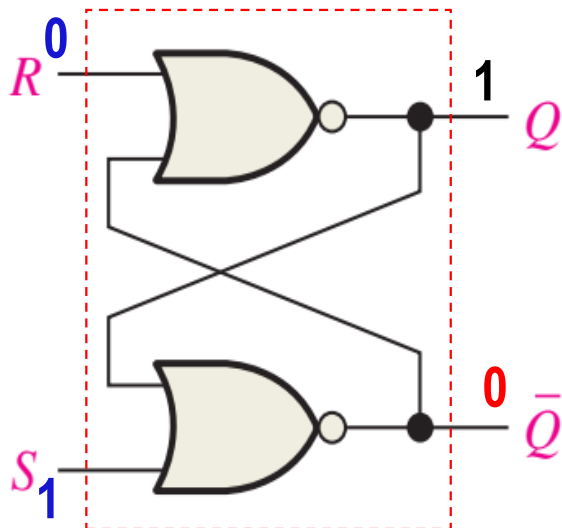


Truth Table

Inputs		Initial Output	Output		Comments
S	R	Q_0	Q	$\overline{Q}$	
0	0	0	0	1	No change, Latch remains in present state
0	0	1	1	0	
0	1	0	0	1	RESET
0	1	1	0	1	
1	0	0	1	0	
1	0	1			
1	1	0			
1	1	1			

SR-Latch (7/11)

□ Operation of Active-HIGH input S-R latch

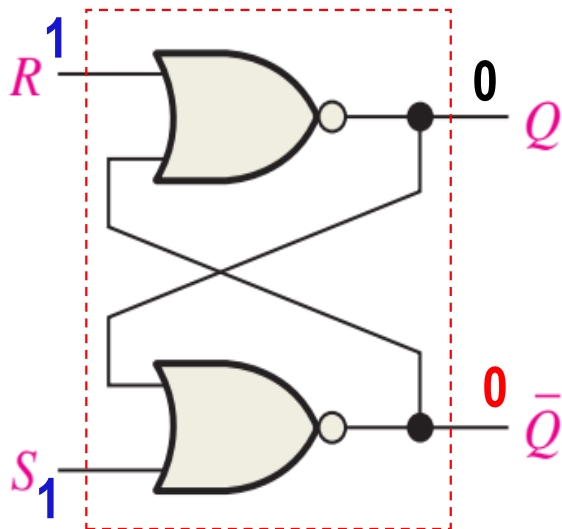


Truth Table

Inputs		Initial Output	Output		Comments
S	R	Q_0	Q	$\overline{Q}$	
0	0	0	0	1	No change, Latch remains in present state
0	0	1	1	0	
0	1	0	0	1	RESET
0	1	1	0	1	
1	0	0	1	0	SET
1	0	1	1	0	
1	1	0			
1	1	1			

SR-Latch (8/11)

□ Operation of Active-HIGH input S-R latch

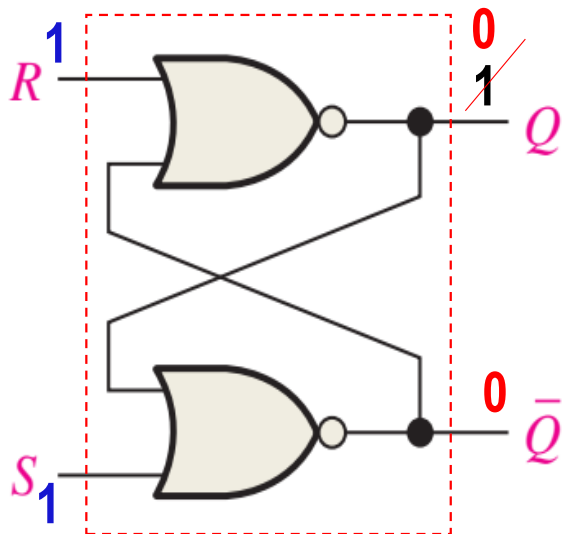


Truth Table

Inputs		Initial Output	Output		Comments
S	R	Q_0	Q	$\overline{Q}$	
0	0	0	0	1	No change, Latch remains in present state
0	0	1	1	0	
0	1	0	0	1	RESET
0	1	1	0	1	
1	0	0	1	0	SET
1	0	1	1	0	
1	1	0	0	0	
1	1	1			

SR-Latch (9/11)

□ Operation of Active-HIGH input S-R latch



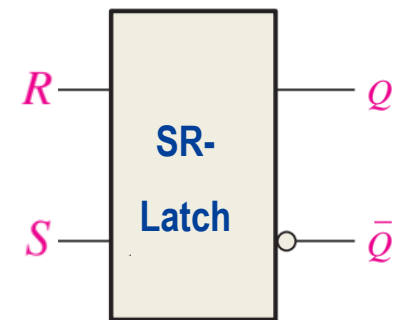
Truth Table

Inputs		Initial Output	Output		Comments
S	R	Q_0	Q	$\overline{Q}$	
0	0	0	0	1	No change, Latch remains in present state
0	0	1	1	0	
0	1	0	0	1	RESET
0	1	1	0	1	
1	0	0	1	0	SET
1	0	1	1	0	
1	1	0	0	0	Invalid Condition
1	1	1	0	0	

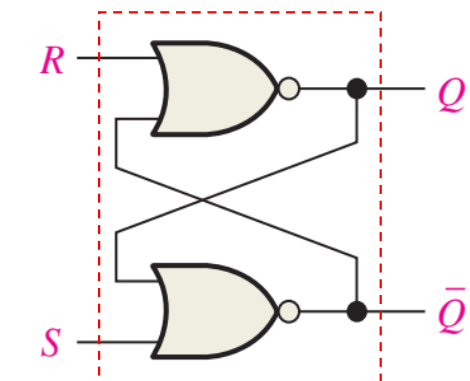
SR-Latch (10/11)



❑ Operation of Active-HIGH input S-R latch



Block Diagram



Internal Circuit

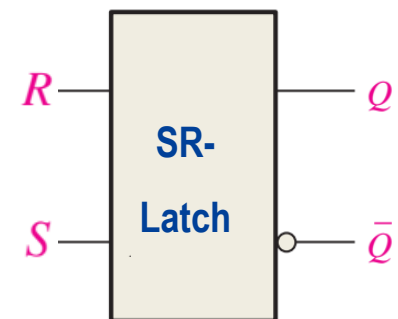
Truth Table

Inputs		Initial Output	Output		Comments
S	R	Q_0	Q	$\bar{Q}$	
0	0	0	0	1	No change, Latch remains in present state
0	0	1	1	0	
0	1	0	0	1	RESET
0	1	1	0	1	
1	0	0	1	0	SET
1	0	1	1	0	
1	1	0	0	0	Invalid Condition
1	1	1	0	0	

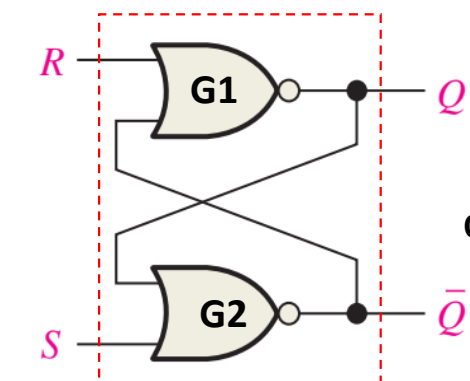
Practically, setting both inputs to 1 is forbidden!

SR-Latch (11/11)

□ Active-HIGH input S-R latch → Summary



Block Diagram



Two cross-coupled NOR gates

Internal Circuit

Truth Table

Inputs		Output		Comments
S	R	Q	$\bar{Q}$	
0	0	Q_0	$\bar{Q}_0$	No change, Latch remains in present state
0	1	0	1	RESET
1	0	1	0	SET
1	1	0	0	Invalid Condition

Practically, setting both inputs to 1 is **forbidden!**

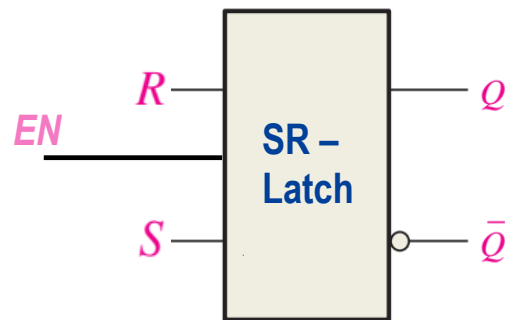
Outlines



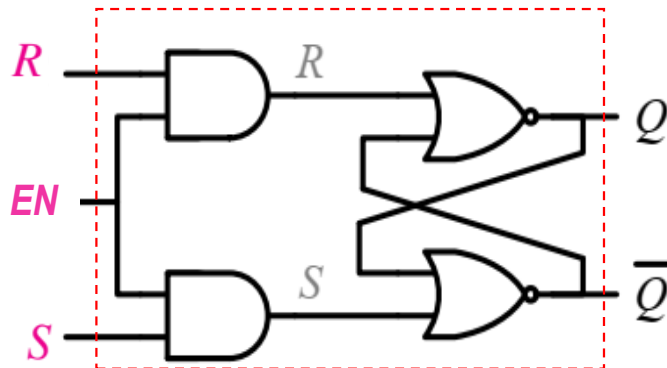
- ☐ Types of digital logic circuits
- ☐ Types of Storage Elements
- ☐ SR-Latch
- ☒ Gated SR-Latch
- ☐ D-Latch
- ☐ D-Flip-Flop
- ☐ JK-Flip-Flop
- ☐ Flip-Flop Applications
- ☐ 555 Timer

Gated SR-Latch

□ A gated latch requires an **Enable input**



Block Diagram



Internal Circuit

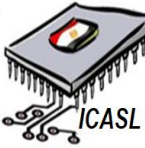
Truth Table

Inputs			Output		Comments
EN	S	R	Q	$\bar{Q}$	
1	0	0	Q_0	$\bar{Q}_0$	No change, Latch remains in present state
1	0	1	0	1	RESET
1	1	0	1	0	SET
1	1	1	0	0	Invalid Condition
0	X	X	Q_0	$\bar{Q}_0$	No change, Latch remains in present state

Outlines



- ☐ Types of digital logic circuits
- ☐ Types of Storage Elements
- ☐ SR-Latch
- ☐ Gated SR-Latch
- ☒ **D-Latch**
- ☐ D-Flip-Flop
- ☐ JK-Flip-Flop
- ☐ Flip-Flop Applications
- ☐ 555 Timer



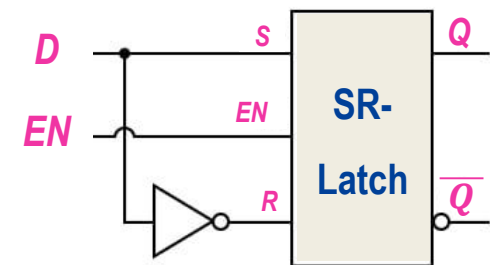
D-Latch (1/3)

❑ D-latch has **only one** input in addition to EN.

❑ This input is called the D (data) input.

- ❑ When the D input is **HIGH** and the EN input is **HIGH**, the latch will set ($Q=D$).
- ❑ When the D input is **LOW** and EN is **HIGH**, the latch will reset ($Q=0$).

The output Q follows the input D when EN is HIGH.



Block Diagram

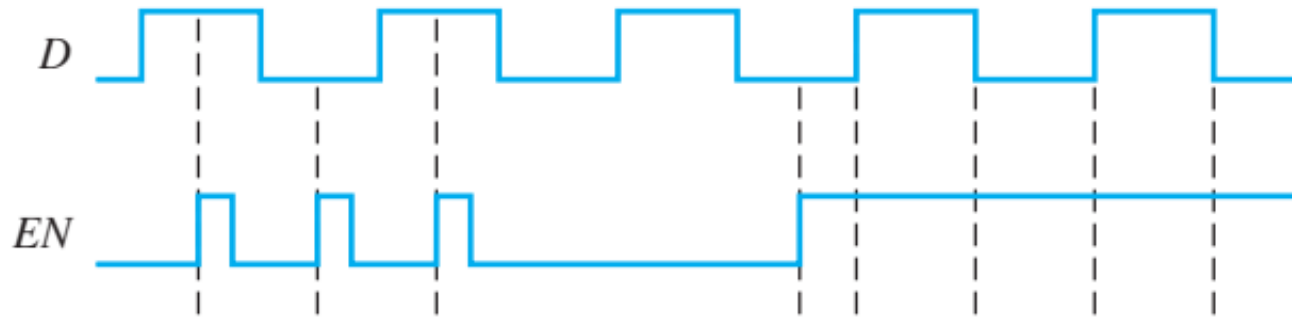
Truth Table

Inputs		Output		Comments
EN	D	Q	$\overline{Q}$	
1	0	0	1	Q=D
1	1	1	0	
0	X	Q_0	Q_0	No change

D-Latch (2/3)



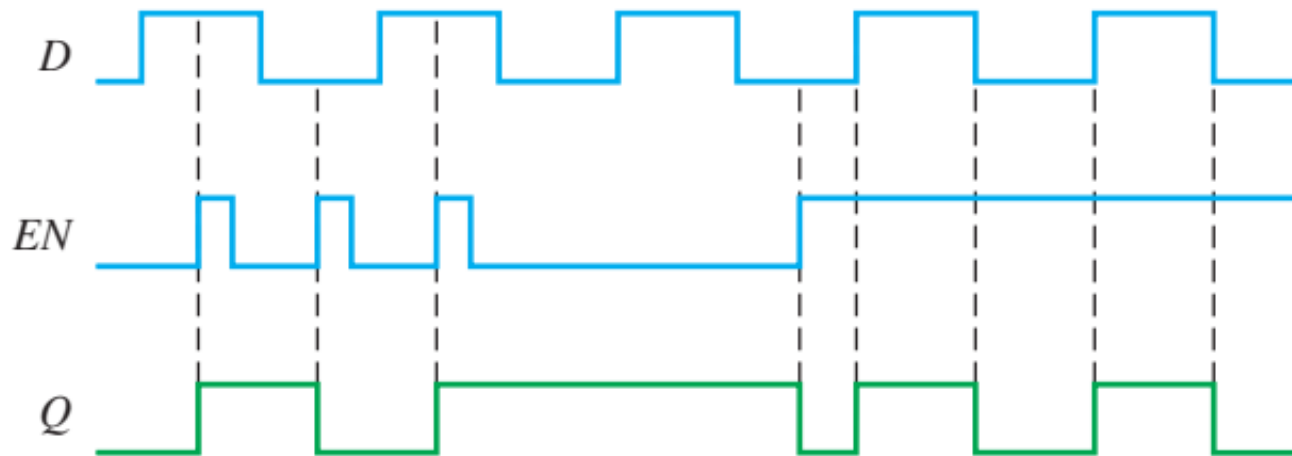
□ Ex.1 Determine the Q output waveform if the inputs shown in Figure are applied to a gated D latch, which is initially RESET



D-Latch (3/3)



- Ex.1 Determine the Q output waveform if the inputs shown in Figure are applied to a gated D latch, which is initially RESET



Solution

- As Long as $EN=1$ the Latch output asynchronously follows input Data D
- When EN is LOW, the state of the latch is not affected by the D input

Outlines

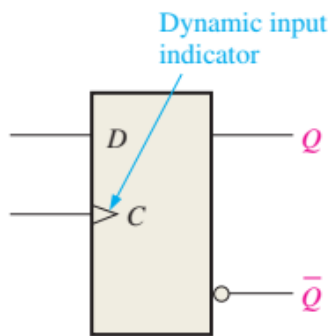


- ☐ Types of digital logic circuits
- ☐ Types of Storage Elements
- ☐ SR-Latch
- ☐ Gated SR-Latch
- ☐ D-Latch
- ☒ **D-Flip-Flop**
- ☐ JK-Flip-Flop
- ☐ Flip-Flop Applications
- ☐ 555 Timer

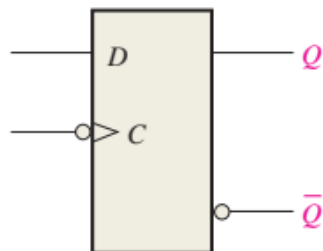
Flip-Flop



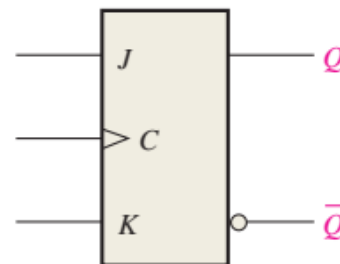
- ❑ The edge-sensitive flip-flop changes state either at the **positive** or at the **negative** edge of the clock pulse.
- ❑ **Two types** of edge-sensitive flip-flops are D-Flip-flop and J-K- Flip-Flop



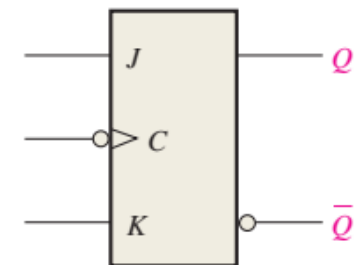
D-Flip-Flop
Positive edge



D-Flip-Flop
Negative edge



JK -Flip-Flop
Positive edge



JK -Flip-Flop
Negative edge



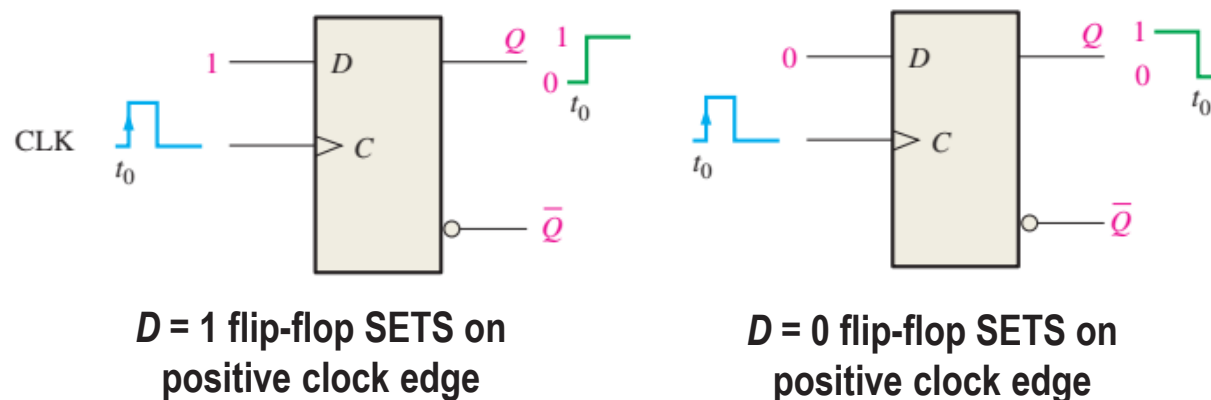
D-Flip-Flop (1/3)

❑ The D input of the D-flip-flop is a **synchronous input** because data on the input are transferred to the flip-flop's output **only** on the triggering edge of the clock.

❑ When D is **HIGH**, the Q output goes **HIGH** on the triggering edge of the clock pulse, and the flip-flop is **SET**.

❑ When D is **LOW**, the Q output goes **LOW** on the triggering edge of the clock pulse, and the flip-flop is **RESET**.

The output Q follows the input D @ triggering edge of the clock



Truth Table of +ve edge-triggered D-Flip Flop

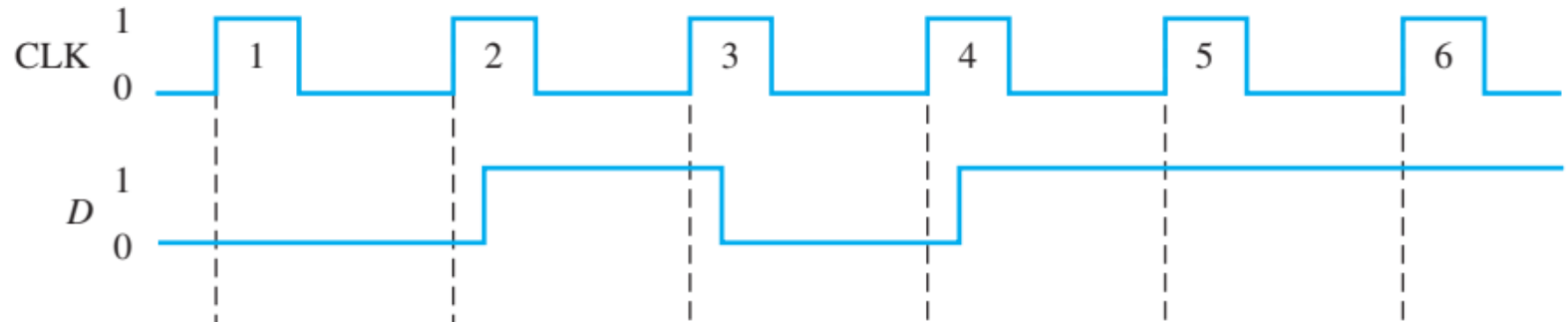
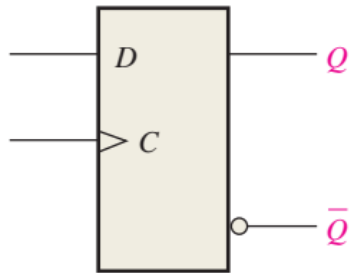
Inputs		Outputs		Comments
D	CLK	Q	$\bar{Q}$	
0	↑	0	1	RESET
1	↑	1	0	SET

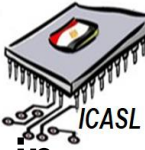
↑: Clock transition LOW to HIGH

D-Flip-Flop (2/3)



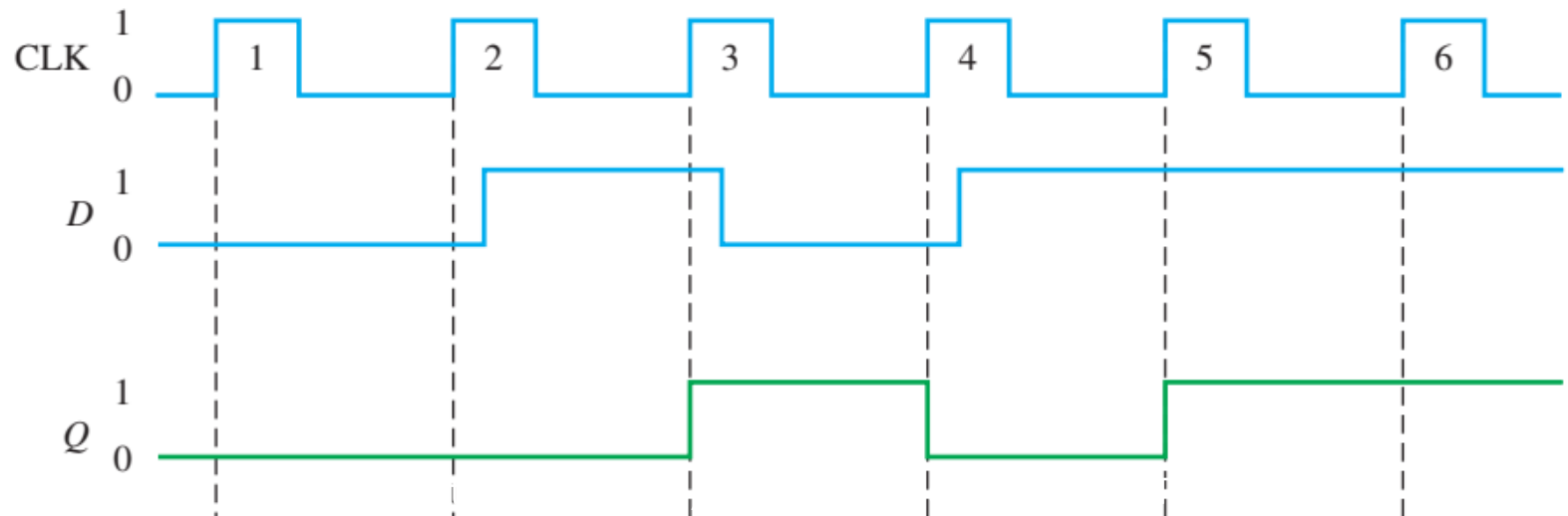
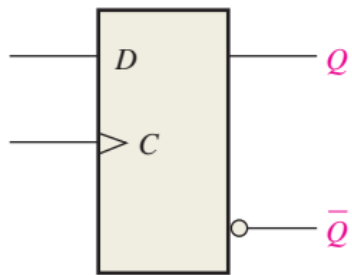
- Determine the Q and $\overline{Q}$ output waveforms of the flip-flop in Figure for the D and CLK inputs in Figure. Assume that the positive edge-triggered flip-flop is initially RESET.





D-Flip-Flop (2/3)

□ Determine the Q and $\overline{Q}$ output waveforms of the flip-flop in Figure for the D and CLK inputs in Figure. Assume that the positive edge-triggered flip-flop is initially RESET.



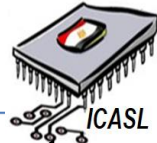
Solution

1. At clock pulse 1, D is LOW, so Q remains LOW (RESET).
2. At clock pulse 2, D is LOW, so Q remains LOW (RESET).
3. At clock pulse 3, D is HIGH, so Q goes HIGH (SET).
4. At clock pulse 4, D is LOW, so Q goes LOW (RESET).
5. At clock pulse 5, D is HIGH, so Q goes HIGH (SET).
6. At clock pulse 6, D is HIGH, so Q remains HIGH (SET).

Outlines

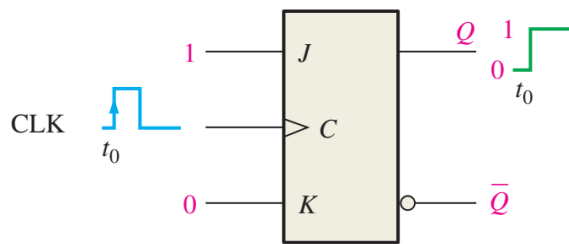


- ☐ Types of digital logic circuits
- ☐ Types of Storage Elements
- ☐ SR-Latch
- ☐ Gated SR-Latch
- ☐ D-Latch
- ☐ D-Flip-Flop
- ☒ JK-Flip-Flop
- ☐ Flip-Flop Applications
- ☐ 555 Timer

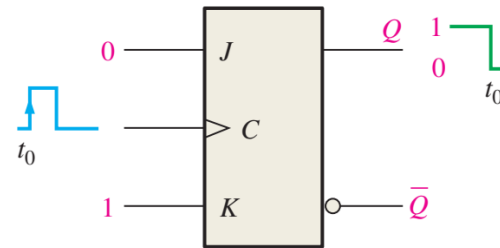


JK-Flip-Flop

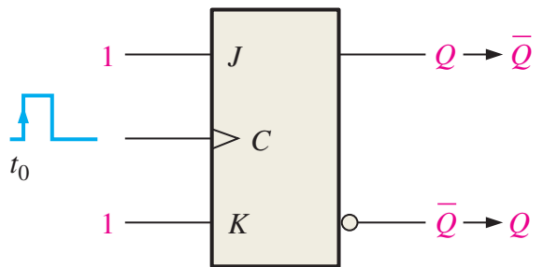
❑ The J and K inputs of the J-K flip-flop are synchronous inputs because data on these inputs are transferred to the flip-flop's output only on the triggering edge of the clock pulse.



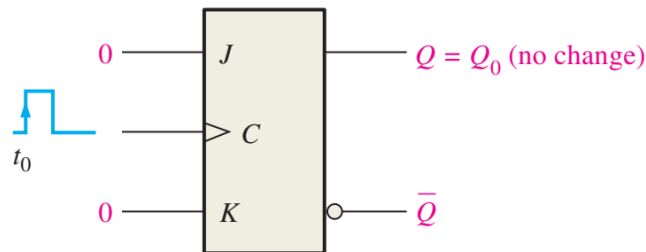
(a) $J = 1, K = 0$ flip-flop SETS on positive clock edge. (If already SET, it remains SET.)



(b) $J = 0, K = 1$ flip-flop RESETS on positive clock edge. (If already RESET, it remains RESET.)



(c) $J = 1, K = 1$ flip-flop changes state (toggle).



(d) $J = 0, K = 0$ flip-flop does not change. (If SET, it remains SET; if RESET, it remains RESET.)

Truth Table of +ve edge-triggered JK -Flip Flop

Inputs			Outputs		Comments
j	k	CLK	Q	$\overline{Q}$	
0	0	↑	Q_0	$\overline{Q_0}$	No change
0	1	↑	0	1	RESET
1	0	↑	1	0	SET
1	1	↑	$\overline{Q_0}$	Q_0	Toggle

↑: Clock transition LOW to HIGH

Outlines



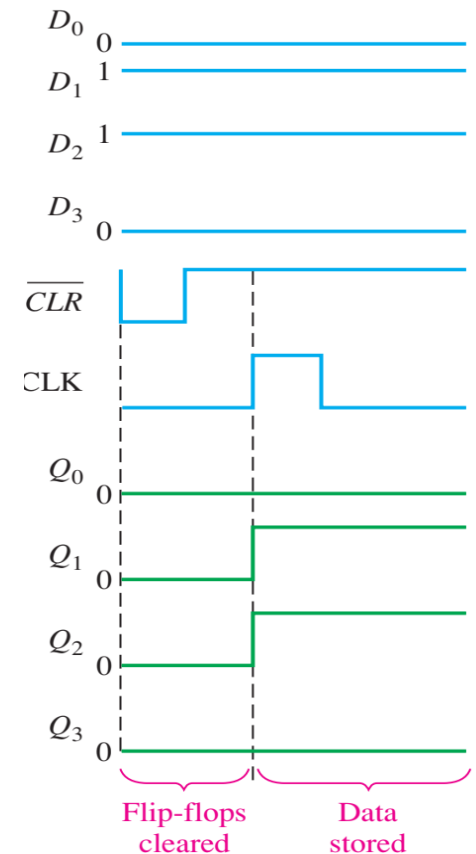
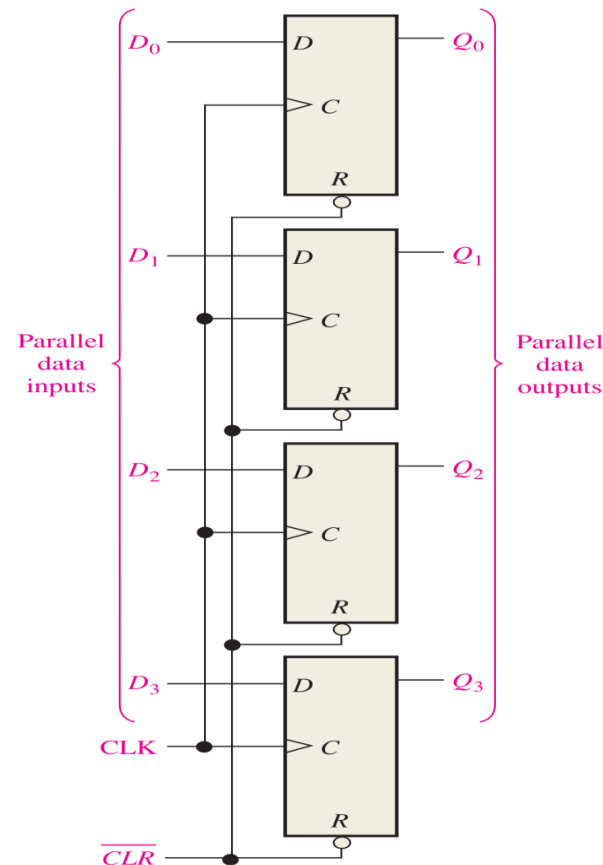
- ☐ Types of digital logic circuits
- ☐ Types of Storage Elements
- ☐ SR-Latch
- ☐ Gated SR-Latch
- ☐ D-Latch
- ☐ D-Flip-Flop
- ☐ JK-Flip-Flop
- ☐ Flip-Flop Applications
- ☐ 555 Timer

Flip-Flop Applications (1/2)

1. Parallel Data Storage

- A common requirement in digital systems is to store several bits of data from parallel lines simultaneously in a group of flip-flops

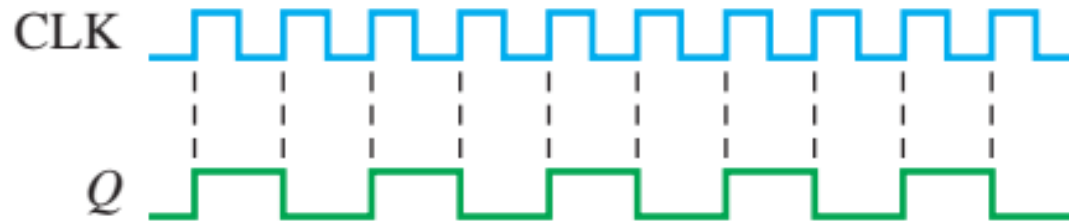
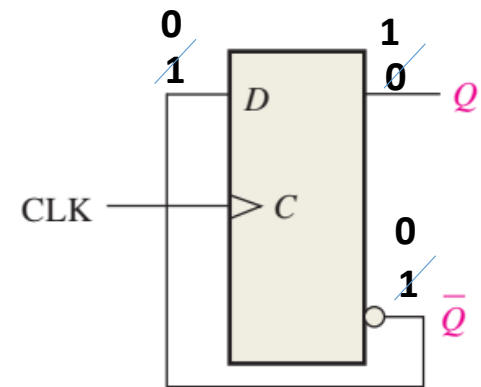
This group of 4 flip-flops is an example of a basic register used for data storage. In digital systems, data are normally stored in groups of bits (usually 8 or multiples thereof) that represent numbers, codes, or other information.



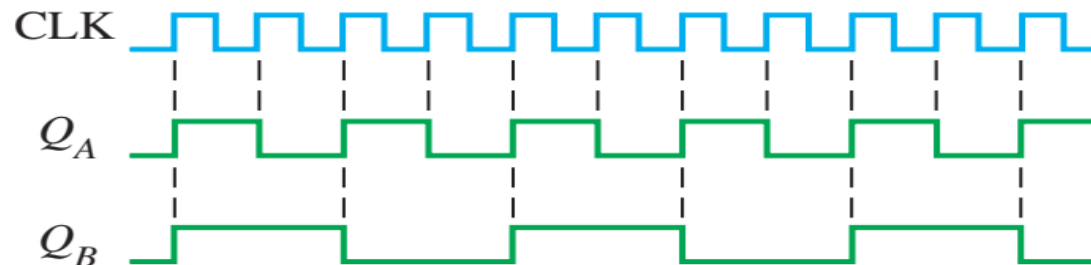
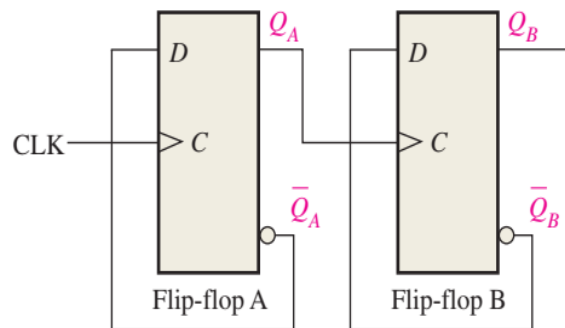
Flip-Flop Applications (2/2)

2. Frequency Division

Another application of a flip-flop is dividing (reducing) the frequency of a periodic waveform



Q is one-half the frequency of CLK



Q_A is one-half and Q_B is one-fourth the frequency of CLK.

By connecting flip-flops in this way, a frequency division of 2^n is achieved, where n is the number of flip-flops. For example, three flip-flops divide the clock frequency by $2^3 = 8$; four flip-flops divide the clock frequency by $2^4 = 16$; and so on.

Outlines



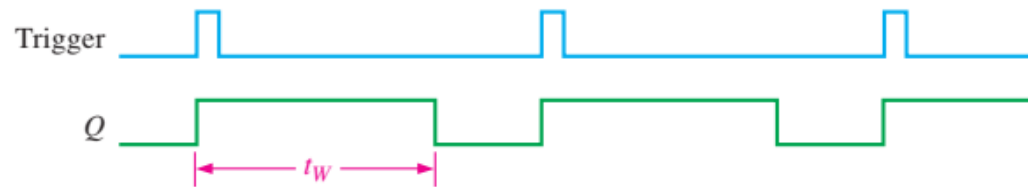
- ☐ Types of digital logic circuits
- ☐ Types of Storage Elements
- ☐ SR-Latch
- ☐ Gated SR-Latch
- ☐ D-Latch
- ☐ D-Flip-Flop
- ☐ JK-Flip-Flop
- ☐ Flip-Flop Applications
- ☐ **555 Timer**

The 555 Timer (1/3)



❑ The 555 **timer** is a versatile and widely used IC device because it can be configured in two different modes

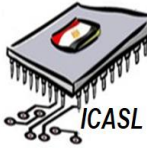
❑ A monostable multivibrator (one-shot).



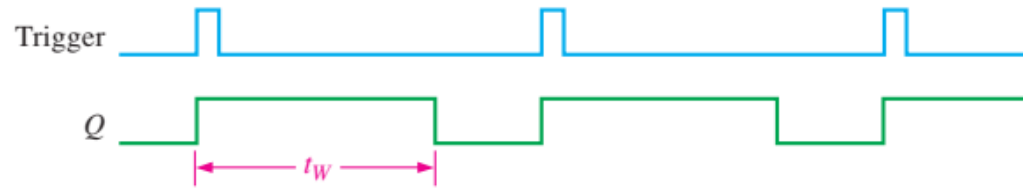
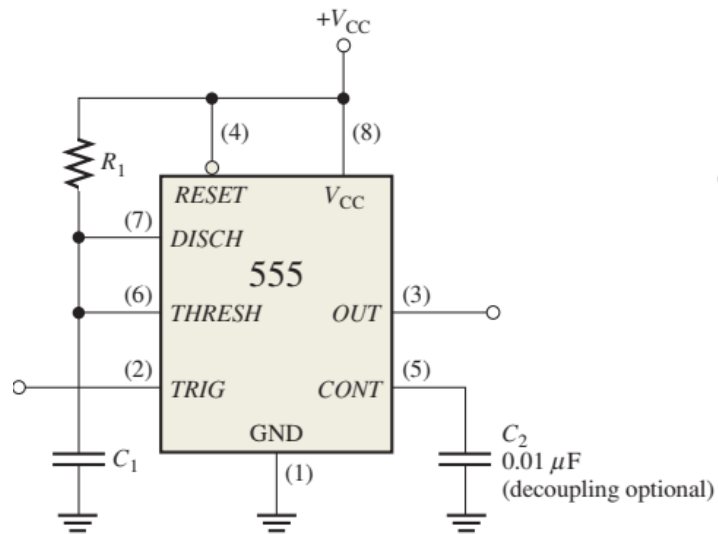
❑ An astable multivibrator (pulse oscillator).



The 555 Timer (2/3)



□ A mono-stable multi-vibrator (one-shot).



$$t_W = 1.1R_1C_1$$

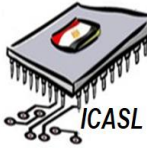
EXAMPLE

What is the output pulse width for a 555 monostable circuit with $R_1 = 2.2 \text{ k}\Omega$ and $C_1 = 0.01 \text{ }\mu\text{F}$?

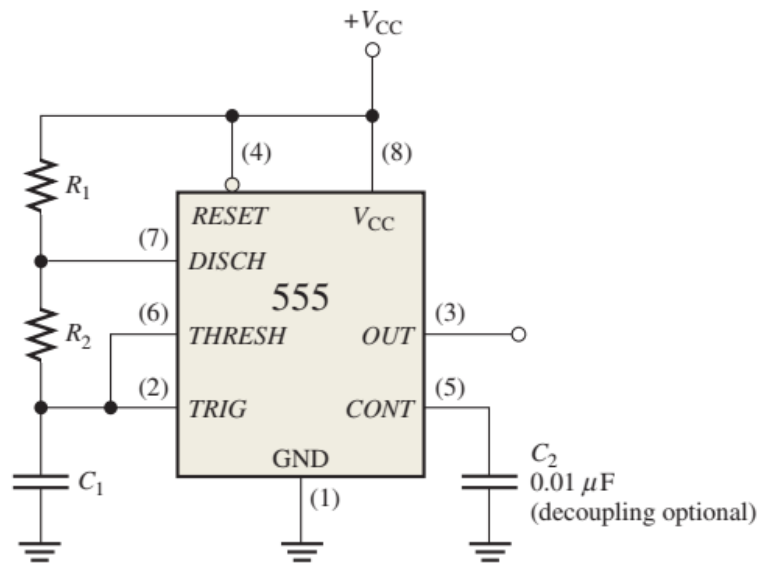
Solution

$$t_W = 1.1R_1C_1 = 1.1(2.2 \text{ k}\Omega)(0.01 \text{ }\mu\text{F}) = 24.2 \text{ }\mu\text{s}$$

The 555 Timer (3/3)



❑ An **astable** multi-vibrator.



$$f = \frac{1.44}{(R_1 + 2R_2)C_1}$$

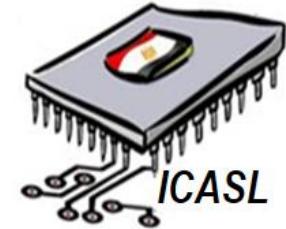
$$\text{Duty cycle} = \left(\frac{R_1}{R_1 + R_2} \right) 100\%$$

EXAMPLE

$$f = \frac{1.44}{(R_1 + 2R_2)C_1} = \frac{1.44}{(2.2 \text{ k}\Omega + 9.4 \text{ k}\Omega)0.022 \text{ }\mu\text{F}} = 5.64 \text{ kHz}$$

$$\text{Duty cycle} = \left(\frac{R_1}{R_1 + R_2} \right) 100\% = \left(\frac{2.2 \text{ k}\Omega}{2.2 \text{ k}\Omega + 9.4 \text{ k}\Omega} \right) 100\% = 59.5\%$$

Thanks!



Digital Design (SET111)

L12

FSM and Counter

Dr. Ahmed Reda Mohamed¹

¹PhD, the MoE Key Laboratory of Artificial Intelligence & Bio-Circuits and Systems Laboratory (**BiCASL**), Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.

Outlines



- ☐ **Finite State Machine**
- ☐ **Asynchronous Counter**
- ☐ **Synchronous Counter**
- ☐ **Design of Synchronous Counters**
- ☐ **Exersice_01**
- ☐ **Exersice_02**
- ☐ **Exersice_03**
- ☐ **Exersice_04**

Outlines

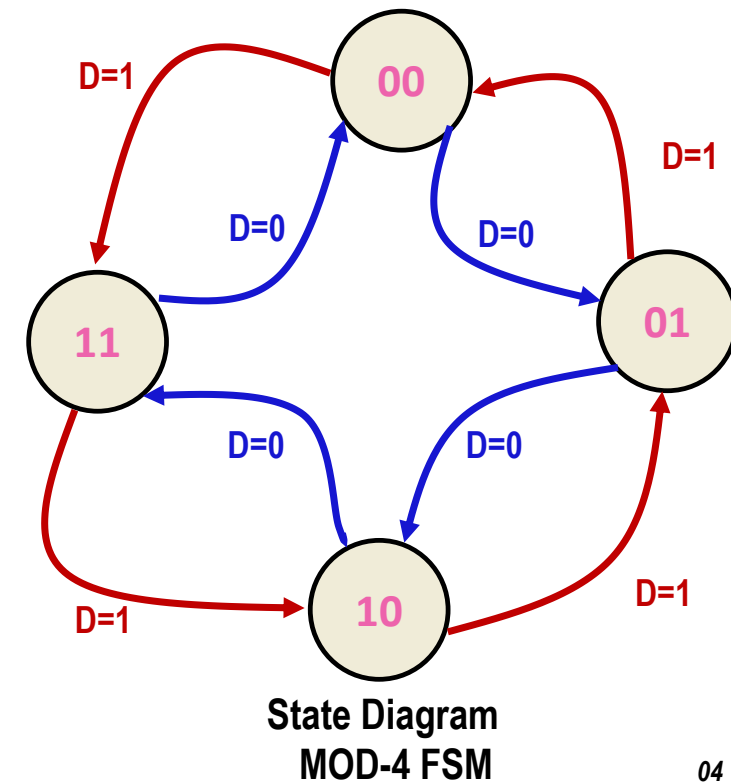
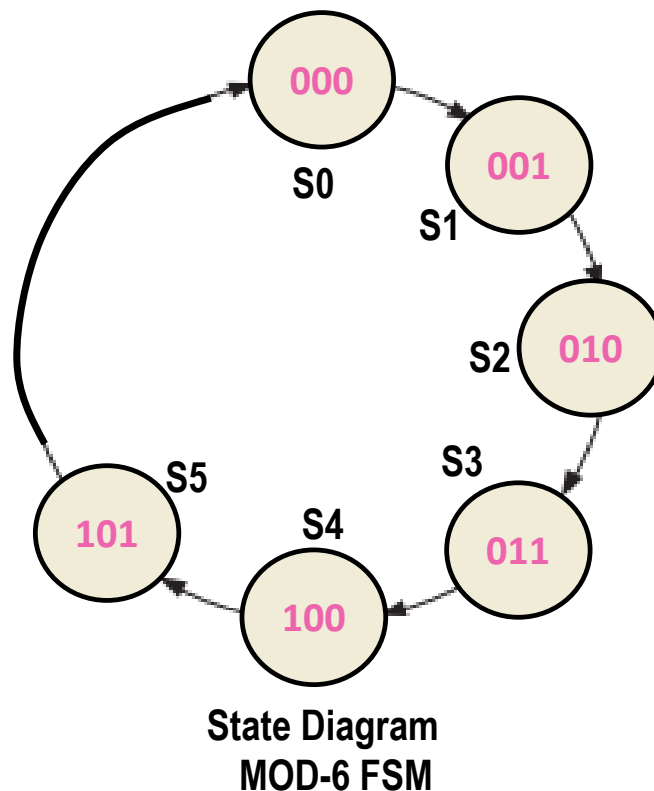
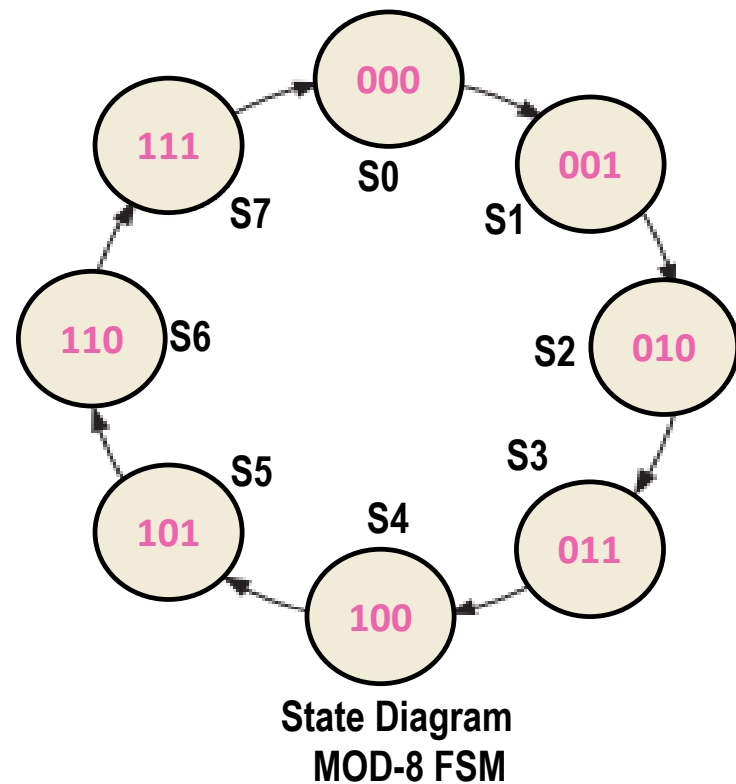


- ☐ **Finite State Machine**
- ☐ **Asynchronous Counter**
- ☐ **Synchronous Counter**
- ☐ **Design of Synchronous Counters**
- ☐ **Exersice_01**
- ☐ **Exersice_02**
- ☐ **Exersice_03**
- ☐ **Exersice_04**



Finite State Machine

- ❑ A **state machine** is a sequential circuit having a limited (**finite**) number of states occurring in a prescribed **order**, so it is called Finite State Machine (**FSM**).
- ❑ A **counter** is an example of a state machine, the number of states is called the **modulus**
- ❑ Two basic types of counters are the **Asynchronous** and **Synchronous** counter.



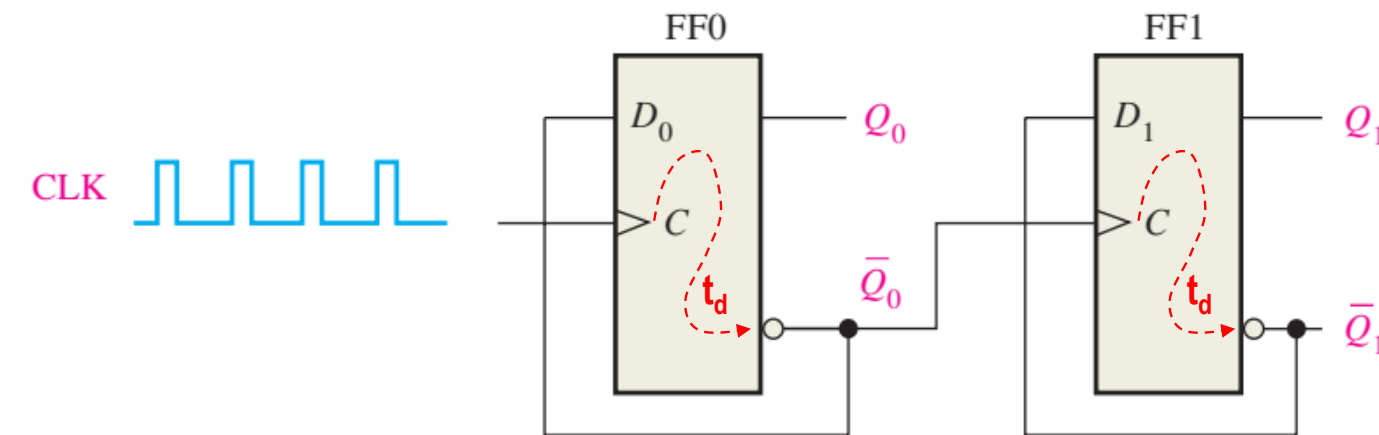
Outlines



- ☐ Finite State Machine
- ☒ Asynchronous Counter
- ☐ Synchronous Counter
- ☐ Design of Synchronous Counters
- ☐ Exercise_01
- ☐ Exercise_02
- ☐ Exercise_03
- ☐ Exercise_04

Asynchronous Counter

- ❑ The term **asynchronous** refers to events that **do not occur at the same time**.
- ❑ An **asynchronous counter** is one in which the flip-flops (FF) within the counter **do not change states** at exactly the same time because they **do not have a common clock pulse**.



A 2-Bit Asynchronous Binary Counter

The clock input of an asynchronous counter is always connected only to the LSB flip-flop

- ❑ Notice that the clock (CLK) is applied to the clock input (C) of **only the first flip-flop**, FF0, which is always the least significant bit (LSB). The second flip-flop, FF1, is triggered by the $\overline{Q_0}$ output of FF0.
- ❑ Because of the inherent propagation delay time (t_d) through a flip-flop, a transition of the input clock pulse (CLK) and a transition of the $\overline{Q_0}$ output of FF0 can never occur at exactly the same time.

Outlines

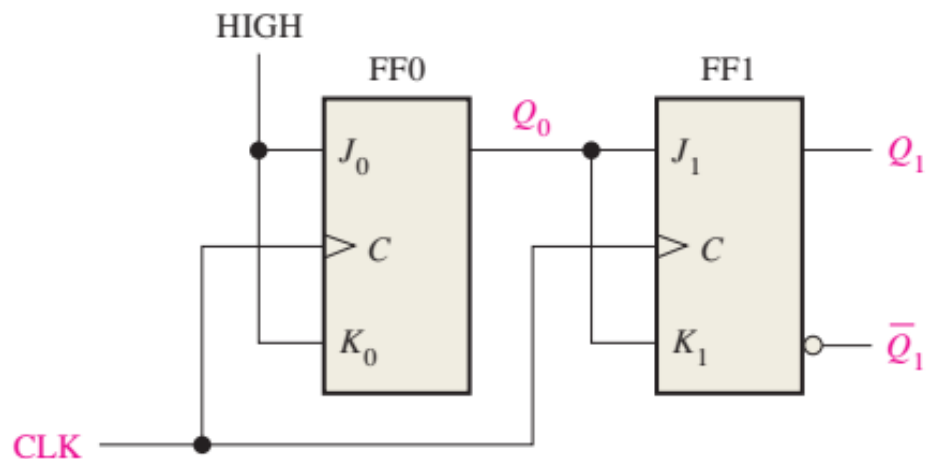


- ☐ Finite State Machine
- ☐ Asynchronous Counter
- ☒ Synchronous Counter
- ☐ Design of Synchronous Counters
- ☐ Exercice_01
- ☐ Exercice_02
- ☐ Exercice_03

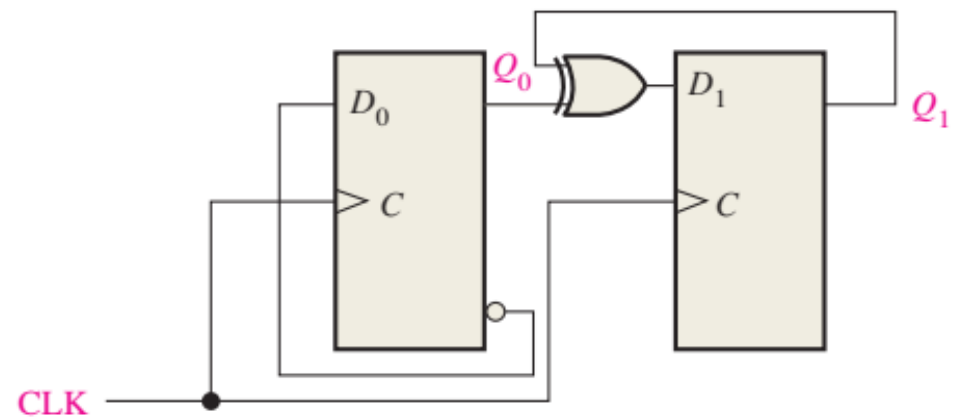
Synchronous Counter (1/4)



- ❑ The term **synchronous** refers to events that **occurs at the same time**.
- ❑ An **synchronous counter** is one in which the flip-flops (FF) within the counter changes states at exactly the same time because all the flip-flops in the counter are clocked by a **common clock pulse**.



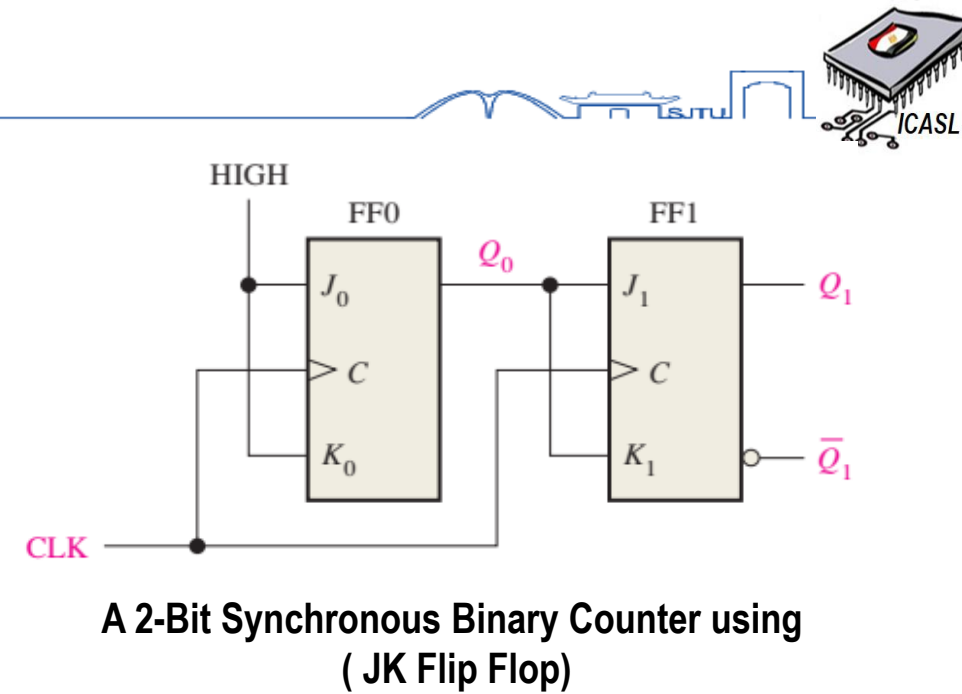
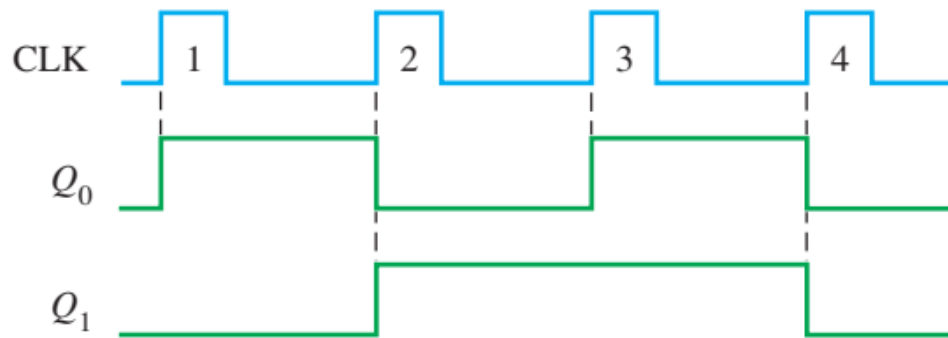
A 2-Bit Synchronous Binary Counter using
(JK Flip Flop)



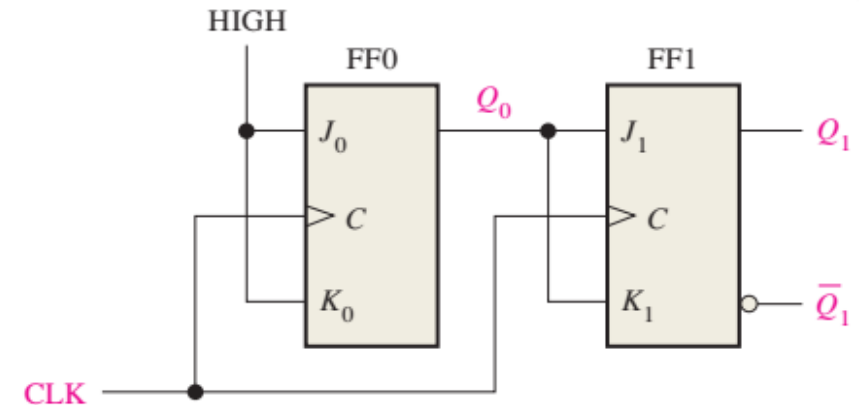
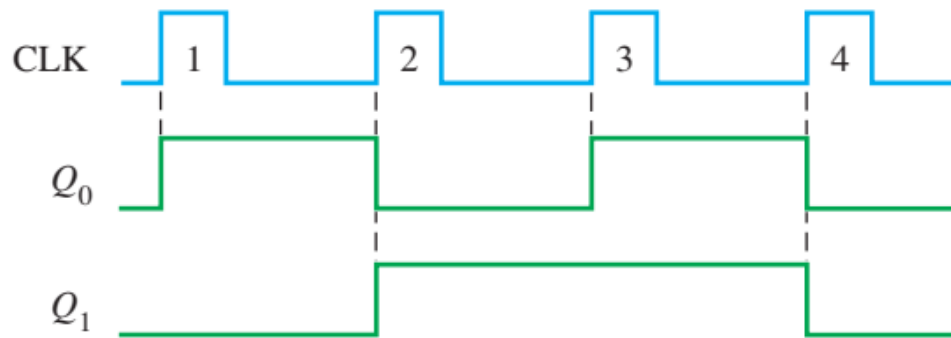
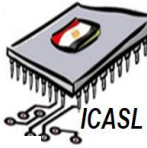
A 2-Bit Synchronous Binary Counter using
(D-Flip Flop)

- ❑ J-K flip-flops are used to illustrate most synchronous counters.
- ❑ D flip-flops can also be used but generally require **more logic** because of having no direct toggle or no-change states.

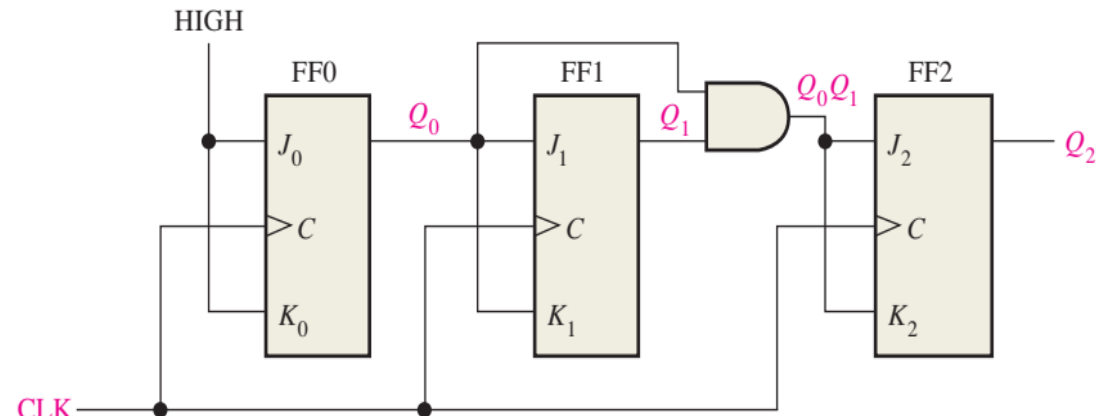
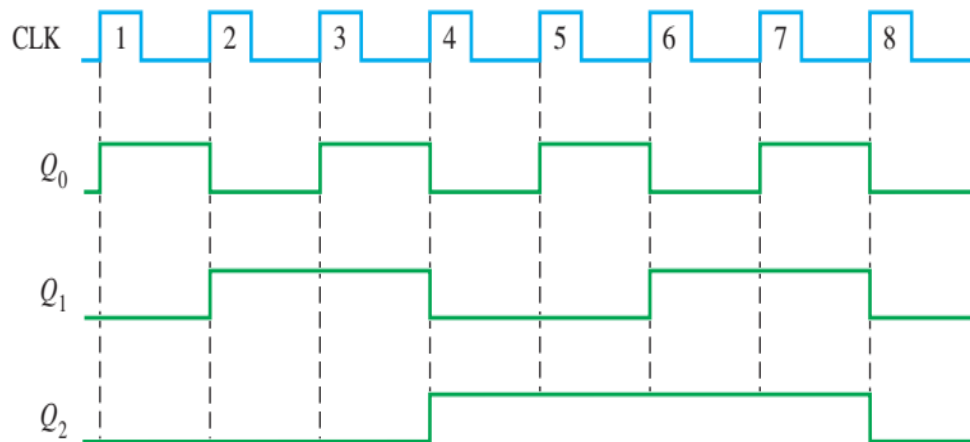
Synchronous Counter (2/4)



Synchronous Counter (3/4)

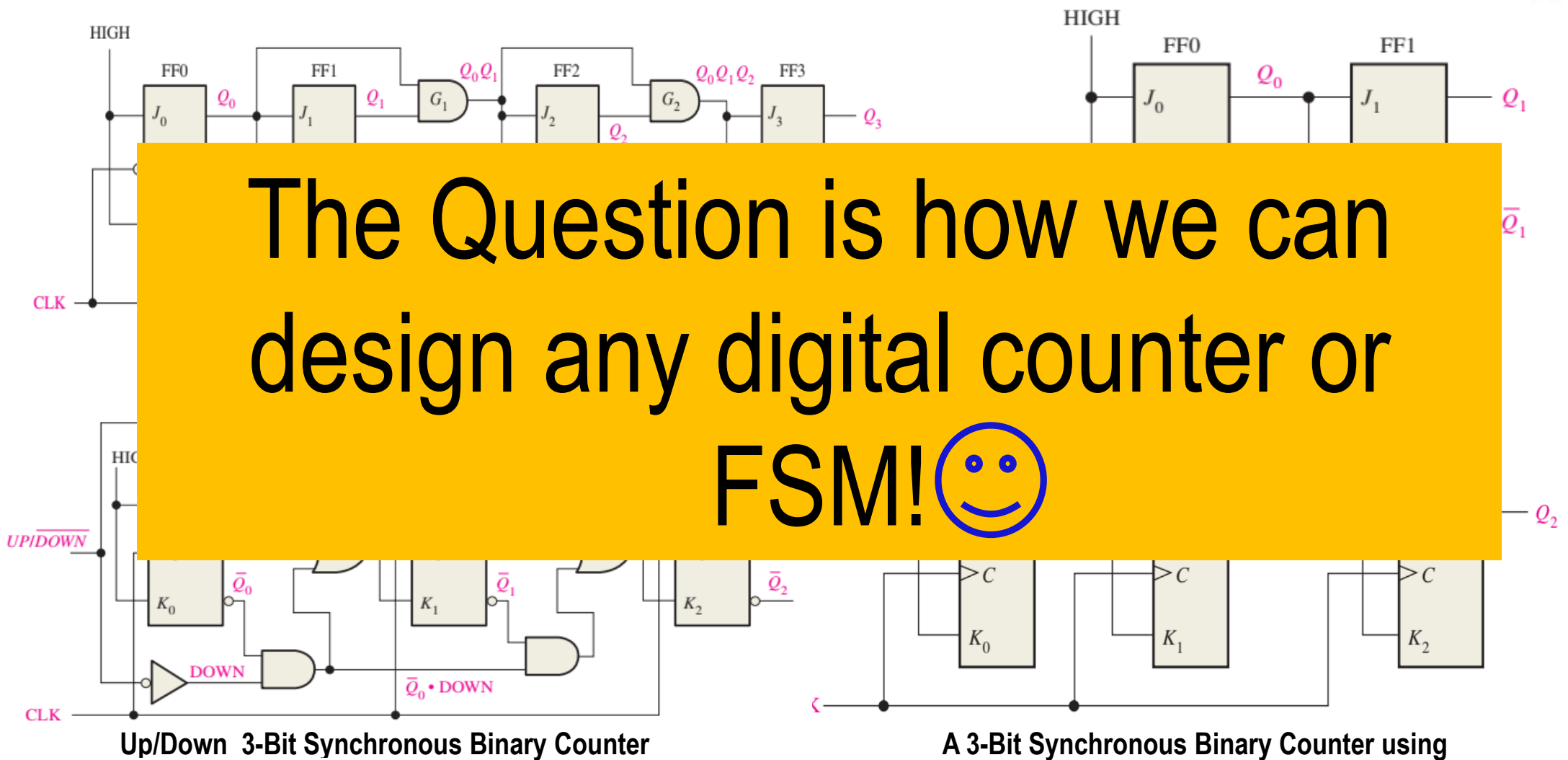
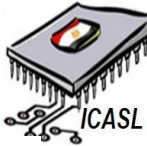


A 2-Bit Synchronous Binary Counter using (JK Flip Flop)



A 3-Bit Synchronous Binary Counter using (JK Flip Flop)

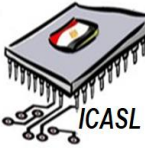
Synchronous Counter (4/4)



Outlines



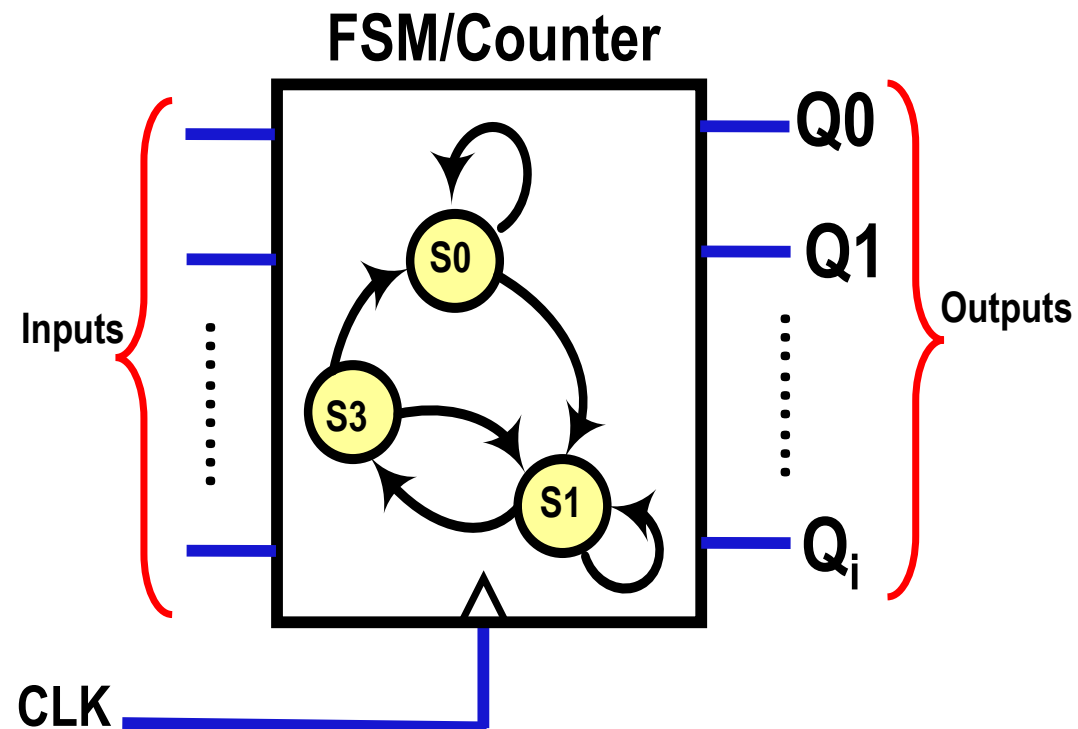
- ☐ Finite State Machine
- ☐ Asynchronous Counter
- ☐ Synchronous Counter
- ☒ Design of Synchronous Counters
- ☐ Exercice_01
- ☐ Exercice_02
- ☐ Exercice_03
- ☐ Exercice_04



Design of Synchronous Counters

❑ You have to execute **Five** steps to design a **counter** Or even any finite state machine (**FSM**).

- ❑ **Step 1:** State Diagram
- ❑ **Step 2:** Next-State Table
- ❑ **Step 3:** Flip-Flop Transition Table
- ❑ **Step 4:** K-Maps
- ❑ **Step 5:** Counter Implementation



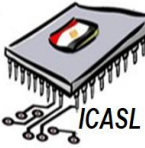
Outlines



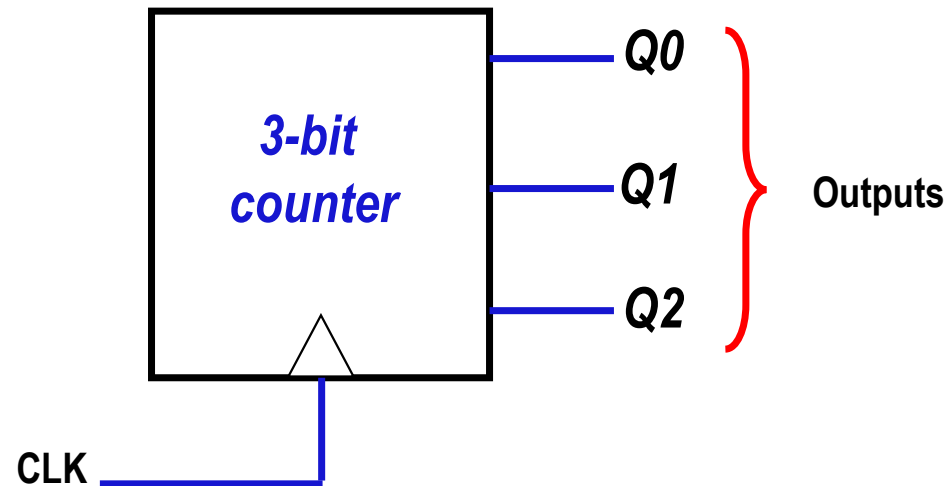
- ☐ Finite State Machine
- ☐ Asynchronous Counter
- ☐ Synchronous Counter
- ☐ Design of Synchronous Counters
- ☒ Exercise_01
- ☐ Exercise_02
- ☐ Exercise_03
- ☐ Exercise_04

Exercise_01

□ Design a 3-bit **synchronous** up counter using JK Flip-Flop.



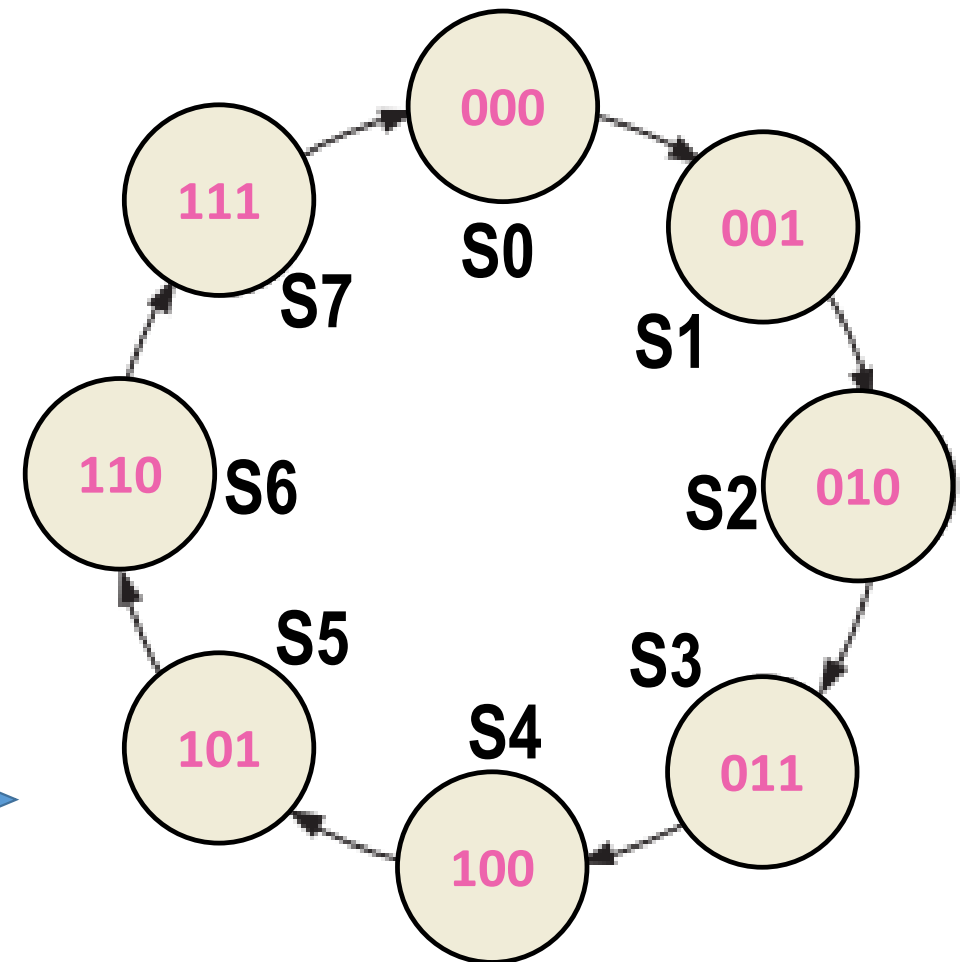
Ans.



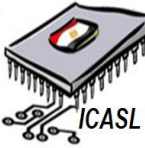
Exercise_01

□ Step 1: State Diagram

3-bit synchronous $\rightarrow 2^3 = 8$ states
Up counter $\rightarrow \{0, 1, 2, 3, 4, 5, 6, 7\}$



State Diagram
3-bit counter



Exercise_01



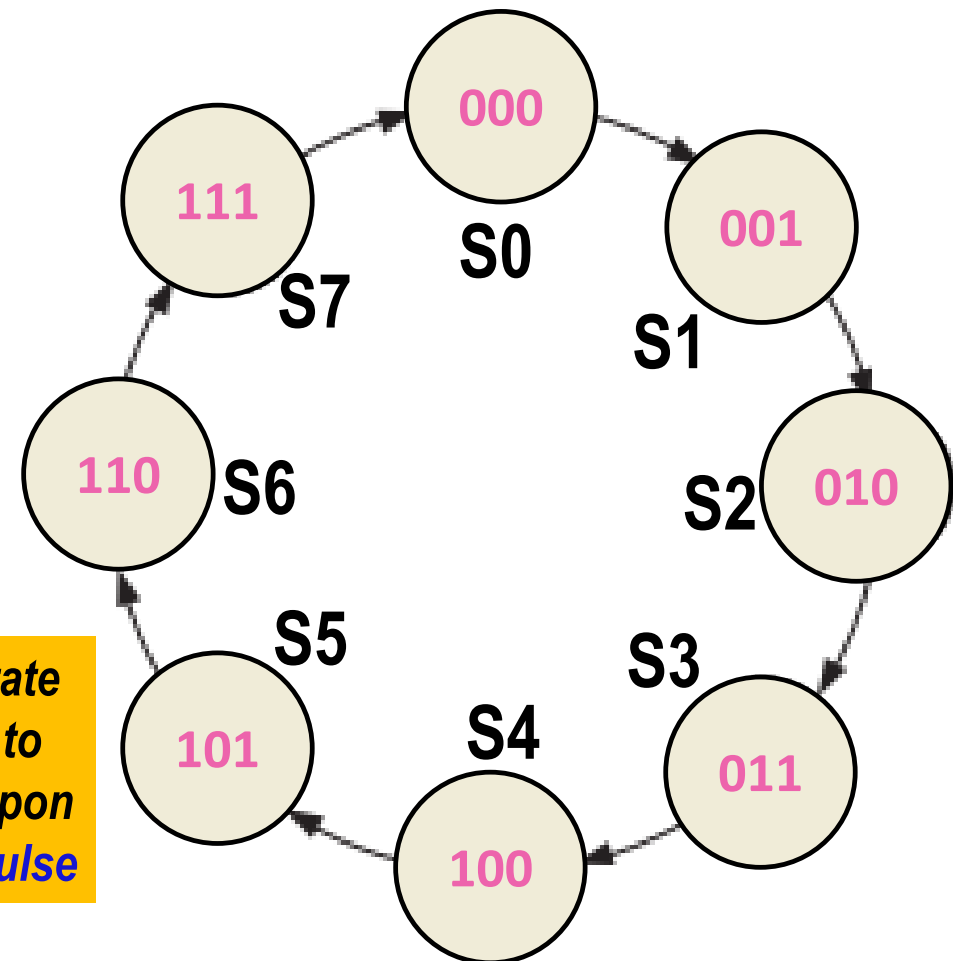
Step 2: Next-State Table

- Once the sequential circuit is defined by a state diagram, the **second step** is to derive a next-state table

Next-state table for 3-bit counter

Present State			Next State		
Q2	Q1	Q0	Q2	Q1	Q0

The **next state** is the state that the counter goes to from its present state upon application of a **clock pulse**



Exercise_01



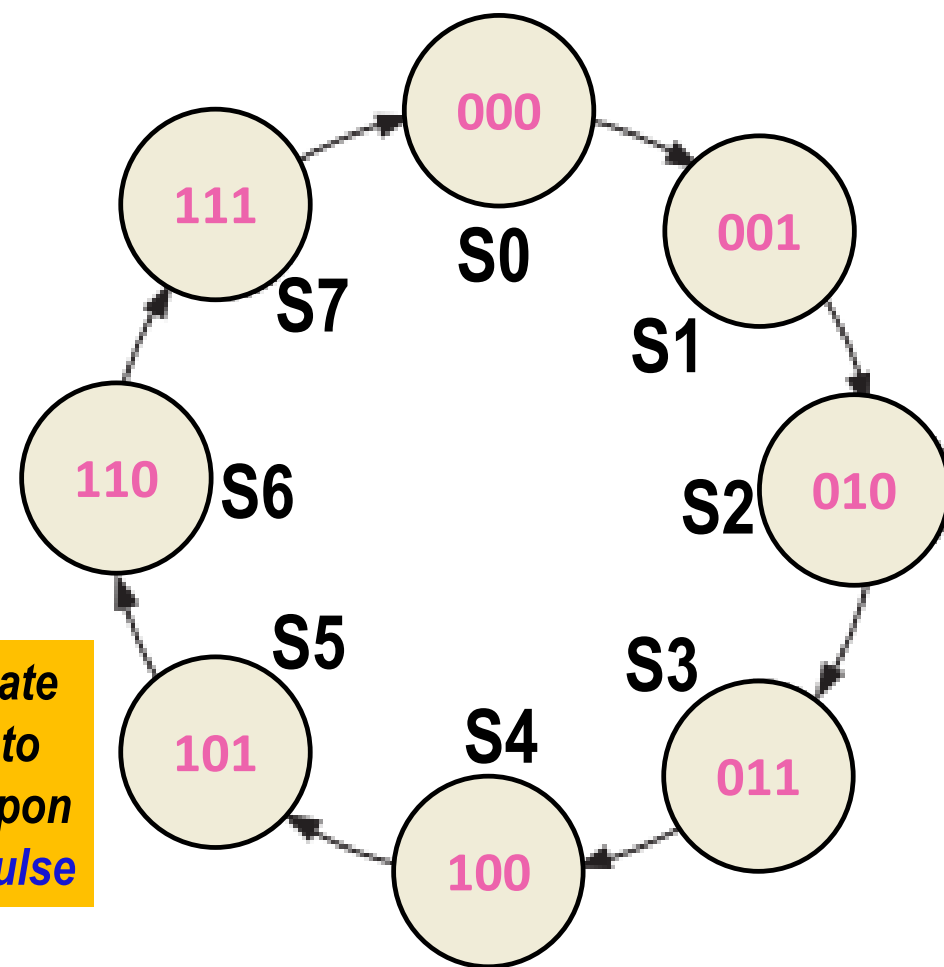
Step 2: Next-State Table

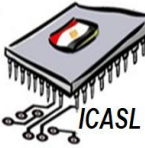
- Once the sequential circuit is defined by a state diagram, the **second step** is to derive a next-state table

Next-state table for 3-bit counter

Present State			Next State		
Q2	Q1	Q0	Q2	Q1	Q0
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	0

The **next state** is the state that the counter goes to from its present state upon application of a **clock pulse**





Exercise_01

Step 3: Flip-Flop Transition Table

- For each **output transition**, the J and K **inputs** that will cause the output transition to occur are always the same and defined in table.

Next-state table for 3-bit counter

Present State			Next State		
Q2	Q1	Q0	Q2	Q1	Q0
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	0

Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q _N		Q _{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care

Exercise_01



Step 3: Flip-Flop Transition Table

of JK Flip-Flop = # of output of the counter. Here, # of JK Flip-Flop=3.

Next-state table for 3-bit counter

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1						
0	0	1	0	1	0						
0	1	0	0	1	1						
0	1	1	1	0	0						
1	0	0	1	0	1						
1	0	1	1	1	0						
1	1	0	1	1	1						
1	1	1	0	0	0						

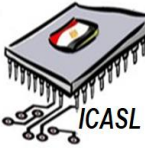
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_01

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table for 3-bit counter

Present State		Next State		Inputs of the JK-Flip Flop		
	Q0		Q0		J0	K0
	0		1			
	1		0			
	0		1			
	1		0			
	0		1			
	1		0			
	0		1			
	1		0			

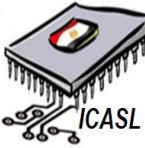
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_01

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table for 3-bit counter

Present State		Next State		Inputs of the JK-Flip Flop		
	Q0		Q0		J0	K0
	0		1		1	X
	1		0		X	1
	0		1		1	X
	1		0		X	1
	0		1		1	X
	1		0		X	1
	0		1		1	X
	1		0		X	1

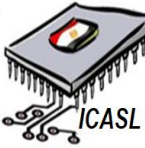
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_01

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table for 3-bit counter

Present State			Next State			Inputs of the JK-Flip Flop		
	Q1			Q1		J1	K1	
	0			0				
	0			1				
	1			1				
	1			0				
	0			0				
	0			1				
	1			1				
	1			0				

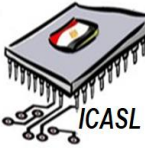
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_01

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table for 3-bit counter

Present State			Next State			Inputs of the JK-Flip Flop		
	Q1			Q1			J1	K1
	0			0			0	X
	0			1			1	X
	1			1			X	0
	1			0			X	1
	0			0			0	X
	0			1			1	X
	1			1			X	0
	1			0			X	1

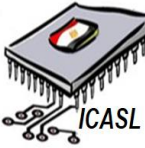
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_01

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table for 3-bit counter

Present State		Next State		Inputs of the JK-Flip Flop	
Q2		Q2		J2	K2
0		0			
0		0			
0		0			
0		1			
1		1			
1		1			
1		1			
1		0			

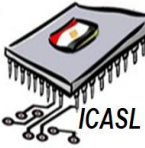
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_01

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table for 3-bit counter

Present State		Next State		Inputs of the JK-Flip Flop	
Q2		Q2		J2	K2
0		0		0	X
0		0		0	X
0		0		0	X
0		1		1	X
1		1		X	0
1		1		X	0
1		1		X	0
1		0		X	1

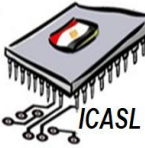
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_01

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop



Next-state table for 3-bit counter

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	1	1	1	X	0	X	0	1	X
1	1	1	0	0	0	X	1	X	1	X	1

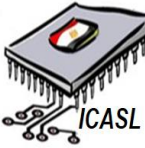
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q _N		Q _{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



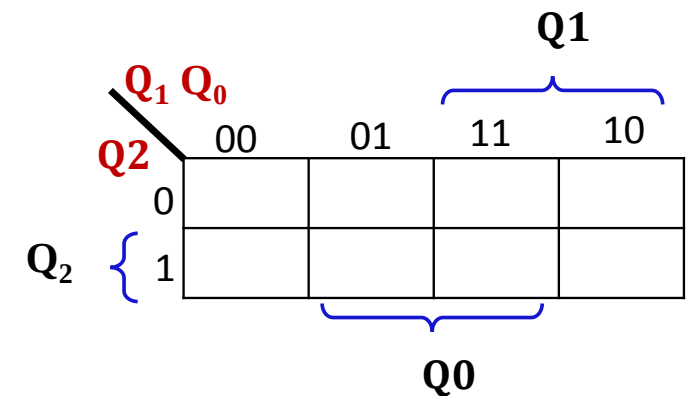
Exercise_01

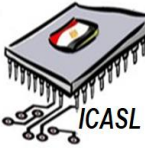
Step 4: k-Maps

K-maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter

Next-state table for 3-bit counter

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	1	1	1	X	0	X	0	1	X
1	1	1	0	0	0	X	1	X	1	X	1





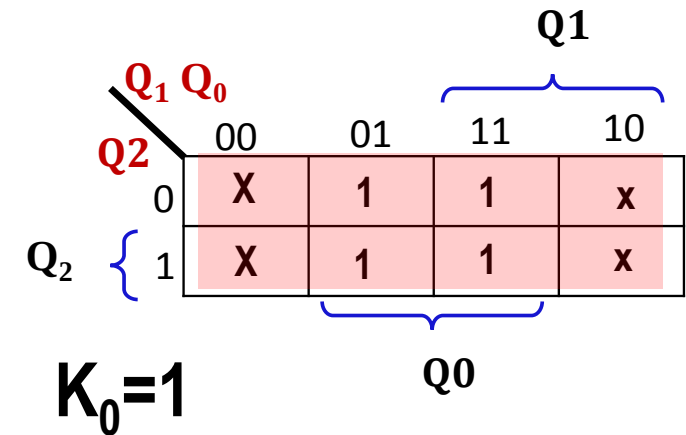
Exercise_01

Step 4: k-Maps

K-maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter

Next-state table for 3-bit counter

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	1	1	1	X	0	X	0	1	X
1	1	1	0	0	0	X	1	X	1	X	1



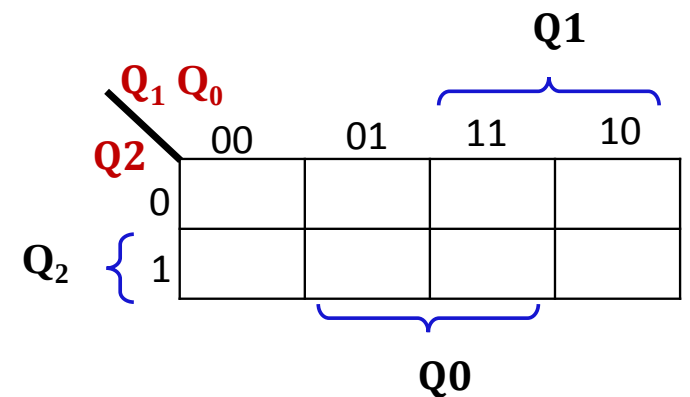
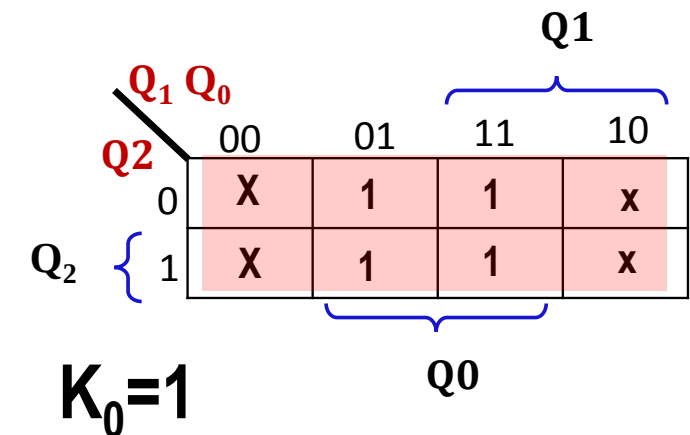
Exercise_01

Step 4: k-Maps

K-maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter

Next-state table for 3-bit counter

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	1	1	1	X	0	X	0	1	X
1	1	1	0	0	0	X	1	X	1	X	1



Exercise_01

Step 4: k-Maps

K-maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter

Next-state table for 3-bit counter

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	1	1	1	X	0	X	0	1	X
1	1	1	0	0	0	X	1	X	1	X	1

K₀=1

		Q ₁ Q ₀			
		00	01	11	10
Q ₂	0	X	1	1	x
	1	X	1	1	x

J₀=1

		Q ₁ Q ₀			
		00	01	11	10
Q ₂	0	1	X	X	1
	1	1	X	X	1

Exercise_01

Step 4: k-Maps

K-maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter

Next-state table for 3-bit counter

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	1	1	1	X	0	X	0	1	X
1	1	1	0	0	0	X	1	X	1	X	1

K₁ = Q₀

		Q ₁			
		00	01	11	10
Q ₂	0	X	X	1	0
	1	X	X	1	0

J₁ = Q₀

		Q ₁			
		00	01	11	10
Q ₂	0	0	1	X	X
	1	0	1	X	X

Exercise_01

Step 4: k-Maps

K-maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter

Next-state table for 3-bit counter

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	1	1	1	X	0	X	0	1	X
1	1	1	0	0	0	X	1	X	1	X	1

K₂ = Q₀ · Q₁

		Q ₁			
		00	01	11	10
Q ₂	0	X	X	X	X
	1	0	0	1	0

J₂ = Q₀ · Q₁

		Q ₁			
		00	01	11	10
Q ₂	0	0	0	1	0
	1	x	x	X	X

Exercise_01



Step 5: Counter implementation

The final step is to implement the combinational logic from the expressions for the J and K inputs

$$J_0=1$$

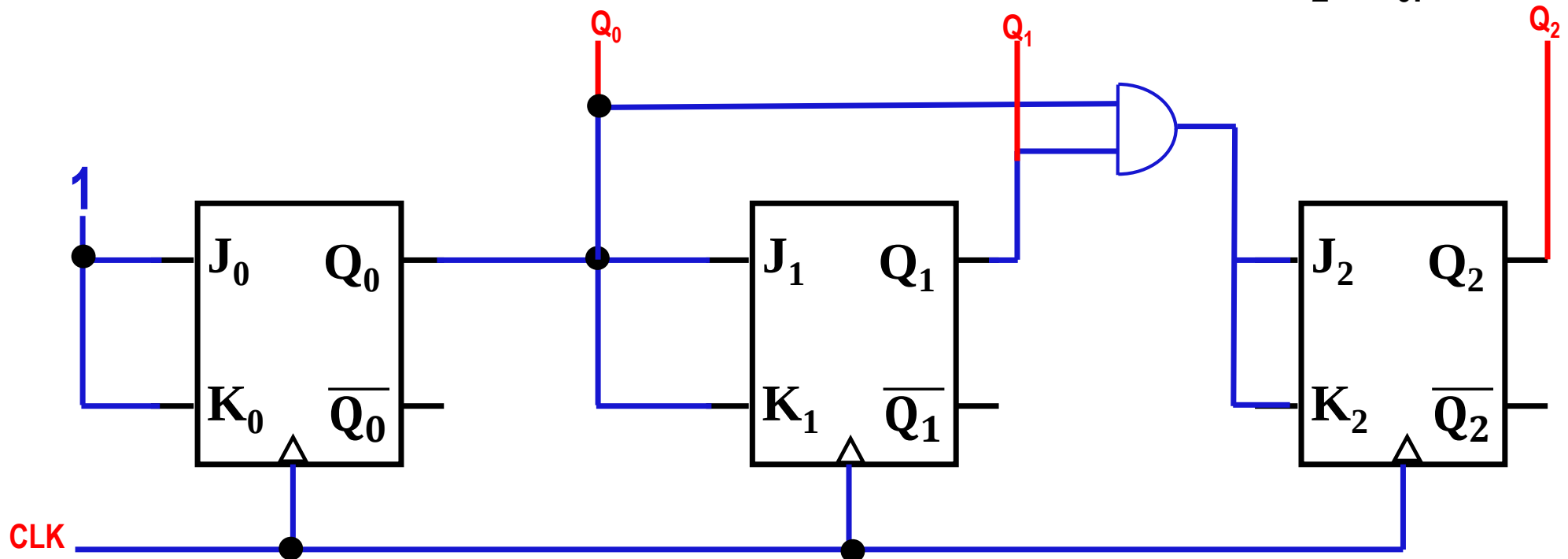
$$K_0=1$$

$$J_1=Q_0$$

$$K_1=Q_0$$

$$J_2=Q_0 \cdot Q_1$$

$$K_2=Q_0 \cdot Q_1$$



Outlines



- ☐ Finite State Machine
- ☐ Asynchronous Counter
- ☐ Synchronous Counter
- ☐ Design of Synchronous Counters
- ☐ Exercise_01
- ☒ Exercise_02
- ☐ Exercise_03
- ☐ Exercise_04

Exercise_02

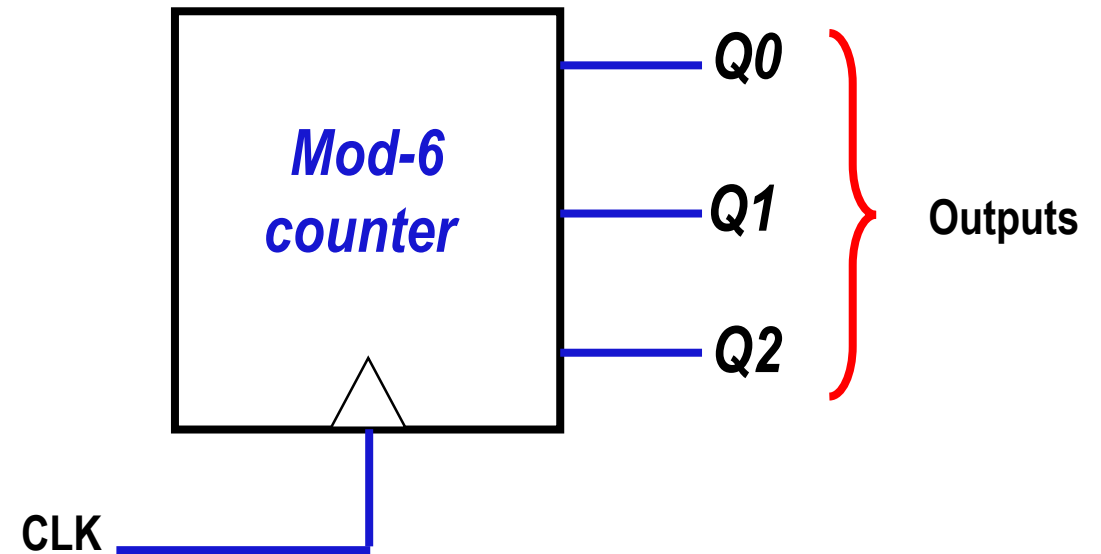


□ Design a **Mod-6** synchronous up Counter using JK flip flops.

Ans.

Mod-6 synchronous → 6 states

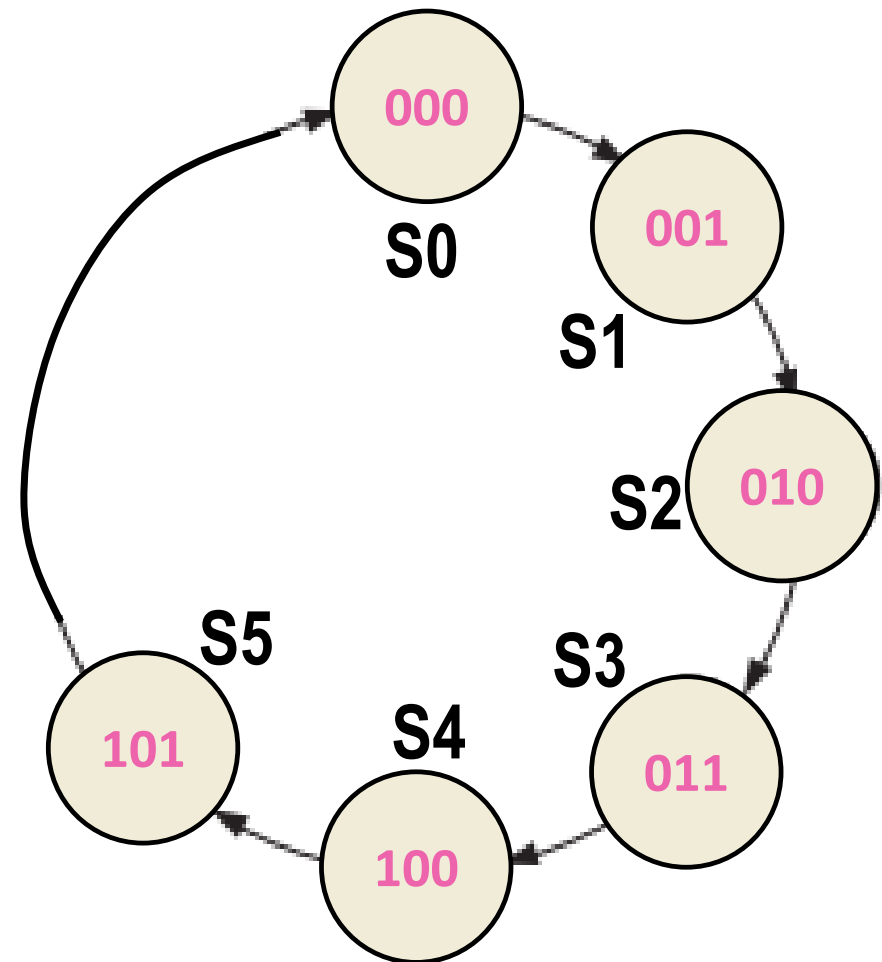
Mod-6 synchronous → needs 3 outputs



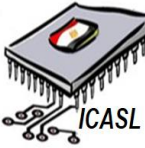
Exercise_02

Step 1: State Diagram

Mod-6 synchronous $\rightarrow$ 6 states
Up counter $\rightarrow \{0,1,2,3,4,5\}$



State Diagram
Mod-6 counter



Exercise_02



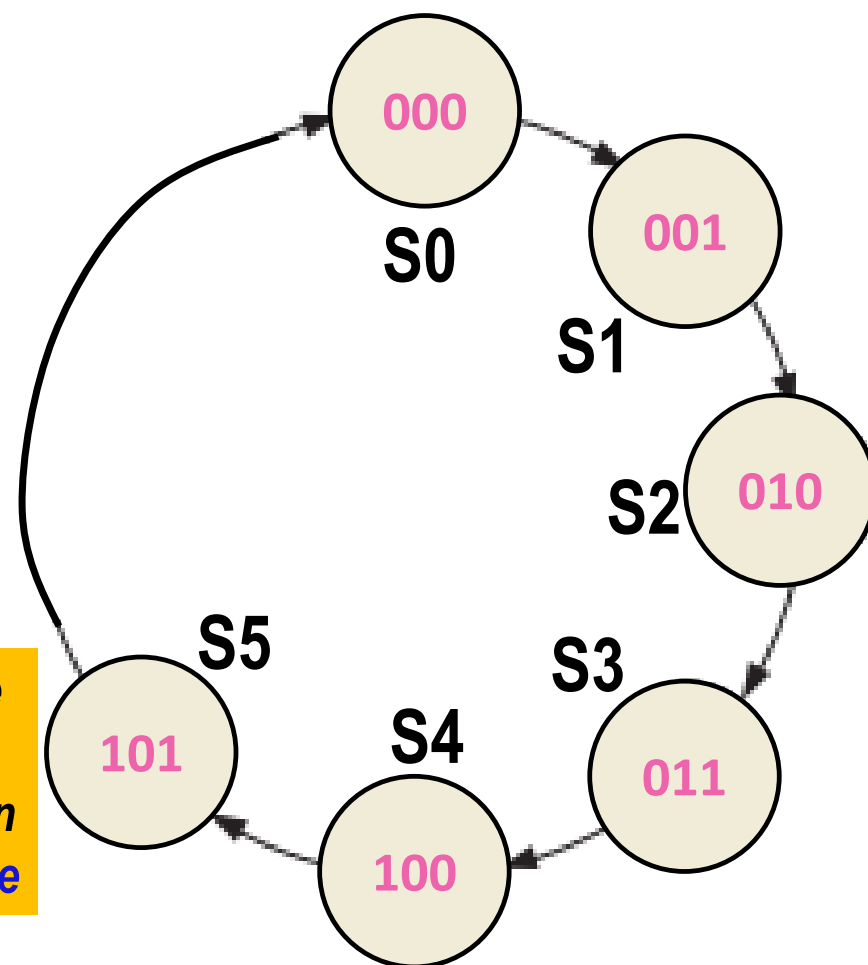
Step 2: Next-State Table

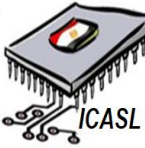
- Once the sequential circuit is defined by a state diagram, the **second step** is to derive a next-state table

Next-state table for Mod-6 counter

Present State			Next State		
Q2	Q1	Q0	Q2	Q1	Q0

The **next state** is the state that the counter goes to from its present state upon application of a **clock pulse**





Exercise_02

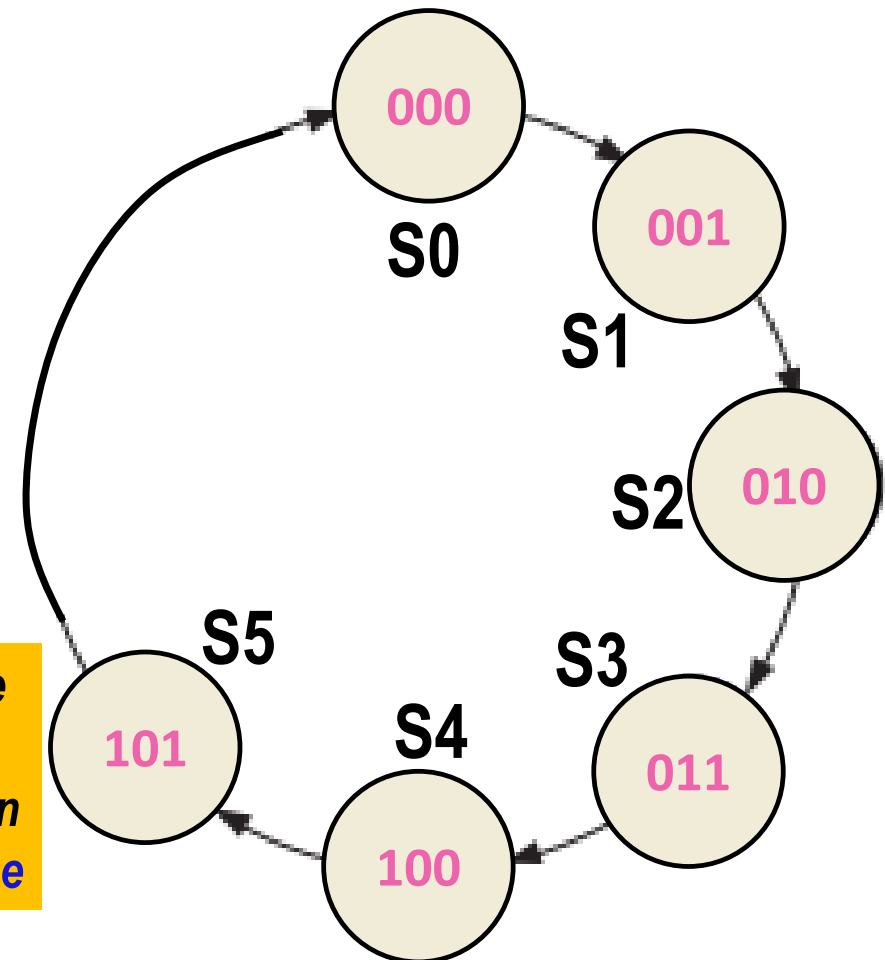
Step 2: Next-State Table

Once the sequential circuit is defined by a state diagram, the **second step** is to derive a next-state table

Next-state table for Mod-6 counter

Present State			Next State		
Q2	Q1	Q0	Q2	Q1	Q0
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	X	X	X
1	1	1	X	X	X

The **next state** is the state that the counter goes to from its present state upon application of a **clock pulse**



Exercise_02



Step 3: Flip-Flop Transition Table

- For each **output transition**, the J and K **inputs** that will cause the output transition to occur are always the same and defined in table.

Present State			Next State		
Q2	Q1	Q0	Q2	Q1	Q0
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	X	X	X
1	1	1	X	X	X

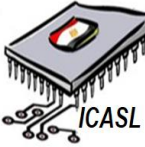
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q _N		Q _{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_02

Step 3: Flip-Flop Transition Table

of JK Flip-Flop = # of output of the counter. Here, # of JK Flip-Flop=3.

Next-state table for Mod-6 counter

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1						
0	0	1	0	1	0						
0	1	0	0	1	1						
0	1	1	1	0	0						
1	0	0	1	0	1						
1	0	1	0	0	0						
1	1	0	X	X	X						
1	1	1	X	X	X						

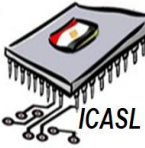
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q _N		Q _{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_02

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table for Mod-6 counter

Present State		Next State		Inputs of the JK-Flip Flop		
	Q0		Q0		J0	K0
	0		1			
	1		0			
	0		1			
	1		0			
	0		1			
	1		0			
	0		X			
	1		X			

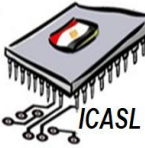
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_02

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table for Mod-6 counter

Present State		Next State		Inputs of the JK-Flip Flop		
	Q0		Q0		J0	K0
	0		1		1	X
	1		0		X	1
	0		1		1	X
	1		0		X	1
	0		1		1	X
	1		0		X	1
	0		X		X	X
	1		X		X	X

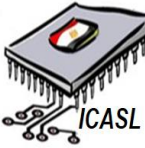
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q _N		Q _{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_02

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table for Mod-6 counter

Present State		Next State		Inputs of the JK-Flip Flop			
	Q1		Q1			J1	K1
	0		0				
	0		1				
	1		1				
	1		0				
	0		0				
	0		0				
	1		X				
	1		X				

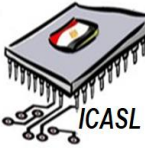
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_02

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table for Mod-6 counter

Present State		Next State		Inputs of the JK-Flip Flop	
Q1	Q1	J1	K1		
0	0	0	X		
0	1	1	X		
1	1	X	0		
1	0	X	1		
0	0	0	X		
0	0	0	X		
1	X	X	X		
1	X	X	X		

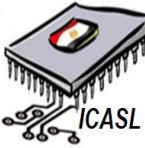
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_02

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table for Mod-6 counter

Present State			Next State		Inputs of the JK-Flip Flop		
Q2			Q2		J2	K2	
0			0				
0			0				
0			0				
0			1				
1			1				
1			0				
1			X				
1			X				

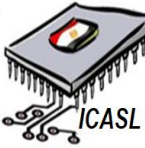
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_02

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table for Mod-6 counter

Present State			Next State			Inputs of the JK-Flip Flop		
Q2			Q2			J2	K2	
0			0			0	X	
0			0			0	X	
0			0			0	X	
0			1			1	X	
1			1			X	0	
1			0			X	1	
1			X			X	X	
1			X			X	X	

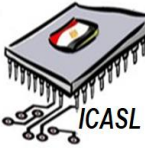
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q _N		Q _{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_02

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table for Mod-6 counter

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	0	0	0	X	1	0	X	X	1
1	1	0	X	X	X	X	X	X	X	X	X
1	1	1	X	X	X	X	X	X	X	X	X

Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q _N		Q _{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care

Exercise_02

Step 4: k-Maps

K-maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter

Next-state table for Mod-6 counter

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	0	0	0	X	1	0	X	X	1
1	1	0	X	X	X	X	X	X	X	X	X
1	1	1	X	X	X	X	X	X	X	X	X

K₀=1

		Q1			
		00	01	11	10
Q2	0	X	1	1	x
	1	X	1	X	x

J₀=1

		Q1			
		00	01	11	10
Q2	0	1	X	X	1
	1	1	X	X	X

Exercise_02

Step 4: k-Maps

K-maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter

Next-state table for Mod-6 counter

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	0	0	0	X	1	0	X	X	1
1	1	0	X	X	X	X	X	X	X	X	X
1	1	1	X	X	X	X	X	X	X	X	X

K₀=1

		Q1			
		00	01	11	10
Q2	0	X	1	1	x
	1	X	1	X	x

J₀=1

		Q1			
		00	01	11	10
Q2	0	1	X	X	1
	1	1	X	X	X

Exercise_02

Step 4: k-Maps

K-maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter

Next-state table for Mod-6 counter

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	0	0	0	X	1	0	X	X	1
1	1	0	X	X	X	X	X	X	X	X	X
1	1	1	X	X	X	X	X	X	X	X	X

K₁ = Q₀

		Q ₁			
		00	01	11	10
Q ₂	0	X	X	1	0
	1	X	X	X	0

J₁ = Q₀ · Q₂

		Q ₁			
		00	01	11	10
Q ₂	0	0	1	X	X
	1	0	0	X	X

Exercise_02

Step 4: k-Maps

K-maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter

Next-state table for Mod-6 counter

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	0	0	0	X	1	0	X	X	1
1	1	0	X	X	X	X	X	X	X	X	X
1	1	1	X	X	X	X	X	X	X	X	X

K₂ = Q₀.

		Q ₁			
		00	01	11	10
Q ₂	0	X	X	X	X
	1	0	1	X	X

J₂ = Q₀.

		Q ₁			
		00	01	11	10
Q ₂	0	0	0	1	0
	1	x	x	X	X

Exercise_02



Step 5: Counter implementation

The final step is to implement the combinational logic from the expressions for the J and K inputs

$$J_0=1$$

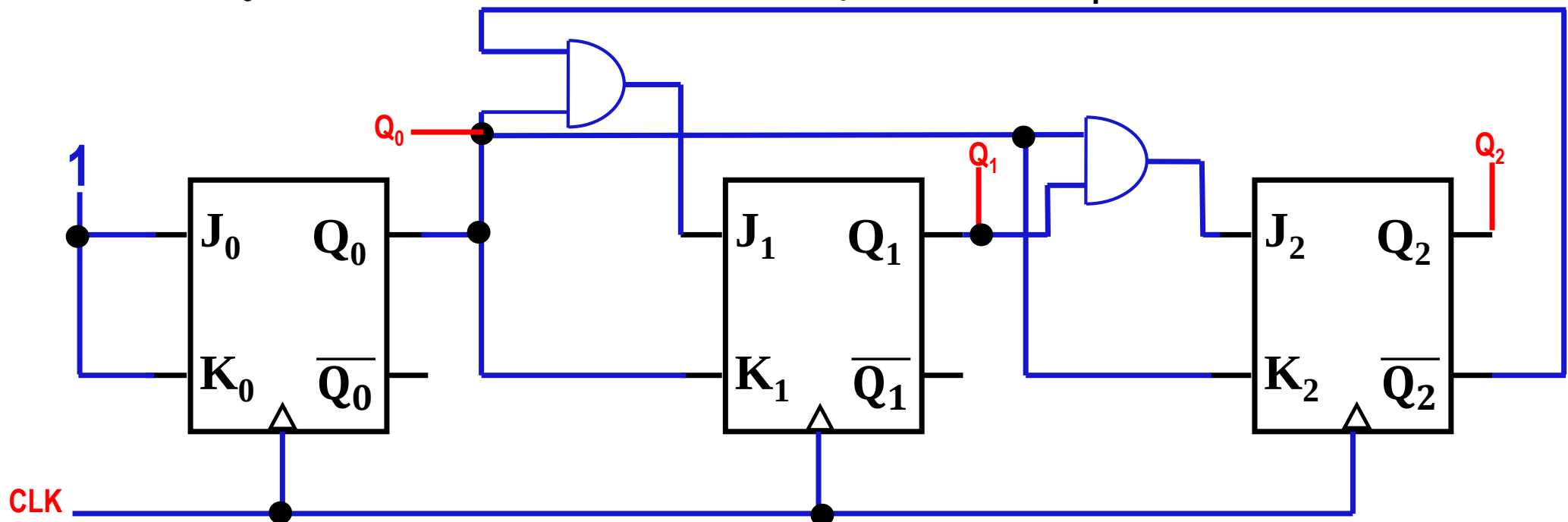
$$K_0=1$$

$$J_1=Q_0 \cdot \overline{Q_2}$$

$$K_1=Q_0$$

$$J_2=Q_0 \cdot Q_1$$

$$K_2=Q_0$$



Outlines



- ☐ Finite State Machine
- ☐ Asynchronous Counter
- ☐ Synchronous Counter
- ☐ Design of Synchronous Counters
- ☐ Exercise_01
- ☐ Exercise_02
- ☒ Exercise_03
- ☐ Exercise_04

Exercise_03

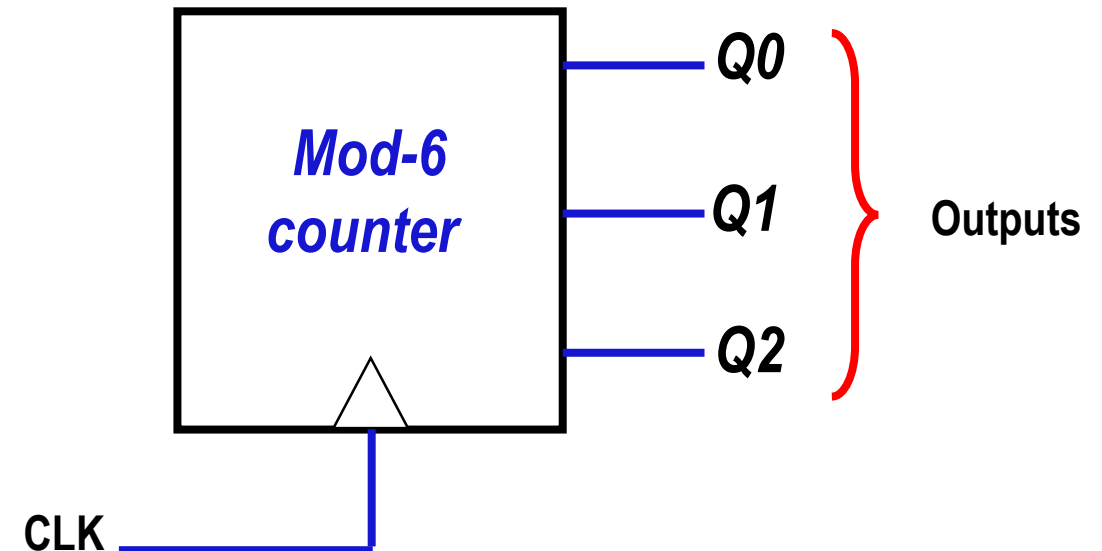


□ Design a synchronous counter with counting sequence: 000, 001, 011, 111, 110, 100, 000,...using JK flip flops.

Ans.

Mod-6 synchronous → 6 states

Mod-6 synchronous → needs 3 outputs

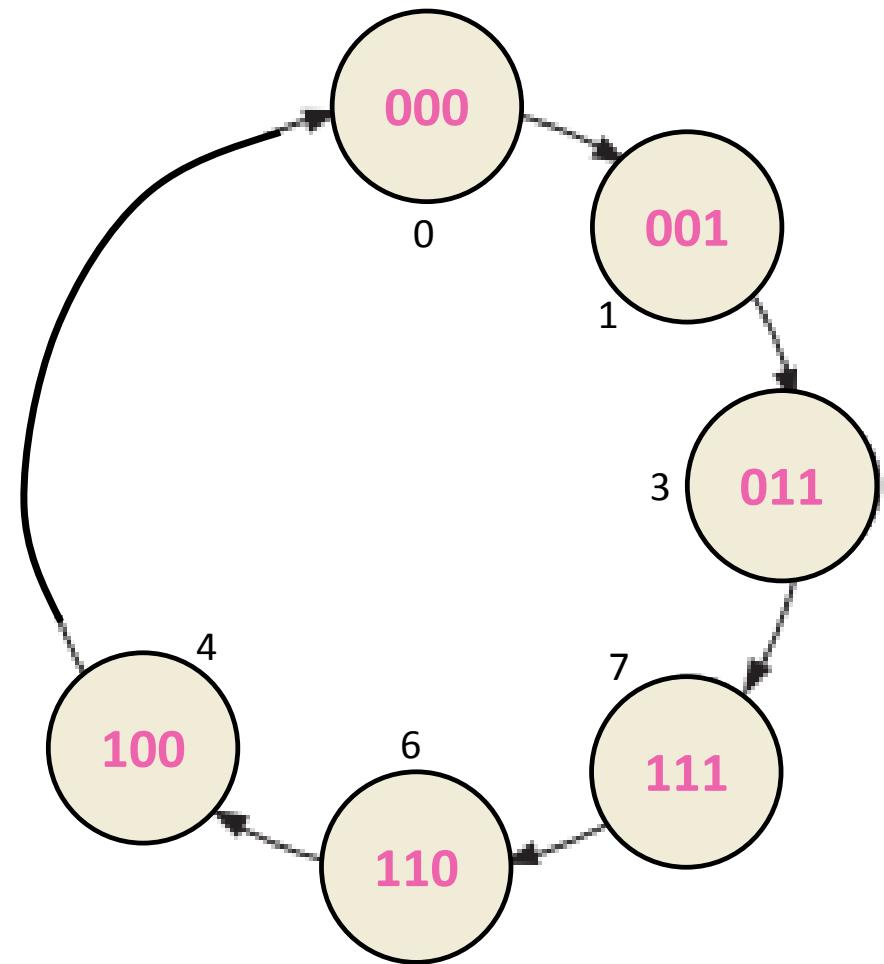
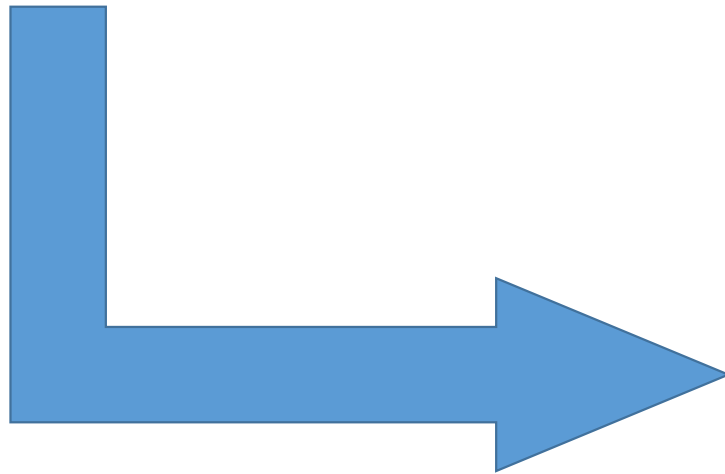


Exercise_03

□ Step 1: State Diagram



Mod-6 synchronous $\rightarrow$ 6 states
Up counter $\rightarrow$ $\{0, 1, 3, 7, 6, 4\}$



State Diagram
Mod-6 counter

Exercise_03



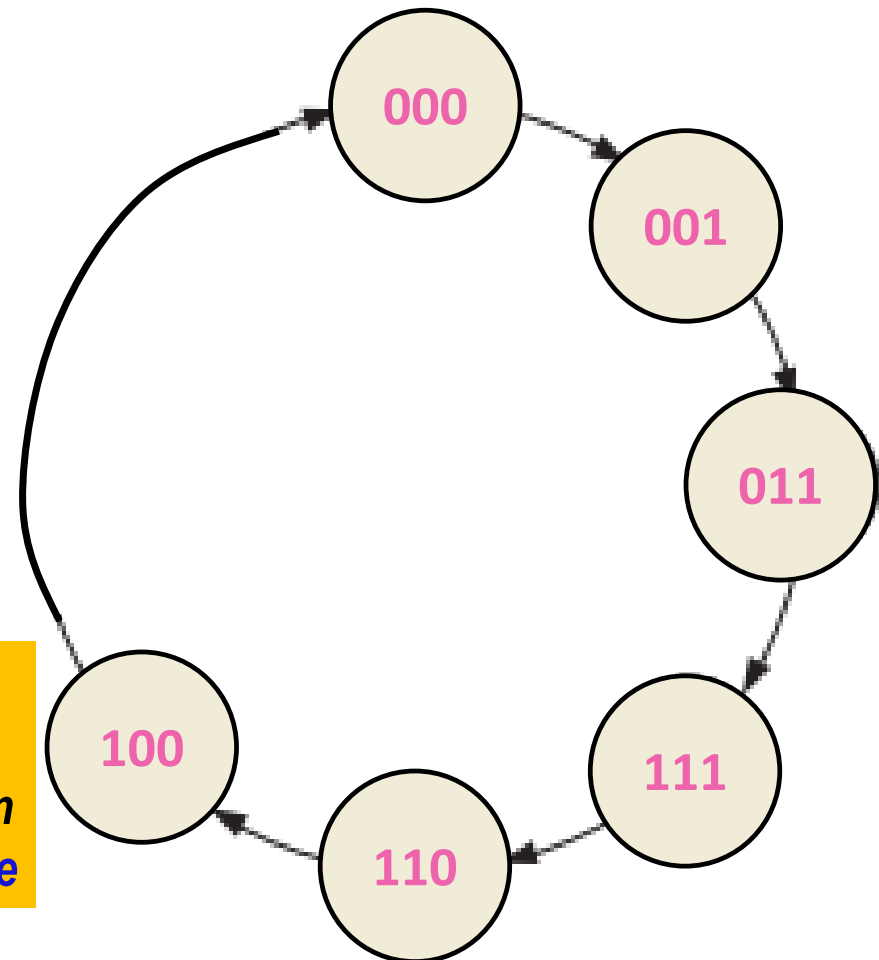
Step 2: Next-State Table

- Once the sequential circuit is defined by a state diagram, the **second step** is to derive a next-state table

Next-state table

Present State			Next State		
Q2	Q1	Q0	Q2	Q1	Q0

The **next state** is the state that the counter goes to from its present state upon application of a **clock pulse**



Exercise_03



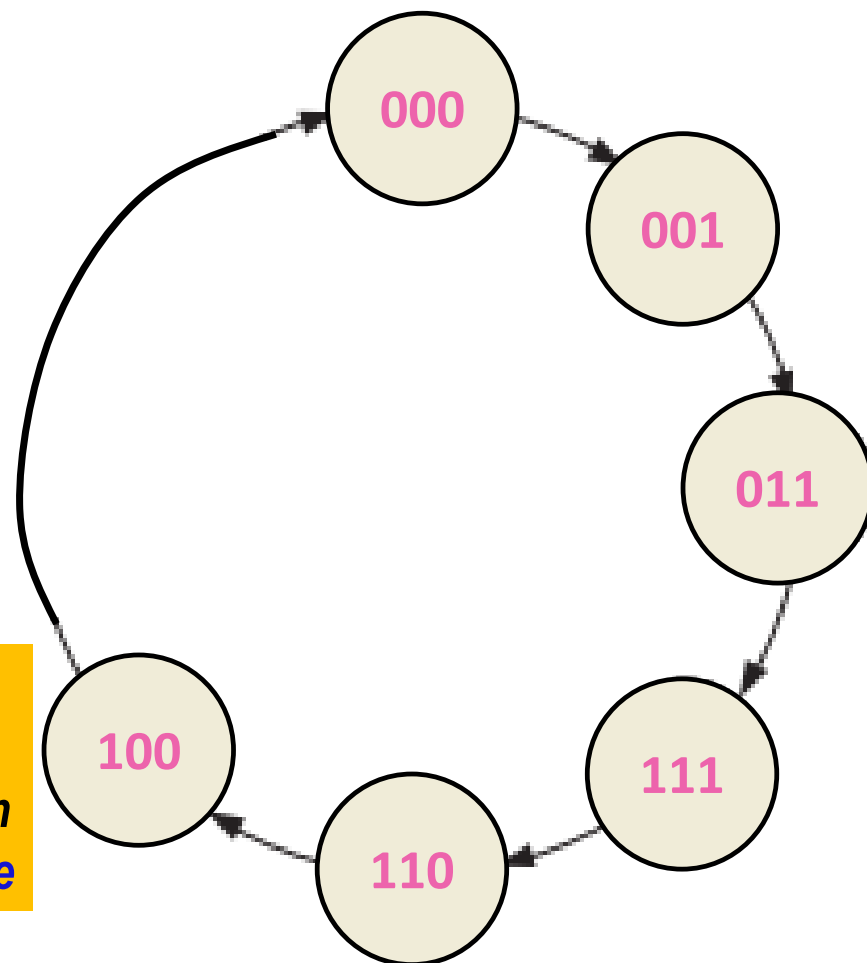
Step 2: Next-State Table

- Once the sequential circuit is defined by a state diagram, the **second step** is to derive a next-state table

Next-state table

Present State			Next State		
Q2	Q1	Q0	Q2	Q1	Q0
0	0	0	0	0	1
0	0	1	0	1	1
0	1	0	X	X	X
0	1	1	1	1	1
1	0	0	0	0	0
1	0	1	X	X	X
1	1	0	0	0	0
1	1	1	1	1	0

The **next state** is the state that the counter goes to from its present state upon application of a **clock pulse**



Exercise_03



❑ Step 3: Flip-Flop Transition Table

- ❑ For each **output transition**, the J and K **inputs** that will cause the output transition to occur are always the same and defined in table.

Next-state table

Present State			Next State		
Q2	Q1	Q0	Q2	Q1	Q0
0	0	0	0	0	1
0	0	1	0	1	1
0	1	0	X	X	X
0	1	1	1	1	1
1	0	0	0	0	0
1	0	1	X	X	X
1	1	0	0	0	0
1	1	1	1	1	0

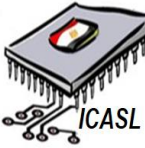
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q _N		Q _{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_03

Step 3: Flip-Flop Transition Table

of JK Flip-Flop = # of output of the counter. Here, # of JK Flip-Flop=3.

Next-state table

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1						
0	0	1	0	1	1						
0	1	0	X	X	X						
0	1	1	1	1	1						
1	0	0	0	0	0						
1	0	1	X	X	X						
1	1	0	0	0	0						
1	1	1	1	1	0						

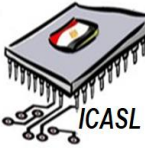
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_03

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table

Present State		Next State		Inputs of the JK-Flip Flop	
	Q0		Q0	J0	K0
	0		1		
	1		1		
	0		X		
	1		1		
	0		0		
	1		X		
	0		0		
	1		0		

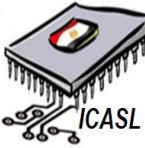
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_03

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table

Present State		Next State		Inputs of the JK-Flip Flop	
	Q0		Q0	J0	K0
	0		1	1	X
	1		1	X	0
	0		X	X	X
	1		1	X	0
	0		0	0	X
	1		X	X	X
	0		0	0	X
	1		0	X	1

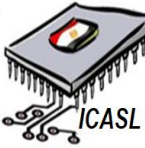
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q _N		Q _{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_03

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table

Present State		Next State		Inputs of the JK-Flip Flop	
	Q1		Q1	J1	K1
	0		0		
	0		1		
	1		X		
	1		1		
	0		0		
	0		X		
	1		0		
	1		1		

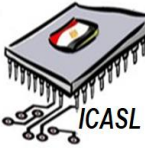
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_03

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table

Present State		Next State		Inputs of the JK-Flip Flop	
	Q1		Q1	J1	K1
	0		0	0	X
	0		1	1	X
	1		X	X	X
	1		1	X	0
	0		0	0	X
	0		X	X	X
	1		0	X	1
	1		1	X	0

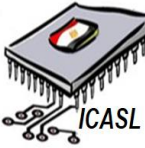
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_03

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table

Present State			Next State			Inputs of the JK-Flip Flop		
Q2			Q2			J2	K2	
0			0					
0			0					
0			X					
0			1					
1			0					
1			X					
1			0					
1			1					

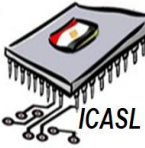
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_03

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table

Present State		Next State		Inputs of the JK-Flip Flop	
Q2		Q2		J2	K2
0		0		0	X
0		0		0	X
0		X		X	X
0		1		1	X
1		0		X	1
1		X		X	X
1		0		X	0
1		1		X	0

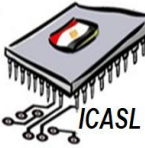
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_03

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	1	0	X	1	X	X	0
0	1	0	X	X	X	X	X	X	X	X	X
0	1	1	1	1	1	1	X	X	0	X	0
1	0	0	0	0	0	X	1	0	X	0	X
1	0	1	X	X	X	X	X	X	X	X	X
1	1	0	0	0	0	X	0	X	1	0	X
1	1	1	1	1	0	X	0	X	0	X	1

Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q _N		Q _{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care

Exercise_03

Step 4: k-Maps

K-maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter

Next-state table

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	1	0	X	1	X	X	0
0	1	0	X	X	X	X	X	X	X	X	X
0	1	1	1	1	1	1	X	X	0	X	0
1	0	0	0	0	0	X	1	0	X	0	X
1	0	1	X	X	X	X	X	X	X	X	X
1	1	0	0	0	0	X	0	X	1	0	X
1	1	1	1	1	0	X	0	X	0	X	1

K₀ = Q₂

		Q ₁ Q ₀			
		00	01	11	10
Q ₂	0	X	0	0	x
	1	X	X	1	x

J₀ = $\overline{Q_2}$

		Q ₁ Q ₀			
		00	01	11	10
Q ₂	0	1	X	X	X
	1	0	X	X	0

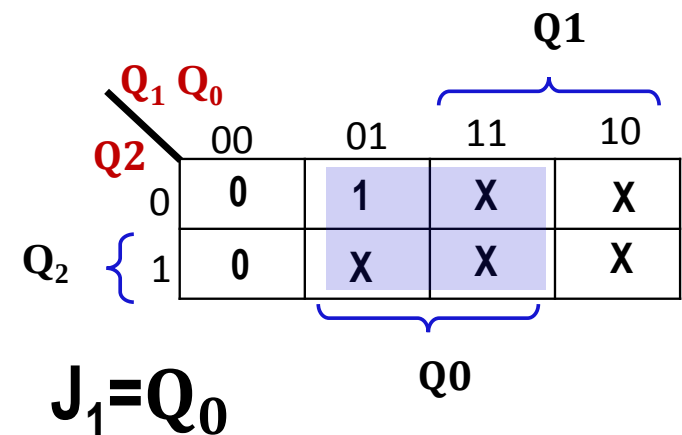
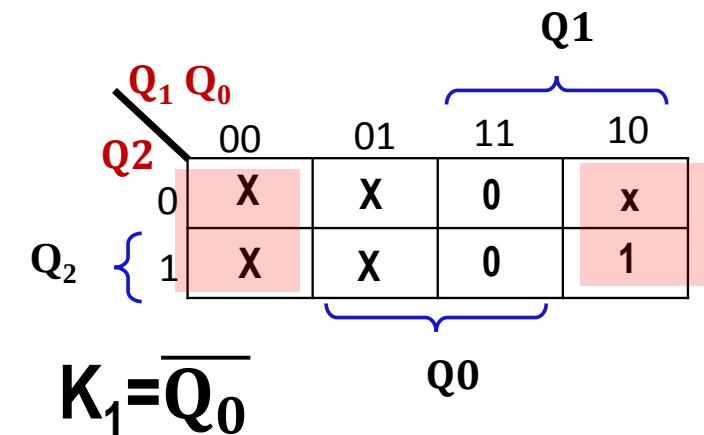
Exercise_03

Step 4: k-Maps

K-maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter

Next-state table

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	1	0	X	1	X	X	0
0	1	0	X	X	X	X	X	X	X	X	X
0	1	1	1	1	1	1	X	X	0	X	0
1	0	0	0	0	0	X	1	0	X	0	X
1	0	1	X	X	X	X	X	X	X	X	X
1	1	0	0	0	0	X	0	X	1	0	X
1	1	1	1	1	0	X	0	X	0	X	1



Exercise_03

Step 4: k-Maps

K-maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter

Next-state table

Present State			Next State			Inputs of the JK-Flip Flop					
Q2	Q1	Q0	Q2	Q1	Q0	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	1	0	X	1	X	X	0
0	1	0	X	X	X	X	X	X	X	X	X
0	1	1	1	1	1	1	X	X	0	X	0
1	0	0	0	0	0	X	1	0	X	0	X
1	0	1	X	X	X	X	X	X	X	X	X
1	1	0	0	0	0	X	0	X	1	0	X
1	1	1	1	1	0	X	0	X	0	X	1

K₂ = $\overline{Q_1}$

		Q ₁			
		00	01	11	10
Q ₂	0	X	X	X	x
	1	1	X	0	0

J₂ = Q₁

		Q ₁			
		00	01	11	10
Q ₂	0	0	0	1	X
	1	X	X	X	X

Exercise_03



Step 5: Counter implementation

The final step is to implement the combinational logic from the expressions for the J and K inputs

$$J_0 = \overline{Q_2}$$

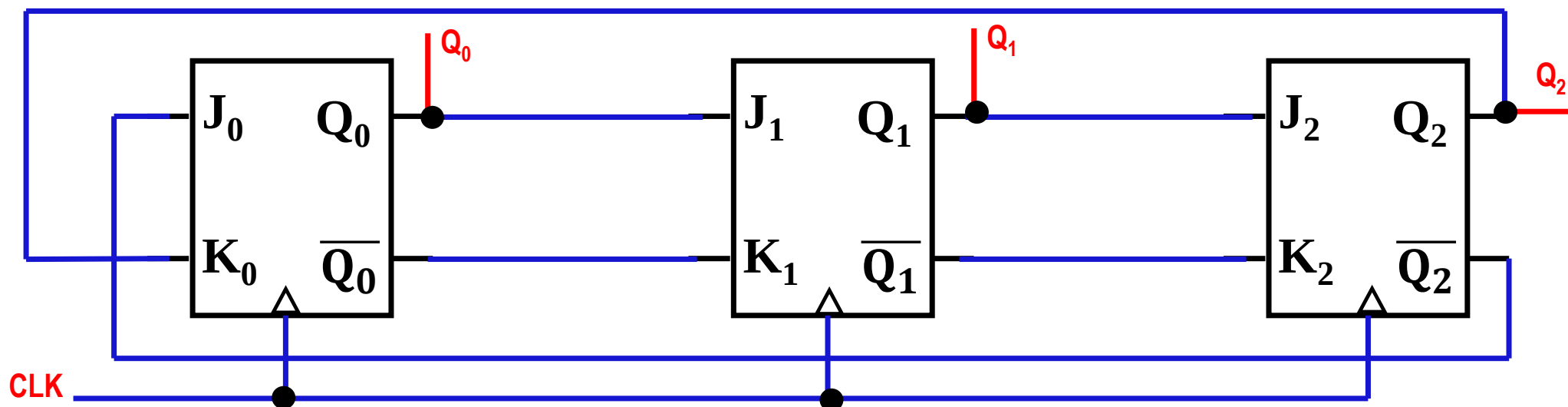
$$K_0 = Q_2$$

$$J_1 = Q_0$$

$$K_1 = \overline{Q_0}$$

$$J_2 = Q_1$$

$$K_2 = \overline{Q_1}$$



Outlines



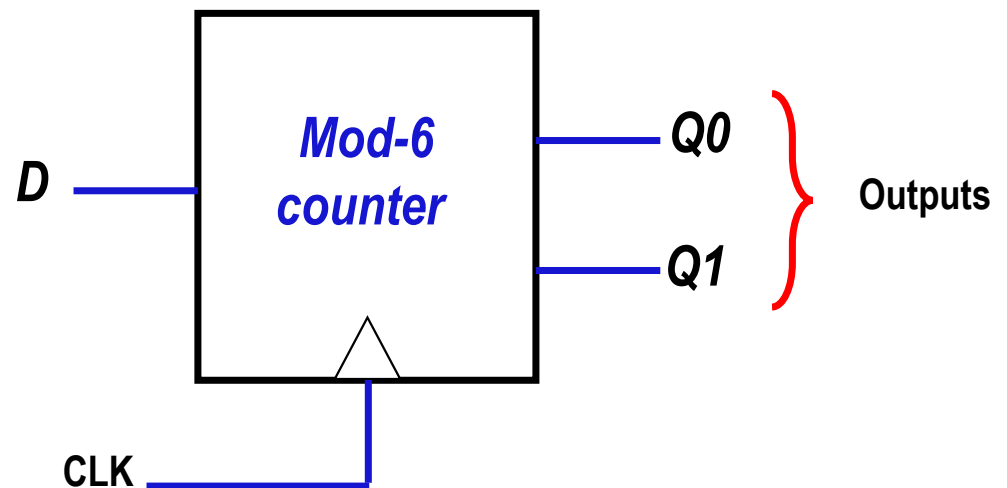
- ☐ Finite State Machine
- ☐ Asynchronous Counter
- ☐ Synchronous Counter
- ☐ Design of Synchronous Counters
- ☐ Exercise_01
- ☐ Exercise_02
- ☐ Exercise_03
- ☐ Exercise_04

Exercise_04

- Design a 2 bit up/down counter with an input D which determines the up/down function. Thus when D=0, the count sequence is 00,01,10,11,00 ... when D=1, the count sequence is 00,11,10,01,00 ...

Ans.

2 bit up/down counter $\rightarrow 2^2=4$ states

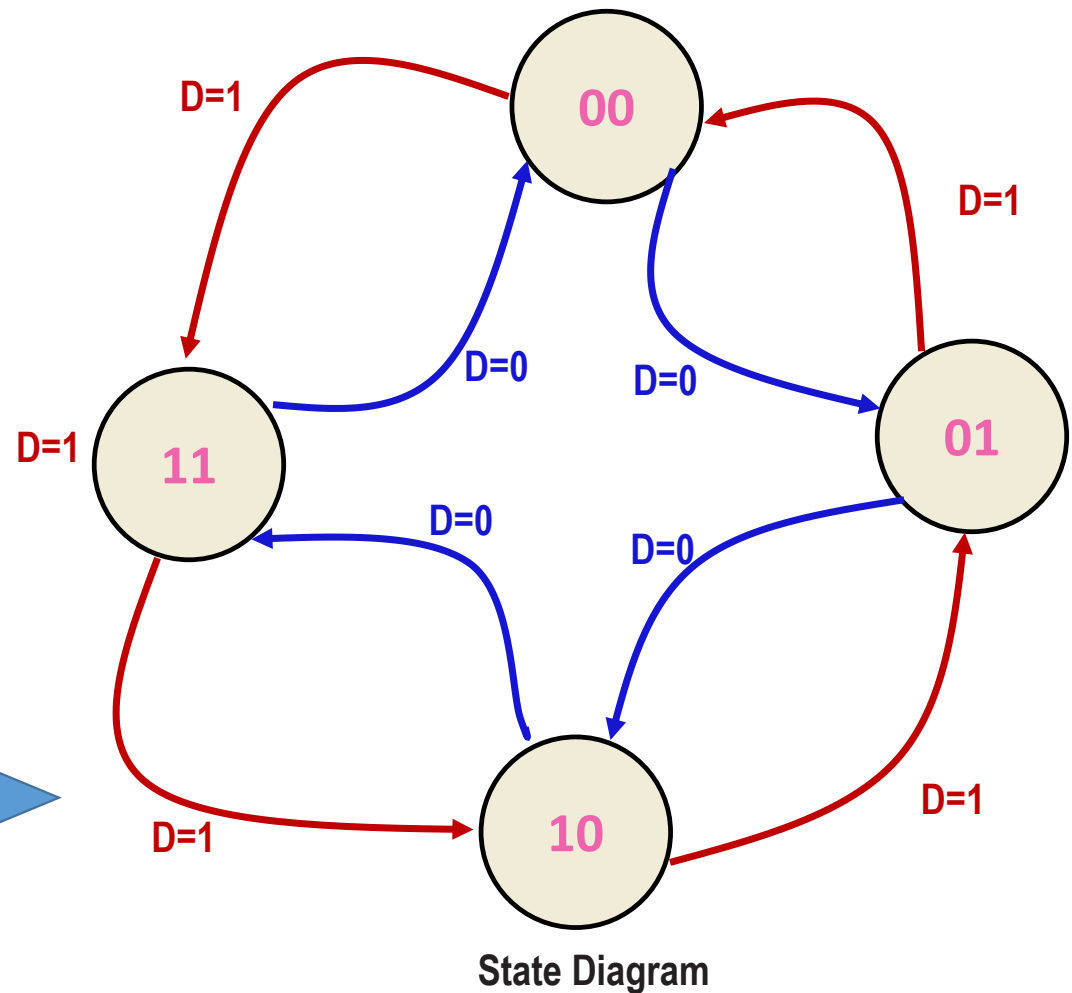


Exercise_04

□ Step 1: State Diagram



2-bit synchronous $\rightarrow 2^2 = 4$ states
Up counter $\rightarrow \{0,1,2,3\}$ @ $D=0$
Down counter $\rightarrow \{0,3,2,1\}$ @ $D=1$



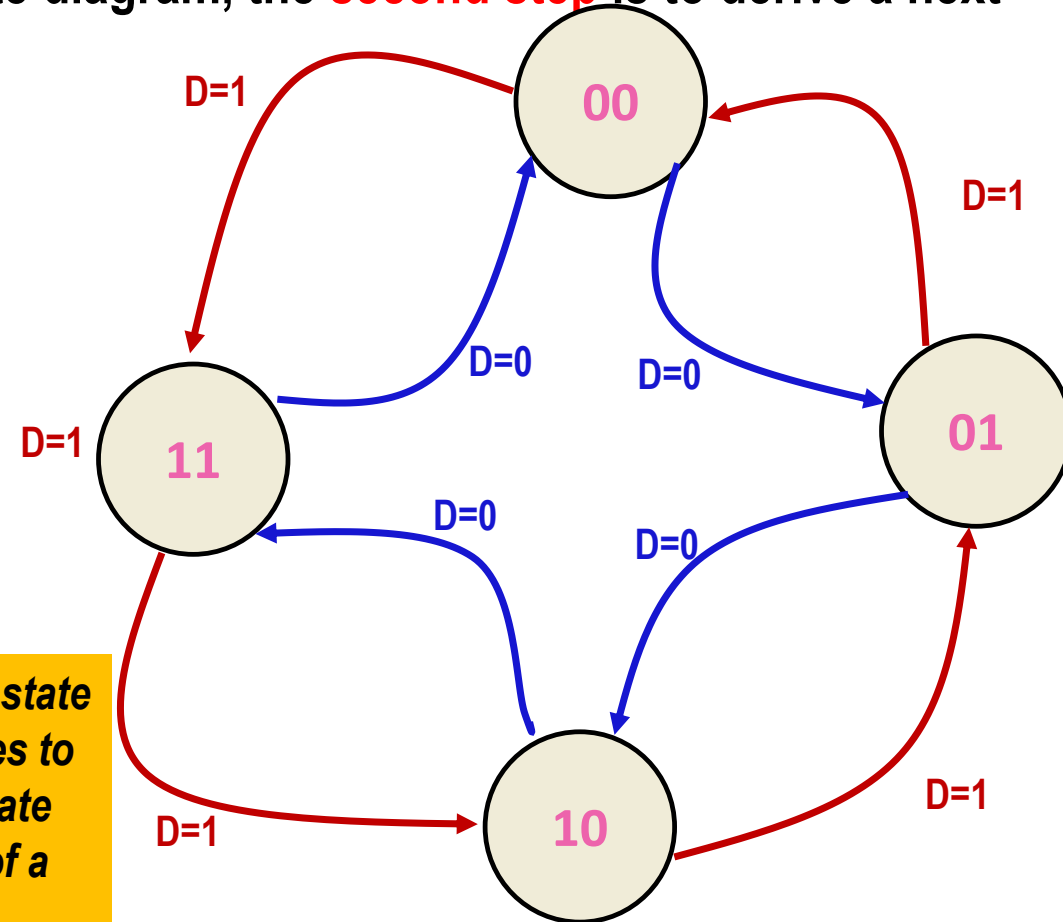
Exercise_04

Step 2: Next-State Table

- Once the sequential circuit is defined by a state diagram, the **second step** is to derive a next-state table

Next-state table

I/P	Present State		Next State	
D	Q1	Q0	Q1	Q0



The **next state** is the state that the counter goes to from its present state upon application of a **clock pulse**

State Diagram

Exercise_04

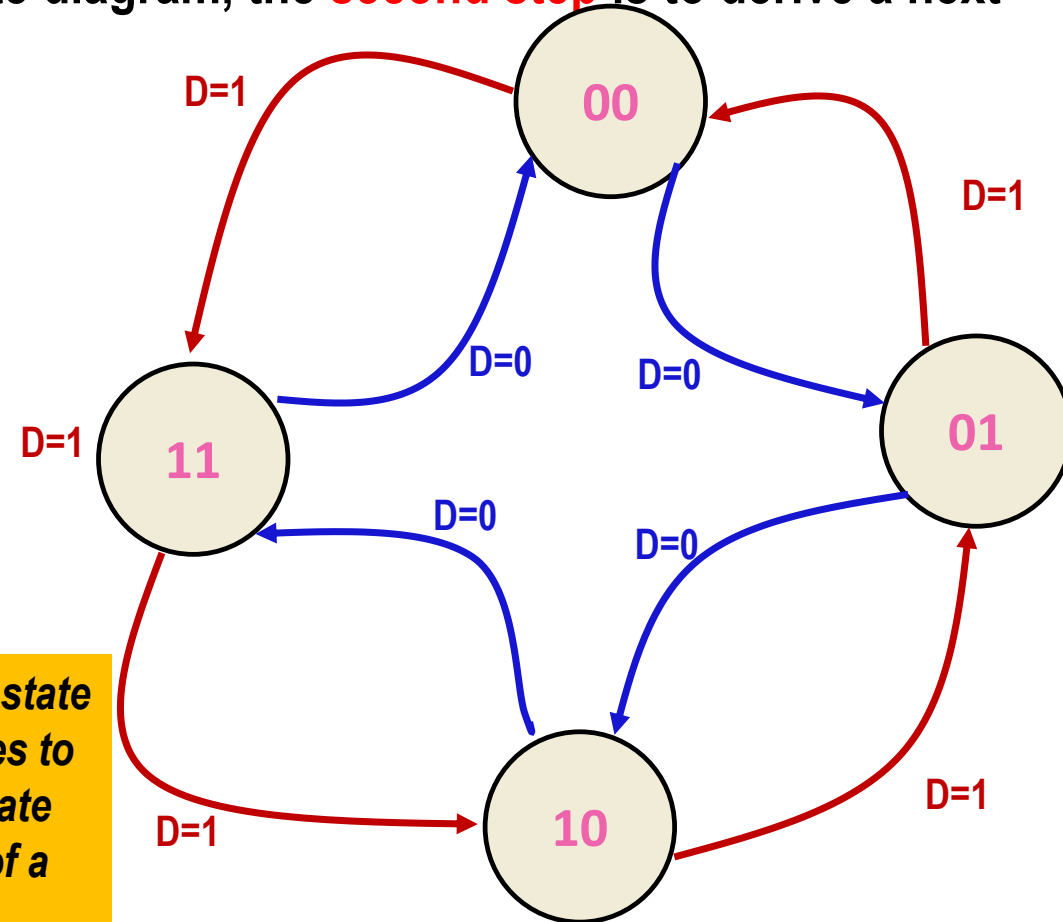
Step 2: Next-State Table

Once the sequential circuit is defined by a state diagram, the **second step** is to derive a next-state table

Next-state table

I/P	Present State		Next State	
D	Q1	Q0	Q1	Q0
0	0	0	0	1
0	0	1	1	0
0	1	0	1	1
0	1	1	0	0
1	0	0	1	1
1	0	1	0	0
1	1	0	0	1
1	1	1	1	0

The **next state** is the state that the counter goes to from its present state upon application of a **clock pulse**



State Diagram

Exercise_04



Step 3: Flip-Flop Transition Table

- For each **output transition**, the J and K **inputs** that will cause the output transition to occur are always the same and defined in table.

Next-state table

I/P	Present State		Next State	
D	Q1	Q0	Q1	Q0
0	0	0	0	1
0	0	1	1	0
0	1	0	1	1
0	1	1	0	0
1	0	0	1	1
1	0	1	0	0
1	1	0	0	1
1	1	1	1	0

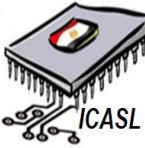
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_04

Step 3: Flip-Flop Transition Table

of JK Flip-Flop = # of output of the counter. Here, # of JK Flip-Flop=2.

Next-state table

I/P	Present State		Next State	
	Q1	Q0	Q1	Q0
0	0	0	0	1
0	0	1	1	0
0	1	0	1	1
0	1	1	0	0
1	0	0	1	1
1	0	1	0	0
1	1	0	0	1
1	1	1	1	0

Inputs of the JK-Flip Flop			
J1	K1	J0	K0

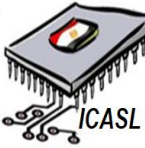
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_04

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table

I/P	Present State		Next State		Inputs of the JK-Flip Flop	
D		Q0		Q0	J0	K0
0		0		1		
0		1		0		
0		0		1		
0		1		0		
1		0		1		
1		1		0		
1		0		1		
1		1		0		

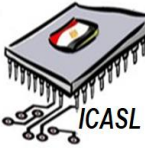
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q _N		Q _{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_04

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table

I/P	Present State		Next State		Inputs of the JK-Flip Flop		
D		Q0		Q0		J0	K0
0		0		1		1	X
0		1		0		X	1
0		0		1		1	X
0		1		0		X	1
1		0		1		1	X
1		1		0		X	1
1		0		1		1	X
1		1		0		X	1

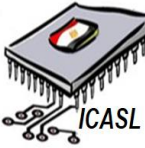
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q _N		Q _{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_04

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table

I/P	Present State		Next State		Inputs of the JK-Flip Flop		
D	Q1		Q1		J1	K1	
0	0		0				
0	0		1				
0	1		1				
0	1		0				
1	0		1				
1	0		0				
1	1		0				
1	1		1				

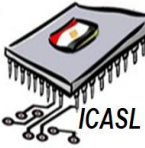
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_04

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table

I/P	Present State		Next State		Inputs of the JK-Flip Flop		
D	Q1		Q1		J1	K1	
0	0		0		0	X	
0	0		1		1	X	
0	1		1		X	0	
0	1		0		X	1	
1	0		1		1	X	
1	0		0		0	X	
1	1		0		X	1	
1	1		1		X	0	

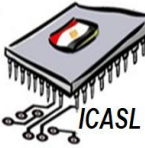
Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care



Exercise_04

Step 3: Flip-Flop Transition Table

Map between the each output transition and the inputs of JK Flip-Flop

Next-state table

I/P	Present State		Next State	
D	Q1	Q0	Q1	Q0
0	0	0	0	1
0	0	1	1	0
0	1	0	1	1
0	1	1	0	0
1	0	0	1	1
1	0	1	0	0
1	1	0	0	1
1	1	1	1	0

Inputs of the JK-Flip Flop			
J1	K1	J0	K0
0	X	1	X
1	X	X	1
X	0	1	X
X	1	X	1
1	X	1	X
0	X	X	1
X	1	1	X
X	0	X	1

Transition table for a J-K flip-flop

Output Transistors			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : Present State

Q_{N+1} : Next State

X: Don't Care

Exercise_04

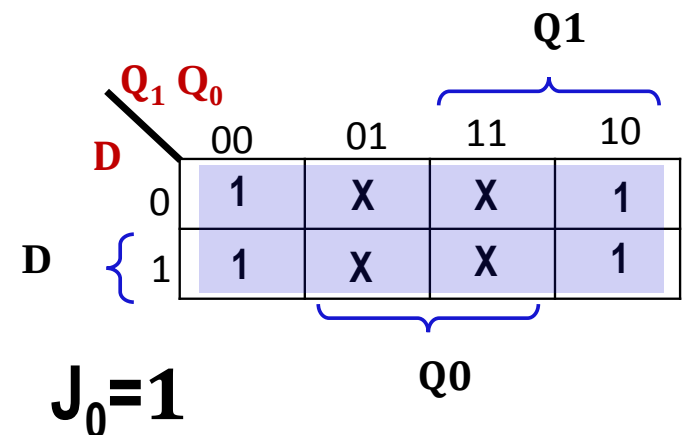
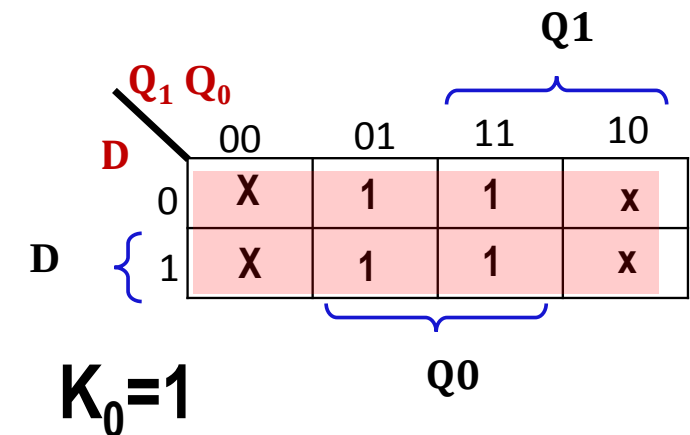
Step 4: k-Maps

K-maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter

Next-state table

I/P	Present State		Next State	
D	Q1	Q0	Q1	Q0
0	0	0	0	1
0	0	1	1	0
0	1	0	1	1
0	1	1	0	0
1	0	0	1	1
1	0	1	0	0
1	1	0	0	1
1	1	1	1	0

Inputs of the JK-Flip Flop			
J1	K1	J0	K0
0	X	1	X
1	X	X	1
X	0	1	X
X	1	X	1
1	X	1	X
0	X	X	1
X	1	1	X
X	0	X	1



Exercise_04

Step 4: k-Maps

K-maps can be used to determine the logic required for the J and K inputs of each flip-flop in the counter

Next-state table

I/P	Present State		Next State	
D	Q1	Q0	Q1	Q0
0	0	0	0	1
0	0	1	1	0
0	1	0	1	1
0	1	1	0	0
1	0	0	1	1
1	0	1	0	0
1	1	0	0	1
1	1	1	1	0

Inputs of the JK-Flip Flop			
J1	K1	J0	K0
0	X	1	X
1	X	X	1
X	0	1	X
X	1	X	1
1	X	1	X
0	X	X	1
X	1	1	X
X	0	X	1

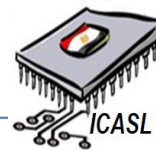
		Q1			
		00	01	11	10
D	0	X	X	1	0
	1	X	X	0	1

$$K_0 = \overline{D} \cdot Q_0 + D \cdot \overline{Q_0} \\ = Q_0 \oplus D$$

		Q1			
		00	01	11	10
D	0	0	1	X	X
	1	1	0	X	X

$$J_0 = \overline{D} \cdot Q_0 + D \cdot \overline{Q_0} \\ = Q_0 \oplus D$$

Exercise_04



Step 5: Counter implementation

The final step is to implement the combinational logic from the expressions for the J and K inputs

$$J_0 = 1$$

$$K_0 = 1$$

$$J_1 = Q_0 \oplus D$$

$$K_1 = Q_0 \oplus D$$

